

AWS 入門

aoki-taquan

2026 年 04 月 21 日

目次

1	はじめに	35
1.1	AWS とは	35
1.2	クラウドの基本概念	35
1.2.1	クラウドコンピューティングの定義	35
1.2.2	サービスモデル	35
1.3	責任共有モデル	36
1.4	グローバルインフラストラクチャ	36
1.4.1	リージョンとアベイラビリティゾーン	36
1.4.2	リージョン選択の考え方	36
1.4.3	リージョンをまたがないリソース	36
1.5	料金モデル	37
1.5.1	従量課金制	37
1.5.2	主な料金構成要素	37
1.5.3	割引の仕組み	37
1.5.4	無料利用枠 (Free Tier)	37
1.6	操作手段	38
1.7	本書の対象読者	38
1.8	前提知識	38
1.9	本書の進め方	38
2	アカウント開設と初期設定	39
2.1	AWS アカウントの開設	39
2.1.1	開設に必要なもの	39
2.1.2	開設手順の概要	39
2.2	ルートユーザーの保護	39
2.2.1	ルートユーザーに必須の設定	39
2.2.2	ルートユーザーでしかできない作業	40
2.3	請求アラートの設定	40
2.3.1	IAM ユーザーによる請求情報の閲覧許可	40
2.3.2	AWS Budgets で予算アラートを作る	40
2.3.3	無料利用枠アラート	40
2.4	IAM Identity Center (旧 AWS SSO)	41
2.4.1	従来の IAM ユーザー方式との違い	41
2.4.2	基本的なセットアップ手順	41
2.4.3	CLI からのログイン	41
2.5	作業用 IAM ユーザーの作成 (Identity Center を使わない場合)	42
2.6	リージョンの選択	42
2.7	AWS CLI のセットアップ	42
2.7.1	インストール	42
2.7.2	認証情報の設定	43

2.7.3	動作確認	43
2.8	初期設定のチェックリスト	43
3	IAM (認証・認可)	45
3.1	IAM の主要コンポーネント	45
3.1.1	4つの基本要素	45
3.1.2	認証と認可の流れ	45
3.2	IAM ポリシーの構造	45
3.2.1	主要フィールド	46
3.2.2	ARN (Amazon Resource Name)	46
3.2.3	条件 (Condition)	46
3.2.4	必要な IAM アクションの調べ方	47
3.3	ポリシーの種類	47
3.3.1	管理ポリシーとインラインポリシー	47
3.3.2	アイデンティティベースとリソースベース	47
3.3.3	SCP (サービスコントロールポリシー)	48
3.4	ロールと一時認証情報	48
3.4.1	なぜロールが必要か	48
3.4.2	ロールの使われ方	48
3.4.3	信頼ポリシーとアクセスポリシー	48
3.4.4	GitHub Actions から OIDC で引き受ける例	48
3.5	ベストプラクティス	49
3.5.1	最小権限の原則	49
3.5.2	権限境界 (Permissions Boundary)	49
3.5.3	実装上の推奨	49
3.5.4	アクセスキーを CI に置かない	50
3.6	実習: 権限不足を体験する	50
3.7	よく使う AWS 管理ポリシー	50
4	VPC (ネットワーク)	51
4.1	VPC の基本構造	51
4.1.1	階層	51
4.1.2	CIDR ブロックの設計	51
4.1.3	デフォルト VPC	51
4.2	サブネットとルートテーブル	52
4.2.1	パブリックサブネットとプライベートサブネット	52
4.2.2	ルートテーブルの役割	52
4.3	インターネット接続	52
4.3.1	Internet Gateway (IGW)	52
4.3.2	NAT Gateway	52
4.3.3	外向き専用 Internet Gateway (Egress-Only IGW)	53
4.3.4	パブリック IP	53
4.4	セキュリティグループとネットワーク ACL	53
4.4.1	セキュリティグループ (SG)	53

4.4.2	SG 間参照	54
4.4.3	ネットワーク ACL (NACL)	54
4.5	VPC エンドポイント	54
4.5.1	ゲートウェイ型エンドポイント	54
4.5.2	インタフェース型エンドポイント	55
4.6	VPC 間・オンプレ接続	55
4.6.1	VPC ピアリング	55
4.6.2	Transit Gateway	55
4.6.3	Site-to-Site VPN	55
4.6.4	Direct Connect	55
4.7	DNS と DHCP	55
4.7.1	VPC のデフォルト DNS	55
4.7.2	DNS ホスト名と DNS 解決	55
4.7.3	プライベートホストゾーン	56
4.8	実践：小規模な Web 構成の VPC 設計	56
4.9	VPC 作成時の注意点	56
5	EC2 (仮想マシン)	57
5.1	EC2 の基本概念	57
5.1.1	インスタンス	57
5.1.2	料金体系	57
5.2	AMI (Amazon Machine Image)	57
5.2.1	主な AMI の出所	57
5.2.2	Amazon Linux 2023	58
5.2.3	カスタム AMI の作り方	58
5.3	インスタンスタイプ	58
5.3.1	主要ファミリー	58
5.3.2	世代と CPU アーキテクチャ	58
5.3.3	t ファミリーのクレジット仕様	59
5.4	ストレージ (EBS / インスタンスストア)	59
5.4.1	EBS (Elastic Block Store)	59
5.4.2	インスタンスストア	59
5.4.3	ルートボリュームの扱い	59
5.5	インスタンスメタデータサービス (IMDSv2)	60
5.6	ネットワーク関連	60
5.6.1	SSM Session Manager (推奨)	60
5.6.2	SSH による接続 (必要な場合のみ)	60
5.6.3	Elastic IP	61
5.7	Auto Scaling とロードバランサ	61
5.7.1	ロードバランサ (ELB)	61
5.7.2	ターゲットグループとヘルスチェック	61
5.7.3	Auto Scaling グループ (ASG)	61
5.8	ユーザーデータと初期化	62

5.9	実習：ALB + EC2 の最小構成	62
5.10	トラブルシューティング	62
6	S3 (オブジェクトストレージ)	63
6.1	S3 の基本概念	63
6.1.1	バケットとオブジェクト	63
6.1.2	耐久性と可用性	63
6.1.3	バケット名の制約	63
6.2	ストレージクラス	63
6.2.1	ライフサイクルルール	64
6.3	操作の基本	64
6.3.1	CLI によるアップロード／ダウンロード	64
6.3.2	プリサインド URL	65
6.4	アクセス制御	65
6.4.1	階層	65
6.4.2	推奨デフォルト	65
6.4.3	バケットポリシーの例	66
6.4.4	ブロックパブリックアクセスのバイパス	66
6.5	バージョニングとオブジェクトロック	67
6.5.1	バージョニング	67
6.5.2	オブジェクトロック (WORM)	67
6.6	静的ウェブサイトホスティング	67
6.7	イベント通知	68
6.8	クロスリージョンレプリケーションと S3 Replication Time Control	68
6.9	コスト最適化のポイント	68
6.10	S3 のよくあるつまずき	69
7	データベース	70
7.1	サービス選択の早見表	70
7.2	RDS (Relational Database Service)	70
7.2.1	サポートエンジン	70
7.2.2	インスタンスクラスとストレージ	70
7.2.3	高可用性：Multi-AZ 配置	71
7.2.4	リードレプリカ	71
7.2.5	バックアップと復元	71
7.2.6	パラメータグループとオプショングループ	71
7.3	Aurora	71
7.3.1	Aurora の特徴	71
7.3.2	Aurora Serverless v2	71
7.3.3	Aurora Global Database	72
7.4	DynamoDB	72
7.4.1	基本構造	72
7.4.2	料金モード	72
7.4.3	Single Table Design	72

7.4.4	DynamoDB Streams	72
7.4.5	Global Tables	72
7.5	ElastiCache	72
7.5.1	選び方	73
7.5.2	典型的なユースケース	73
7.5.3	クラスタモード	73
7.6	DB 選定のガイドライン	73
7.6.1	「まず RDBMS」で始める	73
7.6.2	マネージドかセルフホストか	73
7.6.3	商用ライセンスの扱い	73
7.6.4	ユースケース別の目安	73
7.7	DB 周りのネットワーク	74
7.8	バックアップとリカバリのベストプラクティス	74
7.9	コスト最適化のポイント	74
8	コンテナとサーバーレス	76
8.1	コンピュータ選択のスペクトラム	76
8.1.1	初学者はどこから始めればよいか	76
8.2	Lambda（サーバーレス関数）	76
8.2.1	実行モデル	76
8.2.2	料金モデル	77
8.2.3	コールドスタート	77
8.2.4	シンプルな Lambda の例（Python）	77
8.2.5	Lambda のユースケース	77
8.2.6	制限	78
8.3	API Gateway	78
8.3.1	種類	78
8.3.2	Lambda Proxy 連携	78
8.3.3	認証／認可	78
8.4	ECS / EKS / Fargate	78
8.4.1	ECS（Elastic Container Service）	78
8.4.2	EKS（Elastic Kubernetes Service）	78
8.4.3	Fargate	79
8.4.4	ECR（Elastic Container Registry）	79
8.4.5	ECS タスク定義の例	79
8.5	App Runner / Elastic Beanstalk / Amplify	80
8.5.1	App Runner	80
8.5.2	Elastic Beanstalk	80
8.5.3	Amplify	80
8.6	イベント駆動・非同期アーキテクチャ	80
8.6.1	SQS + Lambda の基本パターン	81
8.6.2	Step Functions	81
8.7	サーバーレス／コンテナの選定指針	81

8.8	参考アーキテクチャ：サーバーレス Web アプリ	81
9	運用と監視	83
9.1	CloudWatch	83
9.1.1	CloudWatch Metrics	83
9.1.2	アラーム	83
9.1.3	CloudWatch Logs	83
9.1.4	Logs Insights	84
9.1.5	ダッシュボード	84
9.1.6	CloudWatch Alarms の通知をメール以外に	84
9.2	CloudTrail	84
9.2.1	イベント種別	84
9.2.2	証跡 (Trail)	84
9.2.3	主要ログフィールド	85
9.3	AWS Config	85
9.3.1	典型的な利用シーン	85
9.4	Systems Manager (SSM)	85
9.4.1	主要機能	85
9.4.2	Session Manager のメリット	86
9.4.3	Parameter Store	86
9.4.4	Patch Manager	86
9.5	監視・運用の全体像	86
9.6	タグ戦略	87
9.7	アラート疲れを防ぐ	87
9.8	運用のベストプラクティス	87
10	セキュリティとコスト管理	88
10.1	セキュリティの全体像	88
10.2	KMS (Key Management Service)	88
10.2.1	キーの種類	88
10.2.2	キーポリシーと IAM	88
10.2.3	エンベロープ暗号化	88
10.3	Secrets Manager と Parameter Store	89
10.3.1	使い分け	89
10.3.2	DB パスワードのローテーション	89
10.4	WAF と Shield	89
10.4.1	AWS WAF	89
10.4.2	Shield / Shield Advanced	89
10.5	GuardDuty / Inspector / Macie / Security Hub	90
10.5.1	GuardDuty	90
10.5.2	Inspector	90
10.5.3	Macie	90
10.5.4	Security Hub	90
10.6	IAM Access Analyzer	90

10.7	AWS Organizations と SCP	90
10.7.1	管理構造	91
10.7.2	SCP (サービスコントロールポリシー)	91
10.7.3	Control Tower	91
10.8	コスト管理サービス	92
10.8.1	AWS Budgets	92
10.8.2	Cost Explorer	92
10.8.3	Cost and Usage Report (CUR)	92
10.8.4	Savings Plans と Reserved Instance	92
10.8.5	Spot Instance	92
10.8.6	コスト最適化のチェックリスト	92
10.9	セキュリティ+コストで最低限やること	93
11	IaC と運用 Tips	94
11.1	なぜ IaC が必要か	94
11.2	CloudFormation	94
11.2.1	基本概念	94
11.2.2	最小テンプレート例	94
11.2.3	適用	95
11.2.4	特徴	95
11.2.5	SAM (Serverless Application Model)	95
11.3	AWS CDK (Cloud Development Kit)	96
11.3.1	特徴	96
11.3.2	TypeScript での例	96
11.3.3	デプロイフロー	96
11.3.4	CloudFormation との使い分け	97
11.4	Terraform	97
11.4.1	特徴	97
11.4.2	例	97
11.4.3	コマンド	98
11.4.4	state 管理の注意	98
11.5	IaC ツール選びのガイドライン	98
11.6	運用 Tips とハマりどころ	99
11.6.1	リージョン設定の一貫性	99
11.6.2	名前衝突の回避	99
11.6.3	削除できないリソース	99
11.6.4	マルチアカウント設計	99
11.6.5	CI/CD での認証	99
11.6.6	よくある事故と回避	99
11.6.7	リソース棚卸しの習慣	100
11.7	さらに学ぶために	100
12	Route 53 と CloudFront	101
12.1	Route 53 の概要	101

12.1.1	DNS の基礎おさらい	101
12.1.2	Route 53 の主要コンポーネント	101
12.2	ホストゾーン	101
12.2.1	パブリックホストゾーン	101
12.2.2	プライベートホストゾーン	102
12.3	レコードと Alias	102
12.3.1	Alias レコードの利点	102
12.4	ルーティングポリシー	102
12.4.1	Weighted の例	103
12.4.2	Failover の例	103
12.5	ヘルスチェック	103
12.6	ドメイン登録と DNSSEC	103
12.7	Route 53 Resolver と内部 DNS	104
12.7.1	Route 53 Resolver DNS Firewall	104
12.8	Route 53 Profiles	104
12.9	Route 53 の料金感	104
12.10	CloudFront の概要	104
12.10.1	主要メリット	104
12.11	ディストリビューションとオリジン	104
12.11.1	オリジンの種類	105
12.12	ビヘイビアとポリシー	105
12.12.1	TTL の優先順位	105
12.13	OAC (Origin Access Control)	105
12.14	署名付き URL / 署名付き Cookie	106
12.15	エッジコンピューティング	106
12.15.1	CloudFront Functions	106
12.15.2	Lambda@Edge	106
12.16	WAF / Shield 連携	107
12.17	ログとメトリクス	107
12.18	カスタムドメインと ACM	107
12.19	キャッシュ無効化	107
12.20	CloudFront の料金感	107
12.21	統合パターン	108
12.21.1	静的サイト	108
12.21.2	API バックエンド	108
12.21.3	マルチリージョン	108
12.22	Global Accelerator との比較	108
12.23	よくあるトラブル	109
12.24	学習の進め方	109
13	追加のストレージサービス	110
13.1	ストレージ選択の早見表	110
13.2	EFS (Elastic File System)	110

13.2.1	主な特徴	110
13.2.2	ストレージクラス	110
13.2.3	スループットモード	111
13.2.4	マウント例	111
13.2.5	アクセスポイント	111
13.2.6	典型ユースケース	111
13.2.7	料金感	111
13.3	FSx ファミリー	112
13.3.1	FSx for Windows File Server	112
13.3.2	FSx for Lustre	112
13.3.3	FSx for OpenZFS	112
13.3.4	FSx for NetApp ONTAP	112
13.3.5	FSx の選び方	112
13.4	Storage Gateway	113
13.4.1	File Gateway	113
13.4.2	Volume Gateway	113
13.4.3	Tape Gateway	113
13.5	AWS Backup	113
13.5.1	対応リソース	113
13.5.2	主要概念	113
13.5.3	クロスリージョン・クロスアカウントコピー	114
13.5.4	ボールトロック (Vault Lock)	114
13.5.5	AWS Backup Audit Manager	114
13.6	Snow ファミリー	114
13.7	DataSync	114
13.8	ストレージ コスト最適化のコツ	115
13.9	ベンチマーク・パフォーマンス計測	115
13.10	よくあるトラブル	115
14	メッセージングと統合	116
14.1	イベント駆動アーキテクチャの全体像	116
14.1.1	同期 vs 非同期	116
14.1.2	Push vs Pull	116
14.1.3	サービス早見表	116
14.2	SQS (Simple Queue Service)	116
14.2.1	Standard キュー vs FIFO キュー	116
14.3	主要パラメータ	117
14.4	Lambda トリガー	117
14.5	ベストプラクティス	117
14.6	SNS (Simple Notification Service)	117
14.6.1	サブスクライバー (プロトコル)	118
14.6.2	ファンアウトパターン	118
14.6.3	FIFO トピック	118

14.6.4	メッセージフィルタリング	118
14.6.5	配信エラーと DLQ	118
14.7	EventBridge	118
14.7.1	バスの種類	118
14.7.2	ルールとターゲット	119
14.7.3	スケジュール	119
14.7.4	アーカイブとリプレイ	119
14.7.5	EventBridge Pipes	119
14.7.6	スキーマレジストリ	119
14.8	Step Functions	120
14.8.1	2 種類のワークフロー	120
14.8.2	ステートタイプ	120
14.8.3	Service Integration Patterns	120
14.8.4	ASL (Amazon States Language) の例	120
14.8.5	Workflow Studio	121
14.8.6	料金感	121
14.9	API Gateway 詳細	121
14.9.1	HTTP API vs REST API vs WebSocket API	121
14.9.2	ステージとデプロイ	122
14.9.3	カスタムドメイン	122
14.9.4	認証	122
14.9.5	マッピングテンプレート (VTL)	122
14.9.6	統合タイプ	122
14.9.7	WebSocket API	123
14.10	AppSync	123
14.10.1	主な特徴	123
14.10.2	いつ使うか	123
14.11	MQ (Managed Broker)	123
14.12	統合パターン集	123
14.12.1	ファンアウト	123
14.12.2	バッファリング	124
14.12.3	Saga / 長時間トランザクション	124
14.12.4	Choreography (コレオグラフィー)	124
14.12.5	Outbox パターン	124
14.12.6	人手承認	124
14.13	よくあるトラブル	124
15	ストリーミングとデータ分析	126
15.1	データ分析の全体像	126
15.1.1	各サービスの位置づけ早見表	126
15.2	Kinesis ファミリー	126
15.2.1	Kinesis Data Streams	126
15.2.2	Kinesis Data Firehose	127

15.2.3	Kinesis Data Analytics for Apache Flink	127
15.2.4	Kinesis Video Streams	127
15.3	MSK (Managed Streaming for Kafka)	127
15.3.1	MSK Provisioned vs MSK Serverless	127
15.3.2	Kinesis Data Streams との使い分け	127
15.4	S3 をデータレイクの中心に	128
15.4.1	Medallion アーキテクチャ	128
15.4.2	ファイルフォーマット	128
15.4.3	パーティショニング	128
15.4.4	Iceberg / Hudi / Delta Lake	128
15.4.5	S3 Tables (2024 年末 GA)	128
15.5	AWS Glue	128
15.5.1	Glue Data Catalog	129
15.5.2	Glue Crawler	129
15.5.3	Glue ETL	129
15.5.4	Glue DataBrew	129
15.5.5	Glue Workflows	129
15.6	Athena	130
15.6.1	パフォーマンス&コスト最適化	130
15.7	Redshift	130
15.7.1	Provisioned vs Serverless	130
15.7.2	Redshift Managed Storage	130
15.7.3	Redshift Spectrum	130
15.7.4	Zero-ETL 統合	131
15.7.5	マテリアライズドビューとデータ共有	131
15.7.6	Concurrency Scaling と RA3	131
15.8	OpenSearch Service	131
15.8.1	Provisioned vs Serverless	131
15.8.2	主要ユースケース	131
15.8.3	OpenSearch Ingestion	131
15.8.4	UltraWarm / Cold	131
15.9	QuickSight	131
15.10	EMR (Elastic MapReduce)	132
15.10.1	形態	132
15.10.2	適用シーン	132
15.11	Lake Formation と DataZone	132
15.11.1	Lake Formation	132
15.11.2	DataZone	132
15.12	典型アーキテクチャ	132
15.12.1	リアルタイムログ分析	132
15.12.2	バッチ ETL	133
15.12.3	リアルタイムダッシュボード	133

15.12.4	レイクハウス	133
15.13	コスト最適化のヒント	133
15.14	よくあるトラブル	133
16	機械学習と生成 AI	135
16.1	AWS AI/ML スタックの 3 層	135
16.2	既製 AI サービス	135
16.2.1	簡単な使用例 (Rekognition)	135
16.3	Amazon Bedrock	136
16.3.1	利用可能な主要モデル系列	136
16.3.2	Bedrock の主要機能	136
16.3.3	Bedrock の料金モデル	137
16.3.4	モデル選択の実践指針	137
16.4	Amazon Q	137
16.5	SageMaker	138
16.5.1	コンポーネント早見表	138
16.5.2	最小ワークフロー	138
16.5.3	推論方式の使い分け	139
16.5.4	Pipelines + Model Registry	139
16.5.5	Feature Store	139
16.5.6	JumpStart	140
16.5.7	HyperPod	140
16.6	Trainium / Inferentia	140
16.7	ベクトル検索の選択肢	140
16.8	典型的な構成	141
16.8.1	RAG (Retrieval-Augmented Generation)	141
16.8.2	バッチ推論パイプライン	141
16.8.3	リアルタイム推論	141
16.8.4	MLOps (CI/CD)	141
16.8.5	カスタム LLM	141
16.9	コストとリスクのよくある罠	141
16.10	学び始めの推奨パス	142
17	認証サービスの詳細	143
17.1	アイデンティティの種類と AWS の選択肢	143
17.2	IAM Identity Center 詳細	143
17.2.1	アイデンティティソース	143
17.2.2	許可セット (Permission Sets)	143
17.2.3	ABAC (Attribute-Based Access Control)	144
17.2.4	セッション管理	144
17.2.5	監査	144
17.2.6	移行 (IAM ユーザーから Identity Center へ)	144
17.3	Cognito	145
17.3.1	User Pools (ユーザーディレクトリ)	145

17.3.2	Identity Pools (フェデレーション)	145
17.3.3	User Pool と Identity Pool の関係	145
17.3.4	典型構成: SPA + API Gateway + Lambda	145
17.3.5	マルチテナンシー設計	146
17.3.6	Cognito の料金感	146
17.4	Directory Service	146
17.5	Verified Permissions	146
17.6	Resource Access Manager (RAM)	147
17.6.1	共有可能な代表リソース	147
17.6.2	マルチアカウント・ネットワークの実例	147
17.7	IAM Roles Anywhere	147
17.7.1	構成要素	147
17.8	STS と一時認証情報	148
17.8.1	主要 API	148
17.8.2	一時認証情報の有効期限	148
17.8.3	Source Identity の伝播	148
17.8.4	ロール連鎖の制約	148
17.9	認証関連のベストプラクティス	148
17.10	マルチアカウント運用とのつながり	149
17.11	よくあるトラブル	149
18	ネットワーク詳細	150
18.1	大規模ネットワーク設計の選択肢	150
18.2	Transit Gateway	150
18.2.1	アタッチメントとルーティング	150
18.2.2	セグメンテーションパターン	150
18.2.3	マルチリージョン	151
18.2.4	料金感	151
18.3	Cloud WAN	151
18.3.1	コア概念	151
18.3.2	いつ Cloud WAN、いつ Transit Gateway か	151
18.4	Direct Connect	151
18.4.1	種類	151
18.4.2	Virtual Interface (VIF)	151
18.4.3	Direct Connect Gateway	151
18.4.4	MACsec 暗号化	152
18.4.5	冗長化	152
18.4.6	いつ VPN ではなく Direct Connect か	152
18.5	Site-to-Site VPN	152
18.6	Client VPN	152
18.6.1	認証方式	152
18.6.2	接続フロー	152
18.7	AWS Verified Access	153

18.8	Global Accelerator	153
18.8.1	CloudFront との違い	153
18.9	VPC Lattice	153
18.9.1	コンセプト	153
18.10	PrivateLink 詳細	153
18.10.1	主要ユースケース	153
18.10.2	Cross-Region Endpoints	154
18.11	Network Firewall	154
18.11.1	配置パターン	154
18.11.2	Firewall Manager との連携	154
18.12	Route 53 Resolver と DNS Firewall	154
18.13	IPv6 戦略	154
18.14	マルチアカウント・マルチリージョンの典型構成	155
18.14.1	中央 Inspection VPC モデル	155
18.14.2	Cloud WAN ハブ&スポーク	155
18.14.3	小規模メッシュ	155
18.15	コスト整理	155
18.16	よくあるトラブル	155
19	セキュリティサービスの詳細	157
19.1	防御層別のサービスマップ	157
19.2	WAF (Web Application Firewall)	157
19.2.1	Web ACL の構成	157
19.2.2	ルールの種類	157
19.2.3	Bot Control / ATP / ACFP	157
19.2.4	CAPTCHA / Challenge	157
19.2.5	Logging	158
19.2.6	料金感	158
19.3	Shield	158
19.3.1	Standard	158
19.3.2	Advanced	158
19.4	Network Firewall	158
19.4.1	ルールタイプ	158
19.4.2	配置	158
19.5	Firewall Manager	159
19.6	GuardDuty 深掘り	159
19.6.1	データソース	159
19.6.2	Findings の例	159
19.6.3	抑制ルール	159
19.6.4	マルチアカウント運用	159
19.6.5	連携	159
19.7	Inspector	159
19.7.1	対象	160

19.7.2	修復推奨	160
19.8	Macie	160
19.9	Detective	160
19.10	Security Hub	160
19.10.1	セキュリティ標準	160
19.10.2	統合先	160
19.10.3	自動修復	161
19.11	Audit Manager	161
19.12	Artifact	161
19.13	KMS 深掘り	161
19.13.1	キーポリシーと IAM	161
19.13.2	自動ローテーション	161
19.13.3	Multi-Region Keys	161
19.13.4	エンベロープ暗号化	161
19.13.5	Grants	162
19.14	CloudHSM	162
19.14.1	用途	162
19.15	Secrets Manager 深掘り	162
19.15.1	自動ローテーション	162
19.15.2	Cross-Region Replication	162
19.15.3	リソースベースポリシー	162
19.15.4	VPC エンドポイント	162
19.15.5	Parameter Store との使い分け（再掲）	162
19.15.6	コードサンプル	163
19.16	Certificate Manager（ACM）	163
19.16.1	パブリック証明書	163
19.16.2	Private CA（ACM PCA）	163
19.16.3	ACM の制限	163
19.17	CloudTrail Lake	163
19.18	IAM Access Analyzer 深掘り	163
19.18.1	外部アクセス分析	163
19.18.2	未使用アクセス分析	164
19.18.3	Policy Generation	164
19.18.4	カスタムポリシーチェック	164
19.18.5	Reachability	164
19.19	セキュリティ運用ベストプラクティス	164
19.20	インシデント対応の典型フロー	164
19.21	よくあるトラブル	164
20	マルチアカウント運用	166
20.1	なぜマルチアカウントか	166
20.2	AWS Organizations	166
20.2.1	基本構造	166

20.2.2	主な機能	167
20.2.3	管理アカウントの守り方	167
20.2.4	SCP の設計パターン	167
20.2.5	タグポリシー	168
20.3	Control Tower	168
20.3.1	Control Tower がやってくれること	168
20.3.2	Account Factory	168
20.3.3	AFT (Account Factory for Terraform)	168
20.3.4	既存組織への適用	168
20.4	Landing Zone の典型構成	168
20.5	Resource Access Manager (RAM)	169
20.5.1	共有可能リソース	169
20.5.2	VPC 共有の実例	169
20.6	委任管理者 (Delegated Administrator)	169
20.7	Account Management API	170
20.7.1	Region オプトイン	170
20.8	集中ロギング	170
20.8.1	CloudTrail	170
20.8.2	Config	170
20.8.3	VPC Flow Logs	170
20.8.4	ログのライフサイクル	170
20.9	コスト管理 (マルチアカウント視点)	171
20.9.1	一括請求のメリット	171
20.9.2	コスト按分の方法	171
20.9.3	Budgets の組織展開	171
20.9.4	Compute Optimizer	171
20.10	マルチアカウント自動化	171
20.10.1	StackSets (CloudFormation)	171
20.10.2	CDK Pipelines / Terraform マルチアカウント	171
20.10.3	AWS Config Conformance Pack	171
20.10.4	EventBridge クロスアカウント	171
20.11	委任管理 + 自動化のパターン	172
20.12	ベストプラクティスまとめ	172
20.13	よくある落とし穴	172
21	移行とデータ転送	173
21.1	AWS 移行の段取り	173
21.2	Migration Hub	173
21.3	Application Discovery Service	173
21.4	AWS Application Migration Service (MGN)	173
21.4.1	仕組み	174
21.4.2	カットオーバーの流れ	174
21.4.3	大規模移行プロジェクト	174

21.5	AWS Database Migration Service (DMS)	174
21.5.1	サポートする変換	174
21.5.2	主要モード	174
21.5.3	DMS Serverless	174
21.5.4	Babelfish for Aurora PostgreSQL	175
21.5.5	ヘテロジニアス移行のリアル	175
21.6	Server Migration Service (SMS)	175
21.7	Snow ファミリー	175
21.7.1	利用ステップ	175
21.8	DataSync	175
21.8.1	対応 Source / Destination	176
21.8.2	主な機能	176
21.9	AWS Transfer Family	176
21.9.1	ユースケース	176
21.10	Storage Gateway とのハイブリッド転送	176
21.11	Database Migration の典型シナリオ	176
21.11.1	Oracle → Aurora PostgreSQL	176
21.11.2	SQL Server → Aurora PostgreSQL (Babelfish 経由)	177
21.11.3	MySQL on EC2 → Aurora MySQL	177
21.12	サーバ移行の典型シナリオ	177
21.12.1	VMware → EC2 (MGN)	177
21.12.2	コンテナ化 (モダナイゼーション)	177
21.13	ライセンスと EULA	177
21.14	クラウド間の移行 (GCP / Azure → AWS)	177
21.15	移行の品質を上げるコツ	178
21.16	移行後にやるべきこと	178
21.17	よくあるトラブル	178
22	IoT・エッジ・メディア	179
22.1	AWS IoT サービス群	179
22.2	IoT Core	179
22.2.1	デバイス接続	179
22.2.2	認証	179
22.2.3	Things、Thing Group、Thing Type	179
22.2.4	Device Shadow	180
22.2.5	Rules Engine	180
22.2.6	料金感	180
22.3	IoT Device Management	180
22.4	IoT Greengrass	180
22.4.1	コア機能	180
22.4.2	Greengrass v2 のコンポーネント	181
22.5	IoT Analytics	181
22.6	IoT Events	181

22.7	IoT SiteWise	181
22.8	IoT TwinMaker	181
22.9	FreeRTOS	181
22.10	エッジコンピューティング	181
	22.10.1 AWS Outposts	182
	22.10.2 AWS Local Zones	182
	22.10.3 AWS Wavelength	182
	22.10.4 Snow ファミリー（エッジ視点）	182
22.11	メディアサービス群	182
22.12	MediaConvert（VOD トランスコード）	182
	22.12.1 入力 / 出力	183
	22.12.2 機能	183
	22.12.3 料金	183
22.13	MediaLive（ライブトランスコード）	183
22.14	MediaPackage	183
22.15	MediaTailor	183
22.16	IVS（Interactive Video Service）	184
22.17	配信パイプラインの典型	184
	22.17.1 VOD（YouTube ライク）	184
	22.17.2 ライブ（Twitch ライク）	184
	22.17.3 低レイテンシライブ	184
22.18	ゲーム関連	184
22.19	コスト・運用の注意	184
22.20	よくあるトラブル	185
23	アーキテクチャパターン集	186
23.1	Web アプリの基本パターン	186
	23.1.1 静的サイト + サーバーレス API	186
	23.1.2 古典的 3 層 Web アプリ	186
	23.1.3 コンテナ化 Web	186
23.2	モバイルバックエンド	187
	23.2.1 Amplify + Cognito + AppSync	187
	23.2.2 REST + Cognito + DynamoDB	187
23.3	バッチ・データ処理	187
	23.3.1 サーバーレス ETL	187
	23.3.2 Map-Reduce 大規模バッチ	187
	23.3.3 機械学習パイプライン	188
23.4	ストリーミング処理	188
	23.4.1 リアルタイムログ分析	188
	23.4.2 IoT データパイプライン	188
23.5	イベント駆動・非同期	188
	23.5.1 注文処理（Saga）	188
	23.5.2 ファンアウト処理	189

23.6	マイクロサービス通信	189
23.6.1	ECS / EKS + Service Discovery	189
23.6.2	VPC Lattice	189
23.6.3	App Mesh (Envoy ベース)	189
23.7	マルチアカウント・マルチリージョン	189
23.7.1	中央 Inspection VPC + Workload VPCs	189
23.7.2	Active-Active マルチリージョン	190
23.7.3	マルチアカウント Landing Zone	190
23.8	セキュリティ重視構成	190
23.8.1	ゼロトラスト・ネットワーク	190
23.8.2	機密データレイク	190
23.9	ハイブリッドクラウド	190
23.9.1	Direct Connect + Transit Gateway	190
23.9.2	Storage Gateway	190
23.10	災害復旧 (DR)	191
23.11	コスト最適化テンプレート	191
23.11.1	検証環境の自動停止	191
23.11.2	Spot を使ったバッチ処理	191
23.12	ゲーム配信パターン	191
23.12.1	マルチプレイヤゲームサーバ	191
23.13	チャットボット / カスタマサポート	191
23.13.1	Connect + Bedrock	191
23.14	計算ワークロード	192
23.14.1	HPC	192
23.14.2	ML 推論サーバ	192
23.15	メタパターン：選択の指針	192
23.16	設計時のチェックリスト	192
23.17	Anti-pattern (避けるべき構成)	193
24	ハンズオン 1：静的サイトとサーバーレス API	194
24.1	ゴール	194
24.2	前提	194
24.3	構成図	194
24.4	ステップ 1：プロジェクト初期化	194
24.4.1	Bootstrap (初回のみ)	194
24.5	ステップ 2：Lambda 関数を書く	195
24.6	ステップ 3：CDK スタック	196
24.7	ステップ 4：静的フロントエンド	198
24.8	ステップ 5：デプロイ	199
24.9	ステップ 6：カスタムドメイン	199
24.9.1	ACM 証明書 (us-east-1)	199
24.9.2	CDK スタックに追加	199
24.10	ステップ 7：監視を 1 枚追加	200

24.11	ステップ 8 : GitHub Actions で CI/CD (任意)	200
24.12	ステップ 9 : 検証 → 後片付け	201
24.13	学んだこと	201
24.14	発展課題	201
24.15	トラブルシューティング	202
25	ハンズオン 2 : 3 層 Web アプリ	203
25.1	ゴール	203
25.2	前提	203
25.3	構成図	203
25.4	ステップ 1 : プロジェクト構造	203
25.5	ステップ 2 : VPC モジュール	204
25.6	ステップ 3 : ALB モジュール	207
25.7	ステップ 4 : ASG モジュール	208
25.8	ステップ 5 : DB モジュール	211
25.9	ステップ 6 : ルートで配線	212
25.10	ステップ 7 : 適用	213
25.11	ステップ 8 : SSM 接続	214
25.12	ステップ 9 : HTTPS 化	214
25.13	ステップ 10 : 後片付け	215
25.14	学んだこと	215
25.15	発展課題	215
25.16	トラブルシューティング	215
26	ハンズオン 3 : データレイク基礎	216
26.1	ゴール	216
26.2	構成図	216
26.3	ステップ 1 : S3 バケット準備	216
26.4	ステップ 2 : サンプル生データ投入	217
26.5	ステップ 3 : Glue Database 作成と Crawler	217
26.6	ステップ 4 : Athena でクエリ	218
26.7	ステップ 5 : Glue ETL ジョブで Silver 生成	219
26.8	ステップ 6 : Athena CTAS で Gold 生成	220
26.9	ステップ 7 : QuickSight で可視化	220
26.10	ステップ 8 : 日次自動化	221
26.11	ステップ 9 : Lake Formation でアクセス制御 (任意)	221
26.12	ステップ 10 : 後片付け	221
26.13	学んだこと	222
26.14	発展課題	222
26.15	トラブルシューティング	222
27	ディザスタリカバリ戦略	223
27.1	RPO と RTO	223
27.2	4 つの代表 DR パターン	223
27.3	Backup & Restore	223

27.3.1	構成	223
27.3.2	採用ケース	224
27.3.3	ベストプラクティス	224
27.4	Pilot Light	224
27.4.1	構成	224
27.4.2	採用ケース	224
27.4.3	ポイント	224
27.5	Warm Standby	224
27.5.1	構成	224
27.5.2	採用ケース	225
27.5.3	ポイント	225
27.6	Multi-Site Active-Active	225
27.6.1	構成	225
27.6.2	採用ケース	225
27.6.3	ポイント	225
27.7	コンポーネント別 DR 設計	226
27.7.1	コンピューティング (EC2 / ECS / EKS / Lambda)	226
27.7.2	データベース	226
27.7.3	ストレージ	226
27.7.4	ネットワーク・DNS	226
27.7.5	認証・設定	226
27.8	バックアップ戦略	227
27.8.1	3-2-1 ルール	227
27.8.2	AWS Backup の使い方 (再掲)	227
27.8.3	別アカウント分離 (推奨)	227
27.9	ランサムウェア対策	227
27.10	演習 (Game Day)	227
27.11	AWS Resilience Hub	228
27.12	Route 53 ARC (Application Recovery Controller)	228
27.13	クロスアカウント DR の典型構成	228
27.14	コスト感	228
27.15	チェックリスト	229
27.16	よくある落とし穴	229
28	Well-Architected Framework	230
28.1	6本の柱	230
28.2	Operational Excellence (運用上の優秀性)	230
28.2.1	主要設計原則	230
28.2.2	実践	230
28.3	Security (セキュリティ)	231
28.3.1	主要設計原則	231
28.3.2	実践	231
28.4	Reliability (信頼性)	231

28.4.1	主要設計原則	231
28.4.2	実践	231
28.5	Performance Efficiency (パフォーマンス効率)	232
28.5.1	主要設計原則	232
28.5.2	実践	232
28.6	Cost Optimization (コスト最適化)	232
28.6.1	主要設計原則	232
28.6.2	実践	232
28.7	Sustainability (持続可能性)	233
28.7.1	主要設計原則	233
28.7.2	実践	233
28.8	Well-Architected Tool	233
28.8.1	流れ	233
28.8.2	レンズ	233
28.8.3	利点	234
28.9	6 本柱チェックリスト (抜粋)	234
28.9.1	Operational Excellence	234
28.9.2	Security	234
28.9.3	Reliability	234
28.9.4	Performance Efficiency	234
28.9.5	Cost Optimization	235
28.9.6	Sustainability	235
28.10	WAF (Well-Architected Framework) と他の指針	235
28.11	レビューのリズム	235
28.12	よくある誤解	235
28.13	まとめ	236
29	認定資格と学習リソース	237
29.1	AWS 認定資格の全体像	237
29.1.1	カテゴリ	237
29.1.2	受験制度	237
29.1.3	公式リソース	237
29.2	おすすめ学習順序	237
29.2.1	入門パス	237
29.2.2	開発者パス	238
29.2.3	運用パス	238
29.2.4	アーキテクトパス	238
29.2.5	分野特化	238
29.3	資格別の勉強法	238
29.3.1	Cloud Practitioner (CLF)	238
29.3.2	Solutions Architect Associate (SAA)	238
29.3.3	Developer Associate (DVA)	239
29.3.4	SysOps Administrator Associate (SOA)	239

29.3.5	Solutions Architect Professional (SAP)	239
29.3.6	Specialty 全般	239
29.4	学習リソース	239
29.4.1	公式	239
29.4.2	re:Invent / Summit	240
29.4.3	コミュニティ	240
29.4.4	独立系メディア・ブログ	240
29.4.5	日本語リソース	240
29.4.6	有料トレーニング	240
29.5	実務で学び続けるヒント	241
29.5.1	サンドボックスを持つ	241
29.5.2	1つのサービスを深掘りする	241
29.5.3	公式ブログを定期購読	241
29.5.4	re:Invent セッションを見る	241
29.5.5	他人のアーキテクチャを読む	241
29.5.6	小さく作って壊す	241
29.6	AWS と周辺技術	241
29.7	本書からの卒業	242
29.8	参考：読者のフェーズ別推奨	242
30	Lambda 深掘り	243
30.1	Lambda の実行モデル再確認	243
30.1.1	実行環境のライフサイクル	243
30.1.2	コンテナの再利用	243
30.1.3	同時実行とスケーリング	243
30.2	コールドスタート対策	244
30.2.1	削減の優先順位	244
30.2.2	Lambda SnapStart	244
30.2.3	Provisioned Concurrency	244
30.3	ランタイム	244
30.3.1	コンテナイメージ Lambda	245
30.4	Lambda Powertools	245
30.5	設計パターン	246
30.5.1	単機能・薄い Lambda	246
30.5.2	Single-purpose vs Mono-Lambda	246
30.5.3	ストリーム処理 (SQS / Kinesis / DynamoDB Streams)	246
30.5.4	Async vs Sync	246
30.6	Lambda Function URL	246
30.7	テスト戦略	247
30.7.1	ローカルテスト	247
30.7.2	ユニットテスト	247
30.7.3	統合テスト	247
30.8	監視と可観測性	247

30.8.1	CloudWatch Metrics	247
30.8.2	CloudWatch Logs	248
30.8.3	X-Ray	248
30.8.4	Application Signals	248
30.9	コスト最適化	248
30.9.1	Lambda Power Tuning	248
30.10	セキュリティ	248
30.11	Lambda の制限値（覚えておくべき数値）	249
30.12	よくあるトラブル	249
31	DynamoDB と NoSQL 設計	250
31.1	DynamoDB の本質	250
31.1.1	強み	250
31.1.2	弱み	250
31.2	主要概念の整理	250
31.3	Single Table Design	250
31.3.1	なぜ単一テーブルか	251
31.3.2	実例：注文システム	251
31.3.3	Generic な属性名	251
31.4	セカンダリインデックス	251
31.4.1	GSI (Global Secondary Index)	251
31.4.2	LSI (Local Secondary Index)	251
31.4.3	Sparse Index	252
31.5	容量モード	252
31.5.1	Capacity Unit の計算	252
31.5.2	Auto Scaling	252
31.5.3	Reserved Capacity	252
31.6	トランザクション	252
31.6.1	Optimistic Locking	252
31.7	DynamoDB Streams	253
31.7.1	ユースケース	253
31.8	TTL (Time To Live)	253
31.9	Global Tables	253
31.9.1	ユースケース	253
31.10	バックアップとリカバリ	253
31.11	パフォーマンスチューニング	254
31.11.1	Hot Partition (熱いパーティション)	254
31.11.2	Query vs Scan	254
31.11.3	DAX (DynamoDB Accelerator)	254
31.12	コスト最適化	254
31.13	セキュリティ	254
31.14	監視	255
31.15	Single Table Design vs RDB の判断	255

31.16	よくある罠	255
32	ECS と EKS 深掘り	257
32.1	ECS と EKS の選び方再考	257
32.2	ECS の本格運用	257
32.2.1	クラスタ／サービス／タスク	257
32.2.2	タスク定義のキー要素	257
32.2.3	executionRoleArn vs taskRoleArn	258
32.2.4	ネットワークモード	258
32.3	サービス・ロードバランシング	258
32.3.1	デプロイ戦略	258
32.3.2	Service Discovery	259
32.4	Auto Scaling	259
32.4.1	Service Auto Scaling	259
32.4.2	Cluster Capacity Provider	259
32.4.3	Fargate Spot	259
32.5	EKS の基本	259
32.5.1	クラスタ構成	259
32.5.2	Add-ons	260
32.5.3	EKS Auto Mode (2024 年末～)	260
32.5.4	Karpenter	260
32.6	EKS のセキュリティ	261
32.6.1	IRSA (IAM Roles for Service Accounts)	261
32.6.2	EKS Pod Identity (推奨、2023～)	261
32.6.3	aws-auth ConfigMap → Access Entries	261
32.7	Ingress / Service	261
32.7.1	AWS Load Balancer Controller	261
32.7.2	VPC Lattice + EKS	262
32.8	ストレージ	262
32.9	Service Mesh の選択肢	262
32.10	観測性 (Container Insights)	263
32.11	セキュリティのベストプラクティス	263
32.12	コスト最適化	263
32.13	トラブルシューティング	263
33	Aurora 深掘り	265
33.1	Aurora のアーキテクチャ	265
33.1.1	ストレージ層	265
33.1.2	読み取りスケーリング	265
33.2	クラスタの構成要素	265
33.2.1	Aurora MySQL vs Aurora PostgreSQL	265
33.3	Aurora Serverless v2	266
33.3.1	自動スケーリング	266
33.3.2	0 ACU 自動ポーズ	266

33.3.3	Provisioned との使い分け	266
33.4	Aurora Global Database	266
33.5	Zero-ETL 統合	267
33.5.1	Aurora → Redshift	267
33.5.2	Aurora → S3 / OpenSearch	267
33.5.3	Aurora PostgreSQL → DynamoDB	267
33.6	Aurora ML	267
33.7	Babelfish for Aurora PostgreSQL	267
33.8	I/O Optimized	268
33.9	パフォーマンスチューニング	268
33.9.1	Performance Insights	268
33.9.2	クエリチューニング	268
33.9.3	インデックス戦略	268
33.9.4	コネクションプール	268
33.9.5	RDS Proxy	268
33.10	Aurora の高可用性	269
33.10.1	フェイルオーバー	269
33.10.2	Backtrack (Aurora MySQL のみ)	269
33.10.3	Continuous Backup	269
33.11	セキュリティ	269
33.12	コストの内訳	269
33.12.1	コスト最適化のコツ	269
33.13	移行	270
33.13.1	Oracle / SQL Server → Aurora PostgreSQL	270
33.13.2	MySQL → Aurora MySQL	270
33.13.3	PostgreSQL → Aurora PostgreSQL	270
33.14	モニタリング	270
33.15	Aurora vs RDS の判断	270
33.16	よくあるトラブル	271
34	コスト最適化と FinOps	272
34.1	FinOps とは	272
34.1.1	FinOps の 3 原則	272
34.1.2	FinOps のフェーズ	272
34.2	AWS のコスト関連サービス	272
34.3	コストの可視化	273
34.3.1	コスト配分タグ (Cost Allocation Tags)	273
34.3.2	Cost Categories	273
34.3.3	AWS Cost Explorer	273
34.3.4	Cost Anomaly Detection	273
34.3.5	CUR (Cost and Usage Report 2.0)	273
34.3.6	AWS Cost Optimization Hub	273
34.4	Right-Sizing (適正サイジング)	274

34.4.1	Compute Optimizer	274
34.4.2	手動チェックの観点	274
34.5	購入オプション最適化	274
34.5.1	Compute Savings Plans	274
34.5.2	EC2 Instance Savings Plans	274
34.5.3	Reserved Instances	274
34.5.4	コミット戦略	274
34.5.5	Spot Instance	274
34.6	不要リソースの掃除	275
34.6.1	自動化	275
34.7	サービス別の最適化ポイント	275
34.7.1	EC2	275
34.7.2	RDS / Aurora	275
34.7.3	Lambda	276
34.7.4	コンテナ (ECS / EKS / Fargate)	276
34.7.5	S3	276
34.7.6	データ転送	276
34.7.7	CloudWatch	276
34.7.8	Bedrock / SageMaker	276
34.8	予算管理	277
34.8.1	AWS Budgets	277
34.8.2	階層的予算設計	277
34.9	タグ戦略の運用	277
34.10	組織的な FinOps	277
34.10.1	Cost Center モデル	277
34.10.2	コストレビュー会議	277
34.11	持続可能性とコスト	278
34.12	典型的な節約事例	278
34.13	コスト最適化の優先順位	278
34.14	よくある罠	278
35	可観測性と SRE	280
35.1	監視と可観測性の違い	280
35.2	AWS の可観測性スタック	280
35.3	CloudWatch Metrics の使いこなし	280
35.3.1	Embedded Metric Format (EMF)	280
35.3.2	Anomaly Detection	281
35.3.3	Composite Alarm	281
35.3.4	Metric Math	281
35.4	CloudWatch Logs	282
35.4.1	Logs Insights	282
35.4.2	Live Tail	282
35.4.3	Subscription Filter	282

35.4.4	Vended Logs	282
35.4.5	コスト	282
35.5	AWS X-Ray	282
35.5.1	セグメントとサブセグメント	282
35.5.2	サービスマップ	282
35.5.3	サンプリング	283
35.6	Application Signals	283
35.6.1	主要機能	283
35.6.2	SLO 設定の例	283
35.7	Container Insights	283
35.7.1	Enhanced observability (2024～)	283
35.7.2	料金注意	283
35.8	AWS Distro for OpenTelemetry (ADOT)	283
35.8.1	構成	284
35.9	Managed Prometheus / Managed Grafana	284
35.9.1	Managed Prometheus	284
35.9.2	Managed Grafana	284
35.10	CloudWatch Synthetics	284
35.11	CloudWatch RUM (Real-User Monitoring)	285
35.12	Incident Manager	285
35.13	SRE 的な運用	285
35.13.1	SLI / SLO / SLA	285
35.13.2	Error Budget	285
35.13.3	Burn Rate Alarm	286
35.13.4	Postmortem 文化	286
35.14	オブザーバビリティの設計原則	286
35.15	コスト最適化	286
35.16	サードパーティ統合	286
35.17	よくある落とし穴	287
36	AWS の最新動向と次の一步	288
36.1	2024～2026 年の主要トレンド	288
36.1.1	生成 AI の AWS 統合の加速	288
36.1.2	サーバーレスのさらなる成熟	288
36.1.3	マルチリージョン・マルチアカウントの標準化	288
36.1.4	セキュリティの自動化	288
36.1.5	コスト・FinOps の本格化	288
36.1.6	持続可能性	289
36.1.7	エッジ・ハイブリッド	289
36.1.8	Quantum / Robotics / Industrial	289
36.2	注目の新サービス・機能 (2024～2026)	289
36.3	AWS リリースサイクルへの追従	290
36.3.1	公式ソース	290

36.3.2	二次情報	290
36.3.3	学習リソース（再掲）	290
36.4	個人プロジェクトでのおすすめ構成	290
36.4.1	1. サーバーレス Web アプリ（24 章）	290
36.4.2	2. 個人ブログ（Static Site Generator）	290
36.4.3	3. ML / 生成 AI 体験	291
36.4.4	4. データ収集・分析	291
36.4.5	5. IoT / エッジ	291
36.5	ケースに応じた参考アーキテクチャ集	291
36.6	AWS と他クラウドの併用	291
36.7	AWS とオープンソース	292
36.8	AWS パートナーシップとキャリア	292
36.8.1	AWS Partner Network（APN）	292
36.8.2	キャリアパス	292
36.9	学び続けるための 1 日 / 1 週間のリズム	293
36.9.1	毎日（15 分）	293
36.9.2	毎週（1～2 時間）	293
36.9.3	毎月（半日）	293
36.9.4	毎年（数日）	293
36.10	本書の総まとめ	293
36.11	読者へのメッセージ	294
37	ハンズオン 4：生成 AI チャットアプリ	295
37.1	ゴール	295
37.2	構成図	295
37.3	前提	295
37.4	ステップ 1：プロジェクト初期化	295
37.5	ステップ 2：Knowledge Base 用 S3 と OpenSearch Serverless	296
37.6	ステップ 3：API Lambda	297
37.7	ステップ 4：API Gateway + Cognito	298
37.8	ステップ 5：フロントエンド（簡易版）	299
37.9	ステップ 6：Guardrails 適用	300
37.10	ステップ 7：取り込みジョブの起動	301
37.11	ステップ 8：監視と Guardrails の効果検証	301
37.12	ステップ 9：コスト確認	301
37.13	ステップ 10：後片付け	301
37.14	学んだこと	301
37.15	発展課題	302
37.16	トラブルシューティング	302
38	CloudFormation と CDK 詳細	303
38.1	CloudFormation の基本構造	303
38.1.1	セクションの役割	303
38.2	Intrinsic Functions（組み込み関数）	304

38.3	スタック設計戦略	304
38.3.1	モノリス vs ネスト vs Cross-stack	304
38.3.2	レイヤ分割の典型	304
38.4	Cross-stack 参照	304
38.4.1	SSM Parameter Store 経由	305
38.5	Custom Resource	305
38.6	Drift 検出	305
38.7	StackSets	306
38.7.1	2つの権限モデル	306
38.8	AWS CDK 詳細	306
38.8.1	Construct の 3 層	306
38.8.2	App / Stack / Construct の階層	306
38.8.3	Cross-stack 参照 (CDK)	307
38.8.4	Tag の一括付与	307
38.8.5	Aspects	307
38.8.6	cdk-nag	307
38.8.7	CDK Pipelines	308
38.9	テスト	308
38.9.1	1. snapshot テスト	308
38.9.2	2. ファインマッチング	308
38.9.3	3. 統合テスト	309
38.10	デプロイ戦略	309
38.10.1	Hotswap	309
38.10.2	Watch	309
38.10.3	Bootstrap モダン化	309
38.11	CDK と Terraform の選び方	309
38.12	ベストプラクティス	310
38.13	制限値の頭出し	310
38.14	よくある罠	310
39	Terraform 詳細	312
39.1	Terraform vs OpenTofu	312
39.2	基本構造	312
39.3	State 管理	313
39.3.1	Local backend vs Remote backend	313
39.3.2	S3 backend + ロック	313
39.3.3	State の論理分割	313
39.3.4	State 操作 (注意して使う)	314
39.3.5	Workspace	314
39.4	モジュール	314
39.4.1	呼び出し	314
39.4.2	Source の種類	314
39.4.3	コミュニティモジュール	315

39.4.4	自社モジュールの設計指針	315
39.5	Provider	315
39.5.1	Cross-account (Assume Role)	315
39.6	ワークフロー	316
39.6.1	コマンド一覧	316
39.6.2	Plan / Apply の本格運用	316
39.7	CI/CD 統合	316
39.7.1	GitHub Actions (OIDC + Plan/Apply)	316
39.7.2	HCP Terraform / Terraform Enterprise	317
39.7.3	Atlantis / OpenTofu のコミュニティツール	317
39.8	テスト	317
39.8.1	<code>terraform test</code> (v1.6+)	317
40	Terratest (Go ベース)	319
40.1	Drift と State の整合	319
40.1.1	Drift Detection	319
40.1.2	State Refresh と Lock	319
40.2	セキュリティ	319
40.3	コスト	319
40.4	高度な機能	319
40.4.1	Dynamic Block	319
40.4.2	for_each / count	320
40.4.3	Locals / Functions	320
40.4.4	Data Source	320
40.4.5	Lifecycle	321
40.5	組織運用パターン	321
40.5.1	Mono-repo vs Multi-repo	321
40.5.2	ディレクトリ構造の典型	321
40.5.3	Terragrunt	321
40.6	AWS Provider のヒント	322
40.7	よくあるトラブル	322
41	CI/CD と DevOps	323
41.1	CI/CD の全体像	323
41.2	AWS の CI/CD サービス群	323
41.3	GitHub Actions + AWS (推奨)	323
41.3.1	OIDC でアクセスキー不要	323
41.3.2	典型的な CI Workflow	324
41.3.3	段階的 CD (Environment 機能)	325
41.4	AWS CodePipeline	325
41.4.1	CodePipeline V2 の利点	326
41.5	AWS CodeBuild	326
41.5.1	ビルド環境	327
41.5.2	バッチビルド	327

41.6	AWS CodeDeploy	327
41.6.1	EC2 デプロイ	327
41.6.2	ECS デプロイ	328
41.6.3	Lambda デプロイ	328
41.7	Amazon CodeCatalyst	328
41.8	Amazon Q Developer	328
41.9	セキュリティスキャン	329
41.10	Artifact 管理	329
41.10.1	ECR	329
41.10.2	CodeArtifact	329
41.10.3	S3	329
41.11	デプロイ戦略	330
41.11.1	AppConfig (Feature Flag)	330
41.12	マルチアカウントデプロイ	330
41.13	モニタリング・通知	331
41.14	ベストプラクティス	331
41.15	よくある罠	331
41.16	DevOps 文化	331
42	マルチテナント SaaS 設計	333
42.1	マルチテナンシーの 3 モデル	333
42.1.1	Silo モデル	333
42.1.2	Pool モデル	333
42.1.3	Bridge モデル	334
42.2	AWS SaaS Builder Toolkit / SaaS Boost	334
42.3	テナント識別と分離	334
42.3.1	テナント ID の埋め込み	334
42.3.2	ABAC でデータ分離	334
42.3.3	Lambda での Assume Role-by-tenant	335
42.4	データストアの分離パターン	335
42.4.1	DynamoDB	335
42.4.2	Aurora / RDS	335
42.4.3	S3	336
42.5	認証・ID 管理	336
42.5.1	Cognito User Pool 戦略	336
42.5.2	マルチテナント JWT	336
42.6	オンボーディング	337
42.6.1	Tier 別オンボーディング	337
42.7	課金とメータリング	337
42.7.1	利用量計測	337
42.7.2	AWS Marketplace 統合	337
42.7.3	Stripe / Recurly 等の外部請求基盤	338
42.8	Tier 別リソース割当	338

42.8.1	レート制限	338
42.8.2	物理分離 (Tier 別)	338
42.9	Noisy Neighbor 対策	338
42.9.1	Cell-based Architecture	338
42.10	観測性	339
42.10.1	カーディナリティに注意	339
42.11	セキュリティ	339
42.11.1	Tenant Isolation のテスト	339
42.11.2	暗号化	339
42.11.3	コンプライアンス	340
42.11.4	Per-Tenant の権限境界	340
42.12	運用	340
42.12.1	マイグレーション戦略	340
42.12.2	バックアップとリストア	340
42.12.3	テナント解約・退会	340
42.13	参考実装	340
42.14	設計判断のチェックリスト	340
42.15	よくある罠	341
43	AWS チートシート	342
43.1	リージョンコード	342
43.2	AWS 主要サービス価格感 (東京、目安)	342
43.2.1	EC2	342
43.2.2	ストレージ・ネットワーク	343
43.2.3	サーバーレス	343
43.2.4	AI / ML	344
43.3	主要 ARN フォーマット	344
43.4	必須リソースの上限値 (クォータ)	345
43.5	AWS CLI チートシート	345
43.5.1	よく使うコマンド	346
43.6	コンソールの「どこにあるか」早見表	347
43.7	セキュリティ初期セットアップチェックリスト	347
43.8	トラブルシューティング 早見表	348
43.9	Pricing Calculator URL	348
43.10	公式ドキュメントの場所	348
43.11	緊急時の連絡先	349

1 はじめに

1.1 AWS とは

Amazon Web Services (AWS) は、Amazon.com が提供する総合クラウドコンピューティングサービスである。2006 年に S3 と EC2 の提供を開始して以来、計算・ストレージ・ネットワーク・データベース・機械学習・IoT・セキュリティなど 200 を超えるサービスを展開しており、パブリッククラウド市場で最大級のシェアを持つ。

従来、サーバーを運用するには物理マシンを購入し、データセンターに設置し、OS やミドルウェアを構築する必要があった。AWS はこれらの IT リソースをインターネット経由で必要なときに必要なだけ利用できる形で提供する。初期投資なしに、数分でサーバーを起動し、不要になれば即座に破棄できる。

1.2 クラウドの基本概念

1.2.1 クラウドコンピューティングの定義

米国 NIST（国立標準技術研究所）は、クラウドコンピューティングを次の 5 つの特徴で定義している。

- ・ オンデマンド・セルフサービス：人手を介さず必要なときに利用開始できる
- ・ 幅広いネットワークアクセス：インターネット経由でどこからでもアクセスできる
- ・ リソースの共用：複数のテナントで物理リソースを共有する
- ・ スピーディな拡張性：需要に応じて自動でリソースを増減できる
- ・ サービスの計測可能性：利用量を計測し、従量課金が可能である

1.2.2 サービスモデル

クラウドサービスは、抽象度によって以下のように分類される。

モデル	提供範囲	AWS の例
IaaS	仮想マシン・ストレージ・ネットワーク	EC2、VPC、EBS
PaaS	アプリ実行環境	Elastic Beanstalk、App Runner
SaaS	アプリケーション	WorkMail、Chime
FaaS Serverless	/ 関数単位の実行環境	Lambda

AWS は IaaS から SaaS まで幅広く提供しているため、要件に合わせて組み合わせられる。

1.3 責任共有モデル

AWS のセキュリティ責任は、AWS と利用者の間で分担される。これを **責任共有モデル (Shared Responsibility Model)** と呼ぶ。

担当	責任範囲
AWS	物理施設、ハードウェア、仮想化基盤、マネージドサービス内部の運用（「クラウドのセキュリティ」）
利用者	OS 以上の設定、データ、IAM 権限、ネットワーク設定、アプリケーション（「クラウド内のセキュリティ」）

EC2 のような IaaS では OS パッチ適用まで利用者の責任だが、S3 や RDS のようなマネージドサービスではミドルウェア以下は AWS が責任を負う。サービスごとに境界が異なる点に注意する。

1.4 グローバルインフラストラクチャ

1.4.1 リージョンとアベイラビリティゾーン

AWS のインフラは世界中に分散配置されている。

- ・ **リージョン (Region)** : 地理的に独立したデータセンター群の集まり。互いに物理的・ネットワーク的に分離されている
- ・ **アベイラビリティゾーン (AZ)** : 1つのリージョン内で電源・ネットワーク・冷却が独立した複数のデータセンター群。各リージョンには通常 3 つ以上の AZ がある
- ・ **エッジロケーション** : CloudFront や Route 53 等で使われる配信拠点。リージョンよりはるかに多い

日本には **東京 (ap-northeast-1)** と **大阪 (ap-northeast-3)** の 2 つのリージョンがある。低レイテンシが必要な場合は東京を、災害対策のマルチリージョン構成では東京+大阪を選ぶのが一般的である。

1.4.2 リージョン選択の考え方

- ・ **レイテンシ** : ユーザーに近いリージョンを選ぶ
- ・ **法規制・データ主権** : データの保管場所に制約がある場合は国内リージョンを選ぶ
- ・ **料金** : リージョンごとに料金が異なる（東京はバージニア北部より高めの傾向）
- ・ **サービス可用性** : 新サービスはバージニア北部 (us-east-1) から提供開始されることが多い

1.4.3 リージョンをまたがないリソース

ほとんどの AWS リソースはリージョン固有である。東京リージョンで作成した EC2 インスタンスは、大阪リージョンのコンソールには表示されない。一方で IAM、Route 53、CloudFront、S3 のバケット名空間などは **グローバルサービス** として扱われる。

1.5 料金モデル

1.5.1 従量課金制

AWS は原則として使った分だけ支払う **従量課金制 (Pay-as-you-go)** である。固定の月額契約はなく、利用時間・転送量・リクエスト数などに応じて課金される。

1.5.2 主な料金構成要素

項目	例
計算	EC2 インスタンスの稼働時間、Lambda の実行時間
ストレージ	EBS ボリュームの容量、S3 の保存容量
データ転送	インターネット向け下り（アウトバウンド）転送量
リクエスト	S3 の PUT/GET、API Gateway のリクエスト数
付随サービス	NAT Gateway の時間課金、ロードバランサの稼働時間

初心者がハマりやすいのは、**インスタンスを止め忘れた月末の請求と NAT Gateway を立てっぱなしにした固定費** である。個人利用では特に注意する。

1.5.3 割引の仕組み

長期利用を前提とする場合、以下の割引が使える。

- **Savings Plans / Reserved Instance**：1 年または 3 年のコミットで最大約 70% 割引
- **Spot Instance**：中断許容のワークロード向けに最大 90% 割引
- **ボリュームディスカウント**：S3 や データ転送の利用量階段制

1.5.4 無料利用枠 (Free Tier)

AWS には新規アカウント向けの無料枠が用意されているが、2025 年 7 月 15 日を境に制度が大きく変わっている。新規アカウントを作る時期で適用ルールが異なる点に注意する。

- **旧 Free Tier (2025/7/15 以前に開設したアカウント)**
 - ▶ **12 か月間無料**：EC2 `t2.micro` または `t3.micro` を月 750 時間など
 - ▶ **常時無料**：Lambda の月 100 万リクエスト、DynamoDB の 25GB など
 - ▶ **トライアル**：一部サービスで 30 日無料など
- **新 Free Plan (2025/7/15 以降の新規アカウント)**
 - ▶ 開設時に **\$100 のクレジット** が付与される
 - ▶ 特定のアクティビティ達成で **追加最大 \$100 獲得可能** (合計 \$200 まで)
 - ▶ クレジット消化 **または 6 か月** の早い方で終了
 - ▶ 終了後は従量課金に自動移行 (その前に Paid プランへ切替するかを選択)
 - ▶ **常時無料** 枠 (Lambda 100 万 req/月、DynamoDB 25GB 等) は引き続き利用可能

どちらの制度でも、上限超過に気づきにくい点は変わらない。後述する予算アラートを必ず設定する。

1.6 操作手段

AWS は同じ機能を複数の手段で操作できる。

- マネジメントコンソール：ブラウザベースの GUI。学習や単発作業に向く
- AWS CLI：コマンドラインから操作する。スクリプト化に向く
- AWS SDK：各言語（Python の `boto3` など）からプログラマ的に操作
- CloudShell：ブラウザ上のシェル環境。CLI と SDK がプリインストール済み
- IaC：CloudFormation / CDK / Terraform で宣言的に管理

本書では主にマネジメントコンソールと AWS CLI を用いて解説する。

1.7 本書の対象読者

- これから AWS を学び始める開発者・インフラエンジニア
- 自宅ラボや個人プロジェクトで AWS を試したい方
- クラウド全般の基礎を押さえたい方
- AWS 認定資格（CLF / SAA）の前段として概観をつかみたい方

1.8 前提知識

本書では以下の知識があることを前提とする。

- Linux の基本操作（SSH 接続、シェルコマンド）
- TCP/IP の基礎（IP アドレス、サブネット、ポート）
- Web アプリケーションの基本構成（クライアント／サーバ、HTTP）

上記が不安な場合でも、必要な背景は随所で補足する。

1.9 本書の進め方

本書は、まず AWS のアカウントを開設し（2 章）、最小限の安全設定を行ったうえで、IAM → VPC → EC2 → S3 という基礎 4 サービス（3～6 章）を順に理解していく構成である。7 章以降はデータベース、コンテナ・サーバーレス、運用・監視、セキュリティ・コスト管理、IaC と応用テーマを扱う。

手を動かして試す際は、必ず 2 章の予算アラートを先に設定し、作業後はリソースを削除する習慣を身につけることを強く推奨する。

2 アカウント開設と初期設定

AWS を利用する前に必要な、アカウントの作成と、事故を防ぐための初期設定について解説する。この章の作業は必ず最初に行うこと。数千円単位の請求事故を防ぐ基本動作である。

2.1 AWS アカウントの開設

2.1.1 開設に必要なもの

- ・メールアドレス（他の AWS アカウントで未使用のもの）
- ・クレジットカード（または一部デビットカード）
- ・電話番号（SMS または自動音声による本人確認）
- ・住所

メールアドレスは AWS アカウントのルートユーザーに紐づく重要な識別子である。個人名の Gmail ではなく、長期的に維持できる共用アドレス（`aws-root+company@example.com` のような形式）を使うのが望ましい。

2.1.2 開設手順の概要

1. 公式サイトから **AWS アカウントを作成** を選択
2. メールアドレス、パスワード、アカウント名を入力
3. 連絡先情報（個人 / ビジネス）、住所を入力
4. クレジットカードを登録（即時 1 ドル程度のオーソリがかかる）
5. 電話番号で本人確認（SMS または音声）
6. サポートプランを選択（最初は **ベーシック（無料）** で十分）

数分でアカウントが有効になり、マネジメントコンソールにログインできるようになる。

2.2 ルートユーザーの保護

AWS アカウント作成時のメールアドレスでログインするユーザーを **ルートユーザー** と呼ぶ。ルートユーザーはアカウント解約や支払い方法変更を含む全権限を持つため、**日常作業では絶対に使わない**。

2.2.1 ルートユーザーに必須の設定

1. **強固なパスワードを設定**：パスワードマネージャで生成した長いランダム文字列を使う
2. **MFA（多要素認証）を有効化**：以下の優先順位で選ぶ
 - ・第一候補：passkey（WebAuthn / FIDO2）またはハードウェアキー（YubiKey 等）
 - ・次点：仮想 MFA（Google Authenticator、1Password 等）
3. **アクセスキーを作らない**：ルートユーザーのアクセスキーは使わない、存在すれば削除する
4. **通知先メールを安全に管理**：乗っ取り時の復旧経路になる

2024 年以降、1 アカウントあたり最大 8 個の MFA デバイスを登録できるようになった。個人利用でも **メイン passkey + バックアップの仮想 MFA** のように複数登録しておく、デバイス紛失時の締め出しを防げる。

MFA の設定は **IAM コンソール** の「セキュリティ認証情報」から行う。passkey ならブラウザの WebAuthn プロンプト、仮想 MFA アプリなら QR コードを読み取り 6 桁のコードを 2 回入力すれば有効化される。

2.2.2 ルートユーザーでしかできない作業

以下はルートユーザーでしか実行できない。日常的には発生しないが、どのような操作かを把握しておく。

- ・ アカウント名・ルートメールアドレスの変更
- ・ 支払い方法の変更、税務情報の設定
- ・ AWS サポートプランの解約
- ・ AWS アカウント自体の解約
- ・ S3 バケットのデフォルト暗号化ポリシーの一部設定

2.3 請求アラートの設定

AWS で最初に必ず設定すべきは **予算アラート** である。使いすぎに気づく唯一の自動経路である。

2.3.1 IAM ユーザーによる請求情報の閲覧許可

初期状態では、ルートユーザー以外は請求情報にアクセスできない。以下の手順で IAM ユーザーからも見られるようにする。

1. ルートユーザーで **アカウント** ページを開く
2. **IAM ユーザー／ロールによる請求情報へのアクセス** を有効化
3. この設定はアカウント全体に適用される

2.3.2 AWS Budgets で予算アラートを作る

Budgets は予算超過を検知してメール通知するサービスである。

1. コンソールから **Billing → Budgets** を開く
2. **予算を作成** をクリック
3. **コスト予算** を選択
4. 期間を **月次**、金額を例えば **10 USD** と設定
5. アラートしきい値を **実際の 80%** と **予測の 100%** で 2 段階構えにする
6. 通知先メールアドレスを指定

これで毎月の使用額が設定額に近づくと通知が届くようになる。個人利用では必ず最初にやる。

2.3.3 無料利用枠アラート

Free Tier の消化率でアラートを出す設定も併用できる。

- ・ **Billing → Billing Preferences**

- **Free Tier usage alerts** を有効化してメールアドレスを登録

85% 消化時点で自動通知される。Free Tier はサービスごとに条件が細かいため、この機能の利用を推奨する。

2.4 IAM Identity Center (旧 AWS SSO)

IAM Identity Center は、AWS の公式なシングルサインオン／アイデンティティ基盤である。長期的には IAM Identity Center が推奨パスであり、2023 年以降は AWS コンソール自身が新規の IAM ユーザー作成時に非推奨の旨を警告する。本格運用を見据えるなら **最初から Identity Center で運用する** のが現代的な選択である。

迷ったときの指針：

- 「個人検証、今日中に触りたい」→ まずは後述の IAM ユーザー方式でもよい
- 「業務、複数人、複数アカウント、本番運用」→ Identity Center を最初から使う

2.4.1 従来の IAM ユーザー方式との違い

項目	IAM ユーザー	IAM Identity Center
作成単位	AWS アカウント内	組織全体で一元管理
認証	長期パスワード／アクセスキー	SSO、短期認証情報
MFA	任意で設定	標準で設定
マルチアカウント	各アカウントで別ユーザー	1 ユーザーで複数アカウントにアクセス
CLI アクセス	長期アクセスキー	<code>aws sso login</code> で一時認証情報

2.4.2 基本的なセットアップ手順

1. コンソールで **IAM Identity Center** を開く
2. **有効にする** をクリック（AWS Organizations が自動作成される）
3. **ユーザー** を追加し、メールアドレスを登録
4. **許可セット** を作成（例： `AdministratorAccess`、 `ReadOnlyAccess` ）
5. **AWS アカウント** 画面で、ユーザー／グループに許可セットを割り当てる
6. ユーザーに届いた招待メールから初回パスワードと MFA を設定

以降、コンソールへのアクセスは Identity Center のログインポータル URL（例：`https://d-xxxxxxxxx.awsapps.com/start`）を経由する。

2.4.3 CLI からのログイン

CLI を Identity Center で使う設定は以下の通り。

```
aws configure sso
# SSO start URL: https://d-xxxxxxxxxx.awsapps.com/start
# SSO region: ap-northeast-1
# ブラウザが開くのでログイン
# アカウント・ロールを選択
# プロファイル名を入力(例: dev)

aws sso login --profile dev
aws s3 ls --profile dev
```

短期認証情報(最大12時間)が自動発行され、長期アクセスキーをローカルに置く必要がなくなる。漏洩リスクが大幅に下がるため、可能ならこちらを使う。

2.5 作業用 IAM ユーザーの作成 (Identity Center を使わない場合)

個人検証で Identity Center の設定が重いと感じる場合は、従来型の IAM ユーザーで始めてもよい。ただしセキュリティ上の妥協点であることは認識する。

1. IAM コンソール → ユーザー → ユーザーを作成
2. ユーザー名を入力(例: `your-name`)
3. コンソールアクセスを有効化 をチェック、パスワードを設定
4. 権限では 既存のポリシーを直接アタッチ → `AdministratorAccess` を選択(検証用途のみ)
5. 作成後、MFA を必ず有効化する

本番環境や他人と共有するアカウントでは `AdministratorAccess` を恒常的に付けず、Identity Center の一時権限を使うべきである。

2.6 リージョンの選択

コンソール右上のリージョン切替で、操作対象のリージョンを選ぶ。日本国内でのレイテンシ重視なら 東京 (`ap-northeast-1`) が基本である。一度設定すると、そのリージョンのリソースだけが表示される 点に注意する。

- ・グローバルサービス (IAM、Route 53、CloudFront、S3 の一部) はリージョン切替に関わらず 共通
- ・東京で作ったリソースは、うっかりバージニア北部を開くと「何も無い」ように見えるため混乱しやすい

2.7 AWS CLI のセットアップ

2.7.1 インストール

Linux / macOS では以下で導入できる。

```
# Linux (x86_64)
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscli.zip
unzip awscli.zip
sudo ./aws/install

# macOS (Homebrew)
brew install awscli

# 動作確認
aws --version
```

2.7.2 認証情報の設定

Identity Center を使う場合は `aws configure sso` (前述) を使う。アクセスキー方式の場合は以下。

```
aws configure
# AWS Access Key ID: ...
# AWS Secret Access Key: ...
# Default region name: ap-northeast-1
# Default output format: json
```

設定は `~/.aws/credentials` と `~/.aws/config` に保存される。`--profile` オプションで複数のアカウント／ロールを切り替えられる。

2.7.3 動作確認

```
# 現在の認証情報を確認
aws sts get-caller-identity

# S3 バケット一覧
aws s3 ls

# 東京リージョンの EC2 インスタンス一覧
aws ec2 describe-instances --region ap-northeast-1
```

2.8 初期設定のチェックリスト

作業を進める前に、以下がすべて完了していることを確認する。

- ☐ ルートユーザーに MFA を設定した
- ☐ ルートユーザーのアクセスキーは作成していない／削除した
- ☐ AWS Budgets で月次予算アラートを設定した
- ☐ Free Tier 使用量アラートを有効化した
- ☐ IAM Identity Center または作業用 IAM ユーザーを作成した
- ☐ 作業用ユーザーにも MFA を設定した
- ☐ 東京リージョンをデフォルトに設定した

- [] AWS CLI が動作し、`aws sts get-caller-identity` で自分のユーザーが確認できた
ここまで終われば、3 章以降の実習に進む準備が整っている。

3 IAM（認証・認可）

IAM（Identity and Access Management）は、「誰が（認証）」「何を（認可）」できるかを管理する AWS のセキュリティ基盤である。どのサービスを使うにしても IAM が土台になるため、最初に理解しておきたい。

3.1 IAM の主要コンポーネント

3.1.1 4 つの基本要素

要素	説明
ユーザー（User）	個人や単一のアプリに紐づく長期的なアイデンティティ
グループ（Group）	複数ユーザーをまとめる単位。グループにポリシーをアタッチできる
ロール（Role）	一時的に引き受けるアイデンティティ。EC2 や Lambda、他アカウントに付与する
ポリシー（Policy）	JSON で書かれた許可／拒否のルール

ポリシーを直接ユーザー／ロールにアタッチすることもできるし、グループ経由でアタッチすることもできる。

3.1.2 認証と認可の流れ

1. 認証（AuthN）：API コール時に、アクセスキー or 一時認証情報 or SSO によって「誰か」が特定される
2. 認可（AuthZ）：そのプリンシパルに付与されたポリシーと、対象リソース側のポリシーを評価して許可／拒否が決まる
3. 明示的 Deny はどのポリシーの Allow も上書きする

3.2 IAM ポリシーの構造

ポリシーは JSON で記述される。以下は S3 バケットへの読み取りを許可する最小例である。

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadMyBucket",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
```

```

    "arn:aws:s3::my-bucket",
    "arn:aws:s3::my-bucket/*"
  ]
}
]
}
```

3.2.1 主要フィールド

フィールド	意味
Version	必ず 2012-10-17 と書く（現行仕様の識別子）
Effect	Allow または Deny
Action	許可／拒否する API アクション。s3:* のようにワイルドカードも可
Resource	対象リソースの ARN（Amazon Resource Name）
Condition	IP、MFA、時刻など追加条件を指定
Principal	リソースベースポリシー（後述）で対象プリンシパルを指定

3.2.2 ARN（Amazon Resource Name）

AWS のあらゆるリソースは ARN と呼ばれる一意の識別子を持つ。

```
arn:aws:<service>:<region>:<account-id>:<resource>
```

例：

- arn:aws:s3::my-bucket （S3 はリージョンとアカウント ID が空）
- arn:aws:ec2:ap-northeast-1:123456789012:instance/i-0abcd1234efgh5678
- arn:aws:iam::123456789012:role/EC2-S3-ReadOnly

ポリシーの Resource 指定では ARN を使う。

3.2.3 条件（Condition）

アクセス元 IP、時刻、MFA の有無などで制御を追加できる。典型的なのは「MFA なし／社外 IP からの操作を一律拒否する」ための Deny ポリシーである。

```

{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyWithoutMfaOrOutsideCorpIp",
    "Effect": "Deny",
    "Action": "*",
    "Resource": "*",
    "Condition": {
      "BoolIfExists": { "aws:MultiFactorAuthPresent": "false" },
      "NotIpAddress": { "aws:SourceIp": ["203.0.113.0/24"] }
    }
  }]
}
```

```

    }
  }
}
```

Allow 側の **Condition** ではなく **Deny** 側で書くのがセオリーである。**Allow** + ワイルドカードリソース + **Condition** の組み合わせは、**Condition** が条件を満たすときに **過剰な権限を与えてしまう** 危険がある。アクセス制限は「してはいけないケースを **Deny** で塞ぐ」方向で設計する。

3.2.4 必要な IAM アクションの調べ方

実運用では「権限不足エラーが出た → 何を足せばよいか」を特定する場面が多い。次の経路で調べられる。

- ・ **エラーメッセージ**：not authorized to perform: ec2:TerminateInstances のように必要アクション名がそのまま書かれる
- ・ **CloudTrail**：実行したい操作をまず権限ありで実行し、ログから **eventName** と **eventSource** を抜く
- ・ **IAM Access Analyzer Policy Generation**：実際の使用ログから必要最小のポリシーを自動生成
- ・ **IAM Policy Simulator**：ポリシー適用前に特定アクションが通るか試せる
- ・ **サービスの公式ドキュメント**：各サービスの“Actions, resources, and condition keys”ページに列挙されている

3.3 ポリシーの種類

3.3.1 管理ポリシーとインラインポリシー

種類	説明
AWS 管理ポリシー	AWS が提供する既製ポリシー。AdministratorAccess、ReadOnlyAccess など
カスタマー管理ポリシー	利用者が作成する独立したポリシー。複数のユーザー／ロールにアタッチできる
インラインポリシー	ユーザー／ロールに直接埋め込む。1対1で紐づくため他にアタッチできない

運用上は **カスタマー管理ポリシー** を作って再利用するのが望ましい。

3.3.2 アイデンティティベースとリソースベース

- ・ **アイデンティティベースポリシー**：IAM ユーザー／グループ／ロールにアタッチする
 - ・ **リソースベースポリシー**：S3 バケット、SQS キュー、Lambda 関数などリソース側に定義する
- S3 バケットポリシー、Lambda のリソースベースポリシー、KMS キーポリシーなどは後者の代表例である。

3.3.3 SCP（サービスコントロールポリシー）

AWS Organizations 配下で、アカウントそのものに対して使える機能を制限する。親アカウントから子アカウントへのガードレールとして機能する。例：「東京・大阪以外のリージョンでの API 実行を全面禁止」。

3.4 ロールと一時認証情報

3.4.1 なぜロールが必要か

EC2 で動くアプリが S3 にアクセスする場合、従来はアクセスキーをサーバに置いていた。これは漏洩リスクが高く、管理もしづらい。IAM ロールを使えば、インスタンスに一時認証情報が自動配布され、定期的にローテーションされる。

3.4.2 ロールの使われ方

- EC2 インスタンスプロファイル：EC2 に付与する。インスタンス内からメタデータ経由で取得
- Lambda 実行ロール：Lambda 関数が AWS API を呼ぶときの権限
- クロスアカウントロール：別アカウントから `sts:AssumeRole` で引き受ける
- OIDC / SAML 連携ロール：IdP（GitHub Actions など）からフェデレーションで引き受ける
- サービスリンクロール：AWS 側が管理する特別なロール（ELB、ECS などが自動作成）

3.4.3 信頼ポリシーとアクセスポリシー

ロールには 2 種類のポリシーが紐づく。

- 信頼ポリシー（Trust Policy）：「誰がこのロールを引き受けられるか」
- アクセスポリシー（Permissions Policy）：「このロールが何をできるか」

EC2 がこのロールを引き受けられるようにする信頼ポリシーの例：

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

3.4.4 GitHub Actions から OIDC で引き受ける例

アクセスキーを発行せず、GitHub Actions から AWS を操作するためのモダンな構成。

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
```



```

    "Principal": {
      "Federated": "arn:aws:iam::123456789012:oidc-provider/
token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
        "token.actions.githubusercontent.com:sub": "repo:octo/app:ref:refs/heads/main"
      }
    }
  }
}

```

`sub` の条件に `StringLike` でワイルドカード（例：`repo:octo/app:*`）を使うのは危険である。フォーク PR やタグ push など、想定外のワークフローからもロールを引き受けられてしまう。`StringEquals` で特定ブランチ・環境・タグを明示するのが安全である。複数条件を許したい場合は配列で列挙する。

```

"token.actions.githubusercontent.com:sub": [
  "repo:octo/app:ref:refs/heads/main",
  "repo:octo/app:environment:prod"
]

```

3.5 ベストプラクティス

3.5.1 最小権限の原則

必要最小限の権限だけを付与する。運用で権限不足が発生したら、CloudTrail のログや IAM Access Analyzer で実際に使われた API を確認し、必要なものだけを追加していく。

3.5.2 権限境界（Permissions Boundary）

開発者が自分で IAM ロールを作れるが、特定の権限を超えないようにしたい場合に使う。IAM ロール／ユーザーの「権限の上限」を定義する。

3.5.3 実装上の推奨

- ・ ルートユーザーは使わない、MFA を必ず設定する
- ・ 長期アクセスキーを避ける、代わりにロール／Identity Center の一時認証情報を使う
- ・ グループ経由で権限付与、ユーザーに直接アタッチしない
- ・ カスタマー管理ポリシー を作り、複数ユーザーで共有する
- ・ アクセスキーを定期的にローテーション、未使用のキーは削除
- ・ IAM Access Analyzer で外部公開されたリソースを検出
- ・ CloudTrail で監査ログを取り続ける

3.5.4 アクセスキーを CI に置かない

GitHub Actions や GitLab CI などでは AWS を操作する場合は、長期アクセスキーをシークレットに保存するのではなく **OIDC フェデレーション** でロールを引き受ける。漏洩リスクがゼロに近づく。

3.6 実習：権限不足を体験する

IAM の感覚をつかむには、実際に権限を絞ってエラーを出すのが早い。

1. 管理者ユーザーとは別に、`ReadOnlyAccess` のみを付けた IAM ユーザー `demo-readonly` を作成
2. そのユーザーで AWS CLI の別プロファイルを設定
3. `aws ec2 describe-instances --profile demo-readonly` は成功する
4. `aws ec2 terminate-instances ... --profile demo-readonly` は `UnauthorizedOperation` で失敗する
5. エラー文言には必要なアクション名（例：`ec2:TerminateInstances`）が含まれる

このフィードバックループを通じて、「エラー → 必要アクション特定 → ポリシー更新」の手順が身につく。

3.7 よく使う AWS 管理ポリシー

ポリシー名	用途
AdministratorAccess	全権限。検証・管理者用（本番で一般ユーザーに付けない）
ReadOnlyAccess	全サービスの読み取りのみ。監査・調査用
PowerUserAccess	IAM 以外ほぼ全権限。開発者用
AmazonS3FullAccess	S3 の全操作
AmazonEC2FullAccess	EC2 の全操作
IAMUserChangePassword	自分のパスワード変更を許可。全ユーザーに付けるとよい
AWSSupportAccess	サポートケースの作成・閲覧
Billing	請求情報の閲覧・管理

最初はこれらを組み合わせて始め、慣れてきたらカスタマー管理ポリシーで絞り込む。

4 VPC（ネットワーク）

VPC（Virtual Private Cloud）は、AWS 内に自分専用の仮想ネットワークを作るサービスである。EC2、RDS、Lambda（VPC モード）、EKS など、ほとんどの計算リソースは VPC の中に置かれる。AWS のネットワーク設計は VPC を中心に回る。

4.1 VPC の基本構造

4.1.1 階層

```
アカウント / リージョン
└─ VPC (例: 10.0.0.0/16)
    │
    ├── AZ A
    │   ├── サブネット pub-a (10.0.1.0/24, パブリック)
    │   └── サブネット prv-a (10.0.11.0/24, プライベート)
    └── AZ C
        ├── サブネット pub-c (10.0.2.0/24, パブリック)
        └── サブネット prv-c (10.0.12.0/24, プライベート)
```

- **VPC**：リージョン単位。1つの CIDR ブロック（IPv4 アドレス範囲）を持つ
- **サブネット**：AZ 単位。VPC の CIDR を分割して割り当てる
- サブネットを跨ぐ通信はリージョン内では無料／低遅延

4.1.2 CIDR ブロックの設計

RFC1918 のプライベート IP レンジから選ぶのが基本。

- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16

AWS の VPC は /16 から /28 まで設定できる。将来の拡張を考えて /16 で広めに取り、サブネットは /24 単位で切ると扱いやすい。オンプレや他 VPC と接続する予定があるなら、CIDR 被りを避けるため事前に割り当てルールを決める。

4.1.3 デフォルト VPC

各リージョンには最初から **デフォルト VPC** が用意されている。172.31.0.0/16 のような CIDR で、各 AZ にパブリックサブネットが配置済み。検証・学習には便利だが、本番環境では **専用の VPC を自分で作る** のが望ましい。

4.2 サブネットとルートテーブル

4.2.1 パブリックサブネットとプライベートサブネット

技術的には両者に本質的な違いはない。ルートテーブルが Internet Gateway にルーティングしているかどうかで決まる。

種類	特徴
パブリックサブネット	Internet Gateway へのルートを持つ。パブリック IP を持てば外部から到達可能
プライベートサブネット	Internet Gateway へのルートを持たない。外部からは到達不可能
NAT 経由のプライベート	NAT Gateway 経由で外向き通信のみ可能（着信は不可）
隔離サブネット	外部との通信を一切持たない（DB 専用など）

4.2.2 ルートテーブルの役割

ルートテーブルは、各サブネットからのパケット転送先を決める。

パブリックサブネット用のルートテーブルの例

送信先	ターゲット	
10.0.0.0/16	local	# VPC 内部
0.0.0.0/0	igw-0abc...	# デフォルトルートを IGW に

プライベートサブネット（NAT経由）のルートテーブル

送信先	ターゲット	
10.0.0.0/16	local	
0.0.0.0/0	nat-0def...	# デフォルトルートを NAT Gateway に

1つのサブネットに関連付けられるルートテーブルは1つ。ルートテーブル自体は複数のサブネットで共有できる。

4.3 インターネット接続

4.3.1 Internet Gateway (IGW)

VPC をインターネットに接続するためのコンポーネント。VPC に1つだけアタッチする。IGW 自体は課金されない。

4.3.2 NAT Gateway

プライベートサブネットから **外向きだけ** インターネット接続するためのサービス。EC2 が OS パッケージ更新のためにインターネットに出る、といった用途で必須。

- ・ **時間課金 + データ転送課金** の両方がかかる
- ・ マルチ AZ 構成では、AZ ごとに NAT Gateway を置くのが推奨（耐障害性とデータ転送料削減のため）

- ・代替として NAT インスタンス（EC2 を NAT として使う）もあるが、運用負荷が上がるため推奨されない

4.3.2.1 △ NAT Gateway のコスト事故に注意

NAT Gateway は 個人検証で最も事故になりやすいリソース。東京リージョンでは1台あたり 約 \$0.062/時 (月 \$45〜) + データ処理 \$0.062/GB が発生し、立てっぱなしだと 月に 6,000〜7,000 円以上の固定費になる。VPC を削除してもサブネットや NAT Gateway が残っていれば課金は続く。

回避策：

- ・検証終了後は必ず NAT Gateway を削除する（Elastic IP も合わせて解放）
- ・長期放置するなら S3 / DynamoDB はゲートウェイ型 VPC エンドポイント（無料）で置き換える
- ・どうしても NAT が必要でコストを抑えたい場合は、小さい NAT インスタンス を ASG で必要時間だけ起動する構成に倒す
- ・IaC（CloudFormation / CDK / Terraform）でまとめて `destroy` できる状態にしておく

4.3.3 外向き専用 Internet Gateway（Egress-Only IGW）

IPv6 の外向き専用 IGW。IPv6 には NAT の概念がないため、外向きだけ通す専用 GW が用意されている。

4.3.4 パブリック IP

- ・パブリック IPv4：2024 年 2 月 1 日以降、割り当て・アタッチの有無にかかわらず一律 \$0.005/時 が課金される（月約 \$3.6、Free Tier で 12 か月分は月 750 時間無料）。停止すると動的アドレスは変わる
- ・Elastic IP：固定のパブリック IP。稼働中インスタンスに関連付けている間も同額（\$0.005/時）、未関連付けや停止中インスタンスへの関連付けも同額
- ・IPv6 には現時点で課金はない

パブリック IPv4 は「不要なら持たない」のが基本。外向き通信だけなら NAT Gateway か VPC エンドポイント、着信不要なら Private Subnet + SSM Session Manager 経由で接続する。

4.4 セキュリティグループとネットワーク ACL

4.4.1 セキュリティグループ（SG）

ENI（Elastic Network Interface）単位のステートフル・ファイアウォール。EC2・RDS・ALB など、ネットワークインタフェースを持つリソースに適用される。

- ・ステートフル：戻りトラフィックは自動で許可される
- ・許可ルールのみ：明示的な拒否は書けない
- ・ENI あたり最大 5 つ の SG をアタッチできる

典型的な SG ルールの書き方：

```
# Webサーバ用 SG
インバウンド：
```

```

HTTP (80)   から 0.0.0.0/0
HTTPS (443) から 0.0.0.0/0
SSH (22)    から 203.0.113.0/24 # オフィス IP のみ

```

```

アウトバウンド:
すべて         へ 0.0.0.0/0 (デフォルト)

```

4.4.2 SG 間参照

送信元に **別の SG** を指定できる。これを使うと「ALB の SG から来る通信だけ EC2 が受け付ける」のような層状の設計が IP 依存なしで組める。

```

# EC2 (Webサーバ) 側 SG
インバウンド:
HTTP (80) から sg-xxxxxx (ALB用SG)

```

4.4.3 ネットワーク ACL (NACL)

サブネット単位のステートレス・ファイアウォール。

- ・ステートレス：戻りトラフィックも明示的に許可する必要がある
- ・許可・拒否の両方が書ける
- ・ルールに番号を振り、小さい順に評価される

基本的には **セキュリティグループで制御し、NACL はデフォルトのまま** で運用する。特定 IP を VPC 全体でブロックしたい場合だけ NACL を触る、という使い分けが現実的である。

4.5 VPC エンドポイント

VPC 内から AWS のマネージドサービス (S3、DynamoDB など) にアクセスする際、通常はインターネット経由となる。**VPC エンドポイント** を使うと、AWS 内部ネットワークを経由してサービスに到達できる。

4.5.1 ゲートウェイ型エンドポイント

- ・対象：S3 と DynamoDB のみ
- ・ルートテーブルにエントリが追加される方式
- ・無料

S3 と DynamoDB だけが対象なのは、この 2 つが **歴史的経緯でリージョン単位のパブリックエンドポイント** を持ち、ルートテーブルベースの制御で十分賄えるため。他のサービスは後述のインタフェース型に統一されている。

プライベートサブネットの EC2 から S3 にアクセスするために NAT Gateway を立てているなら、S3 ゲートウェイエンドポイントに置き換えるだけで NAT 通信量 (= 課金) が大幅に減る。個人利用でも設定しておくといよい。

4.5.2 インタフェース型エンドポイント

- ・ 対象：S3・DynamoDB 以外の多数のサービス（KMS、SQS、Secrets Manager 等）
- ・ ENI がサブネット内に作られ、プライベート IP で到達できる
- ・ **時間課金 + データ処理課金**
- ・ S3 もインタフェース型で作れるが、同一リージョン内アクセスならゲートウェイ型を優先する（無料）

4.5.2.1 PrivateLink

インタフェース型エンドポイントの基盤技術。自社サービスを他アカウントからプライベートに公開することもできる。SaaS 事業者がよく使う。

4.6 VPC 間・オンプレ接続

4.6.1 VPC ピアリング

2 つの VPC を直接接続する。シンプルだがハブ&スポーク構成には向かない（推移的ルーティング不可）。

4.6.2 Transit Gateway

複数 VPC、オンプレ、VPN、Direct Connect を 1 つのハブにまとめて接続する。VPC 数が増えたらこちら。

4.6.3 Site-to-Site VPN

オンプレ拠点と VPC を IPsec VPN で接続する。設定は比較的容易で、Direct Connect よりも低コスト。

4.6.4 Direct Connect

オンプレと AWS を専用線で接続する。高帯域・低レイテンシ・安定性が要件のケース向け。初期工事費や月額費用が発生する。

4.7 DNS と DHCP

4.7.1 VPC のデフォルト DNS

VPC には **Route 53 Resolver** が組み込まれており、**10.0.0.2** のような予約アドレスで DNS が解決される。EC2 は自動で `/etc/resolv.conf` にこれが設定される。

4.7.2 DNS ホスト名と DNS 解決

VPC の属性として以下 2 つを有効化しておくのが基本。

- ・ **enableDnsSupport**：VPC 内で Amazon DNS を使う
- ・ **enableDnsHostnames**：パブリック IP を持つインスタンスに `ec2-xx-xx-xx-xx.ap-northeast-1.compute.amazonaws.com` 形式のホスト名を付与

4.7.3 プライベートホストゾーン

Route 53 で VPC 内だけに見える独自ドメインを運用できる。 `internal.example.com` のようなドメインを作って、社内リソースに名前を付けると可読性が上がる。

4.8 実践：小規模な Web 構成の VPC 設計

2 AZ の最小構成で、パブリック層（ALB）・アプリ層（EC2）・DB 層（RDS）を分離する設計例。

VPC: 10.0.0.0/16 (ap-northeast-1)

AZ-a:

Public:	10.0.1.0/24	ALB
App:	10.0.11.0/24	EC2
DB:	10.0.21.0/24	RDS Master

AZ-c:

Public:	10.0.2.0/24	ALB
App:	10.0.12.0/24	EC2
DB:	10.0.22.0/24	RDS Standby

Internet Gateway: VPC にアタッチ

NAT Gateway: 各パブリックサブネットに1つ (AZ-a, AZ-c)

- Appサブネットの 0.0.0.0/0 は同じ AZ の NAT へ

セキュリティグループ:

sg-alb:	0.0.0.0/0 → 80/443
sg-app:	sg-alb → 8080
sg-db:	sg-app → 3306

VPCエンドポイント:

S3ゲートウェイ (App・DBのルートテーブル)

この構造を CloudFormation や Terraform でコード化してしまえば、検証環境を何度でも立て直せる。

4.9 VPC 作成時の注意点

- ・ CIDR は後から広げられるが、狭められない。最初に広め取る
- ・ 1つの AZ に寄せるとその AZ 障害で全滅する。最低 2 AZ を使う
- ・ NAT Gateway は固定費が発生する。検証では削除を忘れずに
- ・ サブネットの IP は、最初と最後の数個が AWS 予約で使えない（ /24 なら使えるのは 251 個）
- ・ Lambda を VPC に入ると ENI 起動分だけコールドスタートが伸びる 点に注意

VPC は一度作ったら壊しにくいので、最初から冗長性と拡張性を意識して設計する。

5 EC2（仮想マシン）

Amazon EC2（Elastic Compute Cloud）は、AWS で仮想マシンを起動・管理するサービスである。AWS の原点とも言える中心サービスであり、他のサービスを理解する上でも基本になる。

5.1 EC2 の基本概念

5.1.1 インスタンス

EC2 の **インスタンス** は、VPC 内のサブネットで起動する 1 台の仮想マシンである。起動時に以下を選択する。

- **AMI**：OS とソフトウェアの雛形
- **インスタンスタイプ**：CPU・メモリ・ネットワーク性能
- **キーペア**：SSH ログイン用の公開鍵／秘密鍵
- **ネットワーク**：VPC、サブネット、パブリック IP の有無
- **セキュリティグループ**：ファイアウォール設定
- **ストレージ（EBS）**：ルートボリュームと追加ディスク
- **IAM インスタンスプロファイル**：インスタンスから AWS API を呼ぶ権限
- **ユーザーデータ**：初回起動時に実行するスクリプト

5.1.2 料金体系

購入オプション	特徴
オンデマンド	秒／時間単位の従量課金。短期・予測困難な用途
Savings Plans	1～3 年のコミットで最大約 72% 割引。柔軟性が高い
Reserved Instance	従来型の予約購入。現在は Savings Plans 推奨
Spot	余剰キャパシティを最大 90% 割引で利用。中断あり
Dedicated Host	物理サーバを専有。BYOL ライセンス等で利用

個人検証では **オンデマンド** が基本。長期本番運用では **Savings Plans**、バッチ処理では **Spot** を組み合わせる。

5.2 AMI（Amazon Machine Image）

AMI は「OS + 初期ソフトウェア + ディスクイメージ」をまとめた雛形である。

5.2.1 主な AMI の出所

- **AWS 提供**：Amazon Linux 2023、Ubuntu、RHEL、Windows Server 等
- **AWS Marketplace**：サードパーティベンダのイメージ（WordPress、各種アプライアンス）
- **コミュニティ**：他ユーザーが公開した AMI
- **カスタム AMI**：自分で構築したインスタンスから作成

5.2.2 Amazon Linux 2023

AWS が提供する独自の Linux ディストリビューション。EC2 で使う場合の第一候補である。

- AWS SDK、CLI、SSM エージェントなどが同梱
- 最低 5 年のサポート
- `dnf` ベースのパッケージ管理（Fedora / RHEL 系）
- セキュリティパッチが AWS リポジトリから即座に配信される

5.2.3 カスタム AMI の作り方

ベースインスタンスに必要なソフトを入れて、そのイメージをスナップショット化する。

```
# 対象インスタンスから AMI を作成
aws ec2 create-image \
  --instance-id i-0abc123... \
  --name "my-app-v1.0" \
  --description "app baked on 2026-04-21"
```

作成した AMI から複数インスタンスを立ち上げれば、初期構成時間が大幅に短縮される。本番運用では Packer などで AMI をビルドパイプライン化するのが一般的。

5.3 インスタンスタイプ

EC2 のインスタンスタイプは `<ファミリー><世代><サイズ>` の命名規則を持つ。例： `m7i.large`、`c7g.xlarge`。

5.3.1 主要ファミリー

ファミリー	用途	例
T	バースト性能。軽量 Web、検証用	t3.micro, t4g.small
M	汎用。メモリと CPU がバランス	m7i.large
C	コンピュート最適化。CPU 重視	c7i.xlarge
R	メモリ最適化。キャッシュ、DB	r7i.large
X	極大メモリ。インメモリ DB	x2iedn.xlarge
I	ローカル NVMe SSD。高 IO DB	i4i.large
G/P	GPU。機械学習、グラフィック	g5.xlarge, p5.48xlarge

5.3.2 世代と CPU アーキテクチャ

世代番号は大きいほど新しく、基本的に同じ価格でも性能が高い。

- サフィックス `g`（例： `t4g`、`m7g`）は **Graviton**（ARM）プロセッサ。x86 よりコスパが良いことが多い
- サフィックス `i` は Intel、`a` は AMD
- サフィックス `n` はネットワーク強化、`d` はローカル NVMe 付き

アプリが ARM 対応なら、まず **Graviton** を試す価値がある。

5.3.3 t ファミリーのクレジット仕様

T インスタンスは **CPU クレジット** でバースト動作する。

- 通常はベースライン CPU 使用率（例：t3.micro は 10%）で稼働
- 余った分がクレジットとして蓄積
- クレジットを消費することで一時的に 100% まで跳ね上げられる
- クレジット枯渇時は **Standard モード**ではベースラインに戻る、**Unlimited モード**では追加課金で持続

CI や機械学習など CPU を継続的に使う用途では t ファミリーを避け、M/C ファミリーを使う。

5.4 ストレージ（EBS / インスタンスストア）

5.4.1 EBS（Elastic Block Store）

ネットワーク越しに接続される永続ブロックストレージ。インスタンスを止めてもデータは残る。

ボリュームタイプ	特徴
gp3	汎用 SSD。コスパが高い。通常はこれ
gp2	旧世代の汎用 SSD。新規では gp3 を使う
io2 / io2 Block Express	高 IOPS 要件の本番 DB 用
st1	スループット最適化 HDD。ログ・DWH
sc1	コールド HDD。低頻度アクセス

- スナップショットを S3 に保存（裏側）してバックアップできる
- スナップショットから別 AZ・別リージョンへボリュームを復元できる
- 暗号化は **常に有効化** を推奨。EC2 の設定 → **データ保護とセキュリティ** → EBS 暗号化 → **デフォルトで常に暗号化** をリージョンごとに ON にする
- 旧世代の **magnetic**（standard）は **新規作成不可**。既存のみ使用可能

5.4.2 インスタンスストア

一部のインスタンスタイプに付属する **ローカル NVMe SSD**。

- 高速だが **インスタンスを停止・終了するとデータが消える**
- キャッシュ・一時ファイル・シャーディングされた DB など、消えてもよいデータに使う

5.4.3 ルートボリュームの扱い

デフォルトでは **インスタンス終了時にルート EBS も削除** される。データ保持したい場合は「終了時に削除」のチェックを外すか、別途データ用ボリュームを使う。

5.5 インスタンスメタデータサービス (IMDSv2)

EC2 内部からは `169.254.169.254` のインスタンスメタデータサービス (IMDS) にアクセスでき、インスタンスの ID、リージョン、IAM ロールの一時認証情報などが取得できる。SDK や CLI が裏で呼んでいる。

IMDS には v1 (token 不要、GET のみ) と v2 (PUT でトークンを取ってから GET) があり、**v1 は SSRF 脆弱性と組み合わせるとクラウド認証情報が盗まれる経路** になる。2024 年以降の新規 AMI はデフォルトで **IMDSv2 のみを受け付ける** 設定だが、既存の起動テンプレートや持ち込み AMI では明示する。

```
# 起動テンプレート／インスタンスで IMDSv2 必須化
aws ec2 modify-instance-metadata-options \
  --instance-id i-0abc... \
  --http-tokens required \
  --http-put-response-hop-limit 2 \
  --http-endpoint enabled \
  --instance-metadata-tags enabled
```

CloudFormation / CDK / Terraform で新規作成する場合も、`MetadataOptions` や `metadata_options` で同等の設定を必ず入れる。

5.6 ネットワーク関連

5.6.1 SSM Session Manager (推奨)

SSH を開かない 現代的な接続方式。Systems Manager 経由でインスタンスにコンソール接続する。新規構築では **第一選択肢** として扱う。

- ポート 22 を開放不要
- SSH 鍵の管理不要
- IAM で接続権限を制御
- 全コマンドが CloudTrail に記録される

前提：インスタンスに **SSM エージェント** (Amazon Linux 2023 / 2、最近の Ubuntu / RHEL 公式 AMI には標準搭載) と、IAM ロール `AmazonSSMManagedInstanceCore` を付与する。

```
# CLIから接続
aws ssm start-session --target i-0abc123...
```

検証・本番ともに、**SSM Session Manager** を**第一選択** にすると運用が軽くなる。

5.6.2 SSH による接続 (必要な場合のみ)

EC2 にキーペアベースの SSH で接続したい場合は、起動時に **キーペア** (公開鍵・秘密鍵のペア) を関連付ける。

```
# キーペアを作成して秘密鍵を取得
aws ec2 create-key-pair --key-name my-key \
  --query 'KeyMaterial' --output text > ~/.ssh/my-key.pem
chmod 400 ~/.ssh/my-key.pem

# Amazon Linux へ SSH
ssh -i ~/.ssh/my-key.pem ec2-user@<パブリックIP>
```

キーペア紛失時は、新しいインスタンスを立てて EBS を付け替えて復旧する、という手順になる。パスワード認証は基本使わない。SSH を開けるとポート 22 スキャンの攻撃対象となるため、**本番用途では SSM Session Manager を優先**し、SSH はどうしても必要な場合（OS 側の診断、rsync、SSH ポートフォワーディング等）に限定する。

5.6.3 Elastic IP

固定のパブリック IPv4 アドレス。インスタンスを再起動しても IP が変わらなくする。

- ・ 2024 年 2 月以降は **関連付けの有無にかかわらず常時課金**（\$0.005/時、約 \$3.6/月）
- ・ **使わないのに確保したまま放置しない**（コンソールの「Elastic IP」一覧で解放する）

5.7 Auto Scaling とロードバランサ

5.7.1 ロードバランサ（ELB）

種類	用途
ALB (Application)	L7 / HTTP(S)。パス・ホスト・ヘッダでルーティング
NLB (Network)	L4 / TCP・UDP。超低レイテンシ・高スループット
GWLB (Gateway)	セキュリティアプライアンスの挿入用
CLB (Classic)	旧世代。新規では使わない

Web アプリなら基本は ALB。/api/* を別のターゲットグループに振り分ける、マイクロサービスのエッジなどでよく使われる。

5.7.2 ターゲットグループとヘルスチェック

ALB / NLB は **ターゲットグループ** を介して EC2 やコンテナにトラフィックを転送する。ヘルスチェック（例：/health に HTTP 200）でターゲットの生死を判定し、不健全なものを切り離す。

5.7.3 Auto Scaling グループ（ASG）

EC2 の台数を自動で増減させる。

- ・ **起動テンプレート** で AMI・インスタンスタイプ・ユーザーデータを定義
- ・ **最小／希望／最大** 台数を設定
- ・ **CPU 使用率** や **リクエスト数** などのメトリクスでスケーリング
- ・ ELB と連携して、新規インスタンスを自動的にターゲットグループに登録

例：平常時 2 台、CPU 70% 超で最大 10 台まで増える、というポリシーが典型的。

5.8 ユーザーデータと初期化

起動時に **ユーザーデータ** にシェルスクリプトを入れると、インスタンス起動直後に一度だけ実行される。

```
#!/bin/bash
dnf update -y
dnf install -y nginx
systemctl enable --now nginx
cat > /usr/share/nginx/html/index.html <<EOF
<h1>Hello from $(hostname)</h1>
EOF
```

繰り返し使う設定は **AMI に焼き込む**、ユーザーデータは **AMI 間の差分** を埋める用途に使うのが基本。

5.9 実習：ALB + EC2 の最小構成

1. 4 章で作ったパブリックサブネット × 2 を使う
2. 起動テンプレートを作成（AMI、`t3.micro`、SG `sg-app`、ユーザーデータで nginx 起動）
3. ASG を最小 1・最大 2 で作成
4. ALB を作成、ターゲットグループに ASG を登録
5. ALB の DNS 名にアクセスして nginx ページが返れば成功
6. 確認後は ASG を 0 台に縮める か まとめて削除

実習後は **Elastic IP**、**NAT Gateway**、**停止中の EBS** を見回って、不要ならすぐ削除する。これが月末の請求事故を防ぐ最大のコツである。

5.10 トラブルシューティング

症状	確認ポイント
SSH できない	SG の 22 番開放、キーペア、パブリック IP、ルートテーブル
起動直後に停止する	インスタンスタイプの空き枯渇、IAM 権限不足、ユーザーデータ失敗
ALB でヘルスチェック失敗	SG (ALB → EC2)、ヘルスチェックのパスとポート、起動の遅延
起動が Pending のまま	AZ のキャパ不足。別 AZ を試す
AMI が世代不一致で失敗	AMI が x86 か ARM か、インスタンスタイプと合っているか
ディスク容量不足	EBS を拡張 → OS 側で <code>growpart</code> / <code>xfs_growfs</code>

6 S3（オブジェクトストレージ）

Amazon S3（Simple Storage Service）は、インターネット経由でアクセスできるオブジェクトストレージである。AWS で最も古いサービスの一つで、事実上無制限の耐久性・スケーラビリティを持ち、ログ保管から動画配信、データレイク、静的サイトホスティング、バックアップまで幅広く使われる。

6.1 S3 の基本概念

6.1.1 バケットとオブジェクト

- ・ **バケット**：オブジェクトを格納する最上位の入れ物。名前はグローバルに一意
- ・ **オブジェクト**：ファイル本体＋メタデータ＋キー（ファイルパスに相当する文字列）
- ・ **キー（Key）**：バケット内でオブジェクトを一意に識別する文字列（例：`logs/2026/04/21/app.log`）

S3 には「ディレクトリ」は存在しない。キーに `/` を含めることで、**あたかもディレクトリ構造があるように見える** だけである。

6.1.2 耐久性と可用性

- ・ **耐久性（Durability）**：99.999999999%（イレブンナイン）。年間で 1 万個に 1 個失う確率が約 0.0000001%
- ・ **可用性（Availability）**：Standard は年間 99.99%（設計上）
- ・ 内部では **複数 AZ に冗長コピー** されるため、AZ 障害を吸収できる

6.1.3 バケット名の制約

- ・ 3～63 文字、英小文字・数字・ハイフンのみ
- ・ **グローバルで一意**（他人が `company-backup` を使っていたら作れない）
- ・ 一度作ったら変更不可。削除して作り直す必要がある

命名規則は `<組織>-<環境>-<用途>-<リージョン>` のようにしておくとう衝突しづらい（例：`acme-prod-logs-apne1`）。

6.2 ストレージクラス

用途に合わせてストレージクラスを選ぶことで、大幅にコスト削減できる。

クラス	特徴
S3 Standard	デフォルト。頻繁アクセス用
S3 Intelligent-Tiering	アクセスパターン不明な場合。自動で階層移動
S3 Standard-IA (低頻度)	月 1 回程度の読み出し。最小 30 日課金
S3 One Zone-IA	1 AZ のみ。再生成可能な二次データ
S3 Glacier Instant Retrieval	年 1 回程度のアクセスで即時取得
S3 Glacier Flexible Retrieval	分～時間単位の取り出し時間を許容

クラス	特徴
S3 Glacier Deep Archive	12 時間程度の取り出し。最安。長期アーカイブ
S3 Express One Zone	超低レイテンシ。ML 訓練などの高 IO 用

個人検証では **Standard** で十分。大量ログやバックアップを貯めるなら **Glacier Deep Archive** にすると TB あたり月 \$1 程度まで落ちる。

6.2.1 ライフサイクルルール

オブジェクトの経過日数に応じて、自動的にストレージクラスを移動したり削除したりできる。

```
{
  "Rules": [{
    "Id": "archive-old-logs",
    "Status": "Enabled",
    "Filter": { "Prefix": "logs/" },
    "Transitions": [
      { "Days": 30, "StorageClass": "STANDARD_IA" },
      { "Days": 90, "StorageClass": "GLACIER" },
      { "Days": 365, "StorageClass": "DEEP_ARCHIVE" }
    ],
    "Expiration": { "Days": 2555 }
  }]
}
```

これは「logs/ 配下を 30 日で IA、90 日で Glacier、1 年で Deep Archive、7 年で削除」というルール。ログ系は必ずライフサイクルルールを入れる。

6.3 操作の基本

6.3.1 CLI によるアップロード／ダウンロード

```
# バケット一覧
aws s3 ls

# バケット作成
aws s3 mb s3://my-bucket-20260421 --region ap-northeast-1

# ファイルアップロード
aws s3 cp ./report.pdf s3://my-bucket-20260421/reports/

# ディレクトリ再帰アップロード
aws s3 sync ./dist s3://my-bucket-20260421/site/ --delete

# ダウンロード
aws s3 cp s3://my-bucket-20260421/reports/report.pdf ./
```



```
# 削除
aws s3 rm s3://my-bucket-20260421/reports/report.pdf

# バケット削除（空でない場合は --force）
aws s3 rb s3://my-bucket-20260421 --force
```

`aws s3api` は低レベル API、`aws s3` は高レベルのラッパ。普段使いは `aws s3` が便利。

6.3.2 プリサインド URL

一時的にオブジェクトへのアクセスを許可する URL。アプリから直接 S3 にアップロード／ダウンロードさせる用途で多用する。

```
aws s3 presign s3://my-bucket-20260421/reports/report.pdf \
  --expires-in 3600
```

URL の有効期間内は、認証なしでそのオブジェクトだけにアクセスできる。ログインユーザーに限定公開するファイル配信などに便利。

6.4 アクセス制御

S3 のアクセス制御は歴史的経緯から **複数のレイヤ** が重なっている。これが初心者の混乱の原因になりがち。

6.4.1 階層

レイヤ	役割
ブロックパブリックアクセス	公開設定を全面禁止する強力なスイッチ（推奨 ON）
IAM ポリシー	IAM ユーザー／ロール側からバケットへの権限
バケットポリシー	バケット側に貼るリソースベースポリシー
オブジェクト ACL	旧方式。原則使わない（無効化が推奨）
S3 Object Ownership	バケット所有者に所有権を寄せる設定（推奨）

6.4.2 推奨デフォルト

新規バケットは以下の設定で作る。

- **Block all public access** : ON（2023 年 4 月以降、新規バケットはデフォルト ON）
- **S3 Object Ownership** : Bucket owner enforced（ACL 無効化。2023 年以降デフォルト）
- **バケット暗号化** : 2023 年 1 月以降、新規バケットは **SSE-S3 が自動有効・無効化不可**。機密性が高い場合は **SSE-KMS（CMK）** に変更する
- **バージョンニング** : 必要に応じて有効化

新規作成時点で既に「全部閉じる」状態に近いが、確認は必須。特別な理由（静的サイト公開など）がなければ、必要な穴だけバケットポリシーで開ける。

6.4.3 バケットポリシーの例

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowReadFromVPCE",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-bucket/*",
      "Condition": {
        "StringEquals": {
          "aws:SourceVpce": "vpce-0abc123..."
        }
      }
    }
  ]
}
```

「特定の VPC エンドポイントから来るリクエストだけ GetObject を許可」という制御。Principal に `*` を書いても Condition で絞っていれば安全。

6.4.4 ブロックパブリックアクセスのバイパス

⚠ **ほとんどの場合、この操作は不要** である。静的サイトの HTTPS 配信は後述の **CloudFront + OAC (Origin Access Control)** でバケットを非公開のまま実現できる。一般公開バケットはバケット名列挙やデータ漏洩事故の温床で、できる限り避ける。

どうしてもバケット自体を公開する必要がある場合（レガシーシステムとの互換など）は、影響を最小化するように **必要な項目だけを解除** する。以下はバケットポリシーによる公開は許すが、ACL 経由の公開は止める、という最小解除の例。

```
aws s3api put-public-access-block \
  --bucket my-public-site \
  --public-access-block-configuration \

  "BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=false,RestrictPublicBuckets=false"
```

4 項目すべてを `false` にするのは **最も危険な設定** で、学習目的でも推奨しない。この操作は **CloudTrail と EventBridge で監視** し、誰かが解除したらすぐ通知が飛ぶようにしておく。アカウントレベルでも `aws s3control put-public-access-block` で同様のガードを入れておくと、個別バケットの誤設定から二重に守れる。

6.5 バージョニングとオブジェクトロック

6.5.1 バージョニング

バケットごとにオン／オフを切り替える。有効にすると、上書きや削除しても過去バージョンが残る。

- ・ 誤削除からの復旧： `DeleteMarker` を外せば元に戻せる
- ・ ランサムウェア対策：攻撃者が削除してもバージョンが残る
- ・ ストレージ料金が増える ため、ライフサイクルルールで古いバージョンを削除する

6.5.1.1 バージョニング × ライフサイクルのセット運用

バージョニング有効バケットでは、通常のオブジェクトとは別に「非現行バージョン」が積み重なる。これを放っておくと **ストレージ料金が想定外に増える** のが S3 料金事故の典型パターンである。バージョニングを有効化するなら、**必ず同時に以下のライフサイクルを設定** する。

```
{
  "Rules": [{
    "Id": "expire-old-versions",
    "Status": "Enabled",
    "Filter": {},
    "NoncurrentVersionExpiration": { "NoncurrentDays": 30 },
    "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
  }]
}
```

- ・ `NoncurrentVersionExpiration` : 30 日以上前の古いバージョンを削除
- ・ `AbortIncompleteMultipartUpload` : 失敗して放置されたマルチパートアップロードの破片を削除 (これも可視化されないまま課金対象になりがち)

6.5.2 オブジェクトロック (WORM)

法規制対応のための Write-Once-Read-Many。一度書いたら指定期間削除・上書き不可にする。

- ・ **Governance** モード：特定権限を持つ者は解除可能
- ・ **Compliance** モード：ルートユーザーでも解除不可

監査ログや金融系のエビデンスを改ざんから守る用途で使う。

6.6 静的ウェブサイトホスティング

S3 バケットで静的サイト (HTML/CSS/JS) を配信できる。

1. バケットを作成、**ブロックパブリックアクセスを解除**
2. **プロパティ** → **静的ウェブサイトホスティング** を有効化
3. インデックスドキュメントを `index.html`、エラーを `error.html` に設定
4. バケットポリシーで `s3:GetObject` を `*` に許可
5. `http://<bucket>.s3-website-<region>.amazonaws.com` でアクセスできる

ただし S3 単体では HTTPS を提供しない。**本番では CloudFront + ACM** を前段に置いて HTTPS 化するのが標準構成。

```
[ユーザー] → CloudFront (HTTPS, キャッシュ)
              ↓ OAC (Origin Access Control)
              [S3 バケット (非公開)]
```

この構成なら S3 はブロックパブリックアクセスを維持したまま、CloudFront 経由だけ許可できる。

6.7 イベント通知

S3 へのオブジェクト作成・削除をトリガーに、Lambda / SQS / SNS / EventBridge を起動できる。

- 画像アップロード → Lambda でサムネイル生成
- ログ到着 → SQS 経由でワーカーへ
- 不正なアップロードを EventBridge で検知・通知

これは「S3 をイベント起点にした非同期処理」の定番パターン。サーバーレス構成と相性がよい。

6.8 クロスリージョンレプリケーションと S3 Replication Time Control

耐災害性のために、別リージョンへ自動コピーする機能。

- **CRR**：クロスリージョンレプリケーション
 - **SRR**：同一リージョンレプリケーション（別バケットへ）
 - バージョニング有効が前提
 - **S3 RTC** 付きなら 99.99% のオブジェクトが 15 分以内にレプリケーションされる SLA を得られる
- DR（Disaster Recovery）要件があれば検討する。

6.9 コスト最適化のポイント

- **ライフサイクルで階層化**：古いものを IA → Glacier → Deep Archive に流す
- **Intelligent-Tiering**：アクセスパターンが読めないデータに一律適用
- **不完全なマルチパートアップロードを消す**：ライフサイクルで自動削除（放置すると永久に残る）
- **バージョニング有効バケットの古いバージョン削除**：同じくライフサイクルで
- **データ転送課金に注意**：別リージョンへの転送やインターネット配信は高い
- **S3 Storage Lens** でバケット全体の使用状況を可視化

6.10 S3 のよくあるつまずき

症状	対処
403 AccessDenied	ブロックパブリックアクセス、バケットポリシー、IAM ポリシーを順に確認
バケットが作れない	グローバル一意の名前にする、DNS 準拠の名前にする
料金が想定外	Cost Explorer で「S3 > リクエスト課金」「データ転送」を分析
削除できない	バージョニング有効バケット内は、全バージョンと DeleteMarker を消す
アップロード遅い	マルチパート、Transfer Acceleration、並列数を調整

7 データベース

AWS は多様なデータベースサービスを提供している。リレーショナル、NoSQL、インメモリ、時系列、グラフなど、データモデルと負荷特性に合わせて選択する「**Purpose-Built Database**」という思想である。本章では代表的な 4 系統を扱う。

7.1 サービス選択の早見表

サービス	用途	モデル
RDS	汎用 RDBMS (MySQL、PostgreSQL、Oracle、SQL Server、MariaDB)	リレーショナル
Aurora	RDS 上位版。クラウドネイティブな MySQL / PostgreSQL 互換	リレーショナル
DynamoDB	サーバーレス KVS / ドキュメント。超スケーラブル	NoSQL (KVS)
ElastiCache	Redis / Valkey / Memcached のマネージド	インメモリ
DocumentDB	MongoDB 互換	ドキュメント
Neptune	グラフ DB	グラフ
Timestream	時系列データ	時系列
Redshift	データウェアハウス	列指向
Athena	S3 上のデータに SQL	サーバーレス SQL
OpenSearch Service	全文検索・ログ分析	検索エンジン

まずは **RDS / Aurora / DynamoDB / ElastiCache** の 4 つを押さえれば、ほとんどのユースケースに対応できる。

7.2 RDS (Relational Database Service)

RDS は定番 RDBMS をマネージドで提供する。バックアップ・パッチ適用・フェイルオーバーを AWS 側に寄せられる。

7.2.1 サポートエンジン

- MySQL、PostgreSQL、MariaDB (オープンソース系)
- Oracle、SQL Server (商用。ライセンス持ち込み / 含みを選べる)

新規アプリでは PostgreSQL か Aurora MySQL / Aurora PostgreSQL が選ばれることが多い。

7.2.2 インスタンスクラスとストレージ

- インスタンスクラス: `db.t3.micro`、`db.m7g.large` など EC2 と同じ命名規則
- ストレージタイプ: `gp3` (汎用 SSD、デフォルト)、`io1/io2` (高 IOPS)、`magnetic` (旧式)
- ストレージは後から拡張可能、縮小は不可

7.2.3 高可用性：Multi-AZ 配置

Multi-AZ には 2 つの方式がある。

- **Multi-AZ DB インスタンス**（従来型）：別 AZ にスタンバイを置き、同期レプリケーション。**スタンバイは読み取り不可**。障害時に自動で DNS 切り替え（通常 1～2 分）
- **Multi-AZ DB クラスター**（2022 年以降の新方式、MySQL / PostgreSQL のみ）：**2 つのリーダブルスタンバイ** を別 AZ に持ち、読み取りにも使える。フェイルオーバーが従来型より高速

本番では **Multi-AZ 必須**、検証では無効にしてコストを抑える。読み取り分散と高可用性を両立したい場合は Multi-AZ DB クラスター、または後述の Aurora を検討する。

7.2.4 リードレプリカ

読み取りを分散するための非同期レプリカ。Multi-AZ と別物で、以下の特徴がある。

- 最大 15 台（MySQL / PostgreSQL）
- 別リージョンにも置ける（DR 用途）
- レプリカを昇格させて独立 DB にすることも可能

7.2.5 バックアップと復元

- **自動バックアップ**：デフォルト 7 日、最大 35 日。ポイントインタイム復元（PITR）が可能
- **手動スナップショット**：任意のタイミングで取る。別リージョンにコピー可
- **AWS Backup** で横断管理も可能

PITR は「3 日前の午前 10 時 15 分」のような粒度で復元できる、便利機能。

7.2.6 パラメータグループとオプショングループ

- **パラメータグループ**：`max_connections` など DB 設定を管理
- **オプショングループ**：Oracle / SQL Server 向けの追加機能を管理

本番環境では **デフォルトのパラメータグループを使わず、自前のものを作る** と後から変更しやすい。

7.3 Aurora

RDS と同じ感覚で使えるが、ストレージが **独自の分散ストレージ層** になっている。

7.3.1 Aurora の特徴

- **MySQL 5.7/8.0 互換、PostgreSQL 互換** の 2 系統
- **3 AZ × 2 コピー = 6 コピー** を AWS 側で自動管理
- 1 つのクラスタ内に最大 15 台のリーダーを置ける
- **最大 128 TiB** まで自動拡張
- **Aurora Serverless v2** では負荷に応じて自動でスケール

7.3.2 Aurora Serverless v2

ACU（Aurora Capacity Unit）単位で自動スケールする。

- ・ 2024 年 11 月以降、対応バージョンで **0 ACU まで自動ポーズ**（Aurora PostgreSQL 13.15+/14.12+/15.7+/16.3+、Aurora MySQL 3.08+）。再開時は約 15 秒のウォームアップが入る
- ・ 検証・低頻度利用・スパイクがあるワークロードに向く
- ・ Multi-AZ 相当の可用性

7.3.3 Aurora Global Database

複数リージョンにまたがるレプリケーション。典型的には主リージョン＋副リージョンで DR を構成する。物理ストレージレベルのレプリケーションで、通常 1 秒未満の RPO を実現する。

7.4 DynamoDB

フルマネージドの NoSQL KVS / ドキュメントストア。サーバーレスで、スキーマ設計さえ合えば極端にスケールする。

7.4.1 基本構造

- ・ テーブル、アイテム（行に相当）、属性（列に相当）
- ・ パーティションキー（必須）と ソートキー（任意）で主キーを構成
- ・ セカンダリインデックス：GSI（グローバル）と LSI（ローカル）

7.4.2 料金モード

モード	特徴
オンデマンド	リクエスト単位の課金。スパイク・不定期アクセスに
プロビジョンド	秒あたりの容量（RCU/WCU）を予約。Auto Scaling 可

新規プロジェクトは **オンデマンド** で始めて、負荷が読めてきたらプロビジョンドに移すのが無難。

7.4.3 Single Table Design

DynamoDB では RDB 的な「テーブルごとに正規化」は不向き。**1 テーブルに複数エンティティを詰め込み、アクセスパターンごとにキー設計する** という思想が推奨される。学習コストは高めだが、これを受け入れるとスループットとコストが大きく変わる。

7.4.4 DynamoDB Streams

テーブルへの変更を 24 時間キャプチャする。Lambda をトリガーに、監査ログ出力、キャッシュ更新、他サービス連携などが組める。

7.4.5 Global Tables

複数リージョンのテーブルを双方向でレプリケーションする。リージョン障害時にもアプリが動き続ける「アクティブ／アクティブ」構成。

7.5 ElastiCache

インメモリデータストアのマネージド。Redis / Valkey / Memcached をサポート。

7.5.1 選び方

- **Valkey / Redis**：機能豊富。永続化、Pub/Sub、トランザクション、各種データ構造
- **Memcached**：シンプル KVS。マルチスレッドで純粋なキャッシュ用途

新規利用では **Valkey**（Redis フォーク、より積極的にメンテされている）か **Redis** を選ぶ。

7.5.2 典型的なユースケース

- **セッションストア**：ALB + EC2 の後ろに置いて、セッションを共有
- **DB 前面キャッシュ**：RDS / Aurora の読み取りをキャッシュ
- **レートリミット**：API Gateway 前段での制限カウント
- **ランキング／リアルタイム集計**：Redis の Sorted Set を活用

7.5.3 クラスタモード

- **クラスタモード無効**：1つのプライマリ+レプリカ。小～中規模に十分
- **クラスタモード有効**：シャーディングで水平スケール。超大規模向け

7.6 DB 選定のガイドライン

7.6.1 「まず RDBMS」で始める

アクセスパターンが読めない段階では、**Aurora PostgreSQL** のような柔軟な RDBMS で始め、ボトルネックが見えてから DynamoDB / ElastiCache を足すのが安全である。初手 DynamoDB は、アクセスパターンを読み違えるとスキーマ設計のやり直しになりやすい。

7.6.2 マネージドかセルフホストか

EC2 に自前で MySQL を立てるのは、ほぼ非推奨。バックアップ、フェイルオーバー、バージョンアップ、パッチ適用 すべて自前で面倒を見ることになり、マネージドサービスの料金差をすぐ上回るコストになる。個人検証でも Aurora Serverless v2 などフルマネージドを使うのがよい。

7.6.3 商用ライセンスの扱い

Oracle / SQL Server は **ライセンスインクルード** か **BYOL（持ち込み）** を選べる。既存ライセンス資産があれば BYOL、なければインクルードで試算する。

7.6.4 ユースケース別の目安

迷ったときの目安。あくまで出発点で、要件次第で見直す。

アプリ例	主 DB	付加
個人ブログ / 社内ツール	Aurora Serverless v2 PostgreSQL (0 ACU 対応)	—
Web アプリ (MAU 数万)	Aurora PostgreSQL r7g.large + リーダ 1 台	ElastiCache (セッション)

アプリ例	主 DB	付加
ソーシャル系の高頻度 KV / 連番 ID 無し	DynamoDB オンデマンド	DynamoDB Streams → Lambda
EC サイト（注文 × 検索）	Aurora MySQL	OpenSearch（商品検索） + ElastiCache
リアルタイムランキング	DynamoDB	ElastiCache（Redis Sorted Set）
大量センサーデータ	Timestream	S3 + Athena（長期保管）
分析・BI	Redshift または Aurora + Athena（S3 レイク）	—
既存 Oracle 資産	RDS for Oracle（BYOL）	段階的に PostgreSQL に移行

7.7 DB 周りのネットワーク

RDS / Aurora / ElastiCache は通常 **プライベートサブネット** に置く。

- DB サブネットグループ（2 つ以上の AZ を指定）を作って、その中に配置
- SG で **アプリサーバの SG からのみ接続可** にする
- **パブリックアクセスを有効化しない**（個人検証でも基本禁止）

社外から接続したい場合は、踏み台 EC2 や **SSM セッションマネージャ** のポートフォワーディングを使う。

```
# ローカルから RDS に SSM 経由でトンネリング
aws ssm start-session \
  --target i-0bastion... \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host":["mydb.xyz.ap-northeast-1.rds.amazonaws.com"],"portNumber":
["5432"],"localPortNumber":["15432"]}'
```

7.8 バックアップとリカバリのベストプラクティス

- RPO / RTO を明確にする：どれくらいのデータ損失と復旧時間を許容するか
- ポイントインタイム復元 を有効化
- 定期的なリストア演習 をする：取っただけのバックアップは「使えるか」分らない
- クロスリージョンスナップショットコピー でリージョン災害に備える
- IAM で削除・暗号化解除を絞る：誰でも `DeleteDBInstance` できる状態にしない

7.9 コスト最適化のポイント

- アイドル DB を止める または削除する。開発用はスケジュールで夜間停止
- Aurora Serverless v2 で低負荷期間を 0 ACU に近づける

- IOPS を無闇に上げない。gp3 の場合、スループット・IOPS も課金対象
- リードレプリカの数を見直す：余計なレプリカは費用がかさむ
- DynamoDB のプロビジョンドキャパシティを Auto Scaling で最適化

8 コンテナとサーバーレス

EC2 で仮想マシンを立てる方法に加えて、AWS では **コンテナ** や **サーバーレス** でアプリを動かす選択肢がある。運用負荷を AWS に寄せるほど、開発者はアプリケーションロジックに集中できる。本章ではこの領域の主要サービスを俯瞰する。

8.1 コンピュート選択のスペクトラム

レイヤ	責任範囲（利用者側）	例
仮想マシン	OS・ミドルウェア・アプリ	EC2
コンテナ（IaaS）	コンテナ・タスク定義	ECS on EC2、EKS on EC2
コンテナ（サーバーレス）	コンテナだけ	ECS on Fargate、EKS on Fargate
サーバーレス関数	関数コードだけ	Lambda
フルマネージド PaaS	アプリコードだけ	App Runner、Elastic Beanstalk、Amplify

下へ行くほど運用負荷が下がり、抽象度が上がる。

8.1.1 初学者はどこから始めればよいか

選択肢が多いので迷いやすい。目安として以下の順序で検討するとよい。

1. **軽い API・イベント処理**：Lambda + API Gateway から始める。インフラがゼロで、Free Tier の範囲で永続的に試せる
2. **既存の Web アプリ（Docker イメージがある）**：App Runner。Git 連携または ECR から 1 コマンドで公開できる
3. **コンテナだが本格制御したい**：ECS on Fargate。タスク定義・ALB・Auto Scaling を明示的に組む
4. **Kubernetes の資産がある、他クラウドと共通化したい**：EKS。学習コスト高
5. **既存の EC2 運用ノウハウを活かす**：ECS on EC2 または EC2 + Auto Scaling

「まずは Lambda か App Runner」で小さく始めて、負荷・要件が見えてから上位へ移るのが事故が少ない。

8.2 Lambda（サーバーレス関数）

AWS Lambda は、関数単位でコードを実行するサーバーレス基盤である。インフラを意識せず、イベント駆動で動くコードを書ける。

8.2.1 実行モデル

- ・コードをランタイム（Python、Node.js、Java、Go、Ruby、.NET、Rust）で動かす
- ・コンテナイメージ（OCI）としてデプロイも可能（最大 10 GB）

- ・ イベント をトリガーに起動：API Gateway、S3、SQS、EventBridge、DynamoDB Streams など
- ・ 同時実行数（Concurrency）でスケール。デフォルト上限はアカウント・リージョンで 1000

8.2.2 料金モデル

- ・ リクエスト数 × 実行時間 × メモリサイズ の従量課金
- ・ メモリを上げると CPU も連動して増える（時間が短縮されれば相殺される場合も）
- ・ 月 100 万リクエスト、40 万 GB 秒 までは **Free Tier**（常時無料）

8.2.3 コールドスタート

しばらく呼ばれていない Lambda は、次の呼び出し時に **コンテナ起動＋ランタイム初期化** が走る。これが **コールドスタート**。

- ・ 数十 ms ～ 数秒（言語・パッケージサイズに依存）
- ・ 解消策：**Provisioned Concurrency**（事前に温めておく）、**SnapStart**（Java で起動時間短縮）
- ・ VPC 内 Lambda は ENI 初期化でさらに長くなる

8.2.4 シンプルな Lambda の例（Python）

```
import json

def lambda_handler(event, context):
    name = event.get("queryStringParameters", {}).get("name", "world")
    return {
        "statusCode": 200,
        "body": json.dumps({"message": f"hello, {name}!"})
    }
```

デプロイは以下の通り。

```
zip -r func.zip lambda_function.py
aws lambda create-function \
  --function-name hello \
  --runtime python3.12 \
  --role arn:aws:iam::123456789012:role/lambda-basic \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://func.zip
```

8.2.5 Lambda のユースケース

- ・ API バックエンド：API Gateway + Lambda
- ・ ファイル処理：S3 アップロード → Lambda でサムネイル生成・ウイルススキャン
- ・ ストリーム処理：DynamoDB Streams、Kinesis
- ・ スケジュールジョブ：EventBridge Scheduler で cron 相当
- ・ ChatOps / Slack Bot：WebHook 受信処理
- ・ IoT バックエンド：IoT Core からの軽い処理

8.2.6 制限

- ・ 実行時間 15 分上限：長時間処理には向かない（Step Functions や Fargate と組み合わせる）
- ・ メモリ上限 10 GB、一時ディスク 10 GB
- ・ 同時実行数の制限：急激なスケールでスロットリングに注意

8.3 API Gateway

HTTP / REST / WebSocket の API をマネージドで提供する。Lambda、HTTP バックエンド、VPC Link 経由で NLB / ALB にも接続できる。

8.3.1 種類

- ・ HTTP API：軽量・低レイテンシ・安価。新規は基本これ
- ・ REST API：機能豊富（使用量プラン、API キー、WAF 連携、リクエスト検証）
- ・ WebSocket API：双方向通信

8.3.2 Lambda Proxy 連携

API Gateway HTTP API の典型パターン。リクエストを Lambda にそのまま渡し、Lambda のレスポンスを HTTP レスポンスに変換する。

8.3.3 認証／認可

- ・ Cognito オートライザー：ユーザー認証（JWT）
- ・ Lambda オートライザー：独自ロジックで認可
- ・ IAM 認可：SigV4 署名付きリクエスト

8.4 ECS / EKS / Fargate

8.4.1 ECS（Elastic Container Service）

AWS 独自のコンテナオーケストレーション。**タスク定義** という概念で Docker コンテナの起動仕様を記述する。

- ・ クラスター > サービス > タスク という階層
- ・ タスクは EC2 上 または Fargate 上で動く
- ・ ELB との統合、Auto Scaling、Blue/Green デプロイ対応

EKS より **シンプル・学習コストが低い**。AWS だけで完結する運用なら ECS の方が扱いやすいことが多い。

8.4.2 EKS（Elastic Kubernetes Service）

Kubernetes（k8s）のコントロールプレーンをマネージドで提供する。

- ・ Kubernetes のエコシステムをそのまま使える
- ・ オンプレや他クラウドとの移植性が高い
- ・ 運用ノウハウが必要。IAM・ENI・CoreDNS・Ingress で AWS 固有の癖あり

- **Karpenter** を使うとノードスケールリングが賢くなる

「Kubernetes の知識があり移植性重視」なら EKS、「シンプルに回したい」なら ECS。

8.4.3 Fargate

コンテナを **サーバーレス** で動かす仕組み。EC2 ノードを自前で管理せず、必要な vCPU・メモリだけ指定してタスクを起動する。

- EC2 ノードの OS パッチやスケールリングから解放される
- **タスクの vCPU × 時間** で課金
- ネットワーク的には VPC の ENI が各タスクに付く

小～中規模で運用負荷を下げたい場合は **ECS on Fargate**、k8s 要件があるなら **EKS on Fargate** が第一候補。

8.4.4 ECR (Elastic Container Registry)

OCI 互換のコンテナレジストリ。Docker Hub の AWS 内版。

- **パブリックレジストリ**（誰でも pull 可）と **プライベートレジストリ**
- IAM でアクセス制御
- 脆弱性スキャン、イメージ署名
- ECS / EKS / Lambda（コンテナイメージ方式）から引かれる

```
# ECR にログイン
aws ecr get-login-password --region ap-northeast-1 \
| docker login --username AWS --password-stdin \
  123456789012.dkr.ecr.ap-northeast-1.amazonaws.com

# タグ付けして push
docker tag myapp:latest \
  123456789012.dkr.ecr.ap-northeast-1.amazonaws.com/myapp:latest
docker push \
  123456789012.dkr.ecr.ap-northeast-1.amazonaws.com/myapp:latest
```

8.4.5 ECS タスク定義の例

```
{
  "family": "web",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "512",
  "memory": "1024",
  "executionRoleArn": "arn:aws:iam::123456789012:role/ecsTaskExecutionRole",
  "containerDefinitions": [{
    "name": "nginx",
    "image": "123456789012.dkr.ecr.ap-northeast-1.amazonaws.com/myapp:latest",
    "portMappings": [{ "containerPort": 80 }],
    "logConfiguration": {
```

```

    "logDriver": "awslogs",
    "options": {
      "awslogs-group": "/ecs/web",
      "awslogs-region": "ap-northeast-1",
      "awslogs-stream-prefix": "web"
    }
  }
}
}
}

```

8.5 App Runner / Elastic Beanstalk / Amplify

より高レベルの PaaS 的サービス群。

8.5.1 App Runner

コンテナまたは GitHub リポジトリから URL 付きの Web アプリを直接起動できる。VPC や ALB の設定も自動でやってくれる。

- シンプルな Web アプリ（API、SSR フロントエンド等）に最適
- 料金は vCPU × 時間 + リクエスト数
- Fargate より抽象度が高い

8.5.2 Elastic Beanstalk

古くからある PaaS。アプリをアップロードすれば EC2 / ALB / RDS を含めて環境を自動構築。

- 機能的には枯れているが、新規採用は減少傾向
- 裏のリソースにアクセスできるのが特徴（Lock-in が少ない）

8.5.3 Amplify

フロントエンド・モバイル向けのフルスタックホスティング。

- Git 連携で自動ビルド・デプロイ
- 認証（Cognito）、API（GraphQL / REST）、ストレージ（S3）、ホスティング（CloudFront）を統合
- React / Vue / Next.js などの SSR / SPA をそのまま乗せられる

Next.js や Nuxt.js の個人プロジェクトをデプロイするなら Amplify または App Runner が楽。

8.6 イベント駆動・非同期アーキテクチャ

サーバーレス／コンテナと組み合わせて使われる重要サービス。

サービス	役割
SQS	メッセージキュー。Standard / FIFO の 2 種

サービス	役割
SNS	Pub/Sub。トピックに発行すると複数宛に配信（メール・SMS・HTTP・SQS・Lambda）
EventBridge	イベントバス。AWS サービス + SaaS を統合
Step Functions	ステートマシン。Lambda / サービスを組み合わせたワークフロー
Kinesis	ストリーミング。Data Streams / Firehose / Analytics
MSK	マネージド Kafka

8.6.1 SQS + Lambda の基本パターン

1. Producer（Web / Lambda）が SQS にメッセージを送る
2. Lambda が SQS をポーリングして 1 件ずつ処理
3. 失敗時は DLQ（Dead Letter Queue）に退避

このパターンは、ピークをキューで吸収し、バックエンドを非同期化する定番。バーストトラフィックを受ける Web アプリで多用する。

8.6.2 Step Functions

Lambda や AWS サービス呼び出しを **ステートマシン** で繋ぐ。長時間処理、分岐、リトライ、並列実行を JSON で宣言できる。

```
[Start] → [画像解析Lambda] → (成功) → [サムネイル生成] → (並列) → [通知]
                        ↳ (失敗) → [エラーログ] → [End]
```

Lambda 単体の 15 分上限を超える処理や、複雑なフローの可視化に有用。

8.7 サーバーレス／コンテナの選定指針

- ・ 単純な HTTP エンドポイント：API Gateway + Lambda、または App Runner
- ・ バッチ処理・非同期ジョブ：SQS + Lambda、または ECS Fargate タスク
- ・ 長時間処理：Step Functions または ECS Fargate タスク
- ・ 既存 Docker アプリを動かす：ECS on Fargate
- ・ Kubernetes 資産を持っている：EKS
- ・ フロントエンド中心：Amplify / CloudFront + S3
- ・ 自前でスケーリング込みで運用できる：EC2 + Auto Scaling

「まずは Lambda → 限界が見えたら ECS / Fargate → さらに複雑なら EKS」という段階的なステップアップが一つの筋道である。

8.8 参考アーキテクチャ：サーバーレス Web アプリ

```
[ユーザー]
  ↓ HTTPS
[CloudFront] — [S3 (静的フロント)]
  ↓ /api/*
[API Gateway]
  ↓
[Lambda]
  ↓
[DynamoDB]    [Cognito (認証)]
```

このスタックは **インフラを1台も立てずに** Web アプリを動かせる。コストもアクセスが少ないうちは Free Tier の範囲内で収まることが多く、学習・個人プロジェクトの定番になっている。

9 運用と監視

AWS で動くシステムを安定運用するには、**メトリクス・ログ・監査・設定・構成管理** の各観点で仕組みを整える必要がある。本章では CloudWatch、CloudTrail、Config、Systems Manager を中心に解説する。

9.1 CloudWatch

CloudWatch は AWS の監視基盤である。メトリクス、ログ、アラーム、イベントをまとめて扱う。

9.1.1 CloudWatch Metrics

- AWS サービスから **標準メトリクス** が自動的に流れてくる（EC2 の CPU、ELB のリクエスト数など）
- **カスタムメトリクス** をアプリから `PutMetricData` で送れる
- EC2 標準メトリクスは **5 分間隔**、**詳細モニタリング** を有効化すると 1 分間隔。カスタム高解像度メトリクスは最短 1 秒粒度
- **15 か月間保持**（古いほど粗い粒度に丸まる）

9.1.2 アラーム

メトリクスに対してしきい値を設定し、違反時にアクションを発動する。

- アクション：SNS 通知（メール、Slack 連携）、Auto Scaling、EC2 停止など
- **複合アラーム**：複数アラームの AND/OR 合成
- **異常検出**：機械学習による動的しきい値

```
# CPU 80% 超で通知するアラームの作成例
aws cloudwatch put-metric-alarm \
  --alarm-name web-cpu-high \
  --metric-name CPUUtilization \
  --namespace AWS/EC2 \
  --statistic Average \
  --period 300 \
  --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 2 \
  --dimensions Name=InstanceId,Value=i-0abc123 \
  --alarm-actions arn:aws:sns:ap-northeast-1:123456789012:ops-alerts
```

9.1.3 CloudWatch Logs

サービスやアプリのログを集約する。

- **ロググループ** と **ログストリーム** の 2 階層
- **保持期間** を明示的に設定しないと無期限（料金が際限なく伸びる）

- ・ 収集源：Lambda（自動）、ECS / EKS のタスク、EC2 の CloudWatch エージェント、VPC Flow Logs 等

9.1.4 Logs Insights

CloudWatch Logs への独自クエリ言語。

```
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 100
```

SQL 的な書き味で、**急な障害解析に強い**。Athena + S3 に流し込む構成にしておくと、長期保管と分析を両立できる。

9.1.5 ダッシュボード

メトリクスとログの可視化をカスタマイズできる。1 アカウント 3 ダッシュボードまで無料。

9.1.6 CloudWatch Alarms の通知をメール以外に

- ・ SNS → **Chatbot** 経由で Slack / Teams
- ・ SNS → Lambda でカスタムフォーマット
- ・ EventBridge → 任意の AWS アクションに分岐

9.2 CloudTrail

CloudTrail は「**誰が・いつ・何を**」の**監査ログ**を記録するサービスである。**CloudTrail コンソール**の「**Event history**」では直近 90 日分の管理イベントが自動で閲覧可能。ただしこれは**証跡 (Trail)**そのものではなく、ビューに過ぎず、長期保存・S3 保管・検索・通知には次節の証跡を別途作成する必要がある。

9.2.1 イベント種別

- ・ 管理イベント：API 操作（`CreateBucket`、`TerminateInstances` など）
- ・ データイベント：S3 の `GetObject` / `PutObject`、Lambda の `Invoke` など高頻度イベント
- ・ インサイトイベント：異常な API 呼び出しパターンの検知

9.2.2 証跡 (Trail)

長期保存や複数リージョン集約には**証跡**を作成する。

- ・ S3 に保存（推奨はログアーカイブ専用アカウント）
- ・ **ログファイル検証**を有効にして改ざん検知
- ・ CloudWatch Logs へ流して Logs Insights で検索
- ・ EventBridge と組み合わせて重要操作を即時通知

組織全体の監査ログは **Organizations 証跡**で一括管理できる。

9.2.3 主要ログフィールド

- `eventName`、`eventTime`、`sourceIPAddress`
- `userIdentity` (IAM ユーザーかロールかルートか、引き受けた経路)
- `userAgent` (AWS CLI、Console、SDK)
- `requestParameters` / `responseElements`

「ルートユーザーからの API が実行されたら即通知」のような監査アラートは、CloudTrail + EventBridge でよく組まれる。

9.3 AWS Config

AWS Config は、リソースの構成変更を時系列で記録し、ルール違反を検知する サービスである。

- 全リソースの **設定履歴** を S3 / DynamoDB に保存
- **マネージドルール** (例:「S3 バケットが公開されていないこと」「EBS が暗号化されていること」)
- **カスタムルール** (Lambda / Guard DSL で書く)
- **コンフォーマンスパック**: ルールの束 (CIS ベンチマーク、PCI DSS など)

9.3.1 典型的な利用シーン

- セキュリティ基準違反の検知 (公開 S3、未暗号化 EBS、MFA なしユーザー)
- 構成変更の時系列追跡 (「このセキュリティグループはいつ誰が変えた?」)
- コンプライアンスレポートの自動生成

コストは **記録対象リソース数 × 変更回数** で増えるため、マルチアカウント環境では **Config アグリゲータ** で全体可視化と、必要なルールだけ有効化するバランスが重要。

9.4 Systems Manager (SSM)

AWS Systems Manager は、EC2 やオンプレサーバの **構成管理・運用自動化** を統合するサービス群である。非常に広いので主要コンポーネントを押さえる。

9.4.1 主要機能

コンポーネント	用途
Fleet Manager	インスタンスの一覧管理・インベントリ取得
Session Manager	ブラウザ/CLI からインスタンスへ安全に接続
Run Command	複数台にコマンドを一斉実行
Patch Manager	OS パッチの自動適用
Parameter Store	設定値・秘密情報の保存 (階層型)
State Manager	定期的に構成をあるべき姿に収束
Automation	多段の運用タスクを Runbook 化
OpsCenter	運用インシデントの一元管理
Change Manager	変更申請・承認ワークフロー

9.4.2 Session Manager のメリット

前章の EC2 でも触れたが、SSH 22 番を閉じたまま 接続できる現代的な運用の起点になる。

- SSH 鍵管理が不要
- 22 番ポートをセキュリティグループで閉じられる
- すべての操作が CloudTrail に残る
- ポートフォワーディングで RDS やプライベート LB への接続も可能

9.4.3 Parameter Store

アプリケーションの設定値や認証情報を階層パスで保存する。

```
# パラメータの登録（暗号化）
aws ssm put-parameter \
  --name "/app/prod/db_password" \
  --value "supersecret" \
  --type SecureString

# 取得
aws ssm get-parameter \
  --name "/app/prod/db_password" \
  --with-decryption \
  --query Parameter.Value --output text
```

シンプルな設定値なら Parameter Store、より高機能なローテーションが必要なら **Secrets Manager**（次章で扱う）を使い分ける。

9.4.4 Patch Manager

EC2 の OS パッチを定期適用する仕組み。

- パッチベースライン（どのパッチを当てるか）
- メンテナンスウィンドウ（いつ当てるか）
- パッチグループ（どのインスタンスに当てるか）

数台なら手動で十分だが、100 台規模になると必須。

9.5 監視・運用の全体像

AWS 上の典型的な構成では、以下が組み合わさる。

```
[アプリ] → CloudWatch Logs（アプリログ）
↓
メトリクス → CloudWatch Alarms → SNS → Slack/メール
↓
CloudTrail（API監査）→ S3 / CloudWatch Logs
↓
Config（構成変更）→ S3 / ルール違反検知
```

これらをまとめて俯瞰する上位のサービスも存在する。

- **CloudWatch Application Insights**：自動でアプリの健全性を検出
- **X-Ray**：分散トレーシング
- **Application Signals**：CloudWatch に統合された APM
- **Amazon Managed Grafana / Prometheus**：OSS エコシステムをマネージドで
- **Security Hub**：セキュリティ観点の検知を集約（次章）
- **CloudWatch Synthetics**：外形監視（canary によるエンドツーエンド確認）

9.6 タグ戦略

運用を楽にする最大の投資が **タグ戦略** である。

- **Name**：人間が識別するための名前
- **Environment**：prod / staging / dev
- **Owner**：運用責任者／チーム
- **CostCenter / Project**：費用按分用
- **ManagedBy**：terraform / cdk / manual

タグはリソース作成時に付けるのが鉄則。後から入れようとすると数百リソースを見直すことになる。**Organizations** の**タグポリシー** で必須タグを強制できる。

9.7 アラート疲れを防ぐ

監視を入れて数週間経つと、**アラートが日常の雑音になって誰も見なくなる** 現象が起きる。

- **本当に起こすべきもの**だけをアラート化する（夜間に叩き起こされる価値があるか？）
- **シビアリティを分ける**：警告は Slack、緊急は PagerDuty
- **抑制・まとめ**：同じアラートが 1 時間で 100 件来たら 1 件に集約
- **アクションを明文化**する：ランブックを Runbook として Automation に登録

9.8 運用のベストプラクティス

- 最初から CloudTrail / Config / GuardDuty（次章）を有効化
- ログ保持期間を必ず設定する（特に CloudWatch Logs）
- Systems Manager Session Manager を第一の接続手段にする
- 主要リソースに監視ダッシュボードを 1 枚作る（最初の障害対応が段違いに楽になる）
- タグ付けルールを決めて自動化する
- 月次で Trusted Advisor / Cost Explorer を見て棚卸しする

10 セキュリティとコスト管理

AWS は「使ってみる」ハードルは低いですが、**放置すると事故になる** 代表例がセキュリティとコストである。本章では、運用を続けるうえで欠かせない両領域のサービスと、運用プラクティスをまとめる。

10.1 セキュリティの全体像

AWS のセキュリティは層状に構成される。

【アイデンティティ層】	IAM / Identity Center / MFA / Access Analyzer
【データ層】	KMS / Secrets Manager / S3 暗号化 / EBS 暗号化
【ネットワーク層】	SG / NACL / WAF / Shield / PrivateLink
【検知・監査層】	CloudTrail / Config / GuardDuty / Security Hub
【対応層】	Systems Manager Incident Manager / EventBridge

個人検証ではここまで全部整える必要はないが、**本番運用では全レイヤに当たり前の対策を入れる**のが前提である。

10.2 KMS (Key Management Service)

KMS は、暗号鍵を管理するマネージドサービス。S3、EBS、RDS、Secrets Manager など多数のサービスが KMS を使って暗号化する。

10.2.1 キーの種類

- ・ **AWS マネージドキー**：サービスごとに自動生成される（例：`aws/s3`）。追加料金なし
- ・ **カスタマーマネージドキー (CMK)**：利用者が作成・管理するキー。ローテーション、キーポリシー、IAM ポリシーを細かく制御可能
- ・ **AWS 所有キー**：AWS 内部で完全に隠蔽されているキー

検証やシンプルな用途では AWS マネージドキーで十分。**監査要件や独立した権限管理が必要な場合は CMK を使う。**

10.2.2 キーポリシーと IAM

CMK は **キーポリシー**（KMS 鍵自身に付与するリソースベースポリシー）と IAM ポリシーの両方で制御される。鍵作成直後は、キーポリシーに明示的な許可がないと誰も使えなくなる点に注意。

10.2.3 エンベロープ暗号化

KMS は **データキー** を生成し、それで実データを暗号化する方式（エンベロープ暗号化）を推奨する。KMS 本体に大きなデータを投げず、データキーだけを守る仕組み。

10.3 Secrets Manager と Parameter Store

10.3.1 使い分け

項目	Parameter Store	Secrets Manager
基本料金	標準は無料	1 秘密あたり月 \$0.40
自動ローテーション	なし	あり (Lambda 連携)
クロスアカウント共有	限定	ネイティブ対応
レプリケーション	なし	複数リージョンへ
用途	設定値・軽い秘密	DB パスワード等、ローテーションが必要な秘密

個人検証では Parameter Store (SecureString) で十分。本番の DB パスワードなど、**ローテーション運用が必要な秘密情報** は Secrets Manager を使う。

10.3.2 DB パスワードのローテーション

Secrets Manager は RDS / Aurora / Redshift / DocumentDB と統合されていて、**ワンクリックで定期的なパスワードローテーション**を設定できる。裏では Lambda が古いパスワードで新しいパスワードを設定し直す。

10.4 WAF と Shield

10.4.1 AWS WAF

Web アプリケーションファイアウォール。CloudFront、ALB、API Gateway、App Runner の前段に貼れる。

- ・ **マネージドルール**：AWS・Marketplace ベンダが提供する既製ルールセット
- ・ **レート制限**：特定 IP からの過剰リクエストをブロック
- ・ **Bot Control**：自動化された不正アクセスの検出
- ・ **Geo ブロック**：国単位でアクセス制御

10.4.2 Shield / Shield Advanced

- ・ **Shield Standard**：AWS ユーザー全員が無料で利用。L3/L4 の基本 DDoS 対策
- ・ **Shield Advanced**：有償。L7 対策の強化、DDoS Response Team (DRT)、急増課金のクレジット保護

個人利用では Standard で十分。大規模・公開サービスで攻撃リスクが高い場合のみ Advanced を検討。

10.5 GuardDuty / Inspector / Macie / Security Hub

10.5.1 GuardDuty

脅威検知 サービス。CloudTrail、VPC Flow Logs、DNS ログを ML で分析し、不審な動きを検知する。

- Crypto マイニング痕跡、通信先の悪性 IP、IAM キー漏洩の兆候など
- アカウント単位でワンクリック有効化
- 個人検証なら最初の 30 日間は無料。その後も **月数ドル程度**

アカウント作成直後に有効化することを強く推奨。

10.5.2 Inspector

脆弱性スキャナ。EC2、ECR、Lambda を対象にパッケージ脆弱性（CVE）を継続検出する。

- EC2 は SSM エージェント経由
- ECR は push 時と定期
- 重大な脆弱性を EventBridge 経由で通知

10.5.3 Macie

S3 の **機密データ発見** を行う。個人情報（PII）やクレジットカード番号などのパターンを機械学習で検出。

10.5.4 Security Hub

上記のセキュリティ検知を **集約・統合** するハブ。CIS や PCI DSS などのセキュリティ基準に対するスコアも出す。

マルチアカウント環境では、**監査専用アカウントに Security Hub を集約** して組織全体を見るのが一般的。

10.6 IAM Access Analyzer

IAM のリソース共有・外部公開を検出するサービス。

- 外部公開されている S3 バケット、Lambda、ロールを洗い出す
- 想定外のクロスアカウントアクセスを警告
- CloudFormation にポリシー検証機能を組み込める

有効化コストは無料に近く、**アカウント作成直後に ON** にしておくとも誤公開事故に早く気づける。

10.7 AWS Organizations と SCP

複数アカウントを運用する場合は **AWS Organizations** が中心になる。

10.7.1 管理構造

```

[管理アカウント (payer)]
├── OU: Security
│   ├── 監査ログ用アカウント
│   └── セキュリティツール用アカウント
├── OU: Workloads
│   ├── OU: Prod
│   │   └── prod-web アカウント
│   └── OU: Dev
│       └── dev-sandbox アカウント

```

- 各アカウントは **リソース・権限の隔壁** として機能
- 一括請求 (Consolidated Billing)：親アカウントに支払いを集約
- **Organizations 証跡** で CloudTrail を一括管理

10.7.2 SCP (サービスコントロールポリシー)

OU / アカウント単位で、そもそも使えるサービスや API を制限 する。IAM ポリシーより上位の「絶対の壁」として機能する。

```

{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": "*",
    "Resource": "*",
    "Condition": {
      "StringNotEquals": {
        "aws:RequestedRegion": ["ap-northeast-1", "ap-northeast-3"]
      }
    }
  }]
}

```

このような SCP を付けておけば、**東京・大阪以外のリージョンで誤ってリソースが作られる** 事故を防げる。放置されたリソースがアラートに引っかからず料金が膨らむ、という典型事故の対策になる。

10.7.3 Control Tower

新規に組織を作るなら **AWS Control Tower** を使うと、Organizations / IAM Identity Center / ログアーカイブ構成 / ベースラインの SCP が一括で展開される。

10.8 コスト管理サービス

10.8.1 AWS Budgets

2 章でも触れた **予算アラート**。個人利用で最初に設定するサービス。

- **コスト予算**：月次・四半期・年次の金額上限
- **使用量予算**：EC2 時間、データ転送量など
- **Reservation 予算**：RI や Savings Plans の使用率管理
- アラートはメール・Slack（SNS 経由）・Chatbot

必ず「予測」ベースのアラートも入れる。実測が閾値に達してから通知では手遅れになる。

10.8.2 Cost Explorer

過去 13 か月～（有償で長期）のコスト・使用量を可視化・分析する。

- サービス別・タグ別・リージョン別で集計
- **Savings Plans 推奨** を自動算出
- カスタムレポートの保存、CSV エクスポート

月初に前月のコストを振り返る習慣をつけるだけで浪費を大きく減らせる。

10.8.3 Cost and Usage Report (CUR)

CUR 2.0（または CUR）は、もっと詳細な使用量・料金データを S3 に出すレポート。Athena / QuickSight と組み合わせて経営層向けのダッシュボードを作る用途が多い。

10.8.4 Savings Plans と Reserved Instance

- **Compute Savings Plans**：EC2 / Fargate / Lambda 横断。最大 66%
- **EC2 Instance Savings Plans**：特定ファミリーに縛る代わりに最大 72%
- **Reserved Instance**：旧来の予約。新規は Savings Plans が基本

安定的に稼働している計算リソースがあるなら、1 年コミットだけでもかなり下がる。

10.8.5 Spot Instance

Auto Scaling グループでオンデマンドと Spot を混ぜる、ECS で Fargate Spot を使う、バッチを AWS Batch で Spot に寄せる、といった使い方で大幅にコストが落ちる。中断耐性のあるワークロードから適用する。

10.8.6 コスト最適化のチェックリスト

- ☐ Budgets を設定した（実測＋予測）
- ☐ Cost Explorer を月 1 回見ている
- ☐ 停止した EC2 に紐づく EBS を残していないか
- ☐ 未使用の Elastic IP を解放したか
- ☐ NAT Gateway を検証用に立てっぱなしにしていないか
- ☐ CloudWatch Logs の保持期間を設定したか

- [] S3 バケットに Lifecycle ルールを設定したか
- [] Savings Plans の推奨を確認したか
- [] 全リージョンの「幽霊リソース」を点検したか

10.9 セキュリティ+コストで最低限やること

個人・小規模でも、最初の 30 分で 以下は終わらせる。

1. ルートユーザーに MFA、アクセスキーなし
 2. 作業用 IAM ユーザー or Identity Center を用意
 3. Budgets で予算アラート
 4. Free Tier 使用量アラート
 5. CloudTrail を有効化、S3 に長期保管
 6. GuardDuty を有効化
 7. IAM Access Analyzer を有効化
 8. S3 のデフォルト暗号化をオンに
 9. EBS のデフォルト暗号化をオンに
 10. 全リージョン向けの SCP（使うのは東京・大阪だけなら他はブロック）
- ここまでやっておけば、大事故はほぼ防げる。あとは 月 1 回の棚卸し を習慣にする。

11 IaC と運用 Tips

AWS のリソースをコンソールで作ると、再現性が低く・構成変更の履歴が追えない という問題が起きる。これを解決するのが Infrastructure as Code (IaC) である。本章では主要 IaC ツールを比較し、最後に運用で知っておきたい Tips と事故回避策をまとめる。

11.1 なぜ IaC が必要か

- ・ 再現性：同じ構成を別リージョン・別アカウントに何度でも作れる
- ・ 履歴管理：Git で変更を追える。レビューできる
- ・ 自動化：CI/CD でデプロイできる
- ・ ドキュメント：コード自体が構成の正となる
- ・ 破壊と再構築：環境を丸ごと作り直せる

個人検証でも IaC を使う最大のメリットは「**まとめて削除**」が簡単な点。コンソールで作ると消し忘れるリソースも、IaC なら `destroy` 一発で片付く。

11.2 CloudFormation

AWS 純正の IaC。YAML / JSON でリソースを宣言する。

11.2.1 基本概念

- ・ テンプレート：リソース定義の YAML / JSON
- ・ スタック：テンプレートを適用して作られたリソース群
- ・ スタックセット：複数アカウント・リージョンに展開するスタック
- ・ 変更セット：適用前の差分プレビュー

11.2.2 最小テンプレート例

```
AWSTemplateFormatVersion: '2010-09-09'
Description: A single S3 bucket

Parameters:
  BucketName:
    Type: String

Resources:
  MyBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: !Ref BucketName
      VersioningConfiguration:
        Status: Enabled
      PublicAccessBlockConfiguration:
        BlockPublicAcls: true
```

```

    BlockPublicPolicy: true
    IgnorePublicAcls: true
    RestrictPublicBuckets: true

Outputs:
  BucketArn:
    Value: !GetAtt MyBucket.Arn

```

11.2.3 適用

```

aws cloudformation create-stack \
  --stack-name my-bucket \
  --template-body file://bucket.yaml \
  --parameters ParameterKey=BucketName,ParameterValue=my-bucket-20260421

# 更新
aws cloudformation update-stack \
  --stack-name my-bucket \
  --template-body file://bucket.yaml \
  --parameters ParameterKey=BucketName,ParameterValue=my-bucket-20260421

# 削除
aws cloudformation delete-stack --stack-name my-bucket

```

11.2.4 特徴

- AWS 純正なので新サービス対応が早い
- IAM 権限がそのまま効く
- ドリフト検知（実リソースとテンプレートの差分）
- ネイティブ機能としてロールバック
- 反面、YAML の記述量が多くなりがち

11.2.5 SAM (Serverless Application Model)

Lambda・API Gateway・DynamoDB を **短い記述** で書ける CloudFormation の拡張。サーバーレス構成に特化。

```

Transform: AWS::Serverless-2016-10-31
Resources:
  HelloApi:
    Type: AWS::Serverless::Function
    Properties:
      Runtime: python3.12
      Handler: app.handler
      CodeUri: ./src
      Events:
        Api:

```

```
Type: HttpApi
Properties:
  Path: /hello
  Method: get
```

11.3 AWS CDK (Cloud Development Kit)

CDK は、TypeScript / Python / Java / Go / C# などの **プログラミング言語** でクラウド構成を書くツール。裏では CloudFormation テンプレートを生成する。

11.3.1 特徴

- 条件分岐・ループ・ヘルパー関数などを自然に書ける
- **Construct** による抽象化（「Web サービス式」を 1 行で書く、のような）
- 型補完が効く
- エコシステム（L2 / L3 Construct、Community Constructs）が豊富

11.3.2 TypeScript での例

```
import { Stack, StackProps, RemovalPolicy } from 'aws-cdk-lib';
import { Bucket, BucketEncryption, BlockPublicAccess } from 'aws-cdk-lib/aws-s3';
import { Construct } from 'constructs';

export class MyStack extends Stack {
  constructor(scope: Construct, id: string, props?: StackProps) {
    super(scope, id, props);

    new Bucket(this, 'MyBucket', {
      versioned: true,
      encryption: BucketEncryption.S3_MANAGED,
      blockPublicAccess: BlockPublicAccess.BLOCK_ALL,
      removalPolicy: RemovalPolicy.DESTROY,
    });
  }
}
```

11.3.3 デプロイフロー

```
cdk init app --language typescript
cdk bootstrap           # 初回のみ、CDK 用の土台を作る
cdk synth               # CloudFormation YAML を生成
cdk diff                # 差分プレビュー
cdk deploy
cdk destroy
```


11.3.4 CloudFormation との使い分け

- プログラマビリティが欲しい、テストを書きたい → CDK
- YAML で宣言的に書きたい、純 AWS で揃えたい → CloudFormation / SAM

新規プロジェクトなら CDK を第一候補に挙げる開発者が増えている。

11.4 Terraform

HashiCorp が提供する **マルチクラウド対応** の IaC ツール。AWS に限定されないため、オンプレや他クラウドと混在する環境で強い。

11.4.1 特徴

- HCL (HashiCorp Configuration Language) で記述
- **プロバイダ** を差し替えれば AWS / GCP / Azure / Kubernetes / SaaS もまとめて扱える
- **state ファイル** が現実のリソースとコードの対応表になる (S3 + DynamoDB でリモート保存)
- OSS 版と HCP Terraform (旧 Terraform Cloud) あり
- ライセンス変更後のフォークとして **OpenTofu** も選択肢

11.4.2 例

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  backend "s3" {
    bucket        = "tfstate-20260421"
    key           = "my-app/terraform.tfstate"
    region        = "ap-northeast-1"
    dynamodb_table = "tfstate-lock"
    encrypt       = true
  }
}

provider "aws" {
  region = "ap-northeast-1"
}

resource "aws_s3_bucket" "app" {
  bucket = "my-bucket-20260421"
}

resource "aws_s3_bucket_versioning" "app" {
  bucket = aws_s3_bucket.app.id
  versioning_configuration {
```

```

    status = "Enabled"
  }
}

```

11.4.3 コマンド

```

terraform init    # プロバイダと backend を初期化
terraform plan    # 差分確認
terraform apply   # 適用
terraform destroy # 削除
terraform fmt     # 整形
terraform validate

```

11.4.4 state 管理の注意

- state ファイルには機密情報が含まれる（パスワードや ARN など）ので、S3 リモート backend を使う
- state の手編集は避ける（`terraform import` / `state rm` を使う）
- 複数人・CI で運用するなら 排他ロック は必須。Terraform 1.10 以降は `backend "s3"` に `use_lockfile = true` を指定することで S3 ネイティブのロック が GA（DynamoDB テーブル不要）。それ以前の構成では従来通り `dynamodb_table` でロックを取る

11.5 IaC ツール選びのガイドライン

状況	おすすめ
AWS 純度重視・小規模	CloudFormation / SAM
サーバーレスだけを素早く	SAM または CDK
プログラマビリティ重視・型安全に書きたい	CDK
マルチクラウド・オンプレ混在	Terraform / OpenTofu
既存チームが Terraform に慣れている	Terraform
K8s マニフェストも同一ツールで	Terraform + Helm プロバイダ or Pulumi

どれか1つに決める必要はなく、チームと用途に合わせて併用するのが現実解。CDK で AWS 基盤を作り、Kubernetes 上のアプリは Helm で運用、といった構成はよくある。

11.6 運用 Tips とハマりどころ

11.6.1 リージョン設定の一貫性

コンソールの右上リージョンは **ブラウザのタブごとに独立** する。スタックを削除したつもりで別リージョンを見ていた、という事故がよくある。IaC を使えばリージョンは明示的にコードで指定されるため、この種の事故が防げる。

11.6.2 名前衝突の回避

- S3 バケット名、CloudFront、Route 53 レコードはグローバル一意
- IAM ロール名、Lambda 関数名、セキュリティグループ名はアカウント＋リージョン内一意
- 環境サフィックス（`-prod`、`-dev`）を入れる／乱数を付与する

11.6.3 削除できないリソース

- S3 バケットが空でない → 全バージョン削除が必要
- RDS に **削除保護**（Deletion Protection）がかかっている → 無効化してから
- VPC を消したいのにできない → ENI や EIP がどこかで紐づいている
- IAM ロールにサービスが紐づいたままになっている

「消すときのコスト」を IaC で下げる意味は大きい。

11.6.4 マルチアカウント設計

個人でも **検証専用の子アカウント** を作るのを推奨する。

- 実験で事故ってもメインのアカウントに影響しない
- 課金を明確に分離できる
- 検証が終わったらアカウントごと閉鎖できる

Organizations + Control Tower で楽に構築できる。

11.6.5 CI/CD での認証

GitHub Actions / GitLab CI から AWS を操作するときは、**アクセスキーを置かず**に **OIDC** で **AssumeRole** する。手順：

1. IAM で OIDC プロバイダ（`token.actions.githubusercontent.com`）を登録
2. 引き受け可能なロールを作成、信頼ポリシーで対象リポジトリを制限
3. Workflow で `aws-actions/configure-aws-credentials@v4` を使い、`role-to-assume` を指定

この方式なら、万一リポジトリが公開されても長期キーが漏れる心配がない。

11.6.6 よくある事故と回避

事故	回避策
NAT Gateway 立てっぱなしで月数千円	検証終了時に <code>destroy</code> 、または平日だけ稼働する IaC
Elastic IP 放置で課金	未使用の EIP を自動検知する Lambda を設置

事故	回避策
S3 バケットを誤って公開	ブロックパブリックアクセス、Access Analyzer、Config ルール
ルートユーザー漏洩	MFA 必須、Organizations で使用を制限、CloudTrail で即時通知
リージョン間違いで不要リソース増殖	SCP で使えるリージョンを限定
CloudWatch Logs が無限に増える	ロググループごとに保持期間を必ず設定
想定外の無料枠超過	Billing Preferences で Free Tier アラート有効化
IAM 権限不足で急な障害	IAM Access Analyzer Policy Generator で必要権限を算出

11.6.7 リソース棚卸しの習慣

月 1 回、以下を確認する。

- **Cost Explorer** で前月比の急増がないか
- 全リージョンで EC2 の停止インスタンス、未使用の EBS、未使用の EIP を点検
- 停止した RDS（自動再起動でコストが戻る）
- 放置された CloudFormation スタック や Terraform state
- IAM ユーザーの未使用アクセスキー

Trusted Advisor（Basic プランでも一部利用可）を見ると、上記の多くが自動リスト化される。

11.7 さらに学ぶために

- **AWS Well-Architected Framework**：運用・セキュリティ・信頼性・パフォーマンス・コスト・持続可能性の 6 本柱で AWS 構成をレビューする公式フレームワーク
- **AWS Skill Builder**：公式の学習プラットフォーム。無料コースだけでもかなりの量
- **AWS Certified Cloud Practitioner (CLF) / Solutions Architect Associate (SAA)**：資格勉強は体系学習の最短コース
- **AWS ブログ / What's New**：新サービス・アップデートのキャッチアップ
- 各サービスの **公式ドキュメント**：情報の正は常にドキュメント。ブログや本は補助

本書は「広く浅く」で全体像を掴むためのものである。実運用では、各サービスのドキュメントをあたり、Well-Architected の柱ごとにチェックしていく姿勢が長期的に効く。

12 Route 53 と CloudFront

ユーザーから AWS 上のリソースへ到達するまでの経路は、DNS 解決（Route 53）と エッジ配信（CloudFront）の 2 つの基盤に支えられている。本章ではこの 2 サービスを順に解説し、最後に組み合わせの典型構成をまとめる。

12.1 Route 53 の概要

Amazon Route 53 は、AWS のフルマネージド **権威 DNS** サービスである。加えてドメイン登録、ヘルスチェック、内部 DNS、リゾルバなど、DNS 周りの機能を一通り備える。SLA 100% が公約されている数少ないサービスのひとつ。

12.1.1 DNS の基礎おさらい

DNS は「ドメイン名 → IP アドレス」を返す分散型データベースである。

- **権威サーバ（Authoritative Server）**：そのドメインの正解を持つサーバ。Route 53 はここに位置する
- **リゾルバ（Resolver）**：クライアントから問い合わせを受けて再帰的に解決するサーバ
- **TTL（Time To Live）**：レスポンスをリゾルバ・クライアントがキャッシュしてよい秒数
- **レコードタイプ**：A（IPv4）、AAAA（IPv6）、CNAME（別名）、MX（メール）、TXT、NS、SOA、SRV、PTR など

12.1.2 Route 53 の主要コンポーネント

要素	説明
ホストゾーン	ドメイン（例： <code>example.com</code> ）に対するレコードの集合
レコードセット	ホストゾーン内の個別レコード
Alias レコード	Route 53 独自。AWS リソースへの「論理的な A/AAAA」
ヘルスチェック	エンドポイントの健全性監視。フェイルオーバーに使う
Resolver	VPC 内の DNS 解決機構
トラフィックポリシー	複雑なルーティングを GUI で設計
ドメイン登録	TLD への登録代行

12.2 ホストゾーン

12.2.1 パブリックホストゾーン

インターネットに公開する DNS ゾーン。 `example.com` を Route 53 で運用するには、まずパブリックホストゾーンを作成し、レジストラ側で **ネームサーバ（NS レコード）** を Route 53 のものに設定する。

```
aws route53 create-hosted-zone \
  --name example.com \
  --caller-reference $(date +%s)
```

作成すると 4 つの NS (`ns-XXX.awsdns-XX.com` など) が割り当てられる。これをレジストラ側に設定すれば委任完了。

12.2.2 プライベートホストゾーン

VPC 内部だけに公開する DNS ゾーン。複数 VPC・複数アカウントから参照可能。
`internal.example.com` のように内部用ドメインを切るのが定石。

```
aws route53 create-hosted-zone \
  --name internal.example.com \
  --caller-reference $(date +%s) \
  --vpc VPCRegion=ap-northeast-1,VPCId=vpc-0abc...
```

12.3 レコードと Alias

通常の A レコードでは IP アドレスを直接書く。AWS リソース (ALB、CloudFront、S3 静的サイト、API Gateway、Global Accelerator など) に対しては **Alias レコード** を使う。

12.3.1 Alias レコードの利点

- ・エンドポイントの IP 変更に追従する (ALB の IP は時間で変わる)
- ・Apex (例: `example.com`) に直接設定可能 (CNAME は Apex に置けない)
- ・追加の DNS クエリ料金がかからない

```
{
  "Name": "example.com",
  "Type": "A",
  "AliasTarget": {
    "HostedZoneId": "Z2FDTNDAQYW2",
    "DNSName": "d1111111abcdef8.cloudfront.net.",
    "EvaluateTargetHealth": false
  }
}
```

CloudFront ディストリビューションへの Alias。 `HostedZoneId` はサービスごとの固定値 (CloudFront はグローバル `Z2FDTNDAQYW2`)。

12.4 ルーティングポリシー

Route 53 では同名・同タイプのレコードに複数のターゲットを登録し、**DNS 応答を制御** できる。

ポリシー	動作
Simple	単一ターゲット
Weighted	重み付け配分 (Blue/Green、A/B テスト)
Latency	リゾルバから近いリージョンを返す
Failover	プライマリ／セカンダリ。ヘルスチェック連動
Geolocation	リクエスト元の国・大陸でターゲット切り替え
Geoproximity	地理的距離 + バイアス
Multivalue Answer	複数 IP をランダムに返す
IP-based	送信元 IP プレフィックスで分岐

12.4.1 Weighted の例

```
example.com → A × 2
  - 重み 90 → ALB-blue
  - 重み 10 → ALB-green
```

Blue/Green デプロイの段階切替や、新環境のカナリアテストに使う。

12.4.2 Failover の例

```
api.example.com
  - Primary   → ALB (東京)   ヘルスチェック付き
  - Secondary → S3 「メンテナンス中」ページ
```

プライマリが落ちると即座にセカンダリへフェイルオーバー。

12.5 ヘルスチェック

Route 53 のヘルスチェックは以下を監視できる。

- HTTP / HTTPS / TCP のエンドポイント疎通
- CloudWatch アラームの状態
- 他のヘルスチェックの集計 (Calculated Health Check)

ヘルスチェック失敗時、Failover ポリシーや Weighted ポリシーでターゲットから自動的に外れる。30 秒・10 秒間隔のいずれか、しきい値 1~10、複数リージョンからの確認が可能。

12.6 ドメイン登録と DNSSEC

Route 53 はレジストラ機能も持つ。`.com` `.jp` `.org` など多数の TLD に対応。既存の他社レジストラから移管も可能 (Auth Code が必要)。

DNSSEC も Route 53 で有効化できる。署名鍵は KMS の非対称キー (ECC_NIST_P256 等) で管理。改ざん検知が必要な業務ドメインで導入する。

12.7 Route 53 Resolver と内部 DNS

VPC 内では予約アドレス（例：10.0.0.2）でリゾルバが動いている。**Resolver Endpoint**を追加することで、オンプレ ↔ VPC 間の名前解決が可能。

- ・ **インバウンドエンドポイント**：オンプレから VPC 内ホスト名を解決
 - ・ **アウトバウンドエンドポイント**：VPC 内から **Resolver Rule** に基づいてオンプレ DNS へ転送
- ハイブリッド環境では必須。

12.7.1 Route 53 Resolver DNS Firewall

DNS クエリのドメイン単位での許可・ブロック。マルウェアが C2 サーバへ問い合わせるのをブロック、特定 SaaS をブロック、といった用途。

12.8 Route 53 Profiles

複数 VPC に対して、プライベートホストゾーン関連付け、Resolver Rule、DNS Firewall ルールなどを **まとめて適用** する仕組み。マルチアカウント・マルチ VPC 環境で有用。

12.9 Route 53 の料金感

- ・ ホストゾーン：\$0.50/月
- ・ 標準クエリ：\$0.40/100 万クエリ（最初の 10 億）
- ・ Latency / Geo / Geoproximity ベースクエリ：\$0.60/100 万クエリ
- ・ ヘルスチェック：基本 \$0.50/月、AWS 外エンドポイントは \$0.75
- ・ DNS Firewall：\$0.60/100 万クエリ + ルールグループ料金

個人運用なら 月 1〜2 ドル程度 に収まる。

12.10 CloudFront の概要

Amazon CloudFront は AWS のグローバル CDN。世界中の **エッジロケーション** と **リージョナルエッジキャッシュ** にコンテンツをキャッシュし、ユーザーに近い場所から配信する。

12.10.1 主要メリット

- ・ **レイテンシ削減**：地理的に近い PoP から応答
- ・ **オリジン負荷軽減**：キャッシュヒットすればオリジンには行かない
- ・ **DDoS 対策**：Shield Standard が無料適用
- ・ **HTTPS 終端**：ACM 証明書で簡単に
- ・ **エッジコンピューティング**：CloudFront Functions / Lambda@Edge

12.11 ディストリビューションとオリジン

CloudFront の単位は **ディストリビューション**。1〜複数の **オリジン** を持つ。

12.11.1 オリジンの種類

- **S3 オリジン**：S3 バケットを直接、または OAC 経由で
- **Custom HTTP オリジン**：ALB、EC2、独自サーバ
- **Origin Group**：プライマリ／セカンダリでフェイルオーバー
- **VPC Origins**：CloudFront から VPC 内のプライベートな ALB / NLB / EC2 に直接接続

12.12 ビヘイビアとポリシー

ディストリビューションごとに **ビヘイビア** を複数定義し、**パスパターン**（`/api/*`、`/static/*`、`*`）で振り分ける。各ビヘイビアに以下のポリシーを設定する。

ポリシー	用途
Cache Policy	キャッシュキーに含める要素と TTL
Origin Request Policy	オリジンに転送する要素
Response Headers Policy	レスポンスヘッダの追加・削除（CORS、HSTS、CSP）

AWS マネージドポリシーが豊富にあり、`CachingOptimized`、`CachingDisabled`、`Managed-AllViewer`、`SecurityHeadersPolicy` 等を使い回すのが定石。

12.12.1 TTL の優先順位

1. オリジン側のキャッシュヘッダ（`Cache-Control`、`Expires`）
2. Cache Policy の Min/Max/Default TTL
3. ビューア側のリクエスト（`Cache-Control: max-age=0`）

12.13 OAC (Origin Access Control)

S3 オリジンでバケットを 非公開のまま CloudFront にだけ読ませる 仕組み。旧来の OAI の後継で、**新規構築では OAC を使う**。

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AllowCloudFrontOAC",
    "Effect": "Allow",
    "Principal": { "Service": "cloudfront.amazonaws.com" },
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::my-static-site/*",
    "Condition": {
      "StringEquals": {
        "AWS:SourceArn": "arn:aws:cloudfront::123456789012:distribution/E1ABCDEF12345"
      }
    }
  }]
}
```

12.14 署名付き URL / 署名付き Cookie

特定ユーザー・期間限定のアクセス制御。動画配信、有料コンテンツ、レポートダウンロードで使う。

- 署名付き URL：1 ファイルだけに有効
- 署名付き Cookie：同一プレフィックス配下の複数ファイルに有効（HLS 動画など）

CloudFront Key Group を作って秘密鍵で署名する。

12.15 エッジコンピューティング

12.15.1 CloudFront Functions

軽量・超高速の JavaScript（ECMAScript 5.1 ベース）。リクエスト／レスポンス時に走る。

- 用途：URL 書き換え、A/B テスト、ヘッダ追加、シンプル認証
- 実行時間：1ms 以下、メモリ 2MB
- 安価：100 万リクエストあたり \$0.10
- ビューアリクエスト・ビューアレスポンスでのみ動作

```
function handler(event) {  
  var request = event.request;  
  var uri = request.uri;  
  if (uri.endsWith('/')) {  
    request.uri += 'index.html';  
  } else if (!uri.includes('.')) {  
    request.uri += '/index.html';  
  }  
  return request;  
}
```

12.15.2 Lambda@Edge

Node.js / Python の Lambda をエッジで実行。

- 用途：複雑な認証、画像変換、SEO 対応の SSR、HTTP ヘッダ精密制御
- ビューアリクエスト／ビューアレスポンス／オリジンリクエスト／オリジンレスポンスの 4 箇所で動作
- **us-east-1 にデプロイ** → グローバルにレプリケート
- 実行時間 5～30 秒、メモリ最大 10GB
- 通常の Lambda よりやや高い

CloudFront Functions で済むなら Functions、SDK 呼び出しや複雑処理が必要なら Lambda@Edge。

12.16 WAF / Shield 連携

CloudFront は WAF の関連付け先として最も自然な配置。グローバルの Web ACL を貼ることで、エッジで不正リクエストを遮断できる。Shield Standard は CloudFront 経由で自動適用。

12.17 ログとメトリクス

- **Standard Logs (v2)** : S3 / Kinesis Firehose / CloudWatch Logs に出力。最新版（2024 年後半 GA）はリアルタイム性とフィルタが向上
- **Real-time Logs** : Kinesis Data Streams 経由でほぼリアルタイム
- **CloudWatch Metrics** : リクエスト数、4xx/5xx 率、転送量
- **CloudFront Functions Metrics** : 実行時間、エラー数

ログは Athena でクエリすると分析が楽。

12.18 カスタムドメインと ACM

CloudFront でカスタムドメインを使うには ACM 証明書を **us-east-1 リージョン** で取得する必要がある（CloudFront はグローバルサービスだが、設定は北バージニア固定）。

```
aws acm request-certificate \  
  --domain-name 'example.com' \  
  --subject-alternative-names '*.example.com' \  
  --validation-method DNS \  
  --region us-east-1
```

DNS 検証用の CNAME レコードを Route 53 に追加する（コンソールから 1 クリックで生成可）。

12.19 キャッシュ無効化

デプロイ後にキャッシュを即時破棄したいときは **Invalidation** を実行する。

```
aws cloudfront create-invalidation \  
  --distribution-id E1ABCDEF12345 \  
  --paths '/*' '/index.html'
```

無料枠は月 **1,000 パス** まで。それを超えると \$0.005/パス。

運用ヒント : パスでのバージョニング（`/static/v123/app.js`）にすればそもそも無効化が要らなくなる。

12.20 CloudFront の料金感

- **データ転送** : リージョン別。東京 PoP からなら \$0.114/GB（最初の 10TB）
- **リクエスト** : HTTPS で \$0.0120/10,000 リクエスト（北米・欧州）
- **無料枠** : 永続無料 1TB/月の HTTP/HTTPS データ転送、1,000 万リクエスト

- **Functions** : \$0.10/100 万リクエスト
- **Lambda@Edge** : 通常 Lambda の数倍

個人サイト規模なら **常時無料枠** に収まることが多い。

12.21 統合パターン

12.21.1 静的サイト

```
[Browser] → Route 53 (alias) → CloudFront → S3 (OAC, 非公開)
                      ↳ ACM (us-east-1)
                      ↳ WAF
```

最も典型的な構成。SPA のホスティングに最適。

12.21.2 API バックエンド

```
[Browser] → Route 53 → CloudFront - /static/* - S3
                      ↳ /api/*      - ALB → ECS/EC2
                                ALB → Lambda
```

静的アセットを長期キャッシュ、API はキャッシュ無効 + Origin Request Policy で必要なヘッダのみ転送。

12.21.3 マルチリージョン

```
example.com (Latency)
- 東京          → CloudFront → ALB-東京
- バージニア → CloudFront → ALB-バージニア
```

CloudFront 自体がグローバルなので、マルチリージョンは **オリジン** を Route 53 で振り分ける。Origin Group + フェイルオーバーとの組み合わせも有効。

12.22 Global Accelerator との比較

項目	CloudFront	Global Accelerator
プロトコル	HTTP/HTTPS 中心	TCP/UDP 任意
キャッシュ	あり	なし
対象	Web コンテンツ・API	ゲーム、IoT、独自プロトコル
IP	配信ごとの DNS 名	固定 Anycast IP × 2
料金モデル	リクエスト + 転送	固定 \$18/月 + 転送

ゲームサーバや独自プロトコルなら Global Accelerator、Web 配信なら CloudFront。

12.23 よくあるトラブル

症状	確認ポイント
DNS が反映されない	TTL、レジストラ側 NS、ローカル DNS キャッシュ（ <code>dig</code> ）
CloudFront が 403	OAC バケットポリシー、Origin Path、署名付き URL 期限切れ
HTTPS が動かない	ACM が us-east-1 か、ドメイン検証完了済みか、CNAME 一致
古いコンテンツが返る	TTL 設定、Invalidation 到達待ち、ブラウザキャッシュ
API レスポンスがおかしい	Cache Policy / Origin Request Policy のヘッダ・Cookie 設定
Lambda@Edge デプロイできない	us-east-1、信頼ポリシーに <code>edgelambda.amazonaws.com</code>
キャッシュヒット率が低い	クエリ文字列・ヘッダ・Cookie がキャッシュキーに入りすぎ

12.24 学習の進め方

1. Route 53 でホストゾーンを作り、無料の `.tk` などを実験
2. S3 + CloudFront + ACM の静的サイトを立てる（Free Tier 内）
3. Failover ポリシーで意図的にプライマリを落とし切り替えを確認
4. CloudFront Functions で URL 書き換えを書いて挙動を見る
5. 商用利用なら Standard Logs を S3 に出して Athena でアクセス分析

13 追加のストレージサービス

EBS（ブロック）と S3（オブジェクト）は前章までで扱った。本章ではそれ以外のストレージサービス、特に **共有ファイルシステム**（EFS / FSx）、**ハイブリッド**（Storage Gateway）、**バックアップ**（AWS Backup）、**物理データ移送**（Snow ファミリー）を扱う。

13.1 ストレージ選択の早見表

種類	主な選択肢
ブロック（単一サーバー）	EBS、FSx for OpenZFS（NVMe）
ファイル共有（NFS）	EFS、FSx for Lustre、FSx for OpenZFS、FSx for NetApp ONTAP
ファイル共有（SMB）	FSx for Windows File Server、FSx for NetApp ONTAP
オブジェクト	S3
バックアップ	AWS Backup（一元管理）
ハイブリッド・既存システム	Storage Gateway
大容量物理移送	Snowball / Snowcone / Snowmobile
テープライブラリ置換	Storage Gateway Tape Gateway

13.2 EFS（Elastic File System）

Amazon EFS は NFSv4.1 互換のフルマネージド共有ファイルシステム。複数の EC2、Fargate タスク、Lambda 等から **同時マウント** できる。

13.2.1 主な特徴

- ・ **マルチ AZ で耐久性**：自動的に複数 AZ に冗長保管（One Zone クラス除く）
- ・ **自動拡張・縮小**：使った分だけ課金、容量プロビジョニング不要
- ・ **POSIX 互換**：UID/GID、パーミッション、シンボリックリンク
- ・ **マウントターゲット**：AZ ごとに ENI を作成し、その IP に NFS マウント

13.2.2 ストレージクラス

クラス	用途
Standard	複数 AZ・頻繁アクセス
Standard-IA	複数 AZ・低頻度アクセス（30 日以上未使用）
One Zone	単一 AZ・頻繁アクセス。安価
One Zone-IA	単一 AZ・低頻度。最安
Archive	年に数回のアクセス。さらに安価（2024～）

ライフサイクル管理 で「30 日触られていないファイルを IA に」「90 日触られていないファイルを Archive に」等を自動化できる。

13.2.3 スループットモード

モード	特性
Bursting	容量に比例した baseline + bursting credit
Provisioned	容量と独立してスループットを予約
Elastic（推奨デフォルト）	需要に応じて自動でスケール。ほとんどのワークロードで第一選択

13.2.4 マウント例

```
# amazon-efs-utils 経由（推奨）
sudo dnf install -y amazon-efs-utils
sudo mkdir -p /mnt/efs
sudo mount -t efs -o tls fs-0123456789abcdef0:/ /mnt/efs

# 標準 NFS クライアント
sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsiz=1048576,hard,timeo=600,retrans=2 \
fs-0123456789abcdef0.efs.ap-northeast-1.amazonaws.com:/ /mnt/efs
```

`/etc/fstab` に登録して恒久マウントする運用が一般的。

13.2.5 アクセスポイント

EFS の **アクセスポイント** は、ファイルシステム内の特定パスとユーザー ID を固定して提供する仕組み。

- ・アプリケーションごとに別々のディレクトリを使わせる
- ・root (UID=0) を強制的に別ユーザーに置き換える
- ・IAM で「このアクセスポイントだけアクセス可」と絞れる
- ・Lambda から EFS をマウントするときに必須

13.2.6 典型ユースケース

- ・WordPress / Drupal などの共有メディア
- ・データサイエンス用の共有データセット
- ・コンテナ (ECS / EKS) の永続ボリューム
- ・共有設定ファイル、共有ログ
- ・ML 訓練の中間成果物

13.2.7 料金感

- ・Standard : \$0.36/GB/月（東京）
- ・Standard-IA : \$0.027/GB/月 + アクセス料
- ・One Zone : Standard より約 20% 安
- ・Archive : Standard-IA よりさらに安
- ・Provisioned スループット : 別途課金

13.3 FSx ファミリー

EFS では合わない要件向けに、FSx は 4 種類の専用ファイルシステムを提供する。

13.3.1 FSx for Windows File Server

SMB プロトコル + Active Directory 統合 の Windows 共有。Windows 業務アプリ、Microsoft SQL Server のファイルシェア、ファイルサーバ移行先として使う。

- マネージド AD 統合（または既存 AD 連携）
- DFS 名前空間、シャドウコピー（VSS）
- マルチ AZ オプションあり
- AzureFiles のような Linux 共有需要には EFS / FSx for OpenZFS を選ぶ

13.3.2 FSx for Lustre

高速並列ファイルシステム。HPC、ML 訓練、ゲノム解析などに。

- スループット数百 GB/s、IOPS 数百万
- S3 とリンク できる：S3 のオブジェクトをファイルとして読み書き、結果を自動同期
- Persistent / Scratch の 2 モード（Scratch は短期計算向け）
- Linux 用（Lustre クライアント）

13.3.3 FSx for OpenZFS

ZFS の機能をフルマネージドで。スナップショット、クローン、圧縮、データ整合性チェック。

- NFSv3 / v4
- 高 IOPS NVMe SSD
- スナップショットの低コスト保持、クローンの瞬時作成
- 開発環境を本番のクローンから瞬時に作る、といった使い方が得意

13.3.4 FSx for NetApp ONTAP

既存 NetApp 資産を AWS でも。SnapMirror、SnapVault、FlexClone、デデュープ、圧縮、Fabric Pool（コールド層を S3 に）対応。

- マルチプロトコル（NFS / SMB / iSCSI）
- オンプレ ONTAP との **SnapMirror** 連携
- 大規模 SAP、データベース、企業の汎用ファイラ

13.3.5 FSx の選び方

要件	推奨
Windows 共有・AD 統合	FSx for Windows File Server
HPC / ML 並列ファイル I/O	FSx for Lustre
ZFS 機能（snap/clone/圧縮）	FSx for OpenZFS
既存 NetApp 資産・マルチプロトコル	FSx for NetApp ONTAP

要件	推奨
汎用 NFS で十分	EFS

13.4 Storage Gateway

オンプレからクラウドへのハイブリッド接続を提供する。3つのタイプがある。

13.4.1 File Gateway

オンプレに NFS / SMB のエンドポイントを提供し、書き込まれたファイルを **S3 オブジェクト** として保存する。

- ・ ローカルキャッシュ（読み出し高速化）
- ・ POSIX メタデータの保持
- ・ バックアップ、メディアアーカイブ、ML データ集約に

13.4.2 Volume Gateway

iSCSI ボリュームを提供し、裏で S3 + EBS スナップショット相当に保管する。

- ・ **Cached モード**：プライマリは S3、ホット部分だけローカルキャッシュ
- ・ **Stored モード**：プライマリはローカル、定期的に S3 へバックアップ

13.4.3 Tape Gateway

仮想テープライブラリ（VTL）。バックアップソフト（Veeam、NetBackup 等）から見ると LTO テープに見えるが、裏は S3 / Glacier。物理テープ運用の置き換え。

13.5 AWS Backup

複数サービスを **横断的に** バックアップ管理する統合サービス。各サービスの個別バックアップ機能を一元化する。

13.5.1 対応リソース

EBS、EC2 イメージ、RDS、Aurora、DynamoDB、EFS、FSx ファミリー、Storage Gateway、Neptune、DocumentDB、S3、Redshift、CloudFormation スタック、SAP HANA on EC2、VMware（Backup Gateway 経由）など。

13.5.2 主要概念

概念	説明
バックアップ計画	いつ取るか、どこに保存するか、何日保持するか
バックアップポルト	バックアップの保存先（KMS で暗号化）
バックアップセレクション	どのリソースを対象にするか（タグや ARN で指定）
リカバリポイント	個別のバックアップ実体
ライフサイクル	Cold storage 移行、削除のスケジュール

13.5.3 クロスリージョン・クロスアカウントコピー

- **DR 用途** でリージョン災害に備える
- **監査用** に別アカウント（バックアップ専用アカウント）にコピーし、本番側からは削除できなくする

13.5.4 ボールトロック（Vault Lock）

バックアップボールトに **WORM** を効かせる機能。

- **Compliance モード**：誰も削除できない（ルートユーザーでも）
- **Governance モード**：特定権限を持つ管理者は解除可能

ランサムウェア対策・規制対応の決定版。

13.5.5 AWS Backup Audit Manager

バックアップが定義したフレームワーク（バックアップ頻度、保持期間、ボールトロック）を満たしているかを継続評価。レポート生成・SNS 通知。

13.6 Snow ファミリー

ペタバイト級データの物理移送、エッジコンピューティング向け。

デバイス	用途
Snowcone	小型・8TB 程度。エッジ・小規模オフライン
Snowball Edge Storage Optimized	80TB 超のオフライン移送
Snowball Edge Compute Optimized	現地でのコンピュータ＋ストレージ
Snowmobile（提供終了）	40 フィートコンテナ。エクサバイト級。後継は大量 Snowball

「100TB を月の回線でアップロードすると数か月かかる、Snowball で送る方が速くて安い」というケースで採用。デバイス物理輸送のリードタイム（1～2 週間）を含めて計画する。

13.7 DataSync

オンプレ ↔ AWS、AWS 間でのデータ同期サービス。

- 対応元／先：NFS / SMB / HDFS / オブジェクトストレージ / S3 / EFS / FSx
- 並列・高速転送、検証付き
- スケジュール実行、増分差分転送
- 帯域制御
- KMS 暗号化、整合性チェック

オンライン回線でリーズナブルな時間で送れる規模なら DataSync、それを超えるなら Snow。

13.8 ストレージコスト最適化のコツ

- S3：ライフサイクルで階層化、Intelligent-Tiering、不完全マルチパート削除、バージョン整理
- EBS：未使用ボリュームの削除、gp2 → gp3 移行（性能割安）
- EFS：ライフサイクルで IA・Archive へ、Elastic スループットで自動最適化
- FSx：Lustre は計算終了後すぐ破棄、OpenZFS のスナップショット定期整理
- Backup：Cold ストレージ移行（Glacier 相当）、リージョン分離戦略の見直し
- Storage Gateway：ローカルキャッシュサイズの妥当性検証

13.9 ベンチマーク・パフォーマンス計測

- fio（Linux）：ブロック・ファイル両方の汎用ベンチ
- iperf：ネットワーク帯域
- AWS 公式パフォーマンス計測スクリプト：EFS 用、Lustre 用に提供あり

ストレージはワークロード特性次第で適合解が変わる。本番投入前に必ず実データで検証する。

13.10 よくあるトラブル

症状	対処
EFS マウントできない	SG で 2049 (NFS) 開放、amazon-efs-utils、IAM Mount 権限
EFS が遅い	Bursting Credit 枯渇 → Elastic スループットに変更
FSx Windows に AD ログインできない	DNS 名解決、AD 同期、SMB 暗号化要件
Storage Gateway がオフラインに	VM のリソース、ネットワーク疎通、アクティベーションキー再発行
AWS Backup ジョブが失敗	IAM ロール、リソースポリシー、ポールド KMS 権限
Snowball が届かない	配送業者追跡、税関、AWS Support に問い合わせ

14 メッセージングと統合

サービスを疎結合に保ち、スケーラビリティと耐障害性を高めるために、AWS は豊富なメッセージング・統合サービスを提供する。本章では SQS / SNS / EventBridge / Step Functions を中心に、API Gateway 詳細、AppSync、MQ も扱う。

14.1 イベント駆動アーキテクチャの全体像

14.1.1 同期 vs 非同期

- **同期**：呼び出し側がレスポンスを待つ。HTTP API、gRPC
- **非同期**：呼び出し側はキューにメッセージを置いて即時離脱。バックエンドが順次処理

14.1.2 Push vs Pull

- **Push**：送信側が宛先に直接配信（SNS、EventBridge）
- **Pull**：受信側がキューを定期的に読みに行く（SQS）

14.1.3 サービス早見表

サービス	向いている用途
SQS	ジョブキュー、バックプレッシャ、Worker パターン
SNS	Pub/Sub、ファンアウト、ユーザー通知
EventBridge	サービス間イベント連携、SaaS 統合、スケジュール
Step Functions	ワークフロー、Saga、長時間処理
API Gateway	HTTP / REST / WebSocket API
AppSync	GraphQL、リアルタイム
MQ	既存 ActiveMQ / RabbitMQ 資産の置き換え
Kinesis	ストリーミング（15 章で扱う）

14.2 SQS（Simple Queue Service）

最も枯れたフルマネージドキュー。サーバー不要、HTTP API で送受信。

14.2.1 Standard キュー vs FIFO キュー

項目	Standard	FIFO
スループット	ほぼ無制限	3,000/秒（バッチ）または 300/秒
配信	少なくとも 1 回（重複あり）	厳密に 1 回
順序	ベストエフォート	厳密な FIFO
料金	基本：\$0.40/100 万	基本：\$0.50/100 万

項目	Standard	FIFO
用途	スループット重視・幂等処理	順序・重複排除が必須

14.3 主要パラメータ

パラメータ	意味
可視性タイムアウト	メッセージ受信後、他のコンシューマから見えなくなる時間。処理時間より長く設定
メッセージ保持期間	最大 14 日（デフォルト 4 日）
遅延キュー	送信後、指定秒数は配信されない
Long polling	<code>ReceiveMessageWaitTimeSeconds</code> で空ポーリング削減
DLQ	N 回処理失敗したメッセージを別キューに退避
メッセージ属性	ヘッダ的な追加情報

14.4 Lambda トリガー

SQS 上のメッセージを Lambda が自動でポーリング・処理する標準パターン。

- ・ バッチサイズ：1~10,000（FIFO は最大 10）
- ・ バッチウィンドウ：最大 5 分待ってからまとめて処理
- ・ **Partial Batch Failure**：一部だけ失敗を返せる（残りは成功扱い）
- ・ **Maximum Concurrency**：同時実行数の上限を SQS イベントソース側で制御

```
def lambda_handler(event, context):
    failures = []
    for record in event['Records']:
        try:
            process(record['body'])
        except Exception:
            failures.append({"itemIdentifier": record['messageId']})
    return {"batchItemFailures": failures}
```

14.5 ベストプラクティス

- ・ 幂等性：Standard は重複配信あり前提。`MessageDeduplicationId` 相当の処理側ガード
- ・ **Long polling 必須**：1 秒/秒のショートポーリングは API コスト無駄
- ・ **DLQ を必ず設定**：失敗の山が Live キューに残らないように
- ・ メッセージは小さく（最大 256KB、超えるなら S3 に置いてキーだけ送る）
- ・ FIFO の `MessageGroupId` は適切な粒度で（ユーザー ID 単位など）。1 グループ＝順序単位

14.6 SNS (Simple Notification Service)

Pub/Sub モデル。トピックに publish、サブスクライバーに fan-out。

14.6.1 サブスクライバー（プロトコル）

- Email、Email-JSON
- SMS（料金高めなので注意）
- HTTPS / HTTP（任意 URL に POST）
- SQS（最も多用）
- Lambda
- Mobile Push（APNs / FCM）
- Kinesis Data Firehose
- Application（モバイルプッシュ）

14.6.2 ファンアウトパターン

```
[Producer] → SNS Topic
               ↳ SQS-A → Lambda-A
               ↳ SQS-B → Lambda-B
               ↳ SQS-C → Lambda-C
```

1 publish で複数の処理系を独立に走らせる。各 SQS は再試行・DLQ を独立管理できるため、片方の処理系が遅れても他に影響しない。

14.6.3 FIFO トピック

順序保証付き SNS。配信先に FIFO SQS を使う。エンドユーザー通知よりも、内部システム連携で順序が必要な場合に。

14.6.4 メッセージフィルタリング

サブスクリプション側で **メッセージ属性** に基づくフィルタを設定。「`event_type=order_placed` のときだけこのキューに流す」等。Producer 側のロジックが軽くなる。

14.6.5 配信エラーと DLQ

サブスクリプションごとに DLQ（SQS）を設定。HTTPS エンドポイントが無応答だった場合などに退避。

14.7 EventBridge

サービス間・SaaS との イベントバス。SNS よりも構造化・宣言的でルーター能力が高い。

14.7.1 バスの種類

- **Default Event Bus**：AWS サービスのイベントが流れてくる（EC2 状態変化、ECS タスク状態など）
- **Custom Event Bus**：自分で定義してアプリ間連携
- **Partner Event Bus**：Datadog、Zendesk、PagerDuty など SaaS からのイベント

14.7.2 ルールとターゲット

ルールでイベントパターンを書き、マッチしたイベントをターゲットに送る。

```
{
  "source": ["aws.ec2"],
  "detail-type": ["EC2 Instance State-change Notification"],
  "detail": {
    "state": ["terminated"]
  }
}
```

ターゲット候補（最大5個）: Lambda、Step Functions、SQS、SNS、Kinesis、ECS タスク、SSM Run Command、API Destination（任意 HTTP）、別 EventBridge バスなど。

14.7.3 スケジュール

旧来は **EventBridge ルール** の **cron 式** で書いていたが、現在は **EventBridge Scheduler** が独立サービスとして推奨される。

- 1 分間隔以下～年単位
- 1 回限り（One-time）スケジュールも可
- 最大1年先まで予約可能
- タイムゾーン指定
- グループ化、暗号化

```
aws scheduler create-schedule \
  --name daily-report \
  --schedule-expression 'cron(0 9 * * ? *)' \
  --schedule-expression-timezone 'Asia/Tokyo' \
  --target '{"Arn":"arn:aws:lambda:...","RoleArn":"arn:aws:iam:..."}' \
  --flexible-time-window '{"Mode":"OFF"}'
```

14.7.4 アーカイブとリプレイ

イベントバス上のイベントを **S3 にアーカイブ** し、後から **リプレイ** してターゲットに再配信できる。障害復旧、テスト環境への流し込みに便利。

14.7.5 EventBridge Pipes

Source → **Filter** → **Enrich** → **Target** の1対1パイプ。SQS、Kinesis、DynamoDB Streams、MSK、Self-managed Kafka、ActiveMQ 等を Source にして、Lambda や ECS タスクで enrich し、Target に流す。Step Functions や Lambda を経由せずシンプルな ETL/連携が組める。

14.7.6 スキーマレジストリ

イベントの JSON スキーマを管理し、SDK のバインディング（Java、Python、TypeScript、Go）を自動生成。

14.8 Step Functions

ステートマシンを宣言する Workflow サービス。

14.8.1 2 種類のワークフロー

項目	Standard	Express
最長実行時間	1 年	5 分
課金	ステート遷移ごと	実行時間 + リクエスト
べき等	exactly-once	at-least-once (Async) , exactly-once (Sync)
履歴	長期保存・可視化	CloudWatch Logs に
向く用途	長時間ビジネスプロセス	高頻度・短時間処理

14.8.2 ステートタイプ

- **Task** : Lambda 呼び出しや AWS サービス統合
- **Choice** : 条件分岐
- **Parallel** : 並列実行
- **Map** : 配列要素ごとに並列実行 (最大 10,000)
- **Wait** : 時間待機 or 特定時刻まで
- **Pass** : データ加工
- **Succeed / Fail** : 終了

14.8.3 Service Integration Patterns

パターン	動作
Request Response	呼び出してすぐ次へ。デフォルト
.sync	サービス完了まで待つ (ECS RunTask など)
.waitForTaskToken	Token を渡し、外部からのコールバックを待つ (人手承認など)

14.8.4 ASL (Amazon States Language) の例

```
{
  "Comment": "Order processing",
  "StartAt": "Validate",
  "States": {
    "Validate": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:::function:validate",
      "Next": "IsValid"
    },
    "IsValid": {
      "Type": "Choice",
      "Choices": [{
```



```

        "Variable": "$.valid",
        "BooleanEquals": true,
        "Next": "Charge"
    }],
    "Default": "Reject"
},
"Charge": {
    "Type": "Task",
    "Resource": "arn:aws:lambda::function:charge",
    "Retry": [{
        "ErrorEquals": ["TransientError"],
        "IntervalSeconds": 2,
        "MaxAttempts": 3,
        "BackoffRate": 2.0
    }],
    "Catch": [{
        "ErrorEquals": ["States.ALL"],
        "Next": "Reject"
    }],
    "End": true
},
"Reject": {
    "Type": "Fail",
    "Cause": "Order rejected"
}
}
}

```

14.8.5 Workflow Studio

GUI で状態マシンを設計できる。生成された ASL は Git 管理し、CI/CD で deploy するのが定石。

14.8.6 料金感

- Standard：100 ステート遷移あたり \$0.025（東京）
- Express：100 万リクエストあたり \$1 + 実行時間
- 高頻度・短時間処理は Express、長時間業務処理は Standard

14.9 API Gateway 詳細

8 章では Lambda Proxy 統合を中心に紹介した。本節ではその他の機能を補完する。

14.9.1 HTTP API vs REST API vs WebSocket API

機能	HTTP API	REST API	WebSocket API
基本料金	\$1.00/100 万	\$3.50/100 万	\$1.00/100 万 + メッセージ

機能	HTTP API	REST API	WebSocket API
API キー	×	○	×
使用量プラン	×	○	×
WAF	○	○	× (CloudFront 経由)
リクエスト変換	基本	VTL でフル制御	—
認証	JWT/Lambda/IAM	Cognito/Lambda/ IAM	IAM/Lambda
プライベート	×	○	×
Mock 統合	×	○	×

新規構築は **基本 HTTP API**、機能が必要なら REST API、双方向通信なら WebSocket。

14.9.2 ステージとデプロイ

REST API では **ステージ** (`dev` / `prod`) にデプロイし、ステージごとにスロットリング・キャッシュ・ロギングを設定。**カナリアリリース** で新バージョンに 10% トラフィックだけ流す、なども可能。

14.9.3 カスタムドメイン

API Gateway のデフォルト URL は不格好なので (`https://abc123.execute-api...`)、カスタムドメインに ACM 証明書を紐づけて使う。エンドポイントタイプは `Edge` / `Regional` / `Private` の 3 種。

14.9.4 認証

方式	用途
IAM 認可	SigV4 署名。サービス間呼び出しに
Cognito オーソライザー	User Pool で発行された JWT を検証
Lambda オーソライザー	独自ロジックで認可 (古いトークン形式、外部 IdP 連携)
JWT オーソライザー (HTTP API)	一般的な JWT を直接検証 (Cognito 以外も OK)
API キー + 使用量プラン	サードパーティ向け公開 API でレート制限・課金

14.9.5 マッピングテンプレート (VTL)

REST API でリクエスト・レスポンスを変換できる。Apache Velocity 構文。 `$input.json('$.path')` のような式で JSON 加工。複雑になりがちなので、必要なら Lambda で変換する設計が無難。

14.9.6 統合タイプ

- **Lambda Proxy** : 素直にイベント全体を Lambda へ
- **Lambda Custom** : マッピングテンプレートで変換
- **AWS Service** : 直接 DynamoDB、SQS 等にプロキシ (Lambda レス)
- **HTTP** : 別の HTTP バックエンドへプロキシ
- **Mock** : 固定レスポンスを返す (モック・ダミー)

14.9.7 WebSocket API

`$connect` `$disconnect` `$default` の 3 つのデフォルトルートと、`action` フィールドベースのカスタムルート。

- 接続 ID を DynamoDB で管理し、配信時に `PostToConnection` で個別送信
- 双方向リアルタイム通信（チャット、ライブ更新、ゲーム）
- レイテンシ重視ならプロデューサ側で Pub/Sub を組む

14.10 AppSync

GraphQL のフルマネージド実装。

14.10.1 主な特徴

- **Subscription**（リアルタイムプッシュ、WebSocket / MQTT）
- データソース：DynamoDB、Lambda、HTTP、RDS、OpenSearch、EventBridge
- 複数データソースから 1 リクエストで取得（GraphQL の本来の利点）
- 認証：Cognito User Pool、IAM、API Key、OIDC、Lambda
- **AppSync Merged APIs**：複数の AppSync API を統合
- **Pipeline Resolvers**：複数データソースを順に呼ぶ

14.10.2 いつ使うか

- フロント／モバイルが **柔軟なクエリ** を必要とする
- リアルタイム更新が要件
- マイクロサービスを統合したい

REST と GraphQL の選び方は宗教戦争になりがちだが、クライアント側の柔軟性 vs サーバー側のシンプルさのトレードオフ。

14.11 MQ（Managed Broker）

ActiveMQ / RabbitMQ をマネージドで提供。

- 既存システムが JMS / AMQP / STOMP / MQTT に依存している場合の移行先
- 新規構築では SQS / SNS / EventBridge を優先（料金・スケール）

14.12 統合パターン集

14.12.1 ファンアウト

```
[API] → SNS → SQS-A → Worker-A
          → SQS-B → Worker-B
          → SQS-C → Worker-C
```

1 イベントで複数処理。各 Worker が独立にスケール、独立に DLQ 管理。

14.12.2 バッファリング

[Burst Traffic] → SQS → Lambda (同時実行数で制御)

スパイクトラフィックを SQS で吸収し、バックエンドを過負荷から守る。

14.12.3 Saga / 長時間トランザクション

Step Functions:

```

予約 → 決済 → 在庫確保 → メール
      ↑           ↓
      └─ キャンセル補償 ← 失敗
  
```

各ステップが幂等で、失敗時に補償トランザクションを実行する分散トランザクション。

14.12.4 Choreography (コレオグラフィー)

```

[Service A] → EventBridge → [Service B]
                        → [Service C]
[Service B] → EventBridge → [Service D]
  
```

サービス間が直接呼び出さず、イベントで疎結合に連動。マイクロサービスの典型。

14.12.5 Outbox パターン

DB トランザクションと外部メッセージ送信の整合性を保つ。DynamoDB Streams → Lambda → EventBridge / SQS の連携。

14.12.6 人手承認

Step Functions の `.waitForTaskToken` を使い、Slack / メールでユーザーに承認 URL を送り、コールバックで再開。

14.13 よくあるトラブル

症状	対処
SQS メッセージが 2 回処理される	Standard は重複配信あり前提。受信側で幂等化
Lambda が SQS を処理しない	可視性タイムアウト > Lambda 実行時間、IAM 権限、Trigger 有効
SNS から HTTPS 配信失敗	HTTPS エンドポイントの応答コード 200、TLS 証明書
EventBridge ルール一致しない	パターンの完全一致 vs 部分一致、Test event で検証
Step Functions のループが多い	Map state に置換、Express を検討

症状	対処
API Gateway 504	Lambda タイムアウト 29 秒、バックエンド遅延
CORS が効かない	API Gateway 側の CORS 設定 + Lambda 側のヘッダ両方必要

15 ストリーミングとデータ分析

本章ではデータの 収集（ストリーミング）→ 蓄積（データレイク）→ 加工（ETL）→ 分析（クエリ・BI）の各レイヤを担うサービス群を扱う。

15.1 データ分析の全体像

```
[Source] → [Ingest] → [Storage] → [Process] → [Serve]
IoT      Kinesis    S3 Lake    Glue       Athena
App      MSK        Iceberg    EMR        Redshift
DB       DMS          Hudi/Delta Spark      OpenSearch
                                           QuickSight
```

15.1.1 各サービスの位置づけ早見表

レイヤ	主な選択肢
リアルタイムストリーム	Kinesis Data Streams、MSK
マネージド配信	Kinesis Data Firehose
ストリーム処理	Kinesis Data Analytics (Flink)、Lambda、EMR Spark Streaming
データレイク	S3 (+ Iceberg / Hudi / Delta)
メタデータ・ETL	Glue Data Catalog / Glue ETL / Glue DataBrew
SQL クエリ（オンデマンド）	Athena
DWH	Redshift (Serverless / Provisioned)
全文検索・ログ分析	OpenSearch Service
BI ダッシュボード	QuickSight
大規模 OSS フレームワーク	EMR (Spark / Hive / Presto / Flink)
ガバナンス	Lake Formation、DataZone

15.2 Kinesis ファミリー

15.2.1 Kinesis Data Streams

シャードベースのリアルタイムデータストリーム。

- ・ **シャード**：1 シャードあたり書き込み 1MB/秒 または 1,000 records/秒、読み出し 2MB/秒
- ・ **保持期間**：標準 24 時間、最大 365 日
- ・ **Producer**：KPL、SDK、Kinesis Agent、Firehose 連携
- ・ **Consumer**：KCL、Lambda、Firehose、Flink、Glue
- ・ **拡張ファンアウト（Enhanced Fan-Out）**：コンシューマあたり 2MB/秒の専用スループット
- ・ **On-Demand モード**：シャード管理不要、自動スケール

15.2.2 Kinesis Data Firehose

ストリームを S3 / Redshift / OpenSearch / Splunk / HTTP にバッファリングしながら配信するマネージドサービス。

- バッファ条件（サイズ・時間）でまとめて配信
- **Lambda 変換**：配信前にレコードを加工
- **動的パーティショニング**：S3 配信時にレコード内容で `year=/month=/day=/` パスを動的決定
- フォーマット変換：JSON → Parquet / ORC（Glue カタログ参照）
- ほぼゼロ運用、バッファサイズと配信先だけ決めれば動く

15.2.3 Kinesis Data Analytics for Apache Flink

旧 KDA SQL は廃止予定。現行は **Managed Service for Apache Flink**。Flink ジョブをマネージドで実行する。

- ステートフルなストリーム処理（ウィンドウ集計、CEP、結合）
- Java / Scala / Python（PyFlink）
- **Studio Notebooks**：Zeppelin で対話的開発

15.2.4 Kinesis Video Streams

動画ストリームの取り込み・保管・再生・分析。Rekognition Video や ML パイプラインと連携。IoT カメラ・監視・スマートホームに。

15.3 MSK（Managed Streaming for Kafka）

フルマネージド Apache Kafka。既存 Kafka エコシステム（Kafka Connect、Schema Registry、ksqlDB、Debezium 等）を使うなら MSK。

15.3.1 MSK Provisioned vs MSK Serverless

項目	Provisioned	Serverless
ブローカー管理	サイズ・台数を指定	自動
スケール	手動 or 設定変更	自動
認証	TLS / SASL / IAM	IAM 必須
向く規模	常時高負荷	スパイク・低頻度

15.3.2 Kinesis Data Streams との使い分け

- **Kafka エコシステムが必要** → MSK
- **AWS ネイティブで完結** → Kinesis Data Streams
- **スループット要件が極端に高い** → どちらも対応可、コスト計算で比較
- **Lambda 統合のシンプルさ** → Kinesis Data Streams が一段楽

15.4 S3 をデータレイクの中心に

15.4.1 Medallion アーキテクチャ

- **Bronze**：生データ（取り込んだまま、変換なし）
- **Silver**：クレンジング・正規化済み
- **Gold**：ビジネス用集計・分析向け

レイヤごとに S3 プレフィクス（`s3://lake/bronze/`、`silver/`、`gold/`）を分ける。

15.4.2 ファイルフォーマット

フォーマット	特性
JSON / CSV	人間可読、互換性高、サイズ大
Parquet	列指向、圧縮効率高、Athena/Spark の標準
ORC	列指向、Hive エコシステム
Avro	行指向、スキーマ進化に強い、Kafka との相性

データレイクは **Parquet + Snappy 圧縮** が定番。

15.4.3 パーティショニング

```
s3://lake/events/year=2026/month=04/day=21/hour=15/file.parquet
```

Hive 形式パスにすることで、Athena / Glue / Spark がパーティションを認識し、不要なパーティションをスキャンしないため **コストとパフォーマンス両方が改善** する。

15.4.4 Iceberg / Hudi / Delta Lake

行レベルの更新・削除、トランザクション、Time Travel、スキーマ進化を S3 上で実現する **テーブルフォーマット**。

- **Apache Iceberg**：AWS が公式に推す。Athena / Glue / EMR / Redshift Spectrum でサポート
- **Apache Hudi**：レコード単位の更新が得意
- **Delta Lake**：Databricks 主導。EMR / Glue でサポート

15.4.5 S3 Tables（2024 年末 GA）

S3 が **Iceberg ネイティブ テーブル** をマネージドで提供する新機能。

- メタデータ、コンパクション、スナップショット管理を AWS 側で
- 通常の S3 General Purpose よりクエリ性能が大幅に向上
- 標準 S3 テーブルバケット、料金は別体系

15.5 AWS Glue

サーバーレスな ETL／メタデータ統合プラットフォーム。

15.5.1 Glue Data Catalog

Hive メタストア互換の **中央メタデータ**。

- データベース → テーブル → パーティション の階層
- Athena / Redshift Spectrum / EMR / Glue ETL から共有参照
- カラム統計、Iceberg メタデータ管理
- **Lake Formation** と連携してきめ細かなアクセス制御

15.5.2 Glue Crawler

S3 上のデータをスキャンしてスキーマ推定 → Data Catalog に登録。新しいパーティションも検出。スケジュール起動も可。

15.5.3 Glue ETL

PySpark / Scala / Python Shell / Ray でジョブを書ける。

- **DynamicFrame** : Spark の DataFrame 拡張、スキーマ揺れに強い
- **Glue Studio** : GUI で ETL パイプラインを設計
- **ジョブブックマーク** : 処理済みデータを記憶し、増分処理を実現
- **ワーカータイプ** : G.1X、G.2X、G.4X、G.8X

```
import sys
from awsglue.transforms import *
from awsglue.context import GlueContext
from pyspark.context import SparkContext

sc = SparkContext()
glueContext = GlueContext(sc)

src = glueContext.create_dynamic_frame.from_catalog(
    database="raw", table_name="events"
)
clean = src.drop_fields(["password"]).resolve_choice(specs=[("amount", "cast:double")])
glueContext.write_dynamic_frame.from_options(
    frame=clean,
    connection_type="s3",
    connection_options={"path": "s3://lake/silver/events/", "partitionKeys":
["year", "month", "day"]},
    format="parquet"
)
```

15.5.4 Glue DataBrew

ノーコードのデータ準備 GUI。300 以上の変換を組み合わせる。

15.5.5 Glue Workflows

複数ジョブ・クローラを **DAG** で連結。Step Functions / EventBridge で代替も可。

15.6 Athena

S3 上のデータに **SQL** を実行するサーバーレスクエリ。

- ・ エンジン：Presto / Trino ベース
- ・ フォーマット：CSV / JSON / Parquet / ORC / Avro / Iceberg / Hudi / Delta
- ・ 料金：スキャン量（\$5/TB）または **Provisioned Capacity**（一定の DPU 確保）
- ・ CTAS（Create Table As Select）：結果を別の S3 + Glue テーブルとして保存
- ・ INSERT：既存テーブルへの追記
- ・ Federated Query：DynamoDB、RDS、Snowflake、HBase 等に SQL でアクセス
- ・ Athena for Apache Spark：PySpark ノートブック
- ・ Workgroup：チームごとにクエリ履歴・コスト・データ範囲を分離

15.6.1 パフォーマンス&コスト最適化

- ・ 列指向（Parquet/ORC）：必要列だけスキャン
- ・ 圧縮：Snappy / Zstd
- ・ パーティション：WHERE 句で絞り込んで不要パーティションをスキップ
- ・ パーティション射影（Projection）：Glue にパーティション登録なしでも高速
- ・ 小ファイル問題：Iceberg コンパクション、Glue ジョブで集約
- ・ Bloom Filter：高選択性カラムに付ける
- ・ 結果再利用（Result Reuse）：直近の結果をキャッシュ

15.7 Redshift

列指向 MPP（Massively Parallel Processing）データウェアハウス。

15.7.1 Provisioned vs Serverless

項目	Provisioned	Serverless
ノード	ra3.xlplus、ra3.4xlarge 等 を選択	選択不要
スケール	手動 or 自動	自動
課金	ノード時間	RPU 時間（必要時のみ）
向き	常時稼働 BI	スパイク・PoC

15.7.2 Redshift Managed Storage

ストレージとコンピュー트가分離されており、データ量と計算リソースを独立にスケール。

15.7.3 Redshift Spectrum

S3 のデータに Redshift から直接 SQL。Athena と似ているが、Redshift の DWH と組み合わせて **レイクハウス** を構成する用途。

15.7.4 Zero-ETL 統合

Aurora / RDS for MySQL / DynamoDB → Redshift への **リアルタイム複製**。ETL ジョブを書かずに DWH 側で分析できる。2024 年～順次拡大。

15.7.5 マテリアライズドビューとデータ共有

- **Materialized View**：頻出クエリの事前集計、自動リフレッシュ
- **Data Sharing**：複数 Redshift クラスタ間でテーブルをコピーなしで共有

15.7.6 Concurrency Scaling と RA3

ピーク時に追加クラスタを自動スピニアップ。料金は 1 日 1 時間まで無料相当（クレジット）。

15.8 OpenSearch Service

Elasticsearch / OpenSearch をマネージドで。

15.8.1 Provisioned vs Serverless

項目	Provisioned	Serverless
ノード	選択	自動
スケール	手動 or UltraWarm	自動
向き	常時稼働ログ基盤	スパイク・短期分析

15.8.2 主要ユースケース

- **ログ分析**：CloudWatch Logs / Firehose / Logstash → OpenSearch → Dashboards
- **全文検索**：商品検索、サポート FAQ
- **ベクトル検索**：KNN プラグインで近傍検索（RAG 用途）

15.8.3 OpenSearch Ingestion

旧 Data Prepper のマネージド版。Source → Processor → Sink のパイプラインで OpenSearch に取り込み。

15.8.4 UltraWarm / Cold

ホット（SSD）→ ウォーム（S3 + キャッシュ）→ コールド（S3 のみ）と自動移行。長期ログ保管のコスト削減。

15.9 QuickSight

サーバーレス BI / ダッシュボード。

- **SPICE**：インメモリ列指向エンジン
- 行・列レベルセキュリティ、データセットでのフィルタ
- 埋め込みダッシュボード（自社サービスへ組み込み）

- Q：自然言語クエリ
- **Generative BI**：プロンプトからダッシュボード自動生成
- 料金：作成者（Author）と閲覧者（Reader）で異なる

データソース：Redshift、Athena、RDS、S3、SaaS（Salesforce 等）、JDBC 接続。

15.10 EMR (Elastic MapReduce)

Hadoop / Spark / Hive / Presto / Trino / Flink / HBase 等の OSS クラスタをマネージドで。

15.10.1 形態

- **EMR on EC2**：従来形。クラスタを EC2 で
- **EMR on EKS**：Kubernetes 上で実行
- **EMR Serverless**：クラスタ管理ゼロ。Spark / Hive を必要時のみ
- **EMR on Outposts**：オンプレ環境

15.10.2 適用シーン

- 既存 OSS Spark / Hive 資産の移行
- Glue より細かい制御や非標準フレームワーク
- 大規模バッチ ETL、ML 訓練前処理

Glue で済むなら Glue、独自要件があるなら EMR。

15.11 Lake Formation と DataZone

15.11.1 Lake Formation

データレイク（S3 + Glue Catalog）の **きめ細かなアクセス制御**。テーブル・列・行・タグベースで権限を付与し、Athena / Redshift Spectrum / EMR / Glue ETL に統一適用。

15.11.2 DataZone

データカタログ＋ガバナンス＋協業のポータル。「データプロデューサ／コンシューマ」モデルで、ドメインごとにデータ製品を公開・申請・承認する仕組み。

15.12 典型アーキテクチャ

15.12.1 リアルタイムログ分析

```
App → CloudWatch Logs → Subscription Filter → Firehose → S3
                                          → OpenSearch
                                          → Lambda（フィルタ・整形）
→ Athena / OpenSearch Dashboards
```

15.12.2 バッチ ETL

```
App → S3 (Bronze) → Glue Crawler → Catalog
                        → Glue ETL (PySpark) → S3 (Silver/Gold)
                                                → Redshift
                                                    → Athena → QuickSight
```

15.12.3 リアルタイムダッシュボード

```
IoT / App → Kinesis Data Streams → Managed Flink
                                           → DynamoDB / OpenSearch
                                           → QuickSight ダッシュボード
```

15.12.4 レイクハウス

```
S3 + Iceberg ↔ Athena
                ↔ Redshift Spectrum
                ↔ EMR Spark
                ↔ Glue ETL
                ↔ Lake Formation (権限)
```

15.13 コスト最適化のヒント

- **Athena** : パーティション、列指向、圧縮、結果再利用、ワークグループの上限
- **Redshift** : Serverless で開発、Provisioned + Reserved Instance で本番
- **Glue** : DPU 数を必要最小、ジョブブックマークで増分処理
- **Kinesis** : On-Demand と Provisioned のクロスオーバーを試算
- **OpenSearch** : UltraWarm / Cold の活用、不要インデックス削除
- **Firehose** : バッファサイズを調整して S3 PUT リクエスト数削減

15.14 よくあるトラブル

症状	対処
Athena が遅い・高い	パーティション、Parquet、CTAS、結果サイズ確認
Glue ジョブ OOM	DPU 数、Worker サイズ、partition、broadcast join 検討
Kinesis でホットシャード	パーティションキー設計の見直し、シャード分割
Firehose 配信失敗	IAM ロール、宛先側容量、KMS 権限
Redshift がスロー	VACUUM/ANALYZE、ディストリビューションキー設計、Concurrency Scaling
OpenSearch の クラスタ Yellow/Red	シャード数、ディスク満杯、ノード追加

症状	対処
QuickSight でデータが見えない	権限、データセット更新、SPICE 容量
Iceberg テーブルが膨らむ	Snapshot expiration、Compaction の定期実行

16 機械学習と生成 AI

AWS の AI/ML スタックは「既製 API」「ML プラットフォーム」「基盤モデル / インフラ」の 3 層に分かれる。本章はこの順で扱い、最後に RAG・MLOps などの典型構成と費用上の注意をまとめる。

16.1 AWS AI/ML スタックの 3 層

層	提供形態	主な利用者
AI Services	既製モデルを REST/SDK で呼ぶだけ	アプリ開発者
ML Services	SageMaker：訓練・デプロイ・MLOps の総合	データサイエンティスト
ML Frameworks & Infra	Trainium / Inferentia / GPU 等の専用チップ・EC2	研究者・基盤チーム

加えて **Amazon Bedrock**（基盤モデルの統合 API）と **Amazon Q**（生成 AI アシスタント）が独立した重要サービスとして位置づけられる。

16.2 既製 AI サービス

「ML を学ばずに AI 機能をアプリに足す」用途。API キー 1 つで使える。

サービス	用途
Rekognition	画像・動画認識（顔・物体・テキスト・モデレーション）
Transcribe	音声 → テキスト（多言語、リアルタイム、医療・通話特化）
Polly	テキスト → 音声（Neural / 生成系 TTS）
Translate	機械翻訳（カスタム用語集、リアルタイム）
Comprehend	テキスト分析（言語・感情・エンティティ・キーフレーズ・PII）
Comprehend Medical	医療テキスト解析
Textract	文書 OCR + 表・フォーム抽出
Personalize	リアルタイムレコメンド
Forecast	時系列予測（廃止予定、SageMaker Canvas に集約方向）
Lex	会話 Bot（Connect と統合）
Kendra	エンタープライズ全文検索（SaaS コネクタ多数）
Fraud Detector	不正検知モデルの学習・推論

すべて **リージョンに制約** がある。東京で利用可能か、データは越境していないかを必ず確認する。

16.2.1 簡単な使用例（Rekognition）

```
aws rekognition detect-labels \
  --image '{"S3Object":{"Bucket":"my-images","Name":"sample.jpg"}}' \
  --max-labels 10
```

```
import boto3
client = boto3.client('rekognition', region_name='ap-northeast-1')
resp = client.detect_text(Image={'S3Object':{'Bucket':'my-images','Name':'menu.jpg'}})
for d in resp['TextDetections']:
    print(d['DetectedText'])
```

16.3 Amazon Bedrock

複数ベンダの 基盤モデル（Foundation Model, FM）を 統一 API で呼べるサーバーレス基盤。

16.3.1 利用可能な主要モデル系列

ベンダー	モデル系列
Anthropic	Claude 4 Opus / Sonnet / Haiku（推奨）。指示追従と長文・コードに強い
Amazon	Nova（Premier/Pro/Lite/Micro）、Titan（テキスト・画像・埋め込み）
Meta	Llama 4 / Llama 3 系。OSS 系の柔軟性
Mistral	Mistral Large、Mistral Small。欧州系
Cohere	Command / Embed
Stability AI	Stable Diffusion（画像生成）
AI21 Labs	Jamba

モデル提供状況とリージョンは頻繁に変わる。コンソールの「Model access」で最新確認。

16.3.2 Bedrock の主要機能

16.3.2.1 Knowledge Bases（RAG）

S3 上のドキュメントを取り込み、自動チャンク化 → 埋め込み生成 → ベクトル DB（OpenSearch Serverless / Aurora pgvector / Pinecone / MongoDB Atlas）に格納し、**Retrieve and Generate API** で 1 コール RAG を実現。

```
import boto3
br = boto3.client('bedrock-agent-runtime')
resp = br.retrieve_and_generate(
    input={'text': '弊社の有給休暇規定を教えて'},
    retrieveAndGenerateConfiguration={
        'type': 'KNOWLEDGE_BASE',
        'knowledgeBaseConfiguration': {
            'knowledgeBaseId': 'KB123456',
            'modelArn': 'arn:aws:bedrock:ap-northeast-1::foundation-model/anthropic.claude-sonnet-4-6-20251201-v1:0',
        }
    }
)
```



```
)
print(resp['output']['text'])
```

16.3.2.2 Agents

ツール呼び出し (Action Group) と **Knowledge Base** を組み合わせて、ReAct 風のエージェントを宣言的に作る。Lambda がツール実装、OpenAPI スキーマでツールを定義。

16.3.2.3 Guardrails

入出力に対して **安全性フィルタ** を適用。トピック禁止、PII マスキング、有害コンテンツ検出、プロンプトインジェクション対策。

16.3.2.4 Model Evaluation

複数モデル・複数プロンプトの自動評価。ヒューマン評価 (自社ワーカー / AWS マネージドワーカー) と自動メトリクスの両方をサポート。

16.3.2.5 Prompt Management と Flows

プロンプトのバージョン管理 (Prompt Management) と、複数モデル・複数プロンプト・複数ツールを **視覚的に組み合わせる** Flows。

16.3.2.6 Custom Model Import / Provisioned Throughput

- **Custom Model Import** : 自社で用意したモデル (Llama 互換、Mistral 互換等) を Bedrock の API で呼べるようにインポート
- **Provisioned Throughput** : モデル容量を時間予約してスループットを保証。大量バッチ推論や SLA が必要な本番で

16.3.3 Bedrock の料金モデル

- **On-Demand** : トークン単位課金。少量・スパイク向け
- **Provisioned Throughput** : 時間単位、モデル単位
- **バッチ推論** : On-Demand の 50% 程度

16.3.4 モデル選択の実践指針

- **軽量・高速・安価** : Claude Haiku、Nova Micro/Lite、Llama 系の小型
- **バランス** : Claude Sonnet、Nova Pro、Mistral Large
- **最高品質** : Claude Opus、Nova Premier
- **画像生成** : Stable Diffusion、Nova Canvas、Titan Image Generator
- **埋め込み** : Titan Embed、Cohere Embed

迷ったら **Claude Sonnet 系** がコストと品質のバランスが良い。

16.4 Amazon Q

生成 AI を **エンドユーザー向けアシスタント** として提供する一連のサービス。

プロダクト	用途
Amazon Q Developer	開発者向け。IDE、コンソール、CLI 連携。コード生成・診断・テスト生成・トランスフォーム
Amazon Q Business	社内ナレッジ統合チャット。100 以上の SaaS コネクタ、RAG
Amazon Q in QuickSight	BI への自然言語問い合わせ・可視化生成
Amazon Q in Connect	コンタクトセンターのリアルタイムアシスト
Amazon Q Apps	社内向けノーコード生成 AI アプリ

エンタープライズで「とにかく社内データに RAG したい」という需要には Bedrock を自前で組むより Q Business が早い。

16.5 SageMaker

ML プラットフォーム。訓練・デプロイ・MLOps を統合。

16.5.1 コンポーネント早見表

機能	用途
Studio	ML 開発の統合 IDE (JupyterLab + Code Editor)
Notebooks	マネージド Jupyter (Studio Notebooks 推奨)
Training Jobs	マネージド学習。分散・Spot・分散ハイパラ
Hyperparameter Tuning	自動ハイパーパラメータ探索
Inference	リアルタイム / Serverless / 非同期 / バッチ変換
Endpoints	マルチモデル、マルチコンテナ、シャドウ、Auto Scaling
Pipelines	MLOps の DAG (前処理→訓練→評価→登録→デプロイ)
Model Registry	モデルバージョン管理・承認フロー
Feature Store	オンライン／オフライン特徴量ストア
Ground Truth	データラベリング (人手・マネージドワーカー)
Clarify	バイアス検出・モデル説明可能性 (SHAP)
Model Monitor	本番モデルのデータドリフト・モデルドリフト監視
Canvas	ノーコード ML (タブラ、時系列、画像)
JumpStart	事前学習モデル・ソリューションテンプレート
HyperPod	大規模分散学習向けのレジリエントなクラスタ
Unified Studio	SageMaker・Bedrock・Glue・EMR・Redshift を 1 つの UI に統合

16.5.2 最小ワークフロー

```
import sagemaker
from sagemaker.sklearn.estimator import SKLearn
```

```

sess = sagemaker.Session()
role = sagemaker.get_execution_role()

est = SKLearn(
    entry_point='train.py',
    role=role,
    instance_type='ml.m5.large',
    framework_version='1.2-1',
    hyperparameters={'max_depth': 5, 'n_estimators': 100},
)
est.fit({'train': 's3://my-bucket/train.csv', 'val': 's3://my-bucket/val.csv'})

predictor = est.deploy(initial_instance_count=1, instance_type='ml.m5.large')
print(predictor.predict([[1.0, 2.0, 3.0]]))

# 後片付け（重要：エンドポイントは時間課金）
predictor.delete_endpoint()

```

16.5.3 推論方式の使い分け

方式	向く用途
Real-time Endpoint	低レイテンシ、定常トラフィック
Serverless Inference	スパイク・低頻度・ピーク不定
Asynchronous Inference	大きな入力・長い処理時間（最大 1 時間）
Batch Transform	大量データを一度に推論

16.5.4 Pipelines + Model Registry

```

from sagemaker.workflow.pipeline import Pipeline
from sagemaker.workflow.steps import ProcessingStep, TrainingStep
from sagemaker.workflow.step_collections import RegisterModel

pre = ProcessingStep(name='Preprocess', ...)
train = TrainingStep(name='Train', ...)
register = RegisterModel(name='Register', model_package_group_name='my-model', ...)

pipeline = Pipeline(name='ml-pipeline', steps=[pre, train, register])
pipeline.upsert(role_arn=role)
pipeline.start()

```

CodePipeline からこの SageMaker Pipeline を起動して、CI/CD と統合する。

16.5.5 Feature Store

オンライン（低レイテンシ KV）とオフライン（S3 + Glue Catalog）の二段構造。訓練と推論で **同じ特徴量** を保証することがゴール（訓練／推論スキューの防止）。

16.5.6 JumpStart

数百の事前学習モデル・ソリューションテンプレートをワンクリックでデプロイ。LLM のホスティング、画像分類、表形式分類、時系列、レコメンドなど。最近では **ファインチューニング機能** も統合され、Bedrock のカスタムモデルとの線引きが微妙になっている（JumpStart は SageMaker エンドポイントが自分のもの、Bedrock はマネージド API）。

16.5.7 HyperPod

大規模 LLM の分散訓練向け。ノード障害時の自動復旧、SLURM ベース、Trainium 統合。最先端モデルの研究開発で使われる。

16.6 Trainium / Inferentia

AWS 独自の ML 専用チップ。

チップ	用途
Trainium 1 (Trn1)	大規模学習。GPU 比 50% 安
Trainium 2 (Trn2)	Trn1 の 4 倍性能。最大数万ノード分散
Inferentia 1 (Inf1)	推論。低コスト
Inferentia 2 (Inf2)	大規模 LLM 推論。Inf1 の数十倍性能

利用には **AWS Neuron SDK** (PyTorch / TensorFlow / JAX 互換のコンパイラ) が必要。コスト重視の本番推論で導入する。

16.7 ベクトル検索の選択肢

RAG で必須となるベクトル DB は AWS でもいくつか選べる。

選択肢	特徴
OpenSearch Service KNN	既存ログ基盤と統合、フィルタ豊富
OpenSearch Serverless (Vector Engine)	Bedrock Knowledge Bases の標準
Aurora PostgreSQL pgvector	既存 RDBMS と一体管理、SQL JOIN 可
DocumentDB ベクトル	MongoDB 互換
Neptune Analytics	グラフ + ベクトル
Kendra	全文検索エンジン側で意味検索
外部 (Pinecone / Weaviate / MongoDB Atlas)	PrivateLink 経由で利用可

迷ったら **Bedrock Knowledge Bases + OpenSearch Serverless** が最短。RDBMS 中心なら pgvector。

16.8 典型的な構成

16.8.1 RAG (Retrieval-Augmented Generation)

```
[文書 S3] → Knowledge Base 取り込み → 埋め込み生成 → OpenSearch Serverless
[User] → API Gateway → Lambda → Bedrock RetrieveAndGenerate → Claude
                                     ← 回答 + 引用
```

16.8.2 バッチ推論パイプライン

```
[新規データ S3] → EventBridge → SageMaker Batch Transform → 推論結果 S3
                                                         → SNS / Slack 通知
```

16.8.3 リアルタイム推論

```
[Client] → API Gateway → Lambda → SageMaker Real-time Endpoint → Result
                                     ↓
                               Feature Store オンライン取得
```

16.8.4 MLOps (CI/CD)

```
[Git push] → CodePipeline →
    • コードビルド
    • ユニットテスト
    • SageMaker Pipelines (前処理→訓練→評価)
    • Model Registry に登録
    • 人手承認
    • Endpoint デプロイ (カナリア)
```

16.8.5 カスタム LLM

- 軽い適応：Bedrock の Provisioned + プロンプトエンジニアリング
- 中程度：Bedrock Custom Model Import or SageMaker JumpStart の LoRA
- 重め：SageMaker HyperPod + Trainium で Continued Pre-training

16.9 コストとリスクのよくある罠

罠	対策
SageMaker Notebook の停止忘れ	Lifecycle Configuration で自動シャットダウン、IDLE 検知
Real-time Endpoint の放置	使い終わったら必ず <code>delete_endpoint</code> 、CloudWatch アラーム
Bedrock のトークン爆発	Guardrails、最大トークン制限、入力プロンプトの圧縮
Bedrock の Provisioned 過剰	On-Demand で性能と頻度を測ってから判断

罣	対策
データ流出	VPC エンドポイント、Bedrock 入出力ログ、Guardrails の PII マスキング
モデルドリフト	Model Monitor、定期評価バッチ
プロンプトインジェクション	Guardrails、システムプロンプトのサニタイズ、出力パース時の検証

16.10 学び始めの推奨パス

1. 既製 AI を 1 つ呼んでみる（Rekognition で画像認識、Translate で翻訳）
2. **Bedrock** で Claude を呼ぶ最小コード（boto3 で `invoke_model`）
3. Bedrock Knowledge Base で **社内ドキュメント RAG** を 1 本作る
4. SageMaker JumpStart で事前学習モデルを **エンドポイント化**
5. 必要に応じて SageMaker Pipelines で **MLOps** に踏み込む

最初から SageMaker フルスタックを組まなくてよい。多くのケースで **Bedrock + 既製 API + RAG** で要件が満たせる。

17 認証サービスの詳細

3章ではIAM そのものを扱った。本章はIAM を補完・拡張する周辺サービス、特に **Cognito** (自社アプリのエンドユーザー)、**IAM Identity Center** 詳細、**Directory Service**、**Verified Permissions**、**Resource Access Manager**、**IAM Roles Anywhere** を扱う。

17.1 アイデンティティの種類と AWS の選択肢

アイデンティティ種別	AWS の選択肢
AWS リソースを操作する人間	IAM Identity Center (推奨) or IAM ユーザー
マシン・ワークロード	IAM Role + 一時認証情報、IAM Roles Anywhere
自社アプリのエンドユーザー	Cognito User Pools
自社アプリ → AWS リソース利用	Cognito Identity Pools (フェデレーション)
社員 (社内 IdP)	IAM Identity Center + 外部 IdP 連携
オンプレ AD 統合	Directory Service / AD Connector
きめ細かなアプリ内認可	Verified Permissions

「人間は Identity Center、マシンは Role、エンドユーザーは Cognito」が現代の指針。

17.2 IAM Identity Center 詳細

2章では概要を、本節は実運用の細部を扱う。

17.2.1 アイデンティティソース

3つの中から1つを選ぶ。

ソース	用途
Identity Center directory	Identity Center 自前のディレクトリ。小～中規模
Active Directory	既存 AD (オンプレ or AWS Managed AD)
外部 IdP (SAML 2.0 / SCIM)	Microsoft Entra ID、Okta、Google Workspace、OneLogin、JumpCloud 等

外部 IdP 連携が現代企業の標準。SCIM プロビジョニングでユーザー・グループを自動同期し、SSO で SAML 認証する。

17.2.2 許可セット (Permission Sets)

許可セットは「IAM ロール+ポリシーの組み合わせを Identity Center 用にラップしたもの」と理解してよい。

- **AWS マネージドポリシーベース** : `AdministratorAccess`、`ReadOnlyAccess` を貼るだけ
- **カスタマー管理ポリシーベース** : 自前ポリシー名を指定 (各アカウント側にもポリシーが存在している前提)

- ・ カスタムインラインポリシー：許可セット内に直書き
- ・ Permissions Boundary 付き

許可セットを AWS アカウント × ユーザー／グループ に割り当てると、対象アカウントに自動的に `AWSReservedSSO_<許可セット名>_<ハッシュ>` というロールが作成される。

17.2.3 ABAC (Attribute-Based Access Control)

属性ベースアクセス制御。IdP 側のユーザー属性を **セッションタグ** として渡し、IAM ポリシーの `Condition` で評価する。

```
{
  "Effect": "Allow",
  "Action": "s3:*",
  "Resource": "arn:aws:s3:::project-*/*",
  "Condition": {
    "StringEquals": {
      "aws:ResourceTag/Project": "${aws:PrincipalTag/Project}"
    }
  }
}
```

「ユーザーの `Project` 属性と同じ値のリソースタグなら許可」というポリシー。アカウント・ロールを増やさずに、属性追加だけで権限管理が回る。

17.2.4 セッション管理

- ・ 許可セットごとにセッション期間（1～12 時間）
- ・ 強制サインアウト、リフレッシュトークン
- ・ CLI で `aws sso login` 後、`~/.aws/sso/cache/` に短期トークンが置かれる

17.2.5 監査

Identity Center 側の操作は CloudTrail に `eventSource = sso.amazonaws.com` で記録される。許可セット変更、ユーザー作成、SSO ログインなどを監視。

17.2.6 移行 (IAM ユーザーから Identity Center へ)

1. Organizations を有効化（未なら）
2. Identity Center を有効化（リージョン 1 つを選定）
3. アイデンティティソースを決定（最初は Identity Center directory で開始可）
4. 許可セットを作成（最低 `AdministratorAccess` と `ReadOnlyAccess`）
5. ユーザー／グループに割り当て
6. 既存 IAM ユーザーから順次切替、切替完了後にアクセスキー失効

17.3 Cognito

エンドユーザー向けのサインアップ・サインイン・認可基盤。User Pools と Identity Pools の 2 系統がある。

17.3.1 User Pools (ユーザーディレクトリ)

サインアップ・ログイン・MFA・パスワードポリシー・パスワードリセット・属性管理を一手に提供。

主な機能：

- サインアップ／サインイン（メール・電話番号・ユーザー名）
- メール／SMS の検証
- パスワードポリシー
- MFA（TOTP・SMS）、**passkey（WebAuthn）も対応**
- カスタム属性（`custom:tenant_id` など）
- **Lambda トリガー**：Pre Sign-up、Post Confirmation、Pre Token Generation、Custom Message など
- **Hosted UI**：AWS が提供するサインインページ（カスタマイズ可）
- ソーシャル連携：Google、Apple、Facebook、Amazon
- 企業 IdP 連携：SAML、OIDC
- **Advanced Security Features**：適応型認証、リスクスコア、漏洩 Cred 検知

17.3.2 Identity Pools (フェデレーション)

認証済みユーザー（Cognito UP・Google・Apple・Twitter・SAML / OIDC IdP）に AWS リソースアクセス用の一時認証情報を発行する。

ユースケース：モバイルアプリから **DynamoDB / S3 へ直接アクセス** したい場合に、ユーザーごとに細かく権限を切る。

17.3.3 User Pool と Identity Pool の関係

[Cognito User Pool] → JWT

↓

[Identity Pool] → AWS 一時認証情報 → S3 / DynamoDB

User Pool は「認証」、Identity Pool は「AWS リソースの認可」。両方使う構成も、User Pool だけ・Identity Pool だけの構成もありえる。

17.3.4 典型構成：SPA + API Gateway + Lambda

[Browser] → Cognito Hosted UI → JWT

[Browser] → API Gateway (JWT Authorizer) → Lambda → DynamoDB

最も多用される。API Gateway HTTP API の JWT オーソライザーで `Bearer <id_token>` を検証する。

17.3.5 マルチテナンシー設計

- **物理分離**：テナント = Cognito User Pool。完全分離だが管理コスト高
- **論理分離**：1つの User Pool でカスタム属性 `tenant_id` を保持、Pre Token Generation Lambda で JWT に埋め、ABAC でリソース分離

17.3.6 Cognito の料金感

- **MAU 課金**（無料枠：50,000 MAU/月）
- **Advanced Security Features** は別料金（MAU 単価が上がる）
- **M2M**（マシン間）クライアントクレデンシャルフロー：API コール課金

17.4 Directory Service

Active Directory 連携の 3 形態。

種類	用途
AWS Managed Microsoft AD	マネージド AD。本格的な Windows 統合
AD Connector	既存オンプレ AD への中継。AD のみ提供
Simple AD	Samba ベース。小規模・機能限定

利用例：

- **FSx for Windows File Server**：AD ユーザー認証
- **Workspaces**（仮想デスクトップ）：AD でユーザー管理
- **RDS for SQL Server**：Windows 認証
- **EC2 ドメイン参加**：Windows / Linux

オンプレ AD があるなら **AD Connector** で AWS 側からも参照、新規なら **Managed Microsoft AD**。

17.5 Verified Permissions

Cedar 言語 によるきめ細かな認可サービス。アプリケーション内の「誰が／どのリソースに／何ができるか」を中央管理する。

```
permit(
  principal in Group::"editors",
  action == Action::"updateDocument",
  resource
)
when {
  resource.owner == principal ||
  resource.collaborators.contains(principal)
};
```

- API Gateway の Lambda オートライザーから呼び出し
- アプリ内のチェックポイントから SDK で呼び出し

- **Open Source** な Cedar をローカルでも使え、Verified Permissions はマネージドホスティング層

ABAC が IAM レベルなら、Verified Permissions はアプリレベル。複雑な業務認可ロジックを **コード分散させない** のが利点。

17.6 Resource Access Manager (RAM)

アカウント間でリソースを共有する仕組み。

17.6.1 共有可能な代表リソース

- Subnet (VPC 共有)
- Transit Gateway
- License Manager License Configuration
- Resolver Rule
- Route 53 Profiles
- AWS Config Aggregator
- Aurora DB Cluster snapshot
- Glue Data Catalog database / table

Organizations 配下のアカウントには **自動承認** で共有可能。明示的に外部アカウントにも共有できる。

17.6.2 マルチアカウント・ネットワークの実例

中央のネットワーク管理アカウントで Transit Gateway と Subnet を作成し、RAM で各ワークロードアカウントに共有 → 各アカウントは「自分の VPC」として Subnet を見る。ネットワーク変更を中央集約できる。

17.7 IAM Roles Anywhere

オンプレ・他クラウドのワークロードに **X.509 証明書ベース** で AWS 一時認証情報を払い出す。

17.7.1 構成要素

- **Trust Anchor**：信頼する CA（プライベート CA か Customer CA）
- **Profile**：付与する IAM ロールと制約
- **credential-helper**：オンプレ側で証明書を提示し一時認証を取得するクライアント

```
# AWS CLI に credential_process として組み込む
[profile onprem-app]
credential_process = /opt/aws_signing_helper credential-process \
  --certificate /etc/pki/cert.pem \
  --private-key /etc/pki/key.pem \
  --trust-anchor-arn arn:aws:rolesanywhere:ap-northeast-1:123456789012:trust-anchor/
xxxx \
```

```
--profile-arn arn:aws:rolesanywhere:ap-northeast-1:123456789012:profile/yyyy \
--role-arn arn:aws:iam::123456789012:role/onprem-role
```

オンプレで 長期アクセスキーを持たずに AWS リソースを使えるのが最大の利点。

17.8 STS と一時認証情報

AWS Security Token Service (STS) が一時認証情報の発行を司る。

17.8.1 主要 API

API	用途
AssumeRole	別ロールに切り替え（自分 → 別アカウントロール、開発 → 本番ロール）
AssumeRoleWithWebIdentity	OIDC IdP (GitHub Actions、Cognito、Google) からロール引き受け
AssumeRoleWithSAML	SAML IdP からロール引き受け
GetSessionToken	MFA 強制つき同一プリンシパルの一時認証
GetFederationToken	既存 IAM ユーザーからフェデレーテッドユーザー作成

17.8.2 一時認証情報の有効期限

- 標準：15 分～12 時間（ロール側の MaxSessionDuration で制御）
- ロール連鎖（Role Chaining）：最大 1 時間
- IAM Identity Center 経由：許可セットで設定

17.8.3 Source Identity の伝播

`AssumeRole` 時に `SourceIdentity` を渡すと、後続のすべての API 呼び出しに引き継がれ、CloudTrail に記録される。「フェデレーション元の人間」をログから特定できる。

17.8.4 ロール連鎖の制約

A ロールから B ロールを Assume すると、B から C を Assume するときの最大期間が **1 時間に制限** される。長時間処理ではロール構造を見直す。

17.9 認証関連のベストプラクティス

- 人間は Identity Center、マシンは IAM Roles + STS、エンドユーザーは Cognito
- ルートユーザーには passkey/FIDO2、アクセスキーなし
- 外部 IdP (Entra/Okta/Google Workspace) と SCIM で自動同期
- ABAC で権限管理を属性ベースに（タグ／属性が増えても権限が爆発しない）
- フェデレーション (SAML / OIDC) を優先、長期キーは避ける
- MFA / passkey をすべてのプリンシパルに
- Cognito の Pre Token Generation Lambda は短く保つ（コールドスタートが UX を壊す）

- Verified Permissions でアプリ内認可ロジックを中央化

17.10 マルチアカウント運用とのつながり

Identity Center は Organizations と切っても切れない関係にある。マルチアカウント運用は 20 章で扱うが、認証の観点では「Identity Center を中央アカウントで有効化し、Organizations 配下の全アカウントへ許可セットを割り当てる」のが基本形。

17.11 よくあるトラブル

症状	対処
Cognito Hosted UI のドメイン取れない	グローバル一意性、AWS 既存ドメインとの衝突回避
SAML 連携でロール候補が出ない	Role Mapping の SAML attribute、 https://aws.amazon.com/SAML/Attributes/Role
SCIM プロビジョニングが進まない	属性マッピング、グループ階層、API トークン期限
Identity Center CLI が AccessDenied	<code>aws sso login</code> 再実行、許可セット再割り当て後はログイン更新必須
AssumeRole 失敗	信頼ポリシーの Principal、ExternalId、SourceIdentity 整合
Pre Token Generation で JWT 壊れる	Lambda の戻り値が claims を上書きしている、ペイロード超過
Cognito の `User does not exist`	サインインフロー (USER_PASSWORD_AUTH 等) の有効化、ユーザー存在検出設定
IAM Roles Anywhere 401	Trust Anchor の証明書チェーン、CRL、時刻ずれ

18 ネットワーク詳細

4 章では VPC 単体の基礎を、12 章では Route 53 と CloudFront を扱った。本章は **VPC 間・VPC ⇔ オンプレ・グローバル分散** など、より大規模なネットワーク領域を扱う。

18.1 大規模ネットワーク設計の選択肢

シナリオ	主な選択肢
VPC 2～数個	VPC ピアリング
VPC 多数 / 複数アカウント	Transit Gateway
グローバル統合管理	Cloud WAN
オンプレ ⇔ AWS（簡易）	Site-to-Site VPN
オンプレ ⇔ AWS（高品質・高帯域）	Direct Connect
リモートワーク端末	Client VPN
サービス間アプリ通信	VPC Lattice、PrivateLink
グローバル低レイテンシ	Global Accelerator

18.2 Transit Gateway

VPC・VPN・Direct Connect Gateway をハブ&スポークで接続するリージョナルサービス。VPC ピアリングの「推移的ルーティング不可」「N×N の管理コスト」を解決する。

18.2.1 アタッチメントとルーティング

- ・ **アタッチメント**：VPC、VPN、Direct Connect Gateway、Transit Gateway Peering、Connect（GRE 経由 SD-WAN）
- ・ **ルートテーブル**：複数作成し、アタッチメントごとに「使うルートテーブル」と「伝播するルートテーブル」を選ぶ
- ・ **関連付け（Association）**：そのアタッチメントが参照するルートテーブル
- ・ **伝播（Propagation）**：そのアタッチメントのルートを掲載するルートテーブル

18.2.2 セグメンテーションパターン

「Prod 同士は通信、Dev 同士は通信、Prod ⇔ Dev は不可」という設計：

- ・ ルートテーブル `tgw-rt-prod` と `tgw-rt-dev` を作る
- ・ Prod の VPC は `tgw-rt-prod` に関連付け、`tgw-rt-prod` にだけ伝播
- ・ Dev も同様
- ・ Prod ⇔ Dev のルートは存在しないので通信不可

18.2.3 マルチリージョン

Inter-Region Peering : 別リージョンの Transit Gateway とピアリングし、AWS バックボーン経由で接続。データ転送は AWS 内ネットワーク。

18.2.4 料金感

- アタッチメント時間課金 : \$0.07/時/アタッチメント (東京)
- データ処理 : \$0.02/GB
- アタッチメントが 10 個あれば月 \$500 を超える固定費。検証で立てっぱなしは厳禁

18.3 Cloud WAN

グローバルネットワーク全体 を宣言的に管理するメタ層。Transit Gateway を内包し、ポリシーベースで複数リージョン・複数アカウントの接続を一元化する。

18.3.1 コア概念

- **Core Network** : グローバル単一論理ネットワーク
- **Network Policy** : JSON ベースで定義 (リージョン、セグメント、共有、ルーティング)
- **Segment** : 論理的なネットワーク区切り (≒ Transit Gateway のルートテーブル相当)
- **Attachment** : VPC、VPN、Direct Connect Gateway、Transit Gateway Connect

18.3.2 いつ Cloud WAN、いつ Transit Gateway か

- 単一リージョン or 数個まで → Transit Gateway
- 3+リージョンを統合管理したい、ポリシーで自動化したい → Cloud WAN

18.4 Direct Connect

専用線で AWS に接続。1 Gbps~400 Gbps、専有 / 共有。

18.4.1 種類

- **Dedicated Connection** : 物理ポートを丸ごと借りる (1/10/100/400 Gbps)
- **Hosted Connection** : パートナー経由で割り当てられた論理線 (50 Mbps~10 Gbps)

18.4.2 Virtual Interface (VIF)

- **Private VIF** : 自社 VPC への接続 (VGW or Direct Connect Gateway 経由)
- **Public VIF** : AWS パブリックサービス (S3、DynamoDB 等) への直接到達
- **Transit VIF** : Direct Connect Gateway → Transit Gateway 経由で複数 VPC に分配

18.4.3 Direct Connect Gateway

複数 VPC・複数リージョンに専用線を分配する論理ゲートウェイ。

18.4.4 MACsec 暗号化

10/100 Gbps ポートで MACsec (L2 暗号化) に対応。金融・医療等で要求される場合に。

18.4.5 冗長化

- 1 拠点 + 1 接続：SLA なし
- 1 拠点 + 2 接続：SLA 99.9%
- 2 拠点 + 2 接続：SLA 99.99%

本番では 2 拠点以上に分散する。バックアップとして Site-to-Site VPN を併用するのも一般的。

18.4.6 いつ VPN ではなく Direct Connect か

- 帯域 1 Gbps 以上の安定要件
- レイテンシが安定して低い必要がある
- 大量データ転送 (VPN 経由は転送料金が低い)
- セキュリティ・コンプライアンス上の要請

18.5 Site-to-Site VPN

IPsec で AWS とオンプレを接続。

- Customer Gateway (オンプレ側ルータ・FW) と AWS 側 VGW or Transit Gateway
- 1 接続あたり 2 トンネル (同一 BGP 配下、片系障害時に自動フェイルオーバー)
- Static Route or BGP
- **Accelerated Site-to-Site VPN**：Global Accelerator 経由でレイテンシ改善
- 帯域の上限：1 トンネル 1.25 Gbps (合計 2.5 Gbps)。それ以上は複数トンネル束ね

設定が比較的簡単で、料金も Direct Connect より安いので、まず VPN で始めて要件に応じて Direct Connect に進むのが定番。

18.6 Client VPN

リモートワーク端末向け OpenVPN ベース。

18.6.1 認証方式

- 相互認証 (証明書ベース)
- **Active Directory** (AD Connector or Managed AD)
- SAML 2.0 (外部 IdP)

18.6.2 接続フロー

ユーザー → AWS Client VPN Endpoint → ENI → ターゲット (VPC、VPC 間、オンプレ)

接続あたり時間課金 + アクティブ接続時間課金。長時間ずっとつながっぱなしは高くつくため、ゼロトラスト系のソリューション (AWS Verified Access) も検討。

18.7 AWS Verified Access

社内アプリへの VPN レス アクセスを実現するゼロトラストサービス。Identity Center / 外部 IdP からの認証 + デバイスポスチャ (Jamf / CrowdStrike 等の連携) でアクセス可否を判定。アプリは ALB / NLB / Endpoint Groups で公開。

18.8 Global Accelerator

AWS グローバルネットワークを **Anycast IP** で利用するアクセラレーター。

- 静的アニーキャスト IP × 2 (複数リージョン宛先のフロントとして)
- AWS バックボーン経由で最寄り PoP からエンドポイントへ
- 自動フェイルオーバー、ヘルスチェック
- Custom Routing (多人数のクライアントを特定 EC2 ヘバインド)

18.8.1 CloudFront との違い

- **CloudFront** : HTTP/HTTPS、キャッシュあり、ウェブ向け
 - **Global Accelerator** : TCP/UDP 任意、キャッシュなし、ゲーム / IoT / 独自プロトコル / API も可
- 両者を **併用** することもできる : CloudFront → ALB (オリジン) を Global Accelerator 経由にして、ALB へのレイテンシも安定させる。

18.9 VPC Lattice

VPC・アカウントをまたぐ **アプリケーション層のネットワーキング**。Service Mesh 的な機能をマネージドで。

18.9.1 コンセプト

- **Service Network** : 論理的なネットワーク
- **Service** : 1 つの実サービス (ALB / NLB / Lambda / EC2 を背後に)
- **Listener / Rule / Target Group** : Path / Header ルーティング
- **Auth Policy** : IAM ベース認可

VPC ピアリングや Transit Gateway を使わずに、**アプリケーション単位**でサービスを共有する。マイクロサービスのインターネット代替として注目されている。

18.10 PrivateLink 詳細

4 章で簡単に触れたが、PrivateLink は NLB (または GWLB) の前段に **Endpoint Service** を作り、別アカウントの Interface Endpoint からプライベート IP でアクセスする 仕組み。

18.10.1 主要ユースケース

- AWS マネージドサービス (KMS、SQS、Secrets Manager 等) への VPC 内アクセス
- 自社 SaaS の他アカウントへの提供 (PrivateLink 経由でテナントに公開)
- 他ベンダ SaaS (Snowflake、Datadog 等) の VPC 内取り込み

18.10.2 Cross-Region Endpoints

PrivateLink が リージョンをまたぐ 形でも提供できるようになり、グローバル SaaS との統合が楽に。

18.11 Network Firewall

VPC 内の ステートフル FW。Suricata 互換ルール、IP set、ドメインリスト。

18.11.1 配置パターン

- 分散モデル：各 VPC 内に Network Firewall
- 中央集約モデル：Inspection VPC を作り、Transit Gateway でトラフィックを集約 → Network Firewall → 外向き

中央集約は管理が楽だがレイテンシ・コストが上がる。規制要件次第で選択。

18.11.2 Firewall Manager との連携

組織横断で Network Firewall ポリシーを一元管理。

18.12 Route 53 Resolver と DNS Firewall

12 章でも触れたが、ネットワーク詳細の文脈で再整理。

- Resolver Endpoint（インバウンド／アウトバウンド）：オンプレ ↔ VPC の名前解決
- Resolver Rule：条件付きフォワーディング（*.onprem.example.com はオンプレ DNS へ）
- DNS Firewall：ドメイン単位の許可・ブロック

18.13 IPv6 戦略

AWS は段階的に IPv6 化を進めている。

要素	IPv6 対応
VPC	デュアルスタック / IPv6 専用 / IPv4 専用
Subnet	デュアルスタック / IPv6 専用
EC2 / ENI	対応
ALB / NLB	デュアルスタック対応
RDS	デュアルスタックは限定。新世代エンジンから対応
S3	対応（デュアルスタック / IPv6 専用エンドポイント）
Egress-only IGW	IPv6 の外向き専用ゲートウェイ
課金	IPv6 アドレスは無料（IPv4 は 2024 年から課金）

新規構築では デュアルスタック または IPv6 専用 を検討する価値がある。IPv4 アドレス枯渇とコスト増の両面から。

18.14 マルチアカウント・マルチリージョンの典型構成

18.14.1 中央 Inspection VPC モデル



すべての外向き通信を中央で検査。コンプライアンス要件で多用される。

18.14.2 Cloud WAN ハブ&スポーク

Network Policy で宣言：

- Region: ap-northeast-1, us-east-1
- Segments: prod, dev, shared
- Sharing: shared 共通、prod ↔ dev 禁止

ポリシー JSON で全体構造を定義し、リージョン追加もポリシー更新だけで完結。

18.14.3 小規模メッシュ

VPC が 3～5 個ならピアリング全網で十分。Transit Gateway 固定費を回避できる。

18.15 コスト整理

転送	目安料金
同一 AZ 内	無料（プライベート IP 経由）
別 AZ（同一 VPC・同一リージョン）	\$0.01/GB（送受双方）
VPC ピアリング・Transit Gateway 越え（同一リージョン）	\$0.01/GB + TGW \$0.02/GB
別リージョン	\$0.02～\$0.09/GB（リージョン組み合わせで異なる）
インターネットへ送信	\$0.114/GB（東京、最初の 10TB）
CloudFront へ S3 から	無料（CloudFront 経由のオリジンプル）
VPC エンドポイント（Gateway 型 S3/DynamoDB）	無料
VPC エンドポイント（Interface 型）	\$0.014/時 + \$0.01/GB

18.16 よくあるトラブル

症状	確認ポイント
VPC 間で疎通しない	Transit Gateway ルート、SG、NACL、ルートテーブル

症状	確認ポイント
Direct Connect が落ちた	BGP セッション、物理リンク、AWS 障害情報
VPN トンネルが片系のみ Up	Customer Gateway 設定、IKE/IPsec パラメータ
Client VPN で接続できない	証明書、CRL、AD トラスト、エンドポイントルート
Global Accelerator のヘルス チェック失敗	リスナー設定、SG、ターゲットへの疎通
PrivateLink で名前解決失敗	Endpoint の Private DNS 有効化、Route 53 ホストゾーン
TGW のデータ転送料が高い	本当に必要なルートか、別 AZ 越えが多すぎないか確認
IPv6 の外向きが通らない	Egress-only IGW ルート、SG の IPv6 ルール

19 セキュリティサービスの詳細

10 章ではセキュリティの全体像を扱った。本章は各サービスを深く掘り下げ、運用視点での設計と検知パイプライン、KMS / Secrets Manager / ACM の詳細、IAM Access Analyzer の使いこなしまでを対象にする。

19.1 防御層別のサービスマップ

【アイデンティティ】	IAM / Identity Center / Access Analyzer / Verified Permissions
【ネットワーク】	SG / NACL / WAF / Shield / Network Firewall / DNS Firewall
【データ】	KMS / CloudHSM / Secrets Manager / Macie / S3 Block Public Access
【コンピューート】	Inspector / IMDSv2 / GuardDuty Runtime Monitoring
【監査・検知】	CloudTrail / Config / GuardDuty / Detective / Audit Manager
【統合・運用】	Security Hub / Firewall Manager / Incident Manager
【コンプライアンス】	Artifact / Audit Manager

19.2 WAF (Web Application Firewall)

CloudFront、ALB、API Gateway、AppSync、App Runner、Cognito User Pool に関連付けられる L7 FW。

19.2.1 Web ACL の構成

- ・ **ルール**：個別の検査ロジック
- ・ **ルールグループ**：複数ルールの束
- ・ **優先度**：上から評価、最初に Match した Action (Allow / Block / Count / Captcha / Challenge) が確定
- ・ **デフォルトアクション**：どのルールにもマッチしない場合の挙動 (Allow か Block)

19.2.2 ルールの種類

- ・ **マネージドルールグループ**：AWS 公式 (Core、Known Bad Inputs、Linux、PHP、SQL Injection、XSS、Bot Control など) と Marketplace (F5、Fortinet、Imperva 等)
- ・ **カスタムルール**：IP set、文字列マッチ、正規表現、サイズ制約
- ・ **レート制限ルール**：5 分間に N リクエスト超で Block / Challenge

19.2.3 Bot Control / ATP / ACFP

- ・ **Bot Control**：自動化されたボット検出 (簡易・ターゲット)
- ・ **Account Takeover Prevention (ATP)**：ログイン総当たり防御
- ・ **Account Creation Fraud Prevention (ACFP)**：偽アカウント大量作成の防御

19.2.4 CAPTCHA / Challenge

- ・ **CAPTCHA**：画像チャレンジ、有効期限長め

- **Challenge** : JavaScript 検証、ユーザー透明（自動）

19.2.5 Logging

Web ACL のログを **Kinesis Data Firehose** / S3 / CloudWatch Logs に送付。Athena でクエリして攻撃傾向を分析。

19.2.6 料金感

- Web ACL : \$5.00/月
- ルール : \$1.00/月
- 100 万リクエスト : \$0.60
- マネージドルールグループ : 別料金（種類による）

19.3 Shield

DDoS 対策。

19.3.1 Standard

- 無料・全 AWS ユーザー自動適用
- L3/L4 の典型的な DDoS（SYN flood、UDP reflection 等）を吸収
- CloudFront / Route 53 / Global Accelerator で特に効果

19.3.2 Advanced

- 月額 \$3,000 / 1 組織
- L7 攻撃の高度検出、DRT（DDoS Response Team）、急増課金のクレジット保護
- Health-based Detection（CloudWatch アラームと連動）
- Global threat dashboard
- 常時 Web 公開で攻撃リスク高い用途のみ採用検討

19.4 Network Firewall

VPC 内のステートフル FW。Suricata 互換。

19.4.1 ルールタイプ

- ステートレスルール：パケット単位の許可・拒否、低レイテンシ
- ステートフルルール：5 タプル + 接続状態、ドメインリスト、Suricata IPS シグネチャ

19.4.2 配置

「中央 Inspection VPC」モデル（18 章参照）が定石。Transit Gateway 経由で全 VPC のトラフィックを集約検査。

19.5 Firewall Manager

組織横断のセキュリティポリシー管理。

- WAF / Shield / Network Firewall / DNS Firewall / SG ベースライン を組織レベルで配布
- 自動修復ポリシー（違反 SG を自動補正）
- 新規アカウント作成時に自動適用

19.6 GuardDuty 深掘り

機械学習による継続的脅威検知。

19.6.1 データソース

- CloudTrail Management Events：API 操作の異常
- VPC Flow Logs：通信パターン
- DNS Logs：怪しいドメインへの問い合わせ
- S3 Data Events：S3 バケットへの異常アクセス
- EKS Audit Logs：K8s API への怪しい操作
- Lambda Network Logs：Lambda の通信
- RDS Login Activity：RDS への異常ログイン
- Runtime Monitoring：EC2 / ECS / EKS のホスト内挙動
- Malware Protection：EBS スナップショットの自動マルウェアスキャン

19.6.2 Findings の例

- `CryptoCurrency:EC2/BitcoinTool.B`：マイニング兆候
- `UnauthorizedAccess:IAMUser/ConsoleLoginSuccess.B`：海外からのコンソールログイン
- `Trojan:EC2/DropPoint`：マルウェアの中継
- `Recon:EC2/PortProbeUnprotectedPort`：未保護ポート探索

19.6.3 抑制ルール

「特定の検証用 EC2 からの脅威 IP リストヒットは無視」のような抑制ルールを書く。

19.6.4 マルチアカウント運用

委任管理者アカウント（通常はセキュリティ専用アカウント）に集約し、組織全体の Findings を一元監視。

19.6.5 連携

EventBridge → SNS → Slack / PagerDuty、Security Hub への自動集約、Lambda での自動隔離（SG 切替・スナップショット取得・インスタンス停止）など。

19.7 Inspector

継続的脆弱性スキャン。

19.7.1 対象

- EC2：SSM エージェント経由でパッケージ脆弱性
- ECR：イメージ push 時 + 定期再評価
- Lambda：関数コード + レイヤの脆弱性

19.7.2 修復推奨

CVE ごとに修正バージョンが提示される。重大な脆弱性は EventBridge で通知。

19.8 Macie

S3 上の機密データ発見・分類。

- マネージド識別子：PII（メール、電話、個人 ID）、認証情報、PHI、PCI など多数
- カスタム識別子：正規表現で社内固有の機密
- 許可リスト：誤検知を抑制
- **Automated Sensitive Data Discovery**：継続的に小サンプルをスキャンして感度スコアを更新
- 結果は CloudWatch Events / Security Hub に流せる

19.9 Detective

セキュリティ調査支援。

- VPC Flow Logs / CloudTrail / EKS Audit / GuardDuty Findings をグラフ統合
- **Behavior Graph**：エンティティ間の関係を時系列で可視化
- **Entity Profile**：IAM ユーザー、ロール、IP、Finding ごとに掘り下げる UI

GuardDuty で Finding を検知 → Detective でコンテキスト調査 → 修復、というフロー。

19.10 Security Hub

セキュリティ検出の集約 + CSPM（Cloud Security Posture Management）。

19.10.1 セキュリティ標準

- AWS Foundational Security Best Practices
- CIS AWS Foundations Benchmark v1.4 / v3.0
- PCI DSS
- NIST 800-53 Rev. 5
- NIST CSF

各標準に対するスコアと違反コントロールを表示。

19.10.2 統合先

GuardDuty、Inspector、Macie、Config、Firewall Manager、IAM Access Analyzer、Health などからの Findings を **統一スキーマ ASFF** で集約。サードパーティツール（Splunk、Sumo Logic、Tenable 等）も多数対応。

19.10.3 自動修復

EventBridge で Finding を検知 → Lambda / Systems Manager Automation で修復。

19.11 Audit Manager

統制マッピングの自動化。

- ・ 定義済みフレームワーク：PCI DSS、HIPAA、ISO 27001、SOC 2、GDPR、AWS Operational Best Practices
- ・ 統制ごとに **証跡を自動収集**（CloudTrail、Config、Security Hub のデータから）
- ・ レポート生成
- ・ カスタムフレームワーク作成も可

監査対応の手作業を大きく削減できる。

19.12 Artifact

AWS 自体のコンプライアンスレポート（SOC 1/2/3、PCI DSS、ISO 27001/27017/27018、HIPAA、FedRAMP、IRAP 等）を取得・閲覧。NDA 付き。

19.13 KMS 深掘り

10 章では概要を、ここでは設計・運用面を扱う。

19.13.1 キーポリシーと IAM

KMS の鍵は **キーポリシー**（リソースベース）と **IAM ポリシー**（プリンシパルベース）の両方で評価される。両者の関係：

- ・ キーポリシーで IAM 委任を許可していなければ、IAM ポリシーは効かない
- ・ 作成直後は「ルートユーザーが全権限」のキーポリシーが入っており、ここから絞っていく

19.13.2 自動ローテーション

CMK は **年次自動ローテーション** を有効化できる（2022 年以降カスタマー管理キーで対応）。新しいバックアップキーが作られ、復号は古い・新しい両方で可能。

19.13.3 Multi-Region Keys

複数リージョンで **同じキー ID** を持つ鍵セット。レプリケーション時に各リージョン独立に管理されるが、暗号文を相互に復号できる。マルチリージョン構成で必須。

19.13.4 エンベロープ暗号化

KMS は **データキー（DEK）** を発行し、それで実データを暗号化する方式を推奨する。

```
KMS マスターキー
↓ GenerateDataKey
```

Plaintext DEK + Encrypted DEK

↓ DEK でアプリが暗号化

Ciphertext + Encrypted DEK (一緒に保存)

復号時: Encrypted DEK を KMS Decrypt → Plaintext DEK → 復号

KMS 自体は小さなデータしか扱わない。実データの暗号化はクライアント側で。

19.13.5 Grants

一時的な権限委譲。Lambda が一時的に KMS 鍵を使う必要があるが IAM ポリシーで広く付与したくない、といった場合に。

19.14 CloudHSM

FIPS 140-3 Level 3 のシングルテナント HSM。KMS では満たせない厳格な規制要件向け。クラスター管理、PKCS11 / JCE / KSP / CNG 経由で利用。

19.14.1 用途

- 認証局 (CA) の秘密鍵保管
- BYOK (KMS の Import Key Material) の元鍵
- 規制要件で「自社が物理鍵を所有」と求められる場合

19.15 Secrets Manager 深掘り

19.15.1 自動ローテーション

Lambda を使って RDS / Aurora / Redshift / DocumentDB のパスワードを自動ローテーション。AWS 提供のローテーションテンプレートを利用するのが楽。カスタムリソースでも書ける。

19.15.2 Cross-Region Replication

マルチリージョン DR で必須。プライマリ Secret の更新が自動でセカンダリへ伝播。

19.15.3 リソースベースポリシー

別アカウントからのアクセスを許可。

19.15.4 VPC エンドポイント

`com.amazonaws.<region>.secretsmanager` のインタフェースエンドポイントで、プライベートサブネットからインターネット経由なしで取得。

19.15.5 Parameter Store との使い分け (再掲)

- Parameter Store: 設定値・軽い秘密。基本無料
- Secrets Manager: 自動ローテーション・本格的な秘密。1 秘密 \$0.40/月

19.15.6 コードサンプル

```
import boto3, json
sm = boto3.client('secretsmanager')
resp = sm.get_secret_value(SecretId='prod/db/master')
creds = json.loads(resp['SecretString'])
db_password = creds['password']
```

19.16 Certificate Manager (ACM)

19.16.1 パブリック証明書

DNS 検証 / メール検証で発行。自動更新（残り 60 日でドメイン検証チェック → 30 日で更新）。

- ALB / NLB / API Gateway / App Runner：同一リージョンで取得
- CloudFront：us-east-1 で取得（必須）

19.16.2 Private CA (ACM PCA)

社内 PKI のマネージド CA。エンドエンティティ証明書、サブ CA、CRL、OCSP を全部管理。料金は CA \$400/月 + 証明書発行ごと。

19.16.3 ACM の制限

- 自動更新のためには ALB / CloudFront 等の ACM 統合サービスで使用中である必要
- 単独で証明書だけダウンロードしてサーバに置く、はパブリック証明書では不可（PCA なら可）

19.17 CloudTrail Lake

CloudTrail の SQL ベース分析サービス。

- イベントデータストア：保管期間最長 10 年
- SQL でクエリ（Athena 不要）
- フェデレーテッドクエリ：CloudTrail Lake から Glue Data Catalog 経由で他データソースに JOIN

監査調査での「あの API を誰がいつ呼んだ」を高速に。

19.18 IAM Access Analyzer 深掘り

19.18.1 外部アクセス分析

S3、IAM、KMS、Lambda、SQS、Secrets Manager、SNS、ECR、EFS リソースが **外部**（別アカウント・別組織・パブリック）からアクセス可能になっていないかを継続検出。

19.18.2 未使用アクセス分析

IAM ロール・ユーザーの権限のうち、過去 X 日間使われていないアクション・サービスを検出。最小権限への絞り込みに活用。

19.18.3 Policy Generation

CloudTrail ログから 実際に使われたアクション を抽出して必要最小ポリシーを生成。

19.18.4 カスタムポリシーチェック

CI/CD 統合用 API。「このポリシー変更が 新たな外部アクセスを許可していないか」を Pull Request で自動チェック。

19.18.5 Reachability

VPC リソース（EC2 等）にどの経路で到達可能かを可視化。

19.19 セキュリティ運用ベストプラクティス

- ・ 監査専用アカウント に GuardDuty / Security Hub / CloudTrail / Config を集約
- ・ 検知 → トリアージ → 修復のループを **EventBridge + Lambda** で自動化
- ・ **IaC でガードレール**：SCP・Config Rules・Permission Boundary・ABAC
- ・ 「想定外の操作を起こさせない」設計（事後対応より事前防止）
- ・ インシデント対応 **Runbook** を **Systems Manager Automation** に登録
- ・ 定期的な ペンテスト・カオス演習
- ・ 人間のアクセス記録 を Source Identity で追跡可能に

19.20 インシデント対応の典型フロー

1. 検知 GuardDuty Finding → EventBridge
2. 通知 SNS → Slack / PagerDuty
3. 隔離 Lambda: SG を quarantine SG に切替、IAM 権限剥奪
4. 調査 Detective でグラフ調査、CloudTrail Lake で操作履歴
5. 修復 SSM Automation でパッチ・キーローテーション
6. 復旧 バックアップから復元、Auto Scaling で再構築
7. 振り返り 報告書作成、Runbook / SCP / Rule 改善

19.21 よくあるトラブル

症状	対処
WAF が誤検知で正常を Block	Logs を見て対象ルールを Count に切替、例外 IP / URI 追加
GuardDuty 通知が止まらない	抑制ルール、検証用環境のタグ・IP 除外
Inspector で誤検知	Suppression Rule、CVE スコアでフィルタ
Macie がスキャンしない	対象バケットのジョブ作成、IAM 権限

症状	対処
KMS で AccessDenied	キーポリシー優先、IAM 委任、CloudTrail で kms:* イベント確認
ACM 自動更新が走らない	統合サービスで使用中か、DNS 検証レコードが残っているか
CloudTrail Lake が遅い	パーティション設計、データストアの段階的圧縮

20 マルチアカウント運用

組織で AWS を本格利用するときに必須となる、複数アカウントを束ねた管理基盤を扱う。Organizations / Control Tower / Landing Zone / Resource Access Manager / Account Management API、コスト按分・SCP・委任管理の実例まで踏み込む。

20.1 なぜマルチアカウントか

AWS では **アカウントが最強の隔壁** である。1つのアカウントに何でも詰め込むと、以下の問題が起こる。

- ・ **爆風範囲が大きい**：誤操作で本番が止まる、不正アクセスの被害が全業務に及ぶ
- ・ **コストが混在**：プロジェクトごとの請求按分が困難
- ・ **権限が複雑化**：IAM ポリシーが肥大化、誰が何を触れるか把握できない
- ・ **リソース上限に当たる**：1アカウントの API レート、リソース数に達する
- ・ **監査・コンプライアンス対応が困難**：環境ごとに違う基準を 1 アカウントで満たすのは現実的でない

これらを解決する基本戦略が **環境・組織単位での分離**。本番／検証／開発／監査用／ログ保管用、といった単位で別アカウントに切り出す。

20.2 AWS Organizations

複数アカウントを束ねる基盤サービス。

20.2.1 基本構造

```
[Management アカウント (Payer / Root)]
├── OU: Security
│   ├── Log Archive アカウント
│   └── Security Tooling アカウント
├── OU: Workloads
│   ├── OU: Prod
│   │   ├── prod-web アカウント
│   │   └── prod-data アカウント
│   └── OU: NonProd
│       ├── dev アカウント
│       └── staging アカウント
├── OU: Sandbox
│   └── sandbox-* アカウント
└── OU: Suspended
    └── 解約予定アカウント
```

20.2.2 主な機能

- **一括請求 (Consolidated Billing)** : 管理アカウントに支払い集約。ボリューム割引・Savings Plans を組織全体で共有
- **SCP (Service Control Policy)** : OU・アカウント単位で「使えるサービス・API」を制限
- **委任管理者** : セキュリティ系・ネットワーク系サービスを管理アカウント以外で運用可能
- **タグポリシー** : 必須タグ・タグ値の強制
- **バックアップポリシー** : AWS Backup の設定を組織レベルで配布
- **AI サービスオプトアウトポリシー** : AI サービスでのデータ利用許可・拒否

20.2.3 管理アカウントの守り方

管理アカウント (旧称 Master) は **最も重要なアカウント**。請求・SCP・組織自体の管理権限を持つ。

- **ワークロードを置かない** : Organizations / 請求 / セキュリティ管理ツールだけ
- **ルートユーザーには passkey 必須、アクセスキーなし**
- **人は IAM Identity Center 経由でしかアクセスしない**
- **緊急用 Break Glass 手順を別途確立** (金庫保管の認証情報など)

20.2.4 SCP の設計パターン

SCP は **Allow を絞る** のではなく、**禁止事項を明確化** するのに使うのが基本 (`FullAWSAccess` をデフォルトで継承し、危険な操作を Deny で塞ぐ)。

```
// SCP: Sandbox OU の許容リージョン制限
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyOutsideAllowedRegions",
    "Effect": "Deny",
    "NotAction": [
      "iam:*", "sts:*", "support:*", "organizations:*",
      "cloudfront:*", "route53:*", "globalaccelerator:*",
      "waf:*", "wafv2:*", "shield:*", "trustedadvisor:*"
    ],
    "Resource": "*",
    "Condition": {
      "StringNotEquals": {
        "aws:RequestedRegion": ["ap-northeast-1", "ap-northeast-3"]
      }
    }
  }]
}
```

その他典型的な SCP :

- ルートユーザーの利用禁止 (`Condition: aws:PrincipalArn = root`)
- 組織から離脱不可 (`organizations:LeaveOrganization` の Deny)
- CloudTrail / Config の停止禁止

- Region 制限
- 大型インスタンスタイプの利用制限（`ec2:RunInstances` + Condition）
- MFA なしでの API 拒否

20.2.5 タグポリシー

組織で必須タグ（Project、Environment、Owner、CostCenter）を強制。違反タグの自動修復は別途。

20.3 Control Tower

ベストプラクティスに沿った Landing Zone を自動構築するサービス。

20.3.1 Control Tower がやってくれること

- Organizations 有効化
- IAM Identity Center 有効化
- 監査用アカウント・ログアーカイブアカウント自動作成
- CloudTrail / Config を全リージョン・全アカウントで有効化、ログを集約
- 標準 OU（Security / Sandbox など）作成
- マンダトリーガードレール（変更不可の SCP）と 推奨ガードレール

20.3.2 Account Factory

新規アカウントを **セルフサービス** で作る仕組み。Service Catalog を使ったり、`AccountFactory CT API` で IaC からプロビジョニングできる。Terraform Cloud と組み合わせる例も多い。

20.3.3 AFT（Account Factory for Terraform）

Control Tower と Terraform を統合。新規アカウント作成時に Terraform モジュールが自動実行され、ベースラインを構築。

20.3.4 既存組織への適用

すでに Organizations を運用中でも、Control Tower を「後付け」で有効化できる（Landing Zone 拡張機能）。ただし既存アカウントを Account Factory 管理下にするには Enroll 操作が必要。

20.4 Landing Zone の典型構成

Control Tower を使うかどうかに関わらず、組織レベルでの推奨構成は以下。

[管理]	Organizations、請求、SCP
[Identity Center]	人のアクセスポータル
[Log Archive]	CloudTrail / Config / VPC Flow Logs を集約
[Security Tooling]	Security Hub / GuardDuty / Detective の委任管理
[Network Hub]	Transit Gateway / Direct Connect / DNS の中央管理

[Shared Services]	共通 AMI、Service Catalog、Build パイプライン
[Workloads/Prod-*]	本番ワークロードごとに分離
[Workloads/Dev-*]	開発ワークロード
[Sandbox]	個人検証

20.5 Resource Access Manager (RAM)

17 章で触れた RAM をマルチアカウント文脈で再整理。

20.5.1 共有可能リソース

- **Subnet** (VPC 共有)：中央 VPC を作り、各ワークロードアカウントから Subnet を借りる
- **Transit Gateway**：中央ネットワークアカウントの TGW を全 VPC で共有
- **Resolver Rule / Route 53 Profiles**：DNS 設定の集中管理
- **Glue Database / Table**：データレイクのカatalogを共有
- **Aurora Cluster snapshot、EBS snapshot**：バックアップを別アカウントへ
- **License Manager License Configuration**

20.5.2 VPC 共有の実例

中央ネットワークアカウントで VPC + Subnet を作成 → RAM で各ワークロードアカウントに共有。各ワークロードは「自分の VPC のように」Subnet を見て EC2 / RDS を起動できる。

メリット：

- ネットワーク変更を中央集約
- IP アドレス管理が一元化
- データ転送コストの削減（同一 VPC 内通信になる）

20.6 委任管理者 (Delegated Administrator)

管理アカウントで全部やるとリスクが高いため、サービスごとに **委任管理者アカウント** を指定して運用を委譲する。

サービス	委任先 (典型)
GuardDuty	Security Tooling
Security Hub	Security Tooling
Detective	Security Tooling
Inspector	Security Tooling
Macie	Security Tooling
Audit Manager	Security Tooling
IAM Access Analyzer	Security Tooling
Config	Log Archive
CloudTrail (組織証跡)	Log Archive

サービス	委任先（典型）
Firewall Manager	Network Hub
Network Manager / Cloud WAN	Network Hub
License Manager	License 管理アカウント
Service Catalog	Shared Services
Cost Optimization Hub	Finance

委任管理者は管理アカウントから `aws organizations register-delegated-administrator` で指定。

20.7 Account Management API

近年（2023～2024）追加された API で、別アカウントの代替連絡先・プライマリ連絡先・地域オプトインなどを Organizations 越しに管理できる。新規アカウント作成時のメタデータ整備に有用。

20.7.1 Region オプトイン

2019年以降の新リージョン（メキシコ、ハイデラバード、ジャカルタ、メルボルン、テルアビブ、UAE、チューリッヒ、スペイン、マラガなど）は **opt-in が必要**。SCP で許可リージョンを絞っているなら、新リージョン追加前に opt-in 解除も検討。

20.8 集中ロギング

組織内のログを **Log Archive アカウント** に集約する。

20.8.1 CloudTrail

組織証跡を作成すると、全アカウント・全リージョンの CloudTrail が **Log Archive アカウントの S3 バケット** に集約される。バケット側でログファイル整合性検証 + ボールトロック。

20.8.2 Config

組織アグリゲータで全アカウント・全リージョンの Config レコーダ結果を Log Archive に集約。

20.8.3 VPC Flow Logs

各 VPC で Flow Logs を有効化し、共通 S3 バケット（or CloudWatch Logs）へ。Athena で分析。

20.8.4 ログのライフサイクル

S3 ライフサイクルで Standard → IA → Glacier → Deep Archive と階層化。法定保存期間（一般に 7 年）を超えたら自動削除。

20.9 コスト管理（マルチアカウント視点）

20.9.1 一括請求のメリット

- ・ ボリュームディスカウント：S3、EC2、データ転送が組織全体の合計使用量で階段適用
- ・ Savings Plans / Reserved Instance の共有：余剰分が組織内で再利用される
- ・ CloudFront の階段料金 も組織合算

20.9.2 コスト按分の方法

- ・ コスト配分タグ (Cost Allocation Tags) :Project / CostCenter タグを有効化し、Cost Explorer で按分
- ・ Cost Categories :複数のディメンション（アカウント・タグ・サービス）で論理的なグループを定義し、レポートやダッシュボードで使う
- ・ AWS Cost and Usage Report 2.0 :詳細データを S3 に配信、Athena / QuickSight で経営報告

20.9.3 Budgets の組織展開

組織レベルで「OU ごとの月次予算」「ワークロードごとの予算」を Budgets に設定し、超過時に SNS / Chatbot で通知。

20.9.4 Compute Optimizer

EC2 / Auto Scaling Group / EBS / Lambda / RDS / ECS Fargate に対して **右サイジング推奨** を機械学習で算出。組織レベルで集計可能。

20.10 マルチアカウント自動化

20.10.1 StackSets (CloudFormation)

1 テンプレートを **複数アカウント・複数リージョン** に展開。Service Managed mode (Organizations 自動連携) と Self Managed mode。

20.10.2 CDK Pipelines / Terraform マルチアカウント

CDK Pipelines は Cross-Account Cross-Region デプロイをネイティブサポート。Terraform は `provider` を Assume Role で別アカウントを指定。

20.10.3 AWS Config Conformance Pack

組織レベルでセキュリティ標準のルール束を配布。

20.10.4 EventBridge クロスアカウント

イベントバス間で別アカウントへイベント転送。「全アカウントの GuardDuty Finding を Security Tooling アカウントに集約」など。

20.11 委任管理 + 自動化のパターン

[管理アカウント]	Organizations + SCP + 請求
↓ 委任	
[Security Tooling]	GuardDuty / SecurityHub / Inspector / Macie 集約
[Log Archive]	CloudTrail / Config / VPC Flow Logs 集約
[Network Hub]	TGW / 共有 VPC / Direct Connect / Cloud WAN
[Shared Services]	Service Catalog / 共通 AMI / CI/CD パイプライン
[Workloads/Prod]	プロダクション
[Workloads/Dev]	開発
[Sandbox]	個人実験

20.12 ベストプラクティスまとめ

- ・新規組織は Control Tower で開始 → 自前で Organizations を設定する手間を回避
- ・管理アカウントにワークロードを置かない
- ・環境別・チーム別にアカウントを分ける（粒度はチームの規模次第）
- ・SCP はガードレール、IAM ポリシーは細かい権限 と役割分担
- ・ログ集約・セキュリティ集約は専用アカウント
- ・タグ戦略を最初に決める（後から付け回るのは大変）
- ・Sandbox を提供 する（個人検証を本番から完全分離）
- ・Break Glass 手順 を文書化・演習

20.13 よくある落とし穴

問題	対処
SCP で全部止めた	FullAWSAccess を継承前提、Deny でガードレール
新規アカウントが裸	Account Factory + AFT で baseline 自動適用
請求が見えない	子アカウントの IAM ユーザーに請求アクセス許可が必要
SCP が反映されない	親 OU から継承、変更後数分待つ、CloudTrail で確認
委任管理者解除で混乱	Resource を残したまま委任先を変えない、移管手順を踏む
子アカウントを誤って解約	解約は 90 日間取り消し可、待機 OU で隔離して放置でも可
リージョン opt-in 漏れ	新リージョン公開後にチーム周知、SCP 更新

21 移行とデータ転送

オンプレ・他クラウドから AWS への移行、または AWS 内・AWS 間でのデータ転送を扱う。サーバ移行 (MGN)、データベース移行 (DMS / SCT)、物理データ転送 (Snow ファミリー)、ファイル転送 (DataSync / Transfer Family)、アプリケーションリファクタ (Migration Hub) など、多数のサービスを横断的に解説する。

21.1 AWS 移行の段取り

Gartner の 6R フレームワークが定番。

戦略	意味
Rehost (Lift & Shift)	そのまま EC2 に持っていく。最速。最小変更
Replatform (Lift & Reshape)	マネージド DB 化など軽い改造で乗せる
Repurchase	SaaS に置き換える
Refactor / Re-architect	クラウドネイティブに作り直す
Retain	そのままオンプレで残す
Retire	廃止

最初の波は **Rehost** で量を稼ぎ、次に **Replatform** で運用負荷を下げ、戦略的なシステムだけ **Refactor** するのが現実解。

21.2 Migration Hub

移行のポートフォリオ管理。各種 AWS 移行サービスの進捗を 1 画面で。

- ・ アプリケーション・サーバー単位の移行ステータス管理
- ・ **Migration Hub Strategy Recommendations** : オンプレ環境を分析して 6R を提案
- ・ **Migration Hub Refactor Spaces** : モノリスから少しずつマイクロサービス化を進める

21.3 Application Discovery Service

オンプレ環境の発見・依存関係マッピング。

- ・ **Discovery Agent** : 各サーバに導入して詳細データを収集
- ・ **Discovery Connector** : vCenter 経由でエージェントレス収集
- ・ 結果は Migration Hub に集約され、Strategy Recommendations の入力に

21.4 AWS Application Migration Service (MGN)

サーバ単位の Lift & Shift の主力サービス。旧 CloudEndure Migration の後継。

21.4.1 仕組み

オンプレサーバ（Linux / Windows / VMware / Hyper-V / 物理）に **レプリケーションエージェント** を入れると、ブロックレベルでリアルタイムに AWS の **Staging Area** (EBS) にレプリケーション。テストで仮想 EC2 を起動して動作確認、本番カットオーバー時に **DNS / IP 切替** で新 EC2 に切り替える。

21.4.2 カットオーバーの流れ

1. レプリケーション確立（初回フルコピー → 差分継続）
2. Test インスタンスを起動して動作確認
3. アプリ停止・最終差分同期
4. 本番 EC2 起動、DNS / IP 切替
5. 旧サーバ廃止

ダウンタイムを **分単位** に抑えられる。

21.4.3 大規模移行プロジェクト

数百～数千台の規模では、**Migration Hub** の **Application Group** と組み合わせて、関連サーバ一括カットオーバー。

21.5 AWS Database Migration Service (DMS)

DB 移行・継続レプリケーション。

21.5.1 サポートする変換

- ・ **同種**：Oracle → Oracle、PostgreSQL → PostgreSQL
- ・ **異種**：Oracle → Aurora PostgreSQL、SQL Server → MySQL など

異種の場合は **AWS Schema Conversion Tool (SCT)** でスキーマ・ストアドプロシージャを変換 → DMS でデータ移行、という二段構え。

21.5.2 主要モード

モード	用途
Full Load	一括コピー
Full Load + CDC	一括コピー後、変更を継続適用（カットオーバーまで）
CDC Only	既存スナップショット復元後、変更だけ追従

CDC を使うとカットオーバー時のダウンタイムを **秒～分** に圧縮できる。

21.5.3 DMS Serverless

2024 年以降、サーバ管理不要のサーバーレス DMS が GA。短期・スパイクの移行や検証で便利。

21.5.4 Babelfish for Aurora PostgreSQL

SQL Server T-SQL を Aurora PostgreSQL で **そのまま受け入れる** 互換レイヤ。アプリ改修なしで SQL Server から移行できる場合も。

21.5.5 ヘテロジニアス移行のリアル

DMS / SCT で完全自動化はできない。実プロジェクトでは：

- ・ **スキーマ・コード**：SCT で 7～8 割自動変換、残りは手作業
- ・ **データ**：DMS で大半を自動移行
- ・ **アプリ側変更**：データ型差異・関数差異への対応
- ・ **テスト**：本番並み負荷での性能・互換確認

これらを段階的にやる。

21.6 Server Migration Service (SMS)

VMware からの移行。MGN に統合・置き換え進行中だが、既存案件で言及されることがある。

21.7 Snow ファミリー

13 章でも触れたが、移行視点で再整理。

デバイス	用途
Snowcone	～8TB。エッジコンピュート + 小規模オフライン
Snowball Edge SO	80TB クラス。中規模オフライン
Snowball Edge CO	計算強化 (GPU/EC2 互換)。エッジ AI、現地データ前処理

ペタバイト級は **複数台 Snowball を並列発注**。Snowmobile (40 フィートトラック) は新規受注停止。

21.7.1 利用ステップ

1. AWS コンソールから Job 作成、配送先入力
2. 数日で配送
3. 現地で電源・ネットワーク接続、AWS OpsHub (GUI) でセットアップ
4. データコピー (NFS / SMB / S3 互換 API)
5. 集荷依頼 → AWS データセンターへ返送
6. AWS 側で S3 にロード
7. 廃棄処理 (NIST 準拠で複数回上書き)

21.8 DataSync

オンライン回線でのファイル・オブジェクト同期。

21.8.1 対応 Source / Destination

- NFS / SMB（オンプレ）
- HDFS
- オブジェクトストレージ（S3 互換、Azure Blob、Google Cloud Storage）
- AWS：S3、EFS、FSx 全種、SMB / NFS

21.8.2 主な機能

- 並列・チャンク化転送で高速
- 完全性検証（チェックサム）
- メタデータ・ACL 保持
- スケジュール起動・継続的同期
- 帯域制御、マルチタスク
- KMS 暗号化、PrivateLink

オンプレからの ETL データ集約、クロスリージョン S3 同期、FSx 系の移行などに。

21.9 AWS Transfer Family

SFTP / FTPS / FTP / AS2 のフルマネージド受け口。

21.9.1 ユースケース

- パートナー企業との **EDI 連携**
- 既存 SFTP クライアントから受信したファイルを S3 / EFS に
- AS2 で **B2B 取引メッセージ** を受信

認証は IAM、AD、カスタム ID プロバイダ（Lambda）、API Gateway。Workflows でアップロード後の処理（変換、解凍、PII 検出）を自動化。

21.10 Storage Gateway とのハイブリッド転送

13 章で触れたが、移行コンテキストでは：

- **File Gateway**：オンプレに NFS/SMB 提供、裏は S3。徐々にデータを S3 に移しながら使い続けられる
- **Tape Gateway**：物理テープ運用を仮想化、S3/Glacier に保存。テープライブラリ廃止プロジェクトで使う

21.11 Database Migration の典型シナリオ

21.11.1 Oracle → Aurora PostgreSQL

1. SCT でスキーマ・PL/SQL の変換、変換不可コードを洗い出し
2. アプリ側の Oracle 固有 SQL を PostgreSQL 互換に修正
3. DMS で **Full Load** + CDC レプリケーション開始

4. 並行運用で動作検証
5. カットオーバー：アプリ停止 → 最終差分 → アプリを Aurora 接続に切替 → 起動

21.11.2 SQL Server → Aurora PostgreSQL (Babelfish 経由)

1. Babelfish 有効化した Aurora PostgreSQL を構築
2. アプリは T-SQL のまま で接続変更だけで動作 (Babelfish が翻訳)
3. 段階的に PostgreSQL ネイティブ機能に置き換え

21.11.3 MySQL on EC2 → Aurora MySQL

1. Aurora MySQL クラスタ作成
2. DMS Full Load + CDC (または Aurora の `mysqldump` リストア)
3. アプリ接続切替

21.12 サーバ移行の典型シナリオ

21.12.1 VMware → EC2 (MGN)

1. 各 VM に MGN エージェント導入
2. レプリケーション確立、Staging Area で数日同期
3. Test 起動して動作・性能確認
4. 移行ウェーブ計画 (業務関連のサーバ群を 1 単位で)
5. 計画停止時間内にカットオーバー
6. 本番 EC2 を Auto Scaling / Load Balancer 配下に登録

21.12.2 コンテナ化 (モダナイゼーション)

- App2Container : Java / .NET の既存アプリを Docker イメージ化するツール
- Porting Assistant for .NET : .NET Framework → .NET Core 移行支援
- Microservice Extractor for .NET : モノリスから API 候補を抽出

21.13 ライセンスと EULA

商用ソフト (Windows / SQL Server / Oracle / SAP) の移行ではライセンスが大問題。

- **License Included** : AWS 提供 AMI のライセンス込み (Windows、SQL Server)
- **BYOL** : 自社ライセンスを持ち込む
- **Dedicated Host** : BYOL で **物理サーバ専有** が必要なケース (Oracle の特定エディション、Windows のソケット課金)

License Manager で利用状況を可視化。

21.14 クラウド間の移行 (GCP / Azure → AWS)

公式サービスは少ないが、以下を組み合わせる。

- ・ **DMS**：DB 移行（Cloud SQL → Aurora など）
- ・ **DataSync**：オブジェクトストレージ間（GCS / Azure Blob → S3）
- ・ **MGN**：VM 移行（Compute Engine / Azure VM → EC2）
- ・ **Migration Hub Strategy Recommendations**：移行戦略立案

21.15 移行の品質を上げるコツ

- ・ 最初に **Discovery** を徹底：知らないサーバ・依存は移行できない
- ・ 本番を 1 台移して教訓を得る（パイロット）
- ・ 並行稼働期間を設ける：1 日でアプリ全切替は事故の元
- ・ **DNS TTL** を事前に短くする：切替時の伝播時間短縮
- ・ **ロールバック計画** を最初に決めておく
- ・ **セキュリティベースライン** を移行先で先に整備
- ・ **監視・ログ** を移行先で稼働させてから本番切替

21.16 移行後にやるべきこと

- ・ **コスト最適化**：Reserved Instance / Savings Plans
- ・ **マネージド化**：EC2 → ECS Fargate / Lambda / Aurora Serverless へ段階的に
- ・ **監視刷新**：CloudWatch / X-Ray / Application Signals 導入
- ・ **Well-Architected Review**（28 章）

21.17 よくあるトラブル

症状	対処
MGN レプリケーションが進まない	帯域、エージェント側のディスク、SG、エージェント再インストール
DMS の CDC ラグが大きい	ターゲットの IOPS、テーブル並列度、長時間トランザクション排除
SCT で変換不能コード多数	手動修正、ロジックを Lambda / アプリ側へ寄せる
Snowball が温度警告	輸送中の振動・温度、AWS Support に連絡
DataSync 失敗	権限、検証エラー、ファイル名特殊文字、SMB v3 必須
移行後に性能劣化	Right-sizing、ディスクタイプ、リージョン選定
Babelfish 互換性問題	サポート関数リスト、限定機能の代替実装

22 IoT・エッジ・メディア

業界向けの専門領域として、**IoT**（モノとクラウド連携）、**エッジコンピューティング**（クラウドの拡張）、**メディア配信**（動画ストリーミング）を扱う。これらは個別のドメインだが、AWS の主要サービス領域として無視できない。

22.1 AWS IoT サービス群

サービス	用途
IoT Core	デバイスとの MQTT/HTTPS/WebSocket メッセージング、デバイス認証
IoT Device Management	デバイスのプロビジョニング・グループ管理・ファーム更新
IoT Device Defender	デバイス挙動の異常検知
IoT Greengrass	エッジ実行環境（Lambda / Docker / 機械学習をデバイスで）
IoT Analytics	IoT データの収集・前処理・分析
IoT Events	複雑イベント処理、デバイス状態管理
IoT SiteWise	産業用 IoT データの収集・モデリング
IoT TwinMaker	デジタルツイン構築
FreeRTOS	マイコン向けリアルタイム OS

22.2 IoT Core

IoT の中核。

22.2.1 デバイス接続

- MQTT（推奨、軽量パブサブ）
- MQTT over WebSocket（ファイアウォール越え）
- HTTPS（送信のみ、デバイス少なめ）
- LoRaWAN（IoT Core for LoRaWAN）

22.2.2 認証

- X.509 証明書：デバイス 1 台ごとに発行
- IAM 認証：SDK / モバイルアプリ
- Custom Authorizer：独自のトークンベース

22.2.3 Things、Thing Group、Thing Type

デバイスを **Thing** として登録し、グループ・タイプで分類。属性（製造番号、ファーム版、所属拠点）を付ける。

22.2.4 Device Shadow

デバイスの **最新状態** を JSON で AWS 側に保持。オフライン中に変更されても、復帰時に同期される。

```
{
  "state": {
    "desired": { "temperature": 22 },
    "reported": { "temperature": 21 },
    "delta": { "temperature": 22 }
  }
}
```

22.2.5 Rules Engine

デバイスから来る MQTT メッセージを SQL 風に **選択** → **加工** → **ターゲット送信**。

```
SELECT *, topic(2) as device_id FROM 'iot/+/telemetry'
WHERE temperature > 30
```

ターゲット：Lambda、Kinesis、DynamoDB、S3、SNS、SQS、IoT Analytics、CloudWatch、再 publish など。

22.2.6 料金感

- ・ 接続：\$0.08/100 万分（接続時間）
- ・ メッセージング：\$1.00/100 万メッセージ（5KB ブロック）
- ・ Device Shadow / Registry / Rules：別料金
- ・ 数千デバイス規模なら月数十ドル～

22.3 IoT Device Management

- ・ **Fleet Provisioning**：大量デバイスを安全にプロビジョニング
- ・ **Jobs**：デバイス群への一斉指示（再起動、設定更新、ファーム更新 OTA）
- ・ **Secure Tunneling**：NAT 越しにデバイスへ SSH 相当アクセス
- ・ **Fleet Hub**：デバイス可視化ダッシュボード

22.4 IoT Greengrass

エッジ実行環境。クラウドで開発したコンポーネントをデバイス上で動かす。

22.4.1 コア機能

- ・ Lambda 関数のローカル実行
- ・ Docker コンテナの実行
- ・ 機械学習推論（クラウドで訓練したモデルをエッジへデプロイ）
- ・ ローカル MQTT ブローカ（オフライン中もデバイス間通信）

- **Stream Manager**（バッファリングしてオンライン時にクラウドへ送信）

工場・店舗・車両・建設現場など、ネットワーク不安定な環境で活躍。

22.4.2 Greengrass v2 のコンポーネント

宣言的に「このデバイスにはこれらのコンポーネント」と指定し、デプロイ。バージョン管理付き。

22.5 IoT Analytics

IoT データの ETL + 分析。Kinesis + Glue で組むよりも IoT 特化で楽。

- **Channel**：生データ受け口
- **Pipeline**：フィルタ・変換
- **Datastore**：処理済みデータ
- **Dataset**：SQL クエリ結果

QuickSight で可視化、SageMaker で ML パイプラインに連携。

22.6 IoT Events

複雑なイベント処理（CEP）。

- **Detector Model**：状態機械
- 例：「温度 > 50°C が 10 分続いたら警告」「3 つのセンサーが同時に異常なら緊急停止」

22.7 IoT SiteWise

産業用設備（工場ライン、発電設備）のデータモデリング・収集に特化。OPC-UA、Modbus などのプロトコルから収集。階層的なアセットモデルでデータを構造化。

22.8 IoT TwinMaker

物理空間の **デジタルツイン** を構築。3D シーン + リアルタイムデータ + AI 推論を統合。Grafana プラグインでダッシュボード化。

22.9 FreeRTOS

組込み用 RTOS（リアルタイム OS）。Cortex-M、ESP32 等の MCU 向け。AWS と直接接続するためのライブラリ（IoT Device SDK）を含む。

22.10 エッジコンピューティング

IoT を超えた、より広いエッジ系サービス。

22.10.1 AWS Outposts

AWS のラックを自社データセンターに置く。EC2 / EBS / S3 / RDS / EKS / ECS が **オンプレ**で動く。低レイテンシ要件、データ主権、ネットワーク制約があるユースケースで。

- **Outposts ラック**：標準サーバラック型
- **Outposts サーバ**：1U / 2U の小型版

22.10.2 AWS Local Zones

主要都市内に AWS のミニリージョンを配置。リージョンより **さらに低レイテンシ**（数 ms）。ゲーミング、メディア制作、リアルタイム ML 推論に。日本では現時点で Local Zones なし、東京リージョンを直接使う。

22.10.3 AWS Wavelength

5G キャリアのモバイルネットワーク内に AWS インフラを置く。モバイル端末から **キャリア網内で完結**したアクセスで超低レイテンシ。AR/VR、自動運転、ゲームストリーミング。

22.10.4 Snow ファミリー（エッジ視点）

Snowball Edge Compute Optimized は **現地で計算** もできる。砂漠・洋上・宇宙・戦地などの極端なエッジ環境で。

22.11 メディアサービス群

動画配信・放送のユースケース向け。

サービス	用途
Elemental MediaConvert	VOD（ファイルベース）動画変換
Elemental MediaLive	ライブ動画変換
Elemental MediaPackage	パッケージング（HLS / DASH / CMAF）+ DRM
Elemental MediaTailor	動的広告挿入（SSAI）
Elemental MediaStore	低レイテンシメディアオブジェクトストア（廃止予定、S3 へ移行推奨）
Elemental MediaConnect	プロフェッショナル放送用 IP トランスポート
Elemental Link	現地のエンコーダー機器（クラウド連携専用）
Interactive Video Service (IVS)	低レイテンシ・インタラクティブライブ配信
Nimble Studio	クラウドベースのコンテンツ制作スタジオ

22.12 MediaConvert（VOD トランスコード）

ファイルベースの動画変換。

22.12.1 入力 / 出力

- ・ 入力：ほぼ全ての主要フォーマット（MP4、MOV、MXF、TS、3GP、AVI、ProRes、MXF、IMF、MKV など）
- ・ 出力：HLS、DASH、CMAF、Smooth Streaming、MP4、MOV、MXF など

22.12.2 機能

- ・ マルチビットレート ABR：1 ジョブで複数解像度・複数ビットレートを生成
- ・ DRM：DASH/HLS で Widevine / FairPlay / PlayReady
- ・ キャプション・字幕：抽出・埋め込み・別ファイル出力
- ・ 画像オーバーレイ：ロゴ、ウォーターマーク
- ・ CMAF：HLS と DASH を 1 ファイルセットでサポート

22.12.3 料金

時間ベース（出力 1 秒ごと、コーデック・解像度で単価変動）。1 時間の HD 動画を ABR 5 段で変換すると \$0.5～\$2 程度。

22.13 MediaLive（ライブトランスコード）

リアルタイム動画変換。

- ・ 入力：RTMP、HLS、RTP、MediaConnect、ファイル
- ・ 出力：MediaPackage、HLS、UDP/TS、RTMP、SRT、Frame Capture
- ・ Pipeline：Standard（2 系統冗長） / Single Pipeline（コスト重視）
- ・ Resource Pool：チャンネル数の柔軟管理

ライブイベント、24 時間放送、ニュース、スポーツ中継などで使用。

22.14 MediaPackage

オリジン・パッケージング層。

- ・ MediaPackage v2：CMAF ネイティブ、低レイテンシ HLS（LL-HLS）対応
- ・ DVR（タイムシフト）：過去のライブを巻き戻し再生
- ・ Time-shifted Content：開始時刻ずらし配信
- ・ Speke で DRM 統合

22.15 MediaTailor

動的広告挿入（SSAI: Server-Side Ad Insertion）。

- ・ VAST/VMAP 広告サーバと連携
- ・ 視聴者ごとに 別の広告 を挿入してパーソナライズ
- ・ フリーケンシーキャップ、地域ターゲティング

22.16 IVS (Interactive Video Service)

低レイテンシ（～3 秒）のライブ配信を **マネージド** で。ライブコマース、Q&A、ゲーム配信、推し活ライブなど。チャット、Stages（多人数同時放送）も統合。

22.17 配信パイプラインの典型

22.17.1 VOD (YouTube ライク)

```

[アップロード] → S3 → MediaConvert → S3 (HLS/DASH) → CloudFront → [視聴者]
                                     ↑
                                   DRM (Widevine)
  
```

22.17.2 ライブ (Twitch ライク)

```

[配信者 RTMP] → MediaLive → MediaPackage → CloudFront → [視聴者]
                ↑
            MediaTailor (広告挿入)
  
```

22.17.3 低レイテンシライブ

```

[配信者] → IVS → IVS CDN → [視聴者] (LL-HLS、3秒以内)
  
```

22.18 ゲーム関連

メディアと近いカテゴリで、ゲーム特化サービスもある。

- **GameLift**：ゲームサーバホスティング、マッチメイキング
- **GameLift Anywhere**：自社サーバ・他クラウドのゲームサーバを GameLift で管理
- **GameLift Streams**（旧 Lumberyard ベースの再構築）：クラウドゲーミング基盤

22.19 コスト・運用の注意

- **MediaLive のチャンネル時間課金**：使わないライブ中継チャンネルは止める。アイドル中も課金される
- **MediaConvert の出力解像度・コーデック**：4K HEVC は高い、必要解像度に絞る
- **CloudFront のデータ転送**：動画は GB 単位の通信になる。地域単価を把握して料金見積
- **DRM ライセンス料**：別途ベンダ契約が発生する場合あり
- **IoT のメッセージ単価**：5KB ブロック課金。大きなペイロードは 1 メッセージで複数ブロック

22.20 よくあるトラブル

症状	対処
IoT MQTT 接続できない	証明書、ポリシー、エンドポイント名（リージョン違い）、時刻同期
Greengrass コンポーネント反映されない	デプロイ Status、Recipe バージョン、ログ <code>/greengrass/v2/logs/</code>
Device Shadow 反映遅い	トピック権限、Delta 受信ロジック、Connection 維持
MediaConvert ジョブ失敗	入力フォーマット、コーデック対応、IAM ロール (S3 アクセス)
MediaLive 配信が止まる	入力 RTMP の安定性、Pipeline 冗長化、CloudWatch アラーム
HLS が再生できない	CORS、m3u8 のパス、CloudFront キャッシュ TTL (短く)、ABR レンジ
DRM 鍵取得失敗	Speke エンドポイント、ライセンスサーバ疎通、コンテンツ ID マッピング
IVS の遅延が大きい	LL-HLS プレーヤー、配信ビットレート、視聴端末のバッファ設定

23 アーキテクチャパターン集

実務で頻出する AWS 構成を「なぜそうするか」とセットで紹介する。各パターンに **構成図**、**主要サービス選択の理由**、**コスト感**、**拡張ポイント**、**変形パターン** を添える。

23.1 Web アプリの基本パターン

23.1.1 静的サイト + サーバーレス API

```
[User] → Route 53 → CloudFront → S3 (静的フロント、OAC)
                                     ↓ /api/*
                               API Gateway HTTP API → Lambda → DynamoDB
                                     ↓
                               Cognito (認証)
```

- ・ **特長**：常時稼働インフラゼロ、スケール無制限、Free Tier 内に収まることが多い
- ・ **向く規模**：個人サービス～MAU 数十万
- ・ **コスト感**：MAU 1 万なら月数ドル
- ・ **弱点**：複雑なバックエンド処理、長時間処理、SQL 必須要件には向かない
- ・ **変形**：認証を Auth0 に置換、DynamoDB → Aurora Serverless v2、Lambda → App Runner

23.1.2 古典的 3 層 Web アプリ

```
[User] → Route 53 → ALB → EC2 (Auto Scaling, multi-AZ) → RDS (Aurora, Multi-AZ)
                                     ↓
                               ElasticCache (セッション・DB キャッシュ)
                                     ↓
                               S3 (画像・添付) + CloudFront
```

- ・ **特長**：既存技術スタック (PHP/Rails/Spring 等) をそのまま乗せやすい
- ・ **向く規模**：MAU 数万～数百万
- ・ **コスト感**：t3.medium × 2 + Aurora db.r6g.large + ALB + NAT で月 \$300～500
- ・ **拡張**：CloudFront を前段に、Bot 対策に WAF、CDN/Cookie 認証を追加
- ・ **変形**：EC2 → ECS Fargate、RDS → Aurora Serverless v2

23.1.3 コンテナ化 Web

```
[User] → CloudFront → ALB → ECS Fargate → Aurora
                                     ↓
                               Service Discovery / App Mesh
                                     ↓
                               ECS Fargate (マイクロサービス)
```

- ・ **特長**：CI/CD で頻繁にデプロイ、サーバ管理ゼロ

- ・ 向く規模：マイクロサービス化されたアプリ
- ・ 拡張：EKS にすると Helm/operator など k8s エコシステム活用

23.2 モバイルバックエンド

23.2.1 Amplify + Cognito + AppSync

```
[Mobile App] → Amplify Hosting (Web 配信)
               → Cognito (認証)
               → AppSync (GraphQL) → DynamoDB
                                   → Lambda → 任意サービス
               → S3 (添付)
               → Pinpoint (プッシュ通知)
```

- ・ 特長：フロント開発者だけで完結。GraphQL でクエリ柔軟
- ・ 向く規模：B2C モバイル、新規プロダクト

23.2.2 REST + Cognito + DynamoDB

```
[Mobile App] → API Gateway HTTP API (JWT Authorizer)
               → Lambda
               → DynamoDB / S3 / RDS
```

- ・ 特長：シンプル、REST 慣れている開発者向け

23.3 バッチ・データ処理

23.3.1 サーバーレス ETL

```
[源データ S3] → EventBridge → Step Functions
                  | Glue Crawler
                  | Glue ETL (PySpark)
                  | Athena CTAS
                  ↓
                [変換後 S3] → QuickSight
```

- ・ 特長：常時稼働ゼロ、スケール自動
- ・ コスト感：1日1回 100GB 処理で月 \$50~100
- ・ 変形：EMR Serverless / Spark on K8s

23.3.2 Map-Reduce 大規模バッチ

```
[源データ S3] → EMR on EC2 (Spark) → S3
                  → Spot で低コスト
```

- ・ 特長：超大規模、独自フレームワーク利用
- ・ 拡張：EMR Serverless にして運用負荷低減

23.3.3 機械学習パイプライン

```

[新規データ S3] → SageMaker Pipelines
    ↳ 前処理 (Processing Job)
    ↳ 訓練 (Training Job、Spot)
    ↳ 評価
      ↳ Model Registry 登録
        ↓
      CodePipeline 承認 → SageMaker Endpoint デプロイ
  
```

23.4 ストリーミング処理

23.4.1 リアルタイムログ分析

```

[App] → CloudWatch Logs → Subscription Filter → Kinesis Firehose
                                             → S3 (長期)
                                             → OpenSearch (短期)
                                             → Athena → QuickSight
  
```

23.4.2 IoT データパイプライン

```

[Devices] → IoT Core → Rules Engine
    ↳ DynamoDB (最新値)
    ↳ Kinesis Data Streams → Flink → Aurora (集計)
    ↳ Firehose → S3 (履歴) → Athena
  
```

23.5 イベント駆動・非同期

23.5.1 注文処理 (Saga)

```

[Web] → API → SQS → Lambda (注文受付)
        ↓
      EventBridge → Step Functions
                    ↳ 在庫確認 → 在庫サービス
                    ↳ 決済処理 → 決済サービス
                    ↳ 配送手配 → 物流サービス
                    ↳ 通知      → SES / SNS
                    ↑
                  (失敗時は補償トランザクション)
  
```

23.5.2 ファンアウト処理

```
[Producer] → SNS Topic → SQS-A → Worker A (メール送信)
                      → SQS-B → Worker B (DB 更新)
                      → SQS-C → Worker C (外部 API 通知)
```

23.6 マイクロサービス通信

23.6.1 ECS / EKS + Service Discovery

```
[ALB / Ingress] → Service A
                  ↓ Service Discovery (Cloud Map / k8s Service)
Service B → DynamoDB
Service C → Aurora
```

23.6.2 VPC Lattice

```
[Service Network]
├ Service A (Lambda)
├ Service B (ECS Fargate)
└ Service C (EC2)
  ↑
Auth Policy (IAM) で認可
```

VPC ピアリング・PrivateLink・ALB を使わず、サービス間通信を抽象化。

23.6.3 App Mesh (Envoy ベース)

ECS / EKS の上で **Envoy** サイドカーを使った Service Mesh。リトライ、サーキットブレーカ、可観測性、mTLS。AWS は VPC Lattice に重心移動中だが、既存 App Mesh 利用は継続。

23.7 マルチアカウント・マルチリージョン

23.7.1 中央 Inspection VPC + Workload VPCs

```
[Workload VPC] → Transit Gateway → Inspection VPC (Network Firewall) → Internet
[Workload VPC] → Transit Gateway → Inspection VPC (Network Firewall) → Internet
                                                    ↑
                                                    Egress VPC
```

すべての外向き通信を中央検査。コンプライアンス重視構成。

23.7.2 Active-Active マルチリージョン

```
[User] → Route 53 (Latency) → 東京 ALB → ECS → Aurora Global Cluster Primary
                                   → バージニア ALB → ECS → Aurora Global Cluster Secondary
                                   ↓
                               CloudFront でキャッシュ統合
```

- DynamoDB Global Tables、Aurora Global Database、S3 Cross-Region Replication
- Route 53 Failover で災害時の切替

23.7.3 マルチアカウント Landing Zone

20 章参照。Control Tower で構築。

23.8 セキュリティ重視構成

23.8.1 ゼロトラスト・ネットワーク

```
[User] → AWS Verified Access (IAM Identity Center + デバイス検証)
                                   → 内部 ALB → ECS → DB
```

VPN なしで社内アプリにアクセス。コンテキスト評価で動的認可。

23.8.2 機密データレイク

```
[源] → KMS 暗号化 → S3 (プライベート、CMK)
                                   ↓ Glue Crawler
                                   Glue Catalog (Lake Formation で行・列単位制御)
                                   ↓
                                   Athena / Redshift Spectrum / EMR (Lake Formation で認可)
                                   ↓
                                   QuickSight
```

23.9 ハイブリッドクラウド

23.9.1 Direct Connect + Transit Gateway

```
[On-prem DC] ↔ Direct Connect ↔ Direct Connect Gateway ↔ Transit Gateway
                                                         ↔ VPC × N
```

23.9.2 Storage Gateway

```

[On-prem NAS] → File Gateway (NFS/SMB) → S3
                ↓
                ライフサイクル → Glacier

```

23.10 災害復旧 (DR)

27 章で詳述。代表 4 パターン (Backup & Restore、Pilot Light、Warm Standby、Multi-Site Active-Active)。

23.11 コスト最適化テンプレート

23.11.1 検証環境の自動停止

```

EventBridge Scheduler (平日 22:00) → Lambda → EC2/RDS Stop
EventBridge Scheduler (平日 08:00) → Lambda → EC2/RDS Start

```

夜間・週末停止で月コスト 1/3 程度に。

23.11.2 Spot を使ったバッチ処理

```

[Job Queue (SQS)] → AWS Batch (Spot) → EC2 Spot Fleet
                  → 中断時の再キュー

```

GPU 計算、ML 訓練、ETL バッチで適用。

23.12 ゲーム配信パターン

23.12.1 マルチプレイヤーゲームサーバ

```

[Player] → Global Accelerator (Anycast)
          → GameLift FleetIQ → EC2 Spot
          → Matchmaking
[ゲーム状態] → Redis (ElastiCache) / DynamoDB
[資産・配信] → S3 + CloudFront

```

23.13 チャットボット / カスタマサポート

23.13.1 Connect + Bedrock

```

[Customer Call] → Amazon Connect → Lex Bot
                ↓

```

```

Lambda → Bedrock (生成 AI)
        → Knowledge Base (社内 RAG)
↓
必要時にエージェント転送

```

23.14 計算ワークロード

23.14.1 HPC

```

[Job] → AWS Batch / ParallelCluster
      → C7i / Hpc7g など計算ファミリー
      → FSx for Lustre (高速ファイル)
      → S3 永続保存
      → Spot で大幅コスト減

```

23.14.2 ML 推論サーバ

```

[Client] → ALB → ECS Fargate (軽量推論)
          → ALB → SageMaker Endpoint (重い推論、Inferentia)

```

23.15 メタパターン：選択の指針

要件	第一選択
常時負荷低・スパイクあり	サーバーレス (Lambda、Fargate Spot、Aurora Serverless)
予測可能な常時負荷	EC2 + Savings Plans / RI、Aurora Provisioned
既存資産活用	Lift & Shift (EC2、RDS)
新規・モダン化前提	サーバーレス + マネージド DB
マルチクラウド要件	Kubernetes (EKS) + Terraform
規制・隔離強い	マルチアカウント + 中央 Inspection VPC
グローバル展開	Aurora Global / DynamoDB Global / Route 53 Latency
低レイテンシ要件	Wavelength / Outposts / Local Zones

23.16 設計時のチェックリスト

新規構成を組むときに自問する項目：

- ・データの所在 と 責任共有モデル は明確か
- ・単一障害点 がないか (AZ、リージョン、IAM、データ)
- ・スケール戦略 は何か (垂直、水平、自動)
- ・セキュリティ層 が適切か (IAM、ネットワーク、データ、検知)
- ・監視と通知 がデフォルトで仕込まれているか

- ・バックアップ・リストア演習を行ったか
- ・コスト構造を把握しているか（時間課金、データ転送、リクエスト）
- ・IaCで再現可能か
- ・終了・削除の手順が決まっているか

すべてに「はい」と答えられない構成は、本番投入前にレビュー必須。

23.17 Anti-pattern（避けるべき構成）

- ・EC2 1 台に全部乗せ（DB、アプリ、キャッシュ、Web）→ SPOF
- ・キーをコードにハードコード → IAM Role、Secrets Manager に
- ・本番アカウントで全部やる → マルチアカウントで隔離
- ・NAT Gateway を立てっぱなしの個人検証 → 検証後即削除
- ・Lambda 単位で 5 個のテーブル JOIN → 設計の見直し（DynamoDB なら Single Table）
- ・巨大 Glue ジョブで全部前処理 → 段階的に Bronze→Silver→Gold
- ・監視を後回し → 最初の 1 日目から CloudWatch Dashboard と GuardDuty
- ・バックアップを取っただけで終わり → 必ず復元演習

24 ハンズオン 1：静的サイトとサーバーレス API

ここから 3 つのハンズオン章では、実際に手を動かして AWS を使う流れを示す。本章は最小コストで完結する **静的サイト + サーバーレス API** の構成。Free Tier 内に収まることを目指す。

24.1 ゴール

- ・静的フロントエンド（HTML/CSS/JS の SPA）を S3 + CloudFront で HTTPS 配信
- ・バックエンド API を API Gateway + Lambda + DynamoDB で構築
- ・カスタムドメインを Route 53 + ACM で適用
- ・全体を AWS CDK（TypeScript）で IaC 化
- ・終了時に `cdk destroy` で完全削除

24.2 前提

- ・AWS アカウント開設済（2 章）、Identity Center または管理者 IAM ユーザーあり
- ・AWS CLI v2 インストール、`aws sts get-caller-identity` で確認できる状態
- ・Node.js 20 以上、npm
- ・自前のドメイン（Route 53 でも他社レジストラでも可）。なければ DNS 部分はスキップしてもよい
- ・想定所要時間：2～3 時間

24.3 構成図

```

[Browser]
  ↓ HTTPS
[Route 53] → [CloudFront] → [S3 (private, OAC)]    ← 静的フロント
                        → /api/* → [API Gateway HTTP API]
                                → [Lambda]
                                → [DynamoDB]
  
```

24.4 ステップ 1：プロジェクト初期化

```

mkdir -p ~/aws-handson/serverless-app && cd $_
npm init -y
npm install -D typescript ts-node @types/node aws-cdk-lib constructs
npx cdk init app --language typescript
  
```

CDK の `bin/` と `lib/` ができる。`lib/serverless-app-stack.ts` に構築を書いていく。

24.4.1 Bootstrap（初回のみ）

CDK はリージョンごとに **Bootstrap**（CDK 用の S3 バケットや IAM ロール）が必要。

```
npx cdk bootstrap aws://<アカウントID>/ap-northeast-1
```

24.5 ステップ 2：Lambda 関数を書く

lambda/api/index.ts を作成。

```
mkdir -p lambda/api
npm install --prefix lambda/api @aws-sdk/client-dynamodb @aws-sdk/lib-dynamodb
```

```
// lambda/api/index.ts
import { DynamoDBClient } from '@aws-sdk/client-dynamodb';
import { DynamoDBDocumentClient, PutCommand, GetCommand, ScanCommand } from '@aws-sdk/lib-dynamodb';
import { randomUUID } from 'crypto';

const ddb = DynamoDBDocumentClient.from(new DynamoDBClient({}));
const TABLE = process.env.TABLE_NAME!;

export const handler = async (event: any) => {
  const method = event.requestContext?.http?.method;
  const path = event.requestContext?.http?.path;

  try {
    if (method === 'GET' && path === '/api/notes') {
      const r = await ddb.send(new ScanCommand({ TableName: TABLE }));
      return ok(r.Items ?? []);
    }
    if (method === 'POST' && path === '/api/notes') {
      const body = JSON.parse(event.body ?? '{}');
      const item = { id: randomUUID(), text: String(body.text ?? ''), createdAt:
Date.now() };
      await ddb.send(new PutCommand({ TableName: TABLE, Item: item }));
      return ok(item);
    }
    if (method === 'GET' && path?.startsWith('/api/notes/')) {
      const id = path.split('/').pop();
      const r = await ddb.send(new GetCommand({ TableName: TABLE, Key: { id } }));
      return r.Item ? ok(r.Item) : { statusCode: 404, body: 'not found' };
    }
    return { statusCode: 404, body: 'not found' };
  } catch (e: any) {
    return { statusCode: 500, body: JSON.stringify({ error: e.message }) };
  }
};

const ok = (body: unknown) => ({
  statusCode: 200,
```

```
headers: { 'content-type': 'application/json' },
body: JSON.stringify(body),
});
```

24.6 ステップ 3 : CDK スタック

```
// lib/serverless-app-stack.ts
import { Stack, StackProps, RemovalPolicy, Duration, CfnOutput } from 'aws-cdk-lib';
import { Construct } from 'constructs';
import * as s3 from 'aws-cdk-lib/aws-s3';
import * as cf from 'aws-cdk-lib/aws-cloudfront';
import * as origins from 'aws-cdk-lib/aws-cloudfront-origins';
import * as ddb from 'aws-cdk-lib/aws-dynamodb';
import * as lambdaNode from 'aws-cdk-lib/aws-lambda-nodejs';
import * as lambda from 'aws-cdk-lib/aws-lambda';
import * as apigw from 'aws-cdk-lib/aws-apigatewayv2';
import * as apigwInteg from 'aws-cdk-lib/aws-apigatewayv2-integrations';
import * as s3deploy from 'aws-cdk-lib/aws-s3-deployment';
import * as path from 'path';

export class ServerlessAppStack extends Stack {
  constructor(scope: Construct, id: string, props?: StackProps) {
    super(scope, id, props);

    // 1. DynamoDB
    const table = new ddb.Table(this, 'NotesTable', {
      partitionKey: { name: 'id', type: ddb.AttributeType.STRING },
      billingMode: ddb.BillingMode.PAY_PER_REQUEST,
      removalPolicy: RemovalPolicy.DESTROY,
      pointInTimeRecovery: true,
    });

    // 2. Lambda
    const fn = new lambdaNode.NodejsFunction(this, 'ApiFn', {
      entry: path.join(__dirname, '..', 'lambda', 'api', 'index.ts'),
      runtime: lambda.Runtime.NODEJS_20_X,
      memorySize: 256,
      timeout: Duration.seconds(10),
      environment: { TABLE_NAME: table.tableName },
    });
    table.grantReadWriteData(fn);

    // 3. API Gateway HTTP API
    const httpApi = new apigw.HttpApi(this, 'HttpApi', {
      corsPreflight: {
        allowOrigins: ['*'],
        allowMethods: [apigw.CorsHttpMethod.ANY],
        allowHeaders: ['content-type'],
      },
    });
```

```

    },
  });
  httpApi.addRoutes({
    path: '/api/{proxy+}',
    methods: [apigw.HttpMethod.ANY],
    integration: new apigwInteg.HttpLambdaIntegration('ApiInt', fn),
  });

  // 4. S3 (静的サイト用、非公開)
  const siteBucket = new s3.Bucket(this, 'SiteBucket', {
    blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL,
    removalPolicy: RemovalPolicy.DESTROY,
    autoDeleteObjects: true,
    enforceSSL: true,
  });

  // 5. CloudFront + OAC
  const dist = new cf.Distribution(this, 'Dist', {
    defaultBehavior: {
      origin: origins.S3BucketOrigin.withOriginAccessControl(siteBucket),
      viewerProtocolPolicy: cf.ViewerProtocolPolicy.REDIRECT_TO_HTTPS,
      cachePolicy: cf.CachePolicy.CACHING_OPTIMIZED,
    },
    additionalBehaviors: {
      '/api/*': {
        origin: new origins.HttpOrigin(`${httpApi.apiId}.execute-api.
${this.region}.amazonaws.com`),
        viewerProtocolPolicy: cf.ViewerProtocolPolicy.REDIRECT_TO_HTTPS,
        allowedMethods: cf.AllowedMethods.ALLOW_ALL,
        cachePolicy: cf.CachePolicy.CACHING_DISABLED,
        originRequestPolicy: cf.OriginRequestPolicy.ALL_VIEWER_EXCEPT_HOST_HEADER,
      },
    },
    defaultRootObject: 'index.html',
    errorResponses: [
      { httpStatus: 403, responseHttpStatus: 200, responsePagePath: '/index.html' },
      { httpStatus: 404, responseHttpStatus: 200, responsePagePath: '/index.html' },
    ],
  });

  // 6. 静的サイトのデプロイ
  new s3deploy.BucketDeployment(this, 'DeploySite', {
    sources: [s3deploy.Source.asset(path.join(__dirname, '..', 'site'))],
    destinationBucket: siteBucket,
    distribution: dist,
    distributionPaths: ['/*'],
  });

  new CfnOutput(this, 'CloudFrontUrl', { value: `https://
${dist.distributionDomainName}` });

```

```

    }
  }
}

```

24.7 ステップ 4：静的フロントエンド

最小限の SPA を `site/index.html` として作る。

```
mkdir -p site
```

```

<!-- site/index.html -->
<!doctype html>
<html lang="ja">
<head>
<meta charset="utf-8">
<title>Notes</title>
<style>
  body { font-family: sans-serif; max-width: 600px; margin: 2em auto; }
  input, button { font-size: 1em; padding: 0.4em; }
  ul { padding: 0; }
  li { list-style: none; padding: 0.5em; border-bottom: 1px solid #ddd; }
</style>
</head>
<body>
<h1>メモ</h1>
<input id="text" placeholder="新しいメモ">
<button id="add">追加</button>
<ul id="list"></ul>
<script>
async function load() {
  const r = await fetch('/api/notes');
  const items = await r.json();
  document.getElementById('list').innerHTML = items
    .map(i => `<li>${i.text}<br><small>${new Date(i.createdAt).toLocaleString()}</small></li>`)
    .join('');
}
document.getElementById('add').onclick = async () => {
  const text = document.getElementById('text').value;
  if (!text) return;
  await fetch('/api/notes', {
    method: 'POST',
    headers: { 'content-type': 'application/json' },
    body: JSON.stringify({ text }),
  });
  document.getElementById('text').value = '';
  load();
};

```

```
load();
</script>
</body>
</html>
```

24.8 ステップ 5：デプロイ

```
npx cdk deploy
```

CloudFrontUrl の出力 URL (`https://dXXXXXXXX.cloudfront.net`) にブラウザでアクセス。「メモを追加」して保存・再読込で残ることを確認。

24.9 ステップ 6：カスタムドメイン

Route 53 でホストゾーンが既にある前提 (`example.com`)。

24.9.1 ACM 証明書 (us-east-1)

```
aws acm request-certificate \
  --domain-name 'notes.example.com' \
  --validation-method DNS \
  --region us-east-1
```

検証 CNAME を Route 53 に追加 (コンソールで「Create records in Route 53」を 1 クリック)。

24.9.2 CDK スタックに追加

```
import * as acm from 'aws-cdk-lib/aws-certificatemanager';
import * as r53 from 'aws-cdk-lib/aws-route53';
import * as r53Targets from 'aws-cdk-lib/aws-route53-targets';

const zone = r53.HostedZone.fromLookup(this, 'Zone', { domainName: 'example.com' });
const cert = acm.Certificate.fromCertificateArn(this, 'Cert',
  'arn:aws:acm:us-east-1:<account>:certificate/<id>');

const dist = new cf.Distribution(this, 'Dist', {
  // ... 上記に加えて
  domainNames: ['notes.example.com'],
  certificate: cert,
  // ...
});

new r53.ARecord(this, 'AliasRecord', {
  zone,
  recordName: 'notes',
```

```
target: r53.RecordTarget.fromAlias(new r53Targets.CloudFrontTarget(dist)),
});
```

`npx cdk deploy` で再適用。 `https://notes.example.com` でアクセスできるようになる。

24.10 ステップ 7：監視を 1 枚追加

```
import * as cw from 'aws-cdk-lib/aws-cloudwatch';

new cw.Dashboard(this, 'Dashboard', {
  dashboardName: 'NotesApp',
  widgets: [
    [
      new cw.GraphWidget({
        title: 'API Requests',
        left: [httpApi.metric('Count', { statistic: 'Sum', period:
Duration.minutes(5) })],
      }),
      new cw.GraphWidget({
        title: 'Lambda Errors',
        left: [fn.metricErrors({ statistic: 'Sum' })],
      }),
    ],
    [
      new cw.GraphWidget({
        title: 'DynamoDB Read/Write Capacity',
        left: [
          table.metricConsumedReadCapacityUnits(),
          table.metricConsumedWriteCapacityUnits(),
        ],
      }),
    ],
  ],
});
```

24.11 ステップ 8：GitHub Actions で CI/CD（任意）

OIDC で AWS にロール引き受け、 `cdk deploy` を自動化する。

```
# .github/workflows/deploy.yml
name: deploy
on:
  push:
    branches: [main]
permissions:
  id-token: write
```



```

contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::<account>:role/github-actions-cdk
          aws-region: ap-northeast-1
      - run: npx cdk deploy --require-approval never

```

ロール側の信頼ポリシーは3章の例を参照（`StringEquals` で `repo:owner/repo:ref:refs/heads/main` に固定）。

24.12 ステップ 9：検証 → 後片付け

確認が終わったら必ず削除する。

```
npx cdk destroy
```

CloudFront ディストリビューションは削除に 10～15 分かかる。完了を待つ。

S3 バケットに残ったオブジェクトがあれば手動で空にしてから再実行。

24.13 学んだこと

- S3 + CloudFront + OAC で安全な静的配信
- API Gateway HTTP API + Lambda でサーバーレス API
- DynamoDB の `PAY_PER_REQUEST` で完全従量
- CDK による IaC、`cdk destroy` でクリーンアップ
- Route 53 + ACM でカスタムドメイン HTTPS
- CloudWatch Dashboard で監視を 1 枚

24.14 発展課題

- Cognito User Pool で認証を追加し、ユーザーごとのメモにする
- DynamoDB Streams + Lambda で更新通知を SES でメール送信
- Lambda レイヤーで共通ライブラリを切り出し
- API Gateway に **使用量プラン** + **API キー** を入れて公開
- CloudWatch RUM でフロントのリアルユーザーモニタリング
- WAF を CloudFront に紐付けて攻撃ブロック

24.15 トラブルシューティング

症状	対処
cdk bootstrap でエラー	IAM 権限不足、AdministratorAccess で再試行
Lambda が Cannot find module	NodejsFunction の bundler 動作確認、esbuild インストール
API が CORS エラー	corsPreflight 設定、フロントから同一ドメイン経由で叩く
CloudFront が 403	OAC ポリシー、S3 オブジェクト存在、ディストリビューション 伝播待ち
DynamoDB 書き込み権限なし	grantReadWriteData 呼び出し済みか
カスタムドメインが反映しない	ACM 検証完了、CloudFront 再デプロイ、DNS TTL 待ち
請求が発生してる	cdk destroy、コンソールで NAT/EIP/EBS 残存確認

25 ハンズオン 2：3 層 Web アプリ

24 章とは対照的に、本章は EC2 + ALB + RDS の古典的な 3 層構成を Terraform で組み立てる。レガシーアプリの移植や「サーバーレスではないけど運用したい」ケースを想定する。

25.1 ゴール

- VPC をマルチ AZ で構築
- ALB → EC2 (Auto Scaling) → RDS (Aurora MySQL) の 3 層
- 踏み台レス で SSM Session Manager 接続
- 全構成を Terraform で管理
- 検証後 `terraform destroy` で完全削除

25.2 前提

- AWS アカウント (2 章)、Terraform 1.10 以上
- AWS CLI v2、`aws sts get-caller-identity` で確認
- 想定所要時間：3～4 時間

25.3 構成図

```
[Browser]
  ↓ HTTPS
[Route 53] → [ALB (public subnets, multi-AZ)]
               ↓
             [EC2 × 2 (private subnets, ASG)]
               ↓ MySQL
[Aurora MySQL Cluster (private subnets, Multi-AZ)]
```

接続用：開発者 → SSM Session Manager → EC2

DB 接続：開発者 → SSM ポートフォワード → RDS

25.4 ステップ 1：プロジェクト構造

```
mkdir -p ~/aws-handson/3tier && cd $_
mkdir -p modules/{vpc,alb,asg,db}
touch main.tf variables.tf outputs.tf versions.tf terraform.tfvars
```

```
# versions.tf
terraform {
  required_version = ">= 1.10"
  required_providers {
```

```

    aws = { source = "hashicorp/aws", version = "~> 5.70" }
  }
  backend "s3" {
    bucket      = "my-tfstate-20260421"
    key         = "3tier/terraform.tfstate"
    region      = "ap-northeast-1"
    use_lockfile = true    # Terraform 1.10+ S3 ネイティブロック
    encrypt     = true
  }
}

provider "aws" {
  region = var.region
  default_tags {
    tags = {
      Project      = "3tier-handson"
      Environment = var.env
      ManagedBy    = "terraform"
    }
  }
}

```

```

# variables.tf
variable "region"      { default = "ap-northeast-1" }
variable "env"         { default = "dev" }
variable "vpc_cidr"    { default = "10.20.0.0/16" }
variable "azs"         { default = ["ap-northeast-1a", "ap-northeast-1c"] }
variable "instance_type" { default = "t3.small" }
variable "db_engine_version" { default = "8.0.mysql_aurora.3.07.1" }
variable "db_instance_class" { default = "db.t4g.medium" }

```

```

# terraform.tfvars
env = "dev"

```

25.5 ステップ 2 : VPC モジュール

```

# modules/vpc/main.tf
variable "name"      {}
variable "cidr"      {}
variable "azs"       {}

locals {
  public_cidrs = [cidrsubnet(var.cidr, 8, 0), cidrsubnet(var.cidr, 8, 1)]
  app_cidrs    = [cidrsubnet(var.cidr, 8, 10), cidrsubnet(var.cidr, 8, 11)]
  db_cidrs     = [cidrsubnet(var.cidr, 8, 20), cidrsubnet(var.cidr, 8, 21)]
}

```

```

resource "aws_vpc" "this" {
  cidr_block      = var.cidr
  enable_dns_hostnames = true
  enable_dns_support = true
  tags = { Name = "${var.name}-vpc" }
}

resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id
  tags = { Name = "${var.name}-igw" }
}

resource "aws_subnet" "public" {
  count = length(var.azs)
  vpc_id      = aws_vpc.this.id
  cidr_block  = local.public_cidrs[count.index]
  availability_zone = var.azs[count.index]
  map_public_ip_on_launch = false
  tags = { Name = "${var.name}-public-${var.azs[count.index]}" }
}

resource "aws_subnet" "app" {
  count = length(var.azs)
  vpc_id      = aws_vpc.this.id
  cidr_block  = local.app_cidrs[count.index]
  availability_zone = var.azs[count.index]
  tags = { Name = "${var.name}-app-${var.azs[count.index]}" }
}

resource "aws_subnet" "db" {
  count = length(var.azs)
  vpc_id      = aws_vpc.this.id
  cidr_block  = local.db_cidrs[count.index]
  availability_zone = var.azs[count.index]
  tags = { Name = "${var.name}-db-${var.azs[count.index]}" }
}

resource "aws_eip" "nat" {
  count = length(var.azs)
  domain = "vpc"
  tags = { Name = "${var.name}-nat-eip-${count.index}" }
}

resource "aws_nat_gateway" "this" {
  count = length(var.azs)
  allocation_id = aws_eip.nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id
  tags = { Name = "${var.name}-nat-${count.index}" }
  depends_on = [aws_internet_gateway.this]
}

```

```

}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.this.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }
  tags = { Name = "${var.name}-rt-public" }
}

resource "aws_route_table_association" "public" {
  count = length(var.azs)
  subnet_id = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table" "app" {
  count = length(var.azs)
  vpc_id = aws_vpc.this.id
  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.this[count.index].id
  }
  tags = { Name = "${var.name}-rt-app-${count.index}" }
}

resource "aws_route_table_association" "app" {
  count = length(var.azs)
  subnet_id = aws_subnet.app[count.index].id
  route_table_id = aws_route_table.app[count.index].id
}

# DB サブネットはインターネット接続不要
resource "aws_route_table" "db" {
  vpc_id = aws_vpc.this.id
  tags = { Name = "${var.name}-rt-db" }
}

resource "aws_route_table_association" "db" {
  count = length(var.azs)
  subnet_id = aws_subnet.db[count.index].id
  route_table_id = aws_route_table.db.id
}

# S3 ゲートウェイエンドポイント（無料）
resource "aws_vpc_endpoint" "s3" {
  vpc_id          = aws_vpc.this.id
  service_name    = "com.amazonaws.${data.aws_region.this.name}.s3"
  vpc_endpoint_type = "Gateway"
}

```

```

    route_table_ids = concat([aws_route_table.app[0].id, aws_route_table.app[1].id,
aws_route_table.db.id])
}

data "aws_region" "this" {}

output "vpc_id"          { value = aws_vpc.this.id }
output "public_subnets" { value = aws_subnet.public[*].id }
output "app_subnets"    { value = aws_subnet.app[*].id }
output "db_subnets"     { value = aws_subnet.db[*].id }

```

25.6 ステップ 3 : ALB モジュール

```

# modules/alb/main.tf
variable "name"      {}
variable "vpc_id"    {}
variable "subnets"  {}

resource "aws_security_group" "alb" {
  name     = "${var.name}-alb-sg"
  vpc_id   = var.vpc_id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_lb" "this" {
  name                = "${var.name}-alb"
  load_balancer_type = "application"
  subnets            = var.subnets
  security_groups     = [aws_security_group.alb.id]
}

resource "aws_lb_target_group" "app" {
  name     = "${var.name}-tg"
  port     = 80
  protocol = "HTTP"
}

```

```

vpc_id      = var.vpc_id
target_type = "instance"
health_check {
  path          = "/health"
  interval      = 30
  timeout       = 5
  healthy_threshold = 2
  unhealthy_threshold = 3
}
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.this.arn
  port              = 80
  protocol          = "HTTP"
  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.app.arn
  }
}

output "alb_sg_id"      { value = aws_security_group.alb.id }
output "target_group_arn" { value = aws_lb_target_group.app.arn }
output "alb_dns_name"   { value = aws_lb.this.dns_name }

```

25.7 ステップ 4 : ASG モジュール

```

# modules/asg/main.tf
variable "name"      {}
variable "vpc_id"    {}
variable "subnets"  {}
variable "instance_type" {}
variable "alb_sg_id" {}
variable "tg_arn"     {}
variable "db_sg_id"  {}

data "aws_ssm_parameter" "ami" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

resource "aws_security_group" "app" {
  name     = "${var.name}-app-sg"
  vpc_id   = var.vpc_id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
  }
}

```



```

    security_groups = [var.alb_sg_id]
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_iam_role" "ec2" {
  name = "${var.name}-ec2-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
      Action = "sts:AssumeRole"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "ssm" {
  role      = aws_iam_role.ec2.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
}

resource "aws_iam_instance_profile" "ec2" {
  name = "${var.name}-ec2-profile"
  role = aws_iam_role.ec2.name
}

resource "aws_launch_template" "app" {
  name_prefix = "${var.name}-lt-"
  image_id    = data.aws_ssm_parameter.ami.value
  instance_type = var.instance_type

  iam_instance_profile {
    name = aws_iam_instance_profile.ec2.name
  }

  metadata_options {
    http_tokens              = "required" # IMDSv2 必須
    http_put_response_hop_limit = 2
    http_endpoint            = "enabled"
    instance_metadata_tags    = "enabled"
  }

  network_interfaces {
    security_groups = [aws_security_group.app.id]
  }
}

```

```

    associate_public_ip_address = false
  }

  user_data = base64encode(<<-EOT
    #!/bin/bash
    dnf update -y
    dnf install -y nginx
    cat > /usr/share/nginx/html/index.html <<EOF
    <h1>Hello from $(hostname)</h1>
    EOF
    cat > /usr/share/nginx/html/health <<EOF
    OK
    EOF
    systemctl enable --now nginx
  EOT
)

  tag_specifications {
    resource_type = "instance"
    tags = { Name = "${var.name}-app" }
  }
}

resource "aws_autoscaling_group" "app" {
  name                = "${var.name}-asg"
  min_size            = 2
  max_size            = 4
  desired_capacity    = 2
  vpc_zone_identifier = var.subnets
  target_group_arns   = [var.tg_arn]
  health_check_type   = "ELB"
  health_check_grace_period = 60

  launch_template {
    id      = aws_launch_template.app.id
    version = "$Latest"
  }

  tag {
    key          = "Name"
    value        = "${var.name}-app"
    propagate_at_launch = true
  }
}

resource "aws_autoscaling_policy" "cpu" {
  name                = "${var.name}-cpu-policy"
  autoscaling_group_name = aws_autoscaling_group.app.name
  policy_type          = "TargetTrackingScaling"
  target_tracking_configuration {

```

```

    predefined_metric_specification {
      predefined_metric_type = "ASGAverageCPUUtilization"
    }
    target_value = 60
  }
}

# DB SG への許可 (DB 側で受ける)
resource "aws_security_group_rule" "to_db" {
  type           = "ingress"
  from_port      = 3306
  to_port        = 3306
  protocol       = "tcp"
  security_group_id = var.db_sg_id
  source_security_group_id = aws_security_group.app.id
}

output "app_sg_id" { value = aws_security_group.app.id }

```

25.8 ステップ 5 : DB モジュール

```

# modules/db/main.tf
variable "name" {}
variable "vpc_id" {}
variable "subnets" {}
variable "engine_version" {}
variable "instance_class" {}

resource "aws_security_group" "db" {
  name = "${var.name}-db-sg"
  vpc_id = var.vpc_id
  egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_db_subnet_group" "this" {
  name = "${var.name}-dbsg"
  subnet_ids = var.subnets
}

resource "random_password" "master" {
  length = 24
  special = true
}

```

```

resource "aws_secretsmanager_secret" "db" {
  name           = "${var.name}/db/master"
  recovery_window_in_days = 0
}

resource "aws_secretsmanager_secret_version" "db" {
  secret_id = aws_secretsmanager_secret.db.id
  secret_string = jsonencode({
    username = "admin"
    password = random_password.master.result
  })
}

resource "aws_rds_cluster" "this" {
  cluster_identifier = "${var.name}-aurora"
  engine             = "aurora-mysql"
  engine_version     = var.engine_version
  database_name      = "appdb"
  master_username    = "admin"
  master_password    = random_password.master.result
  db_subnet_group_name = aws_db_subnet_group.this.name
  vpc_security_group_ids = [aws_security_group.db.id]
  storage_encrypted   = true
  skip_final_snapshot = true
  deletion_protection = false
  serverlessv2_scaling_configuration {
    min_capacity = 0.5
    max_capacity = 4
  }
}

resource "aws_rds_cluster_instance" "writer" {
  identifier      = "${var.name}-aurora-1"
  cluster_identifier = aws_rds_cluster.this.id
  instance_class  = "db.serverless"
  engine          = aws_rds_cluster.this.engine
  engine_version  = aws_rds_cluster.this.engine_version
}

output "db_sg_id" { value = aws_security_group.db.id }
output "endpoint" { value = aws_rds_cluster.this.endpoint }
output "secret_arn" { value = aws_secretsmanager_secret.db.arn }

```

25.9 ステップ 6：ルートで配線

```

# main.tf
module "vpc" {

```

```

source = "./modules/vpc"
name   = "${var.env}-3tier"
cidr   = var.vpc_cidr
azs    = var.azs
}

module "alb" {
  source   = "./modules/alb"
  name     = "${var.env}-3tier"
  vpc_id   = module.vpc.vpc_id
  subnets = module.vpc.public_subnets
}

module "db" {
  source           = "./modules/db"
  name             = "${var.env}-3tier"
  vpc_id           = module.vpc.vpc_id
  subnets         = module.vpc.db_subnets
  engine_version   = var.db_engine_version
  instance_class   = var.db_instance_class
}

module "asg" {
  source           = "./modules/asg"
  name             = "${var.env}-3tier"
  vpc_id           = module.vpc.vpc_id
  subnets         = module.vpc.app_subnets
  instance_type    = var.instance_type
  alb_sg_id        = module.alb.alb_sg_id
  tg_arn           = module.alb.target_group_arn
  db_sg_id         = module.db.db_sg_id
}

# outputs.tf
output "alb_dns" { value = module.alb.alb_dns_name }
output "db_endpoint" { value = module.db.endpoint }
output "db_secret_arn" { value = module.db.secret_arn }

```

25.10 ステップ 7：適用

```

terraform init
terraform plan
terraform apply -auto-approve

```

`alb_dns` の出力 (`xxx.ap-northeast-1.elb.amazonaws.com`) にブラウザでアクセスして「Hello from ip-…」が見えれば成功。

25.11 ステップ 8 : SSM 接続

EC2 にログインしたい場合：

```
aws ssm start-session --target i-0abc123...
```

DB に接続したい場合（手元の端末から）：

```
aws ssm start-session \
  --target i-0bastionが必要ならここに（or app EC2 を踏み台にしてもよい） \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters "host=$(terraform output -raw
db_endpoint),portNumber=3306,localPortNumber=13306"
```

別タームで：

```
SECRET=$(aws secretsmanager get-secret-value --secret-id $(terraform output -raw
db_secret_arn) --query SecretString --output text)
PASS=$(echo $SECRET | jq -r .password)
mysql -h 127.0.0.1 -P 13306 -u admin -p$PASS appdb
```

25.12 ステップ 9 : HTTPS 化

ACM 証明書を 同一リージョン（ALB 用なので東京）で取得し、ALB リスナーを HTTPS（443）に切替、80 は 443 にリダイレクト。Route 53 Alias で `app.example.com` を ALB に向ける。

```
resource "aws_acm_certificate" "alb" {
  domain_name      = "app.example.com"
  validation_method = "DNS"
}

# ... DNS 検証レコード追加（aws_route53_record）と aws_acm_certificate_validation

resource "aws_lb_listener" "https" {
  load_balancer_arn = aws_lb.this.arn
  port              = 443
  protocol          = "HTTPS"
  ssl_policy        = "ELBSecurityPolicy-TLS13-1-2-2021-06"
  certificate_arn    = aws_acm_certificate_validation.alb.certificate_arn
  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.app.arn
  }
}

# 80 → 443 リダイレクト（既存 listener の default_action を redirect に変更）
```

25.13 ステップ 10：後片付け

```
terraform destroy -auto-approve
```

NAT Gateway があるため、destroy せずに放置すると 月数千円～数万円 かかる。必ず実行する。

S3 backend の state ファイルは残る（再利用するため）。本当に不要なら手動で削除。

25.14 学んだこと

- VPC のマルチ AZ 構築：パブリック / アプリ / DB の 3 層サブネット
- NAT Gateway と S3 ゲートウェイエンドポイント の併用
- ALB → ASG → EC2 の伝統的構成
- Aurora Serverless v2 で 0.5 ACU から自動スケール
- IMDSv2 必須化、SSM Session Manager、Secrets Manager で安全運用
- Terraform モジュール構造、S3 ネイティブブロック

25.15 発展課題

- WAF を ALB に紐付け
- Auto Scaling のスケジュールスケーリング（朝起動・夜縮退）
- CloudWatch Synthetics で外形監視
- Aurora にリードレプリカ追加、アプリで Reader/Writer 分割
- CodeDeploy で Blue/Green デプロイ
- Backup でクロスリージョンスナップショット

25.16 トラブルシューティング

症状	対処
terraform apply で AccessDenied	プロファイル、IAM 権限、ロール Assume
ALB ヘルスチェックが OutOfService	SG (ALB→EC2 80)、/health 200、起動猶予時間
EC2 が起動直後に終了	User Data エラーを cloud-init ログで確認
SSM start-session が失敗	SSM エージェント、IAM ロール、VPC エンドポイント
DB に接続できない	SG (App→DB 3306)、Subnet group、Secret パスワード
destroy で詰まる	S3 バケットの空化、ENI 残存、依存関係の手動解除

26 ハンズオン 3：データレイク基礎

S3 を中心としたデータレイク構築のミニ実装。Bronze / Silver / Gold の 3 層と、Glue・Athena・QuickSight を使う。

26.1 ゴール

- **Bronze**：生 JSON ログを S3 に蓄積
- **Silver**：Glue ETL で Parquet 化、パーティション化
- **Gold**：日次集計テーブルを Athena CTAS で生成
- **Catalog**：Glue Data Catalog にメタデータ登録
- **可視化**：QuickSight でダッシュボード
- **自動化**：EventBridge Scheduler で日次実行

26.2 構成図

```
[App / Firehose] → s3://lake/bronze/<table>/year=/month=/day=/
                    ↓ Glue Crawler
                    Glue Data Catalog
                    ↓ Glue ETL Job (PySpark)
                    s3://lake/silver/<table>/year=/month=/day=/ Parquet
                    ↓ Athena CTAS (毎日)
                    s3://lake/gold/<dataset>/                      集計済み
                    ↓
                    QuickSight ダッシュボード
```

26.3 ステップ 1：S3 バケット準備

```
REGION=ap-northeast-1
ACCT=$(aws sts get-caller-identity --query Account --output text)
BUCKET="lake-${ACCT}-${REGION}"

aws s3api create-bucket --bucket $BUCKET \
  --region $REGION \
  --create-bucket-configuration LocationConstraint=$REGION

aws s3api put-bucket-versioning --bucket $BUCKET \
  --versioning-configuration Status=Enabled

aws s3api put-public-access-block --bucket $BUCKET \
  --public-access-block-configuration \

"BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true"
```



```
# ライフサイクル：古いバージョン30日で削除、不完全マルチパート7日で削除
cat > lifecycle.json <<'EOF'
{ "Rules": [{
  "Id": "expire", "Status": "Enabled", "Filter": {},
  "NoncurrentVersionExpiration": { "NoncurrentDays": 30 },
  "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
}]}
EOF
aws s3api put-bucket-lifecycle-configuration --bucket $BUCKET \
  --lifecycle-configuration file://lifecycle.json
```

26.4 ステップ 2：サンプル生データ投入

```
# gen.py
import json, random, uuid, datetime, os, sys
N = int(sys.argv[1]) if len(sys.argv) > 1 else 1000
day = sys.argv[2] if len(sys.argv) > 2 else datetime.date.today().isoformat()
events = []
for _ in range(N):
    events.append({
        "event_id": str(uuid.uuid4()),
        "user_id": f"u{random.randint(1, 100)}",
        "event_type": random.choice(["view", "click", "purchase"]),
        "amount": round(random.uniform(0, 1000), 2),
        "ts": f"{day}T{random.randint(0,23):02d}:{random.randint(0,59):02d}:"
        {random.randint(0,59):02d}Z"
    })
print("\n".join(json.dumps(e) for e in events))
```

```
# 数日分のデータを生成して投入
for d in 2026-04-{18,19,20,21}; do
  YYYY=${d:0:4}; MM=${d:5:2}; DD=${d:8:2}
  python3 gen.py 5000 $d > /tmp/events.jsonl
  aws s3 cp /tmp/events.jsonl \
    s3://$BUCKET/bronze/events/year=$YYYY/month=$MM/day=$DD/events.jsonl
done

aws s3 ls s3://$BUCKET/bronze/events/ --recursive
```

26.5 ステップ 3：Glue Database 作成と Crawler

```
# Database 作成
aws glue create-database --database-input '{"Name":"lake_db"}'

# Crawler 用 IAM ロール作成
```

```

cat > trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":
{"Service":"glue.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
aws iam create-role --role-name GlueLakeRole --assume-role-policy-document file://
trust.json
aws iam attach-role-policy --role-name GlueLakeRole \
--policy-arn arn:aws:iam::aws:policy/service-role/AWSGlueServiceRole

cat > s3policy.json <<EOF
{"Version":"2012-10-17","Statement":[
  {"Effect":"Allow","Action":
["s3:GetObject","s3:PutObject","s3:DeleteObject","s3:ListBucket"],
  "Resource":["arn:aws:s3:::$BUCKET","arn:aws:s3:::$BUCKET/*"]}
]}
EOF
aws iam put-role-policy --role-name GlueLakeRole --policy-name S3Access \
--policy-document file://s3policy.json

# Crawler 作成
aws glue create-crawler \
--name lake-bronze-events \
--role GlueLakeRole \
--database-name lake_db \
--targets '{"S3Targets":[{"Path":"s3://$BUCKET/bronze/events/"}]}'

aws glue start-crawler --name lake-bronze-events
# 数分待つ
sleep 90
aws glue get-table --database-name lake_db --name events --query
'Table.StorageDescriptor.Columns'

```

26.6 ステップ 4：Athena でクエリ

Athena 結果出力用 S3 バケットを設定（`s3://lake-XXXX/athena-results/`）。コンソールで指定してもよいし、CLI でも。

```

# Workgroup の結果出力先設定
aws athena update-work-group \
--work-group primary \
--configuration-updates "ResultConfigurationUpdates={OutputLocation=s3://$BUCKET/
athena-results/}"

# クエリ実行
QID=$(aws athena start-query-execution \
--query-string "SELECT event_type, COUNT(*) FROM lake_db.events GROUP BY event_type"
\
--result-configuration "OutputLocation=s3://$BUCKET/athena-results/" \

```

```
--query QueryExecutionId --output text)
sleep 5
aws athena get-query-results --query-execution-id $QID
```

26.7 ステップ 5 : Glue ETL ジョブで Silver 生成

silver.py (PySpark スクリプト) を S3 に置く。

```
# silver.py
import sys
from awsglue.utils import getResolvedOptions
from awsglue.context import GlueContext
from pyspark.context import SparkContext
from awsglue.dynamicframe import DynamicFrame

args = getResolvedOptions(sys.argv, ['JOB_NAME', 'BUCKET'])
sc = SparkContext()
gc = GlueContext(sc)
spark = gc.spark_session

src = gc.create_dynamic_frame.from_catalog(
    database="lake_db", table_name="events"
)

df = src.toDF()
df = df.withColumn("ts", df["ts"].cast("timestamp"))
df = df.dropna(subset=["event_id"])

dyf = DynamicFrame.fromDF(df, gc, "silver")
gc.write_dynamic_frame.from_options(
    frame=dyf,
    connection_type="s3",
    connection_options={
        "path": f"s3://{args['BUCKET']}/silver/events/",
        "partitionKeys": ["year", "month", "day"]
    },
    format="parquet"
)
```

```
aws s3 cp silver.py s3://$BUCKET/scripts/silver.py

aws glue create-job \
  --name lake-silver-events \
  --role GlueLakeRole \
  --command "Name=glueetl,ScriptLocation=s3://$BUCKET/scripts/silver.py,PythonVersion=3" \
  --default-arguments "{ \"--BUCKET\": \"$BUCKET\", \"--enable-glue-datacatalog\":
```

```
\ "true\"}" \
  --glue-version 5.0 \
  --worker-type G.1X \
  --number-of-workers 2

aws glue start-job-run --job-name lake-silver-events
```

完了したら Crawler を Silver にも仕掛けて Catalog に登録。

```
aws glue create-crawler \
  --name lake-silver-events \
  --role GlueLakeRole \
  --database-name lake_db \
  --targets "{\"S3Targets\": [{\"Path\": \"s3://$BUCKET/silver/events/\"}]}"
aws glue start-crawler --name lake-silver-events
```

これで Athena から `lake_db.events` (Silver, Parquet) が高速にクエリできる。

26.8 ステップ 6 : Athena CTAS で Gold 生成

```
-- Athena コンソールで実行
CREATE TABLE lake_db.events_daily_summary
WITH (
  format = 'PARQUET',
  external_location = 's3://lake-XXXX/gold/events_daily_summary/',
  partitioned_by = ARRAY['day']
) AS
SELECT
  event_type,
  user_id,
  SUM(amount) AS total_amount,
  COUNT(*) AS cnt,
  CONCAT(year, '-', month, '-', day) AS day
FROM lake_db.events
GROUP BY event_type, user_id, year, month, day;
```

定期更新は `INSERT INTO ... SELECT` を Glue Workflow / EventBridge Scheduler で。

26.9 ステップ 7 : QuickSight で可視化

1. QuickSight にサインアップ (Free Trial 30 日)
2. 「Datasets → New dataset → Athena」
3. Workgroup は `primary`、Database は `lake_db`、Table は `events_daily_summary`
4. Direct query または SPICE インポート (小規模なら SPICE)
5. Analysis を作成、棒グラフ・折れ線グラフ・ピボット
6. Dashboard として共有

26.10 ステップ 8：日次自動化

```
# scheduler.tf 抜粋
resource "aws_scheduler_schedule" "daily_etl" {
  name                = "daily-etl"
  schedule_expression = "cron(0 1 * * ? *)" # 毎日 01:00 UTC
  schedule_expression_timezone = "Asia/Tokyo"
  flexible_time_window { mode = "OFF" }
  target {
    arn      = "arn:aws:scheduler::aws-sdk:glue:startJobRun"
    role_arn = aws_iam_role.scheduler.arn
    input    = jsonencode({ JobName = "lake-silver-events" })
  }
}
```

Step Functions でクローラ → ETL → CTAS の連結も可。

26.11 ステップ 9：Lake Formation でアクセス制御（任意）

Lake Formation を有効化し、`lake_db.events` の特定列（例：`user_id` をマスク）を別グループに見せるなどの行・列・タグベース制御を導入できる。本格運用では必須。

26.12 ステップ 10：後片付け

```
# Glue
aws glue delete-job --job-name lake-silver-events
aws glue delete-crawler --name lake-bronze-events
aws glue delete-crawler --name lake-silver-events
aws glue delete-table --database-name lake_db --name events
aws glue delete-table --database-name lake_db --name events_daily_summary
aws glue delete-database --name lake_db

# IAM
aws iam delete-role-policy --role-name GlueLakeRole --policy-name S3Access
aws iam detach-role-policy --role-name GlueLakeRole \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSGlueServiceRole
aws iam delete-role --role-name GlueLakeRole

# QuickSight: コンソールから「Manage QuickSight → Account settings → Unsubscribe」(必要なら)

# S3
aws s3 rm s3://$BUCKET --recursive
aws s3api delete-bucket --bucket $BUCKET
```

26.13 学んだこと

- Medallion アーキテクチャ：Bronze / Silver / Gold の責務分離
- S3 ライフサイクル + バージョニング で安全運用
- Glue Crawler + Catalog でメタデータ集約
- Glue ETL (PySpark) でデータ加工
- Athena CTAS でサーバーレス DWH 風
- QuickSight で BI ダッシュボード
- EventBridge Scheduler で日次自動化

26.14 発展課題

- S3 Tables (Iceberg) で Bronze を Iceberg 化、UPSERT/DELETE をサポート
- Lake Formation で行・列・タグレベル権限
- Glue DataBrew でノーコード前処理
- Athena Federated Query で DynamoDB / RDS と JOIN
- Redshift Spectrum で同じデータを Redshift から
- DataZone でデータカタログ + ガバナンスの組織化
- SageMaker Data Wrangler で ML 用前処理に拡張

26.15 トラブルシューティング

症状	対処
Crawler が Glue ロールでアクセス拒否	S3 バケットポリシー、ロールアタッチ確認
Athena が空	パーティション登録、Crawler 結果、 <code>MSCK REPAIR TABLE</code>
ETL ジョブが OOM	DPU、Worker サイズアップ、partitioning 見直し
Athena が高い	パーティション、列指向 (Parquet)、 <code>SELECT *</code> 回避
QuickSight 接続失敗	Service Role 権限、Athena Workgroup 結果バケット
パスのケース揺れ	<code>year=2026/month=04/day=21</code> のようにゼロ埋め統一

27 ディザスタリカバリ戦略

リージョン障害・AZ 障害・大規模アカウント事故・ランサムウェア・人為ミスから復旧するための AWS 設計。本章では RPO/RTO の決め方、4 つの代表 DR パターン、クロスリージョン構成、バックアップ戦略、演習 を扱う。

27.1 RPO と RTO

DR の話は必ずこの 2 つの数値から始まる。

指標	意味
RPO (Recovery Point Objective)	どこまでデータ損失を許せるか (直近何分・何時間まで戻る)
RTO (Recovery Time Objective)	復旧までに何分・何時間以内に戻すか

RPO = 0、RTO = 0 は理論上のみ。実際にはコストとのトレードオフで決める。経営判断の話で、エンジニアだけでは決まらない。

27.2 4 つの代表 DR パターン

パターン	RPO 目安	RTO 目安	概要
Backup & Restore	時間～日	時間～日	バックアップだけ保管、災害時に再構築
Pilot Light	分～時間	10 分～時間	DR リージョンに最小限を稼働、災害時に拡張
Warm Standby	分	分～10 分	DR リージョンで縮小版が常時稼働
Multi-Site Active-Active	秒	秒～分	両リージョンで本番、Route 53 で切替

下に行くほど RPO/RTO が短く、コストが上がる。

27.3 Backup & Restore

最もシンプル・低コスト。

27.3.1 構成

[Primary] EBS / RDS / S3
 ↓ AWS Backup (クロスリージョンコピー)
 [DR Region] バックアップデータのみ保管
 ↓ (災害時)
 インフラを IaC で立てる + バックアップから復元

27.3.2 採用ケース

- ・ 開発・検証環境
- ・ バッチ系・夜間バッチで RTO 余裕あり
- ・ 法令上のバックアップ義務はあるが、即時復旧は不要

27.3.3 ベストプラクティス

- ・ AWS Backup で複数サービスを横断管理
- ・ クロスリージョンコピー でリージョン災害に備える
- ・ クロスアカウントコピー で本番アカウント乗っ取りに備える
- ・ Vault Lock (Compliance モード) でランサムウェア対策
- ・ S3 バケットは Versioning + MFA Delete + Replication
- ・ リストア演習 を定期実施

27.4 Pilot Light

DR リージョンに「種火」だけ残す。

27.4.1 構成

[Primary Region]	[DR Region]
ALB + ASG + Aurora Cluster	Aurora Global DB Secondary (読み取り専用)
↓	AMI / コンテナイメージ レプリケート済
継続的レプリケーション	ASG は0台、ALB は未稼働
	↓ (災害時)
	ASG を起動、Aurora を昇格、Route 53 切替

27.4.2 採用ケース

- ・ 中規模 Web、本番停止が 10～30 分許容
- ・ データは継続レプリケーション、計算は止めて節約

27.4.3 ポイント

- ・ Aurora Global Database を Pilot Light の中核に
- ・ DynamoDB なら Global Tables (Active-Active 風だが Pilot Light 設計でも有効)
- ・ AMI / コンテナイメージは EC2 AMI Cross-Region Copy / ECR レプリケーション
- ・ IaC (CDK / Terraform) で DR 構築テンプレートを整備し、災害時に数分で立ち上げ

27.5 Warm Standby

DR リージョンで縮小版が常時稼働。

27.5.1 構成

[Primary]	[DR]
ALB + ASG min=2,max=20	ALB + ASG min=1,max=2 (縮小)
Aurora Writer + Reader×2	Aurora Global Secondary + Reader×1
継続レプリ + 同期確認	ヘルスチェック合格状態
	↓ (災害時)
	ASG を拡張、Aurora を昇格、Route 53 切替

27.5.2 採用ケース

- ・ 重要 Web、本番停止が 5～10 分許容
- ・ 切替後の負荷を最初から捌くため、最低限の Warm を持つ

27.5.3 ポイント

- ・ DR 側も **常時アプリが動いている** のでデプロイミスなどに気付ける
- ・ スケール拡張に **Auto Scaling グループの Predictive Scaling** を仕掛けるのも手
- ・ DR 側 Aurora で **Read 負荷を一部分散** して稼ぐ運用も可

27.6 Multi-Site Active-Active

両リージョンで本番。

27.6.1 構成

```
[User] → Route 53 (Latency or Geo)
      ↳ Tokyo:    ALB → ECS → DynamoDB Global Tables
      ↳ Virginia: ALB → ECS → DynamoDB Global Tables
                ↑
          CloudFront (共通フロント)
```

27.6.2 採用ケース

- ・ グローバル B2C、停止が秒単位で許されない
- ・ DynamoDB / Aurora Global / S3 Cross-Region Replication で **双方向書き込み**
- ・ 競合解決ロジックがアプリに必要 (最後勝ち、CRDT など)

27.6.3 ポイント

- ・ **最も高コスト・最も複雑**。本当に必要かを先に検討
- ・ データ整合性のレベルを明確化 (Eventually Consistent でいいか)
- ・ Route 53 のヘルスチェックでフェイルオーバー
- ・ **スプリットブレイン** 対応 (両側が独立稼働した場合の合流)

27.7 コンポーネント別 DR 設計

27.7.1 コンピュート（EC2 / ECS / EKS / Lambda）

- **AMI Cross-Region Copy**：定期的に DR リージョンへコピー
- **ECR Cross-Region Replication**：コンテナイメージを自動レプリケート
- **Lambda 関数**：CloudFormation / CDK で DR リージョンにも展開
- **Auto Scaling Plan**：DR 側で Predictive Scaling

27.7.2 データベース

DB	DR 機能
Aurora MySQL/PostgreSQL	Aurora Global Database（複数リージョン、RPO 通常 1 秒未満）
RDS	Cross-Region Read Replica（昇格で DR 切替）
DynamoDB	Global Tables（マルチリージョン Active-Active）
ElastiCache	Global Datastore（Redis）
DocumentDB	Global Cluster
Neptune	Cross-Region Snapshot Copy
Redshift	Cross-Region Snapshot、Concurrency Scaling

27.7.3 ストレージ

- **S3 Cross-Region Replication**：継続的レプリケーション
- **S3 Multi-Region Access Points**：複数リージョンの S3 を 1 エンドポイントで
- **EFS Replication**：別リージョンへ自動レプリ
- **FSx Backup** + クロスリージョンコピー
- **AWS Backup** で横断管理

27.7.4 ネットワーク・DNS

- **Route 53 Health Check + Failover**：プライマリ落下を検知して切替
- **Route 53 Application Recovery Controller（ARC）**：DR 切替の調整・手動切替
- **Global Accelerator**：Anycast IP で 2 リージョンを 1 IP として
- **Transit Gateway Inter-Region Peering**：リージョン間 VPC 接続

27.7.5 認証・設定

- **IAM Identity Center**：管理アカウントのリージョンが落ちると影響大。Break Glass 用 IAM ユーザーを別途用意
- **Secrets Manager Cross-Region Replication**：シークレットを DR リージョンへ
- **Parameter Store**：手動 / IaC でレプリケート
- **KMS Multi-Region Keys**：複数リージョンで同じ鍵 ID

27.8 バックアップ戦略

27.8.1 3-2-1 ルール

- 3つのコピー（本番 + バックアップ × 2）
- 2つの異なるメディア（S3 Standard + Glacier など）
- 1つはオフサイト（別リージョン or 別アカウント）

27.8.2 AWS Backup の使い方（再掲）

- バックアップ計画：頻度・保持・コピー先
- バックアップボールド：保存先
- Vault Lock：Compliance モードで改ざん防止
- クロスリージョンコピー
- クロスアカウントコピー
- Backup Audit Manager：ポリシー遵守の監査

27.8.3 別アカウント分離（推奨）



本番アカウントが乗っ取られても、バックアップ専用アカウントは独立した認証情報で守られる。

27.9 ランサムウェア対策

ランサムウェアの典型攻撃は **バックアップごと暗号化・削除**。これに対抗するには：

- 別アカウントにバックアップを置く
- Vault Lock（Compliance モード）で削除不能化
- S3 Object Lock で WORM 保管
- MFA Delete 有効化
- 最小権限：本番ユーザーがバックアップを削除できない設計
- Backup の暗号化キー も別管理（KMS は別アカウント所有）

27.10 演習（Game Day）

DR は 演習しないと使えない。最低でも年に 1 回、できれば四半期ごとに：

- バックアップからのリストア演習：実際に別アカウントに復元してみる
- リージョン切替演習：Route 53 の切替、DNS 伝播、アプリ動作確認
- データベース昇格演習：Aurora Global の Failover、整合性確認
- 人手承認フロー：誰が判断、誰が実行、連絡経路

- ・ **Runbook の更新**：実施で気づいた手順抜けを反映

AWS Resilience Hub を使うと、構成からレジリエンスポリシー違反を検出し、推奨を提示してくれる。

27.11 AWS Resilience Hub

アプリケーションの **レジリエンス（耐障害性）** を評価・改善するサービス。

- ・ アプリを CloudFormation スタック / Resource Group / EKS / AppRegistry から取り込む
- ・ レジリエンスポリシー（RPO/RTO）と比較
- ・ 違反を検出、推奨修正を提示
- ・ 評価結果のレポート、IaC への組み込み

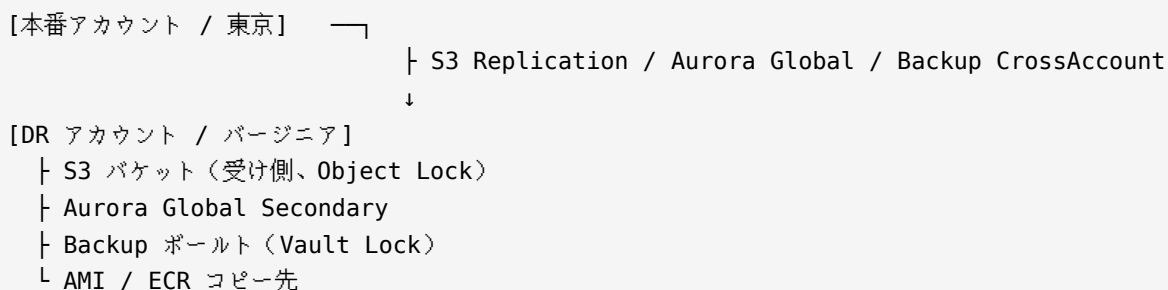
27.12 Route 53 ARC（Application Recovery Controller）

DR 切替を **確実に** 行うための調整サービス。

- ・ **Routing Control**：複数の Route 53 レコードを **まとめて** 切替（フェイルオーバーの整合性）
- ・ **Cluster**：5 つの Cell に分散され、3/5 過半数で操作確定
- ・ **Readiness Check**：DR 側の準備状態を継続評価

クリティカルな DR では Route 53 通常のヘルスチェックよりも ARC が安全。

27.13 クロスアカウント DR の典型構成



本番アカウントが侵害されても、DR アカウントは独立して守られる。

27.14 コスト感

- ・ **Backup & Restore**：本番コストの数 % 程度
- ・ **Pilot Light**：本番コストの 5～15%
- ・ **Warm Standby**：本番コストの 25～50%
- ・ **Active-Active**：本番コストの 100% 超（双方フル稼働）

ビジネス価値（ダウンタイム 1 時間あたりの損失額）と比較して妥当性を判断。

27.15 チェックリスト

新規システムを設計するときに自問する項目：

- ・ ☐ RPO / RTO は文書化されている
- ・ ☐ バックアップは別リージョンと別アカウントに存在
- ・ ☐ バックアップは Vault Lock or Object Lock で守られている
- ・ ☐ リストア演習を直近 6 か月以内に実施
- ・ ☐ DR リージョンで IaC の差分なく構築できる
- ・ ☐ DNS 切替手順が文書化されている
- ・ ☐ DB のフェイルオーバー手順が文書化されている
- ・ ☐ Break Glass 用の認証情報が金庫保管されている
- ・ ☐ インシデント連絡経路が定義されている
- ・ ☐ DR 構成に対する月額コストが把握されている

27.16 よくある落とし穴

落とし穴	対策
バックアップを取っただけで満足	リストア演習、自動検証
DR 側の IaC が古い	本番デプロイ時に DR も同時 apply
KMS 鍵が単一リージョン	Multi-Region Keys に変更
DNS TTL が長い	TTL を短く（60 秒程度）、切替前に短縮
Aurora Global 昇格に時間がかかる	事前に手順書、ARC で自動化
Route 53 ヘルスチェック誤検知	複数リージョンチェック、Calculated Health Check
Identity Center が DR 不可	Break Glass IAM ユーザー、リージョン障害想定
演習せず本番事故で初練習	Game Day 計画、定期演習

28 Well-Architected Framework

AWS Well-Architected Framework は、クラウドシステムを評価する AWS 公式フレームワーク。**6本の柱** でシステム設計をレビューし、改善点を体系的に洗い出す。本章は各柱の要点と、Well-Architected Tool の使い方をまとめる。

28.1 6本の柱

柱	問い
Operational Excellence（運用上の優秀性）	運用をどう改善し続けるか
Security（セキュリティ）	情報・資産をどう守るか
Reliability（信頼性）	期待された機能を正しく実行し続けられるか
Performance Efficiency（パフォーマンス効率）	リソースを効率的に使えているか
Cost Optimization（コスト最適化）	必要以上に払っていないか
Sustainability（持続可能性）	環境負荷を最小化しているか

28.2 Operational Excellence（運用上の優秀性）

28.2.1 主要設計原則

- ・ 運用をコード化する（Infrastructure as Code、Configuration as Code）
- ・ 小さく頻繁な可逆変更を行う
- ・ 手順を定期的に見直す
- ・ 障害を予見し対応する
- ・ 運用失敗から学ぶ

28.2.2 実践

- ・ CloudFormation / CDK / Terraform ですべてを定義
- ・ CI/CD パイプライン（CodePipeline / GitHub Actions）
- ・ ブルーグリーン・カナリアデプロイ
- ・ CloudWatch Dashboard、X-Ray、Application Signals
- ・ Systems Manager Automation で運用手順の自動化
- ・ 定期的な **Game Day**、ポストモテム文化
- ・ **Runbook** と **Playbook** の整備

28.3 Security（セキュリティ）

28.3.1 主要設計原則

- 強固なアイデンティティ基盤
- トレーサビリティを有効化
- すべての層にセキュリティ
- セキュリティのベストプラクティスを自動化
- 転送中・保管中のデータ保護
- データに人を近づけない（アクセスを絞る）
- セキュリティイベントに備える

28.3.2 実践

- **IAM Identity Center** + passkey/FIDO2
- **CloudTrail / Config / GuardDuty / Security Hub** を全リージョン・全アカウントで
- **KMS** で暗号化、**Secrets Manager** で秘密管理
- **WAF + Shield**、**Network Firewall**
- **SCP** でガードレール、**Permission Boundary**
- **IAM Access Analyzer** で外部公開検知
- インシデント対応 **Runbook** を Systems Manager Automation に
- 19 章のセキュリティサービスを組織的に運用

28.4 Reliability（信頼性）

28.4.1 主要設計原則

- 障害からの自動復旧
- 回復手順のテスト
- 水平方向のスケーリング
- キャパシティの推測を止める
- 変更管理を自動化

28.4.2 実践

- マルチ AZ を前提に設計（EC2 / RDS / Aurora / ALB / EFS）
- **Auto Scaling** で水平スケール
- **AWS Backup** + DR 戦略（27 章）
- ヘルスチェック と サーキットブレーカ
- **Chaos Engineering** : AWS Fault Injection Service (FIS) で障害注入
- **Service Quotas** の管理、プロアクティブな上限緩和
- **Resilience Hub** でレジリエンス評価

28.5 Performance Efficiency（パフォーマンス効率）

28.5.1 主要設計原則

- ・ 新技術を民主化する（マネージドサービス活用）
- ・ グローバル展開を数分で
- ・ サーバーレスアーキテクチャ利用
- ・ 頻繁に実験
- ・ 機械的共感を持つ（OS・HW の特性を理解）

28.5.2 実践

- ・ 適切なインスタンスタイプ：Graviton、最新世代
- ・ キャッシュ：ElastiCache、CloudFront、DynamoDB DAX、Global Accelerator
- ・ サーバーレス：Lambda、Fargate、Aurora Serverless
- ・ **Compute Optimizer** で右サイジング推奨
- ・ **Performance Insights**（RDS/Aurora のパフォーマンス診断）
- ・ **CloudWatch Application Signals** でレイテンシ可視化
- ・ 負荷試験（Distributed Load Testing on AWS）

28.6 Cost Optimization（コスト最適化）

28.6.1 主要設計原則

- ・ クラウド財務管理（CCOE 的組織）
- ・ 消費モデルを採用（使った分だけ）
- ・ 全体効率を測定
- ・ 重労働を AWS に寄せる
- ・ コストを分析・帰属させる

28.6.2 実践

- ・ **Budgets + Cost Explorer**
- ・ **Compute Savings Plans / RI**
- ・ **Spot Instance / Fargate Spot**
- ・ Aurora Serverless v2 で低負荷時 0 ACU
- ・ **S3 Intelligent-Tiering** / ライフサイクル
- ・ **CloudWatch Logs** 保持期間
- ・ **Compute Optimizer / Trusted Advisor**
- ・ **コスト配分タグ + Cost Categories**
- ・ **AWS Cost Optimization Hub**（2023～）

28.7 Sustainability（持続可能性）

2021 年追加。環境負荷（CO2 排出量）を意識した設計。

28.7.1 主要設計原則

- ・ 影響を理解する
- ・ 持続可能性目標を設定
- ・ 使用率を最大化する
- ・ より効率的なハードウェアを採用
- ・ 不要なタスクを減らす
- ・ マネージドサービスを使う

28.7.2 実践

- ・ **Customer Carbon Footprint Tool** で排出量可視化
- ・ **Graviton / Inferentia / Trainium** は従来比 CO2 削減
- ・ **Spot Instance** で遊休 HW を活用
- ・ 適切なインスタンスサイズ（オーバースプロビジョニング回避）
- ・ サーバーレス・マネージド化で共用インフラの効率化
- ・ データライフサイクル（古いデータは低炭素な層へ）

28.8 Well-Architected Tool

AWS コンソールの **Well-Architected Tool** で、実際のワークロードを 6 本柱でレビューできる。

28.8.1 流れ

1. **ワークロードを定義**（名前・リージョン・業界・レビュー担当者）
2. **レンズ選択**：AWS Well-Architected Framework（必須）＋業界別レンズ（Serverless / ML / IoT / HPC / SaaS / Financial Services / Healthcare）
3. **質問に回答**：各柱ごとに数十問。該当ベストプラクティスを選択
4. **High Risk Issues（HRI）** と **Medium Risk Issues** が自動集計
5. **改善計画** を立てる（IaC や JIRA に連携可）

28.8.2 レンズ

特定領域向けの追加質問セット。

- ・ **Serverless Lens**
- ・ **SaaS Lens**
- ・ **Machine Learning Lens**
- ・ **IoT Lens**
- ・ **Financial Services Industry Lens**
- ・ **HPC Lens**
- ・ **FTR（Foundational Technical Review）**：APN パートナー向け
- ・ **カスタムレンズ**：自社固有の基準

28.8.3 利点

- 外部レビューを受けなくても自己診断できる
- AWS SA (Solutions Architect) と組んで第三者レビュー (WAR: Well-Architected Review) も可能
- Well-Architected Review 完了のパートナーは AWS クレジット支援 が受けられることがある

28.9 6 本柱チェックリスト (抜粋)

28.9.1 Operational Excellence

- ☐ IaC で全リソースを管理しているか
- ☐ 変更はコードレビューを経てデプロイされるか
- ☐ CloudWatch / X-Ray で可観測性を担保しているか
- ☐ 運用手順が Runbook 化されているか
- ☐ 定期的な改善レビューがあるか

28.9.2 Security

- ☐ ルートユーザーに passkey、アクセスキーなし
- ☐ IAM Identity Center で人のアクセスを管理
- ☐ 最小権限の原則を徹底
- ☐ CloudTrail / Config / GuardDuty が全アカウント有効
- ☐ KMS / Secrets Manager で暗号化・秘密管理
- ☐ WAF / Shield で境界防御
- ☐ インシデント対応手順が文書化

28.9.3 Reliability

- ☐ マルチ AZ 前提の設計
- ☐ Auto Scaling で需要変動に追従
- ☐ バックアップ + 定期リストア演習
- ☐ DR 戦略 (RPO/RTO) が明確
- ☐ Service Quotas を監視
- ☐ Chaos Engineering で障害注入テスト

28.9.4 Performance Efficiency

- ☐ 最新世代インスタンス・Graviton を評価
- ☐ キャッシュ層 (CloudFront、ElastiCache) を活用
- ☐ 負荷試験を実施
- ☐ Compute Optimizer 推奨を反映
- ☐ RDS Performance Insights でクエリ最適化

28.9.5 Cost Optimization

- [] Budgets で予算アラート
- [] Savings Plans / RI を評価
- [] Spot / Fargate Spot をバッチに適用
- [] S3 ライフサイクル、Intelligent-Tiering
- [] CloudWatch Logs 保持期間設定
- [] Cost Explorer で月次レビュー
- [] コスト配分タグ戦略

28.9.6 Sustainability

- [] Customer Carbon Footprint Tool で排出量確認
- [] Graviton / Inferentia を検討
- [] 停止可能なワークロードは Spot / 夜間停止
- [] データライフサイクルで古いデータを低炭素層へ

28.10 WAF (Well-Architected Framework) と他の指針

Well-Architected Framework は **抽象的な原則**。実装に落とすには：

- **AWS Prescriptive Guidance**：具体的な設計ガイド
- **AWS Solutions Library**：業界別の参照アーキテクチャ + CloudFormation テンプレート
- **AWS Architecture Center**：図解アーキテクチャ集
- **AWS Blog**：最新事例・プラクティス
- **re:Invent セッション**：最新の深い話

Well-Architected の質問に答えながら、これらのリソースで実装ノウハウを補う。

28.11 レビューのリズム

- **新規プロジェクト開始時**：設計前のレビュー
- **本番リリース前**：HRI をすべて解決
- **大規模変更時**：影響範囲のレビュー
- **年1回**：全体レビュー + 改善計画更新
- **インシデント後**：該当領域の再評価

28.12 よくある誤解

- **✗** 「Well-Architected レビューは AWS に頼まないとできない」 → **○** 自社だけでも Tool で実施可能
- **✗** 「HRI ゼロを目指す」 → **○** リスクを理解して受容するのも判断。ゼロが必ずしも正解ではない
- **✗** 「全部のベストプラクティスを満たすべき」 → **○** ワークロードの特性・フェーズに応じて優先順位付け

- **×** 「1回やれば OK」 → ○ 継続的プロセス。6 か月～1 年で再評価
- **×** 「Cost Optimization だけやる」 → ○ 6 本柱はトレードオフで成立。安くして可用性を落とすのは本末転倒

28.13 まとめ

Well-Architected Framework は **設計の共通言語**。チーム内の議論を体系化し、経営層への説明資料にもなる。小さいワークロードでも軽く通すだけで盲点が見つかる。本章を読み終えた後、今運用している（あるいは設計中の）システムで、まず **Security と Reliability** を自己評価してみることが推奨する。

29 認定資格と学習リソース

本書の締めくくりとして、AWS の知識を体系的に深めるための **認定資格** と **学習リソース** を紹介する。これから長く AWS を使い続ける上での「次の一步」を具体化する。

29.1 AWS 認定資格の全体像

29.1.1 カテゴリ

カテゴリ	資格
Foundational	Cloud Practitioner (CLF)、AI Practitioner (AIF)
Associate	Solutions Architect (SAA)、Developer (DVA)、SysOps (SOA)、Data Engineer (DEA)、Machine Learning Engineer (MLA)
Professional	Solutions Architect Pro (SAP)、DevOps Engineer Pro (DOP)
Specialty	Advanced Networking (ANS)、Security (SCS)、Machine Learning (MLS)

2024～2025 年にかけて大きく整理された。旧 Data Analytics / Database Specialty は廃止、Data Engineer Associate に統合。AI Practitioner / ML Engineer Associate が新設。

29.1.2 受験制度

- ・ **オンライン受験**（Pearson VUE の自宅監督型）または **テストセンター受験**
- ・ **所要時間**：Foundational 90 分、Associate 130 分、Professional/Specialty 180 分
- ・ **料金**：Foundational \$100、Associate \$150、Professional/Specialty \$300
- ・ **合格ライン**：おおむね 70%（資格により差あり）
- ・ **有効期間**：3 年間。同じか上位資格の合格で自動更新

29.1.3 公式リソース

- ・ **AWS Skill Builder**：公式学習プラットフォーム（無料+Subscription）
- ・ **Exam Readiness コース**：各資格の試験対策コース
- ・ **Sample Questions**：10 問程度の公式サンプル
- ・ **Exam Guide**：試験範囲・配点の公式ドキュメント
- ・ **Practice Exam**（Skill Builder Subscription 含む）

29.2 おすすめ学習順序

29.2.1 入門パス

[目安3か月]

1. Cloud Practitioner (CLF) ... 本書で大部分カバー済み

↓
2. Solutions Architect Associate (SAA) ... アーキテクチャ全般の必携

これで AWS 全体の基盤が身につく。

29.2.2 開発者パス

CLF → SAA → Developer (DVA) → DevOps Engineer Pro (DOP)

DVA は Lambda / DynamoDB / CI/CD / アプリ統合が中心。

29.2.3 運用パス

CLF → SAA → SysOps (SOA) → DevOps Engineer Pro (DOP)

SOA は唯一 **ラボ試験**（実機操作）があったが、2023 年以降ラボ部分は削除されマルチプルチョイスに一本化。

29.2.4 アーキテクトパス

CLF → SAA → Solutions Architect Pro (SAP)

SAP は難関。200 問以上のシナリオ問題、広範囲。

29.2.5 分野特化

SAA → Advanced Networking (ANS) ... ネットワーク設計
 → Security (SCS) ... セキュリティ運用
 → Data Engineer (DEA) ... データ基盤
 → Machine Learning Engineer (MLE) ... ML ワークフロー
 → AI Practitioner (AIF) ... 生成 AI 基礎
 → Machine Learning Specialty (MLS) ... ML 深掘り

29.3 資格別の勉強法

29.3.1 Cloud Practitioner (CLF)

- ・ **対象**：AWS の基本概念、主要サービス、請求、サポート
- ・ **期間**：初学者で 2～4 週間
- ・ **教材**：本書 + AWS Skill Builder 「Cloud Practitioner Essentials」 + サンプル問題
- ・ **実機**：Free Tier で EC2 / S3 / IAM を触るだけで合格ライン届く

29.3.2 Solutions Architect Associate (SAA)

- ・ **対象**：アーキテクチャ設計（信頼性・可用性・パフォーマンス・コスト・セキュリティ）

- ・ 期間：1～3 か月
- ・ 教材：Skill Builder、Udemy (Stephane Maarek、Adrian Cantrill)、Tutorials Dojo の模擬試験、AWS 公式 Exam Readiness
- ・ 実機：VPC / EC2 / ELB / Auto Scaling / RDS / S3 / CloudFront / Route 53 / IAM は手を動かす

29.3.3 Developer Associate (DVA)

- ・ 対象：Lambda、API Gateway、DynamoDB、SAM、CloudFormation、CodeCommit/Build/Deploy/Pipeline
- ・ 期間：1～2 か月（SAA 取得後なら）
- ・ 実機：24 章のハンズオンに近い構成を自作

29.3.4 SysOps Administrator Associate (SOA)

- ・ 対象：監視、運用自動化、セキュリティ、高可用性、コスト最適化
- ・ 期間：1～2 か月
- ・ 実機：CloudWatch、Systems Manager、Config、CloudTrail

29.3.5 Solutions Architect Professional (SAP)

- ・ 対象：大規模エンタープライズアーキテクチャ、マルチアカウント、移行、ハイブリッド、高度な運用
- ・ 期間：3～6 か月
- ・ 難易度：最難関クラス。シナリオが長文、選択肢の差が微妙
- ・ 教材：Adrian Cantrill の詳細コース、Tutorials Dojo、AWS 公式 Exam Readiness
- ・ 体験ベース：実務で大規模設計を経験している人は強い

29.3.6 Specialty 全般

- ・ 専門領域に絞った深い知識
- ・ SAA / SAP 合格後が一般的
- ・ 実務経験が活きる傾向

29.4 学習リソース

29.4.1 公式

- ・ **AWS Skill Builder** : <https://skillbuilder.aws/>
 - ▶ 無料コース多数、Subscription (Individual \$29/月、Team \$449/年) でラボ・Practice Exam
 - ▶ **AWS Jam** : ゲーム形式のスキルチャレンジ
 - ▶ **AWS Cloud Quest** : ロールプレイ形式の学習
- ・ **AWS Documentation** : <https://docs.aws.amazon.com/>
 - ▶ **User Guide / Developer Guide / API Reference**
 - ▶ **Whitepapers** : Well-Architected、Security Pillar、コスト最適化など
- ・ **AWS Blog** : <https://aws.amazon.com/blogs/>

- ▶ 新機能・ベストプラクティス
- **AWS Architecture Center**：参照アーキテクチャ集
- **AWS Solutions Library**：すぐ使える CloudFormation テンプレート
- **AWS Prescriptive Guidance**：テーマ別の設計ガイド
- **AWS Samples (GitHub)**：サンプルコード

29.4.2 re:Invent / Summit

- **AWS re:Invent**：年次グローバルカンファレンス（11 月末～12 月上旬、ラスベガス）
 - ▶ セッション動画は YouTube で無料公開（数千本）
 - ▶ **Keynote**、400 / 500 レベルセッション
- **AWS Summit**：各国の地域カンファレンス（日本では毎年春～夏）
- **AWS Innovate**：オンラインテーマ別カンファレンス

29.4.3 コミュニティ

- **AWS User Group**：JAWS-UG（日本）など世界中に
- **AWS Community Builders**：AWS 公式のアドボケイトプログラム
- **AWS Heroes**：長期貢献者の公式認定
- **Twitter / X の AWS コミュニティ**：@awsreinvent、@jeffbarr
- **Reddit**：r/aws、r/awscertifications
- **Stack Overflow**：aws タグ

29.4.4 独立系メディア・ブログ

- **AWS What's New**：公式の機能発表 RSS
- **Last Week in AWS**（Corey Quinn）：毒舌だが洞察深いニュースレター
- **A Cloud Guru / Pluralsight**：動画コース
- **acloud.guru blog**
- **Medium の AWS タグ**

29.4.5 日本語リソース

- **AWS Black Belt Online Seminar**：AWS JP SA による公式シリーズ
- **classmethod.jp 技術ブログ**（DevelopersIO）：圧倒的な AWS 情報量
- **NRI ネットコム技術ブログ**
- **JAWS-UG**：地域・テーマ別の勉強会
- **書籍**：翔泳社・技術評論社・O'Reilly Japan の AWS 本

29.4.6 有料トレーニング

- **AWS Training and Certification**：公式講師によるクラス
- **Cloud Academy / Pluralsight / A Cloud Guru / Adrian Cantrill**
- **Udemy**：Stephane Maarek、Adrian Cantrill、Neal Davis
- **Whizlabs / Tutorials Dojo**：模擬試験問題集

29.5 実務で学び続けるヒント

29.5.1 サンドボックスを持つ

個人用 AWS アカウント（または勤務先の Sandbox アカウント）を持ち、新機能が出たら手を動かす。Skill Builder Lab や Adrian Cantrill の独自 Lab 環境も有用。

29.5.2 1つのサービスを深掘りする

広く浅くは本書で済んでいる。次は興味のあるサービス1つをドキュメント全部読むくらい深く掘る。S3 の Lifecycle と Intelligent-Tiering を全部、Lambda のイベントソースマッピングの挙動を全部、など。

29.5.3 公式ブログを定期購読

AWS What's New の RSS、各サービスブログ、**Serverless**、**Networking**、**Security**、**Database** など関心分野のブログをフィードリーダーに入れる。毎日 15 分で業界最前線をキャッチアップできる。

29.5.4 re:Invent セッションを見る

年末の re:Invent 後、1～2 か月かけて関心領域のセッションを 10 本程度見る。300 / 400 レベルが実装よりで学びが多い。

29.5.5 他人のアーキテクチャを読む

- AWS Architecture Blog
- Case Study
- re:Invent の “Customer Story” 系セッション
- GitHub 上のオープンソース AWS プロジェクト

自分と違う設計を見ると、自分の選択肢が広がる。

29.5.6 小さく作って壊す

本書のハンズオン（24～26 章）のような小さなシステムを作り、あえて壊して復旧する。学びは実機操作とトラブルシュートから。

29.6 AWS と周辺技術

AWS だけに閉じこもらず、以下の周辺技術も並行して学ぶとキャリアの幅が広がる。

- Terraform / Pulumi : マルチクラウド IaC
- Kubernetes : EKS だけでなく GKE / AKS / 自前でも
- Observability : Datadog、New Relic、Grafana、Prometheus
- データエンジニアリング : Snowflake、Databricks、dbt、Apache Iceberg
- 機械学習 / 生成 AI : PyTorch、Hugging Face、LangChain、MCP (Model Context Protocol)
- セキュリティ : ゼロトラスト、SBOM、SLSA

- **FinOps**：クラウド財務管理の方法論

29.7 本書からの卒業

本書は「幅広く AWS の全体像を掴む」ことを目的に書かれた。ここまで読み通した読者は、AWS のほぼ全領域を俯瞰できる状態にある。次の学習段階として：

1. 個人アカウントで 24～26 章のハンズオンを実施
2. **Cloud Practitioner** → **SAA** の順で受験（2～4 か月）
3. 興味のある特定領域（データ分析 / ML / ネットワーク / セキュリティ）の **Specialty** を目指す
4. 実務で AWS を選ぶ場面を作り、**Well-Architected Review** を通してみる
5. JAWS-UG 等の **コミュニティに参加** し、他社の事例を吸収する

AWS の進化は速い。学び続けることこそが最大のスキル。本書がその第一歩となれば幸いである。

29.8 参考：読者のフェーズ別推奨

読者フェーズ	次にやること
AWS 初心者、本書を読んだ	24 章ハンズオン → CLF 受験 → SAA 教材へ
実務 AWS 半年	SAA 取得 → 担当領域の Specialty 教材
実務 AWS 1～2 年	SAP、Well-Architected Review を通す、re:Invent セッション視聴
実務 AWS 3 年以上	Specialty 複数、コミュニティ登壇、OSS 貢献
ML / データ系に進みたい	DEA → MLA → MLS、Bedrock 実務、SageMaker 深掘り
セキュリティに進みたい	SCS、IAM 深掘り、Well-Architected Security Pillar
独立・フリーランス	複数認定 + ポートフォリオ、AWS Partner 個人登録検討

30 Lambda 深掘り

8 章で概要、14 章でメッセージング統合の文脈で扱った Lambda を、本章では **本番運用品質** で使うための知識として深掘りする。コールドスタート対策、可観測性、Powertools、テスト、コスト、設計パターンを中心に。

30.1 Lambda の実行モデル再確認

30.1.1 実行環境のライフサイクル

1. INIT コンテナ起動 + ランタイム初期化 + ハンドラ外コード実行
2. INVOKE ハンドラ実行（複数回繰り返される）
3. SHUTDOWN コンテナ終了（しばらく未使用后）

INIT は リクエストごとには走らない。同じコンテナが「ウォーム」のうちは INVOKE のみ繰り返される。INIT が走るのが **コールドスタート**。

30.1.2 コンテナの再利用

ハンドラ外で初期化したオブジェクト（DB クライアント、SDK、設定キャッシュ）は、同じコンテナのウォーム呼び出しで **再利用** される。これを意識して書くと性能が上がる。

```
# 良い例: モジュールスコープで初期化
import boto3
ddb = boto3.resource('dynamodb').Table('mytable')

def handler(event, context):
    return ddb.get_item(Key={'id': event['id']})
```

```
# 悪い例: 毎回初期化
def handler(event, context):
    ddb = boto3.resource('dynamodb').Table('mytable') # 毎回作られる
    return ddb.get_item(Key={'id': event['id']})
```

30.1.3 同時実行とスケーリング

- 1 コンテナ = 1 リクエスト同時処理
- スケール上限：アカウント・リージョンの **Reserved/Unreserved Concurrency**
- 急激なスパイク：**Burst Capacity**（東京で 1,000）の後、毎分 +500/秒で拡大
- **Reserved Concurrency**：関数ごとに上限を予約（他関数を圧迫しない）
- **Provisioned Concurrency**：事前ウォーム済みコンテナを確保

30.2 コールドスタート対策

30.2.1 削減の優先順位

対策	効果
パッケージサイズ削減	大。デプロイパッケージを小さく
ランタイム選択	大。Node.js / Python は速い、Java / .NET は遅い
Lambda SnapStart	大（Java、Python、.NET 対応）
Provisioned Concurrency	完全回避できるが課金
Lambda Power Tuning	メモリ最適化で全体短縮
VPC 内 Lambda の改善	Hyperplane ENI で大幅高速化済（2019～）

30.2.2 Lambda SnapStart

JVM 系言語向けが先行、現在は Python / .NET にも拡大。INIT 後の状態をスナップショットして再利用。コールドスタートを **最大 10 倍高速化**。

```
aws lambda update-function-configuration \
  --function-name myfunc \
  --snap-start ApplyOn=PublishedVersions
```

注意：

- **Published Version** に対して有効。**\$LATEST** には適用されない
- スナップショット時の状態が保存されるため、**乱数**や**TLS 接続**は recreate ロジックが必要

30.2.3 Provisioned Concurrency

事前にウォームコンテナを確保。

```
aws lambda put-provisioned-concurrency-config \
  --function-name myfunc \
  --qualifier 1 \
  --provisioned-concurrent-executions 10
```

- 1 コンテナあたり時間課金
- **Auto Scaling** と組み合わせて、業務時間中だけ増やす運用が一般的

30.3 ランタイム

ランタイム	特徴
Node.js 20 / 22	起動速い、エコシステム豊富
Python 3.11 / 3.12 / 3.13	起動速い、データ系・ML 系で人気
Java 17 / 21	JVM 重いが SnapStart で改善
Go (Provided.al2023)	単一バイナリ、高速

ランタイム	特徴
Ruby 3.x	Rails 周辺で
.NET 8	SnapStart 対応
Rust (Provided.al2023 + lambda-runtime crate)	極低レイテンシ・低メモリ
カスタムランタイム	Bash、PHP、Cobol 等
コンテナイメージ (OCI)	最大 10GB、独自バイナリ・依存ライブラリ

30.3.1 コンテナイメージ Lambda

ECR にイメージを push して指定するだけで Lambda として動く。サイズ上限が 10GB と大きく、ML モデルや巨大ライブラリも詰められる。Cold Start は zip より遅め。

```
FROM public.ecr.aws/lambda/python:3.12

COPY requirements.txt .
RUN pip install -r requirements.txt --target ${LAMBDA_TASK_ROOT}
COPY app.py ${LAMBDA_TASK_ROOT}

CMD ["app.handler"]
```

30.4 Lambda Powertools

AWS 公式の **本番運用支援ライブラリ** (Python / TypeScript / Java / .NET)。

主な機能：

- **Logger**：構造化 JSON ログ、相関 ID 自動付与
- **Tracer**：X-Ray のセグメント自動作成
- **Metrics**：CloudWatch EMF 形式で効率的にメトリクス送信
- **Event Source Data Classes**：API GW / S3 / SQS など各イベントの型安全アクセス
- **Parser**：Pydantic ベースのバリデーション
- **Idempotency**：DynamoDB を使った冪等性ヘルパー
- **Feature Flags**：AppConfig 統合
- **Parameters**：Parameter Store / Secrets Manager のキャッシュ付き取得
- **Batch**：SQS / Kinesis / DynamoDB Streams の Partial Batch Failure を簡潔に
- **Validation**：JSON Schema 検証

```
from aws_lambda_powertools import Logger, Tracer, Metrics
from aws_lambda_powertools.metrics import MetricUnit

logger = Logger(service="orders")
tracer = Tracer(service="orders")
metrics = Metrics(namespace="MyApp", service="orders")
```

```
@logger.inject_lambda_context(correlation_id_path="requestContext.requestId")
@tracer.capture_lambda_handler
@metrics.log_metrics(capture_cold_start_metric=True)
def handler(event, context):
    logger.info("order received", extra={"item_count": len(event["items"])})
    metrics.add_metric(name="OrdersProcessed", unit=MetricUnit.Count, value=1)
    return {"statusCode": 200}
```

これだけで構造化ログ、相関 ID 伝播、X-Ray トレース、CloudWatch メトリクス、Cold Start メトリクスが揃う。Powertools は **Lambda 開発の事実上の標準**。

30.5 設計パターン

30.5.1 単機能・薄い Lambda

複雑なロジックは Lambda 外（Step Functions、SDK サービス統合）に出して、Lambda は薄く保つ。

```
[API GW] → Lambda（バリデーション） → DynamoDB
                        → 失敗時 Step Functions で複雑処理
```

30.5.2 Single-purpose vs Mono-Lambda

- **Single-purpose**：エンドポイント 1 つ = Lambda 1 つ。役割明確、独立スケール、独立 IAM
- **Mono-Lambda** (Lambdalith)：複数エンドポイントを 1 Lambda に（Express / FastAPI ベース）。コールドスタートを共有、コード共有が楽

新規はまず Mono-Lambda で始め、性能・隔離要件が出たら分割するのが楽。

30.5.3 ストリーム処理（SQS / Kinesis / DynamoDB Streams）

- **バッチサイズ** と **バッチウィンドウ** で効率調整
- **Partial Batch Failure**：失敗だけを返して残りは成功
- **Maximum Concurrency**：同時実行数を制限してダウンストリーム保護
- **DLQ / On-Failure 宛先**：失敗イベントの隔離

30.5.4 Async vs Sync

- **Sync**：API GW、ALB、CLI Invoke → 戻り値が直接返る
- **Async**：S3、SNS、EventBridge → 裏で実行、戻り値は使われない
- **Stream-based**：SQS、Kinesis、DynamoDB Streams → Lambda が pull して処理

Async では **Destinations**（成功・失敗で別宛先に流せる）と **DLQ**（失敗時の退避）を必ず設定。

30.6 Lambda Function URL

API Gateway なしで **直接 HTTPS エンドポイント** を Lambda に付ける機能。

- 認証：AWS_IAM または NONE（バックエンドに認証ロジック）
- CORS 設定可
- 料金：API Gateway より安い（Lambda 通常料金のみ）
- 制約：使用量プラン、API キー、リクエスト変換などはなし

簡易 Webhook 受け、PoC、内部ツール用に。

30.7 テスト戦略

30.7.1 ローカルテスト

- AWS SAM CLI：`sam local invoke`、`sam local start-api`
- LocalStack：AWS サービスのローカルエミュレータ

```
sam local start-api
curl http://127.0.0.1:3000/api/notes
```

30.7.2 ユニットテスト

依存（DynamoDB、S3）はモック化。`moto`（Python）、`aws-sdk-client-mock`（Node.js）。

```
import boto3
from moto import mock_aws

@mock_aws
def test_handler():
    boto3.client('dynamodb').create_table(
        TableName='mytable',
        KeySchema=[{'AttributeName': 'id', 'KeyType': 'HASH'}],
        AttributeDefinitions=[{'AttributeName': 'id', 'AttributeType': 'S'}],
        BillingMode='PAY_PER_REQUEST',
    )
    from app import handler
    result = handler({'id': '1'}, None)
    assert result['statusCode'] == 200
```

30.7.3 統合テスト

実 AWS にデプロイして E2E。CDK / SAM / Terraform で PR ごとに一時環境を作る運用も一般的。

30.8 監視と可観測性

30.8.1 CloudWatch Metrics

標準で：

- Invocations / Errors / Throttles

- `Duration` / `IteratorAge` (ストリーム系)
- `ConcurrentExecutions`
- `ProvisionedConcurrency*`

30.8.2 CloudWatch Logs

`/aws/lambda/<function-name>` に自動出力。保持期間を必ず設定（デフォルト無期限）。Logs Insights で検索。

30.8.3 X-Ray

Powertools Tracer で自動セグメント作成。downstream（DynamoDB、HTTP）も統合。

30.8.4 Application Signals

CloudWatch の APM 機能。Lambda + Powertools で SLI/SLO ベースのモニタリング(35章参照)。

30.9 コスト最適化

- **Memory tuning : Lambda Power Tuning** (Step Functions のオープンソース) で実行時間 vs メモリの最適点を探す
- **ARM (Graviton)** : x86 より 20% 安く 19% 高速のことが多い
- **Provisioned Concurrency** : 本当に必要な時間帯だけ
- **Tiered Pricing** : 月 60 億 GB 秒以降は割引
- **Compute Savings Plans** でも Lambda は割引対象

30.9.1 Lambda Power Tuning

```
{
  "lambdaARN": "arn:aws:lambda:ap-northeast-1:123456789012:function:myfunc",
  "powerValues": [128, 256, 512, 1024, 1536, 2048, 3008],
  "num": 50,
  "payload": {"key": "value"},
  "strategy": "balanced"
}
```

128MB～3GB の各メモリで実行コストとレイテンシを比較。最適値を選ぶ。

30.10 セキュリティ

- 最小権限の実行ロール
- 環境変数の暗号化 : 機密値は KMS 暗号化または Secrets Manager / Parameter Store
- VPC 内 Lambda : プライベートリソース (RDS など) にアクセス時、Hyperplane ENI で起動時間影響は最小
- **Code Signing** : Lambda パッケージの署名検証
- リソースベースポリシー : 他アカウント / サービスからの呼び出し許可

- Lambda extensions : Datadog、Splunk、New Relic などのオブザーバビリティエージェント

30.11 Lambda の制限値（覚えておくべき数値）

項目	上限
メモリ	128MB～10,240MB（1MB 刻み）
実行時間	最大 15 分
一時ディスク /tmp	512MB～10,240MB
デプロイパッケージ（zip）	50MB（直接） / 250MB（解凍後）
コンテナイメージ	10GB
環境変数	4KB（合計）
同時実行数	アカウント当初 1,000（緩和申請可）
ペイロード（同期）	6MB
ペイロード（非同期）	256KB

30.12 よくあるトラブル

症状	対処
Cold Start が遅い	SnapStart、Provisioned Concurrency、ARM、軽量ランタイム
Throttling	Reserved/Unreserved Concurrency、上限緩和申請
タイムアウト	メモリ増、外部 API 待ち時間、SDK timeouts
VPC 内で外部 API 不通	NAT Gateway、VPC エンドポイント設置
Logs が大量・高コスト	サンプリング、保持期間、Powertools の log level
メモリ不足 OOM	メモリ増、ストリーミング処理、巨大データの S3 経由化
permissions エラー	実行ロール、リソースベースポリシー、暗号化キー権限
Async が再試行され続ける	Destinations / DLQ、最大試行回数調整
SQS でメッセージが消えない	可視性タイムアウト > Lambda 実行時間、partial batch failure

31 DynamoDB と NoSQL 設計

7 章でデータベース全般の中で触れた DynamoDB を、本章では本格的な設計手法と運用面で深掘りする。Single Table Design、GSI/LSI、トランザクション、Streams、容量設計、TTL、グローバル分散まで。

31.1 DynamoDB の本質

NoSQL を「SQL の弱体版」と捉えると失敗する。DynamoDB は **アクセスパターン駆動** の設計で、SQL の正規化思想とは別物。

31.1.1 強み

- ・ 無制限スケール：パーティション分散、自動シャーディング
- ・ ミリ秒台レイテンシ：シングルキー操作で安定 1~10ms
- ・ サーバーレス：容量管理・パッチ・バックアップが自動
- ・ 耐久性：3 AZ に同期書き込み
- ・ グローバル分散：Global Tables でマルチリージョン Active-Active

31.1.2 弱み

- ・ 柔軟なクエリ不可：スキャンは高コスト、JOIN なし
- ・ スキーマ変更が前提のクエリ：あとからアクセスパターンを増やすと再設計
- ・ 容量予測の難しさ：オンデマンドが救いだが青天井
- ・ 学習コスト：SQL とは設計思想が根本的に違う

31.2 主要概念の整理

概念	説明
テーブル	トップレベル
アイテム	テーブル内の 1 レコード（最大 400KB）
属性	アイテム内のキー・値ペア
パーティションキー (PK)	必須。ハッシュ値でアイテムを分散
ソートキー (SK)	任意。同じ PK 内で順序付け
プライマリキー	PK 単独 or PK + SK
セカンダリインデックス	GSI (Global) / LSI (Local)
Read/Write Capacity Unit	プロビジョンド時の単位
On-Demand	リクエスト課金、容量管理不要

31.3 Single Table Design

DynamoDB のベストプラクティス。1 テーブルに**複数エンティティ**を詰める 設計。

31.3.1 なぜ単一テーブルか

- ・ リクエスト数最小化：1 query で関連エンティティをまとめて取得
- ・ トランザクション境界：1 テーブル内で TransactWriteItems が使える
- ・ コスト効率：複数テーブルより容量管理が楽

31.3.2 実例：注文システム

PK	SK	type	attrs
USER#u1	PROFILE	User	name, email
USER#u1	ORDER#o1	Order	total, status
USER#u1	ORDER#o2	Order	total, status
ORDER#o1	ITEM#i1	OrderItem	product_id, qty
ORDER#o1	ITEM#i2	OrderItem	product_id, qty
PRODUCT#p1	META	Product	name, price
PRODUCT#p1	REVIEW#r1	Review	rating, text

これで以下のクエリが **1 query** で解決：

- ・ PK=USER#u1 → ユーザー＋全注文
- ・ PK=USER#u1, SK begins_with ORDER# → 注文一覧
- ・ PK=ORDER#o1 → 注文＋全アイテム
- ・ PK=PRODUCT#p1, SK begins_with REVIEW# → 商品レビュー

31.3.3 Generic な属性名

- ・ PK、SK：プライマリキー
- ・ GSI1PK、GSI1SK：GSI1 のキー
- ・ entity_type：論理エンティティ識別

エンティティごとに属性名を変えると GSI 設計が破綻する。論理名は値（type 属性）で表現する。

31.4 セカンダリインデックス

31.4.1 GSI (Global Secondary Index)

別の PK・SK で別の view を作る。最大 20 個。元テーブルから 非同期で射影 される。

GSI1: PK=email, SK=USER	→ email でユーザー検索
GSI2: PK=ORDER, SK=created_at	→ 注文を時系列で
GSI3: PK=status, SK=updated_at	→ ステータス別注文

31.4.2 LSI (Local Secondary Index)

PK は同じで 別の SK を持つ。テーブル作成時のみ作れる、最大 5 個。同期書き込み（強整合性可）。

GSI が柔軟性高く実用的、LSI は設計初期に固める覚悟があるとき。

31.4.3 Sparse Index

一部のアイテムにしかキー属性が存在しない GSI。「ステータスが `pending` のものだけインデックス」のような絞り込みに有用。

31.5 容量モード

モード	特徴
On-Demand	リクエスト課金、自動スケール、上限なし。新規・スパイク向け
Provisioned	RCU/WCU を予約、Auto Scaling 可。安定負荷向け

31.5.1 Capacity Unit の計算

- RCU：強整合性 1 個 = 4KB/秒、結果整合性 = 半分、トランザクション = 2 倍
- WCU：1 個 = 1KB/秒、トランザクション = 2 倍

例：10 KB のアイテムを毎秒 10 回読む（強整合性）→ $10 \times 3 \text{ RCU} = 30 \text{ RCU}$

31.5.2 Auto Scaling

Provisioned で目標利用率（例：70%）を設定し、自動でスケール。応答が遅いので **スパイク前提** なら On-Demand。

31.5.3 Reserved Capacity

Provisioned の長期予約で割引。

31.6 トランザクション

`TransactWriteItems` / `TransactGetItems` で **最大 100 個のオペレーション** をアトミックに。

```
ddb.meta.client.transact_write_items(TransactItems=[
    {'Put': {'TableName': 't', 'Item': {'PK': 'X', 'SK': 'A'}}},
    {'Update': {'TableName': 't', 'Key': {'PK': 'X', 'SK': 'B'},
               'UpdateExpression': 'SET counter = counter + :one',
               'ExpressionAttributeValues': {':one': 1}}},
    {'ConditionCheck': {'TableName': 't', 'Key': {'PK': 'X', 'SK': 'C'},
                       'ConditionExpression': 'attribute_exists(PK)'}}},
])
```

WCU は 2 倍消費。トランザクション失敗時は全部ロールバック。

31.6.1 Optimistic Locking

`version` 属性を持たせ、`UpdateExpression` で `:expectedVersion` 条件チェック。

31.7 DynamoDB Streams

テーブルへの変更を 24 時間キャプチャ。

- レコードに OLD_IMAGE / NEW_IMAGE / KEYS_ONLY / NEW_AND_OLD_IMAGES の 4 種
- **Lambda トリガー**：変更を非同期処理
- **Kinesis Data Streams へ送信**（拡張オプション）：より長期保持、複数コンシューマ

31.7.1 ユースケース

- **監査ログ**：変更を別ストレージに記録
- **他 DB へのレプリケーション**：DynamoDB → ElastiCache、OpenSearch
- **Outbox パターン**：DB 変更をイベントとして発行
- **マテリアライズドビュー**：別 PK で再保存

31.8 TTL (Time To Live)

特定属性に **Unix エポック秒** を入れておくと、その時刻を過ぎたアイテムが自動削除される（48 時間以内）。

```
import time
item['expires_at'] = int(time.time()) + 30 * 24 * 60 * 60 # 30日後
```

セッション、一時データ、ログのライフサイクル管理に。

31.9 Global Tables

複数リージョンに **双方向レプリケーション**。Active-Active のマルチリージョン構成を実現。

- レプリケーションラグ通常 1 秒未満
- Last-Writer-Wins (LWW) で競合解決
- 各リージョンで独立に書き込み可

31.9.1 ユースケース

- グローバル B2C アプリのレイテンシ削減
- リージョン障害時のフェイルオーバー
- 開発・ステージング環境の地理分散

31.10 バックアップとリカバリ

- **On-Demand Backup**：手動スナップショット、長期保持
- **Point-in-Time Recovery (PITR)**：5 分単位で過去 35 日まで復元
- **AWS Backup** との統合：横断管理、クロスリージョン・アカウントコピー
- **Export to S3**：JSON / Ion / DynamoDB JSON 形式で全件エクスポート、Athena で分析

31.11 パフォーマンスチューニング

31.11.1 Hot Partition（熱いパーティション）

特定の PK に書き込み・読み込みが集中するとスロットルが起きる。

- **書き込みシャーディング**：PK に乱数 suffix を付ける（`USER#u1#0` ～ `USER#u1#9`）→ 読み出し時は 10 回 query して合算
- **Adaptive Capacity**：DynamoDB 側が自動で Hot Partition に余分容量割当（一定範囲）
- **書き込みのバッチ化とバッファリング**

31.11.2 Query vs Scan

- **Query**：PK 指定の効率的な検索
- **Scan**：全件走査。本番では原則禁止。バッチ処理でも **Parallel Scan** で並列化

31.11.3 DAX（DynamoDB Accelerator）

DynamoDB 専用のインメモリキャッシュ。マイクロ秒台レスポンス。

- ItemCache（個別 GET の結果）と QueryCache
- 書き込みは write-through（キャッシュ＋本体両方更新）
- VPC 内に配置、SDK が透過的に利用

31.12 コスト最適化

- **On-Demand vs Provisioned**：使用パターンを Cost Explorer で見て切り替え
- **PITR** は別料金。検証環境では無効化検討
- **DAX** で読み出しコスト削減
- **S3 Export** で長期保管を低単価に
- **TTL** で不要データを自動削除
- **RCU/WCU の予約購入**（Reserved Capacity）

31.13 セキュリティ

- **KMS 暗号化**（デフォルト有効、CMK に切替可）
- **VPC エンドポイント**（Gateway 型、無料）：プライベートサブネットから利用
- **IAM ポリシー**：Condition で `dynamodb:LeadingKeys` を使い **PK ベースの行レベルアクセス** 制御
- **Resource-based policy**：別アカウント共有

```
{
  "Effect": "Allow",
  "Action": ["dynamodb:GetItem", "dynamodb:Query"],
  "Resource": "arn:aws:dynamodb:*:*:table/MyTable",
  "Condition": {
    "ForAllValues:StringEquals": {
```

```

    "dynamodb:LeadingKeys": ["${aws:PrincipalTag/TenantId}"]
  }
}
}

```

31.14 監視

CloudWatch メトリクス：

- ConsumedRead/WriteCapacityUnits
- ThrottledRequests
- SystemErrors、UserErrors
- SuccessfulRequestLatency
- ReturnedItemCount

CloudWatch Contributor Insights：どの PK が熱いかを可視化。

31.15 Single Table Design vs RDB の判断

新規プロジェクトで NoSQL を採用すべきか：

要件	向き
アクセスパターンが固定	DynamoDB 有利
毎週新しい分析クエリ	RDB / 列指向 DB
スパイク・スケール性	DynamoDB 有利
JOIN 多用	RDB 有利
トランザクション複雑	RDB 有利
グローバル分散	DynamoDB Global Tables 有利
厳密な ACID	RDB / Aurora
スキーマレス・進化	DynamoDB 柔軟

「まず Aurora で始めて、ボトルネックや特定要件で DynamoDB に部分移行」が安全。

31.16 よくある罠

罠	対処
Scan で本番遅い・高い	Query 設計、必要なら GSI 追加
Hot Partition でスロットル	シャーディング、Adaptive Capacity
後からアクセスパターンを増やしたい	GSI 追加、テーブル再設計覚悟
UpdateExpression の予約語衝突	ExpressionAttributeNames で別名
サイズ超過 (>400KB)	大きなフィールドを S3 に置き、参照だけ DynamoDB

罣	対処
TTL が即時消えない	削除は最大 48 時間遅延、これは仕様
GSI が遅延	非同期。書き込み直後の整合性は期待しない
Global Tables で競合	LWW、競合検知ロジックをアプリ側で

32 ECS と EKS 深掘り

8章でコンテナ系の概要を扱った。本章では ECS / EKS / Fargate の本格運用に必要な知識を深掘りする。タスク・サービス・ネットワーキング・スケーリング・セキュリティ・監視・トラブルシュート。

32.1 ECS と EKS の選び方再考

基準	推奨
AWS だけで完結 / シンプルさ重視	ECS
Kubernetes エコシステム必要	EKS
複数クラウド・移植性重視	EKS
運用人員少ない	ECS
CD ツール・operator 多用	EKS
サーバ管理ゼロ	いずれも Fargate
GPU・特殊計算	EKS (Karpenter + GPU AMI)

32.2 ECS の本格運用

32.2.1 クラスタ／サービス／タスク

```
[Cluster]
├ [Service-A]
│   ├── Task-A1 (replica)
│   ├── Task-A2 (replica)
│   └─ Task Definition で定義
└ [Service-B]
    └─ Task-B1
```

- ・ **タスク定義**：1～複数コンテナ、CPU/メモリ、IAM ロール、ネットワークモード、ボリューム
- ・ **サービス**：タスク定義のレプリカを N 個維持、ELB 連携、Auto Scaling
- ・ **タスク**：実行中のコンテナグループ。スタンドアロンでも実行可

32.2.2 タスク定義のキー要素

```
{
  "family": "web",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "1024",
  "memory": "2048",
  "executionRoleArn": "arn:aws:iam::...:role/ecsTaskExecutionRole",
```

```

"taskRoleArn": "arn:aws:iam::...:role/ecsTaskRole",
"runtimePlatform": {
  "cpuArchitecture": "ARM64",
  "operatingSystemFamily": "LINUX"
},
"containerDefinitions": [{
  "name": "app",
  "image": "123456789012.dkr.ecr.ap-northeast-1.amazonaws.com/app:v1.0",
  "essential": true,
  "portMappings": [{ "containerPort": 8080, "protocol": "tcp" }],
  "environment": [{ "name": "ENV", "value": "prod" }],
  "secrets": [{ "name": "DB_PASS", "valueFrom": "arn:aws:secretsmanager:..." }],
  "logConfiguration": {
    "logDriver": "awslogs",
    "options": {
      "awslogs-group": "/ecs/web",
      "awslogs-region": "ap-northeast-1",
      "awslogs-stream-prefix": "web",
      "awslogs-create-group": "true"
    }
  }
},
"healthCheck": {
  "command": ["CMD-SHELL", "curl -f http://localhost:8080/health || exit 1"],
  "interval": 30, "timeout": 5, "retries": 3, "startPeriod": 60
}
}]
}

```

32.2.3 executionRoleArn vs taskRoleArn

- **executionRoleArn** : ECS サービス自体が ECR pull ・ CloudWatch Logs 書き込み ・ Secrets 取得に使う
 - **taskRoleArn** : タスク内のアプリケーションコードが AWS API を呼ぶときの権限
- 混同しやすい。アプリ側に S3 書き込み権限が要るなら **taskRoleArn** に付ける。

32.2.4 ネットワークモード

- **awsvpc** : タスクごとに ENI を割当 (Fargate は必須)。SG をタスク単位で
- **bridge** : 従来 Docker のブリッジネットワーク (EC2 のみ)
- **host** : ホストのネットワークを直接 (EC2 のみ、ポート競合注意)

32.3 サービス・ロードバランシング

ALB / NLB と統合。サービス更新時に新タスクを起動・古いを停止する **デプロイ** が走る。

32.3.1 デプロイ戦略

- **Rolling** (標準) : `minimumHealthyPercent` / `maximumPercent` で段階更新

- **Blue/Green** (CodeDeploy 統合) : Blue (旧) / Green (新) で切替、ロールバック容易
- **External** : 自前のオーケストレーション

32.3.2 Service Discovery

- **AWS Cloud Map** : サービス名 → タスク IP の DNS 解決
- **Service Connect** : Envoy ベースの新統合方式 (推奨)。HTTP/2、retry、observability

```
# Service Connect の設定例 (タスク定義 + サービス)
serviceConnectConfiguration:
  enabled: true
  namespace: prod
  services:
    - portName: web
      clientAliases:
        - port: 80
```

32.4 Auto Scaling

32.4.1 Service Auto Scaling

タスク数を自動増減。

- **Target Tracking** : CPU 70%、メモリ 70%、ALB Request Count Per Target など
- **Step Scaling** : 閾値段階的
- **Scheduled** : 時刻ベース

32.4.2 Cluster Capacity Provider

EC2 起動タイプの場合、ノード台数を **タスク需要に応じて** 自動調整。

- **FARGATE / FARGATE_SPOT** : 完全マネージド
- **EC2 ASG** : 自前の ASG をキャパシティプロバイダ化

32.4.3 Fargate Spot

Fargate でも Spot 価格が利用可能 (最大 70% 割引)。中断耐性のあるワークロード (バッチ、ステートレス Web の冗長部分) で。

32.5 EKS の基本

32.5.1 クラスタ構成

```
[EKS Control Plane]   AWS マネージド、SLA 99.95%
    ↓
[Worker Node]
└─ Managed Node Group   EC2 Auto Scaling、AWS が更新管理
```

└ Self-Managed Node	自前 EC2、フルコントロール
└ Fargate	サーバーレス、Pod 単位

クラスタは VPC に紐付く。コントロールプレーンとワーカーノードは ENI で接続。

32.5.2 Add-ons

- **VPC CNI** : Pod に VPC IP を直接割当
- **CoreDNS** : クラスタ内 DNS
- **kube-proxy**
- **EBS CSI / EFS CSI / FSx CSI** : 永続ボリューム
- **Pod Identity Agent** : IRSA の後継 (後述)
- **AWS Load Balancer Controller** : Ingress → ALB、Service → NLB

32.5.3 EKS Auto Mode (2024 年末～)

ノード管理・スケーリング・パッチ・ストレージ・ロードバランサを **AWS が自動運用** するモード。Karpenter / VPC CNI / EBS CSI / LB Controller 等が組み込み済みで **マネージド**。EKS の運用負荷を ECS Fargate 並みに。

32.5.4 Karpenter

ノードオートスケーラー。Pod の要求に合わせて **最適なインスタンスタイプ**を自動選定 し、Spot とオンデマンドを混合。Cluster Autoscaler の後継。

```
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: default
spec:
  template:
    spec:
      requirements:
        - key: kubernetes.io/arch
          operator: In
          values: ["amd64", "arm64"]
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["spot", "on-demand"]
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: default
  limits:
    cpu: "1000"
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 30s
```

32.6 EKS のセキュリティ

32.6.1 IRSA (IAM Roles for Service Accounts)

Pod に IAM ロールを紐付け、AWS API を呼ぶときに **Pod 単位** で権限を絞る。EKS の OIDC プロバイダ経由。

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-app
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/my-app-role
```

32.6.2 EKS Pod Identity (推奨、2023～)

IRSA の後継。OIDC プロバイダを使わず **Add-on (Pod Identity Agent)** 経由で IAM ロールを Pod に渡す。設定がシンプル、セットアップ時間が短縮。

32.6.3 aws-auth ConfigMap → Access Entries

EKS クラスタへの IAM プリンシパルマッピング。従来は `aws-auth` ConfigMap を編集していたが、現在は **Access Entries API** が推奨。

```
aws eks create-access-entry \
  --cluster-name my-cluster \
  --principal-arn arn:aws:iam::...:role/Admin \
  --type STANDARD

aws eks associate-access-policy \
  --cluster-name my-cluster \
  --principal-arn arn:aws:iam::...:role/Admin \
  --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSClusterAdminPolicy \
  --access-scope type=cluster
```

32.7 Ingress / Service

32.7.1 AWS Load Balancer Controller

- **Ingress** リソース → ALB を自動作成
- **Service Type LoadBalancer** → NLB を自動作成
- TargetType `ip` (推奨、Pod IP に直接) または `instance`

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
```

```

name: my-app
annotations:
  kubernetes.io/ingress.class: alb
  alb.ingress.kubernetes.io/scheme: internet-facing
  alb.ingress.kubernetes.io/target-type: ip
  alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS":443}]'
  alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:...
spec:
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: my-app
                port: { number: 80 }

```

32.7.2 VPC Lattice + EKS

VPC Lattice の Service を EKS の Pod から呼ぶ統合（Gateway API Controller 経由）。Service Mesh の代替に。

32.8 ストレージ

種類	用途
EBS CSI	Pod 単一の永続ボリューム（StatefulSet）
EFS CSI	複数 Pod 共有のファイルシステム
FSx CSI	Lustre / OpenZFS / NetApp ONTAP
S3 CSI	S3 をファイルシステムとしてマウント（読み込み中心）

32.9 Service Mesh の選択肢

- **App Mesh**：AWS 純正。Envoy ベース。EKS / ECS / EC2 で動作。新規開発は減速気味
- **Istio**：定番 OSS、機能豊富
- **Linkerd**：軽量・シンプル
- **VPC Lattice**：マネージド、AWS 統合
- **Service Connect**（ECS）：軽量の Service Mesh 代替

複雑な mTLS / トラフィック制御が必要なら Istio、運用負荷下げたいなら VPC Lattice / Service Connect。

32.10 観測性（Container Insights）

- **Container Insights**：CloudWatch によるコンテナ監視（CPU/メモリ/ネットワーク/タスク数/ Pod 数）
- **Container Insights with Enhanced observability**：Prometheus メトリクス自動収集、コスト・パフォーマンス可視化
- **FireLens**：Fluent Bit / Fluentd を使ったログルーティング
- **AWS Distro for OpenTelemetry (ADOT)**：Trace / Metric / Log の統合収集

32.11 セキュリティのベストプラクティス

- 最小権限の Task Role / Pod Role
- **Secrets Manager / Parameter Store** でシークレット注入
- ECR の脆弱性スキャン（Inspector 統合）
- イメージ署名（cosign + Notation）
- **Read-only root filesystem、Non-root user**
- **Security groups for Pods (EKS)**：Pod 単位 SG
- **Network Policy (EKS)**：Calico / Cilium で東西通信制御
- **Falco (EKS)**：ランタイムセキュリティ
- **IMDSv2 必須化（ノード）**

32.12 コスト最適化

- **Fargate Spot / EC2 Spot**
- **Karpenter Consolidation**：未使用ノード自動整理
- **Right-sizing**：Compute Optimizer 推奨を反映
- **Cluster Autoscaler / Karpenter**：必要時のみノード追加
- **CloudWatch Logs 保持期間**
- **Container Insights サンプルング**

32.13 トラブルシューティング

症状	対処
Task が起動失敗	Stopped reason、IAM、ECR pull、ネットワーク
Pod が Pending	リソース不足（CPU/Mem）、ノード spot 中断、Karpenter ログ
ALB ヘルスチェック失敗	SG（ALB→Pod ENI）、/health 200、target type ip と Pod IP 整合
DNS 解決失敗（EKS）	CoreDNS Pod 数、ENI 制限、cluster-dns 設定
Pod が IP を取得できない	VPC CNI の ENI 上限、サブネット IP 枯渇、Custom Networking 検討
ECS Service Connect 通信できない	namespace、port name、Cloud Map

症状	対処
Karpenter で起動しない	NodePool 制約、AMI、 Instance not found ログ
IRSA / Pod Identity が効かない	Service Account annotation、信頼ポリシー、aws-iam-authenticator バージョン
Container Insights が高い	サンプリング、Enhanced observability の取捨

33 Aurora 深掘り

7章で扱った Aurora を、本格運用視点で深掘りする。アーキテクチャの内部構造、Serverless v2、Global Database、Zero-ETL、性能チューニング、Babelfish、移行とコスト。

33.1 Aurora のアーキテクチャ

Aurora は MySQL / PostgreSQL 互換のデータベースエンジンを搭載しつつ、ストレージ層を完全に書き直したクラウドネイティブ DB。

33.1.1 ストレージ層

```
[Aurora インスタンス]
  ↓ ログ送信
[Aurora ストレージ層]
  3 AZ × 2 コピー = 計 6 コピー
  各データブロックは 10GB セグメント
  Quorum: 書き込み 4/6、読み取り 3/6
```

- ・自動修復：故障セグメントを健全な 5 個から再構築
- ・自動拡張：10GB ずつ最大 128TiB
- ・ストレージ I/O 課金：標準は I/O 量、I/O Optimized 設定で固定料金
- ・スナップショット：高速、増分、暗号化、リージョン間コピー

33.1.2 読み取りスケーリング

クラスタ内に最大 15 台のリーダーを配置可能。書き込みはライターから直接ストレージへ、リーダーは差分バッファを最小遅延で受信。

33.2 クラスタの構成要素

- ・DB クラスタ：論理単位。1 つのストレージを共有する複数インスタンス
- ・ライターインスタンス：書き込み可（1 台）
- ・リーダーインスタンス：読み取り専用（最大 15 台）
- ・クラスタエンドポイント：常にライターを指す DNS
- ・リーダーエンドポイント：リーダー間で負荷分散
- ・カスタムエンドポイント：特定のリーダー集合を選んで提供（分析用、報告用など）

33.2.1 Aurora MySQL vs Aurora PostgreSQL

項目	Aurora MySQL	Aurora PostgreSQL
互換	MySQL 5.7 / 8.0	PostgreSQL 13 / 14 / 15 / 16
Babelfish	—	○（SQL Server T-SQL 互換）
pg_vector	—	○

項目	Aurora MySQL	Aurora PostgreSQL
機能性	シンプル	豊富（拡張、JSON、ウィンドウ関数等）

新規開発は **Aurora PostgreSQL が一般的に有利**。MySQL 互換アプリの移植には Aurora MySQL。

33.3 Aurora Serverless v2

ACU（Aurora Capacity Unit）＝約 2 GB メモリ＋比例 CPU。

33.3.1 自動スケーリング

[Min ACU] 0.5（または 0、対応バージョン）
[Max ACU] 1～256

負荷に応じて **秒単位で自動スケール**。

33.3.2 0 ACU 自動ポーズ

2024 年 11 月以降、対応バージョンで **0 ACU まで自動ポーズ**。長時間アクセスがないと完全停止し、再アクセス時に約 15 秒で再開。

対応バージョン：

- Aurora PostgreSQL 13.15+/14.12+/15.7+/16.3+
- Aurora MySQL 3.08+

検証環境・PoC・低頻度バッチで **月コスト数百円～数千円** に圧縮できる。

33.3.3 Provisioned との使い分け

シナリオ	推奨
安定的な常時負荷	Provisioned + Reserved Instance
スパイク・予測困難	Serverless v2
検証・開発	Serverless v2（0 ACU）
マルチテナント SaaS	Serverless v2（テナントごと）

33.4 Aurora Global Database

複数リージョンにまたがるレプリケーション。

[Primary Region: 東京] Writer + Readers ↓ 物理レプリケーション（< 1秒 RPO） Aurora Storage（東京 6コピー）	[Secondary Region: パージニア] Read-only Readers Aurora Storage（パージニア 6コピー）
---	--

- 通常 RPO 1 秒未満、RTO 1 分未満

- ・セカンダリで **読み取り負荷分散**
- ・リージョン障害時、セカンダリを新プライマリに昇格 (**Managed Failover** で自動化)
- ・セカンダリ最大 5 リージョン

33.5 Zero-ETL 統合

ETL ジョブを書かずに、Aurora のデータを **リアルタイム** で他サービスへ複製。

33.5.1 Aurora → Redshift

書き込み発生から **数秒** で Redshift に反映。分析系クエリは Redshift 側で実行、トランザクション系は Aurora で。

33.5.2 Aurora → S3 / OpenSearch

S3 へのエクスポートや OpenSearch への複製も Zero-ETL 化進行中（プレビュー含む）。

33.5.3 Aurora PostgreSQL → DynamoDB

逆方向（DynamoDB → Redshift / OpenSearch）も Zero-ETL 対応。

33.6 Aurora ML

Aurora から **SQL** で SageMaker / Bedrock を呼ぶ。

```
-- Aurora ML 関数の例
SELECT
  customer_id,
  aws_sagemaker.invoke_endpoint(
    'churn-prediction-endpoint',
    NULL, customer_features
  ) AS churn_score
FROM customers;
```

- ・ETL なしで本番 DB に推論結果を組み込める
- ・バッチ処理よりリアルタイム性を求める分析向け

33.7 Babelfish for Aurora PostgreSQL

SQL Server T-SQL を **そのまま受け入れる** 互換レイヤ。Aurora PostgreSQL の追加機能。

- ・TDS（Tabular Data Stream）プロトコルでアプリは SQL Server だと思って接続
- ・既存 .NET / Java アプリの移行コスト大幅削減
- ・100% 完全互換ではない（カバレッジは公式マトリクス）
- ・段階的に PostgreSQL ネイティブに置き換える経路を提供

33.8 I/O Optimized

ストレージ I/O が大量なワークロードで **固定料金**（インスタンス・ストレージ料金が高くなるが、I/O 課金が消える）。

- I/O コストが全体の 25% を超えるなら検討
- 解析系・大量更新系のワークロード向け

33.9 パフォーマンスチューニング

33.9.1 Performance Insights

DB の負荷を **待機イベント単位** で可視化。SQL ごと、ユーザーごと、ホストごとに集計。

- 「どの SQL が遅い」「どのテーブルがホット」
- 過去 7 日（無料） / 24 か月（追加料金）保持
- API で取得して自前ダッシュボードも可

33.9.2 クエリチューニング

- **EXPLAIN / EXPLAIN ANALYZE**：実行計画
- **pg_stat_statements**（Aurora PostgreSQL）：SQL 別統計
- **MySQL Performance Schema**（Aurora MySQL）

33.9.3 インデックス戦略

- B-tree（標準）、Hash、GIN（JSONB / 全文検索）、GiST、BRIN
- 複合インデックスは **選択性高い順**
- 部分インデックス、被覆インデックス
- 過剰なインデックスは書き込み性能を下げる

33.9.4 コネクションプール

Aurora の最大接続数は **インスタンスサイズ依存**。アプリ側で接続プール、または **RDS Proxy**。

33.9.5 RDS Proxy

接続プーリング + フェイルオーバー高速化 + IAM 認証 + Secrets Manager 統合。Lambda から Aurora を叩くときに必須に近い。

```
# Lambda は RDS Proxy のエンドポイントに接続
aws rds-data execute-statement \
  --resource-arn "arn:aws:rds:...:db-proxy:my-proxy" \
  --secret-arn "arn:aws:secretsmanager:..." \
  --sql "SELECT * FROM users LIMIT 10"
```

33.10 Aurora の高可用性

33.10.1 フェイルオーバー

ライター障害時は **リーダーから自動昇格**（通常 30～60 秒）。

- ・ アプリは **クラスタエンドポイント** に接続することで自動追従
- ・ 接続エラー時のリトライロジックが重要

33.10.2 Backtrack（Aurora MySQL のみ）

過去 72 時間まで **DB を巻き戻せる**。誤更新からの即時復旧（バックアップ&リストアより速い）。

33.10.3 Continuous Backup

PITR が標準で有効、過去 35 日まで秒単位で復元。

33.11 セキュリティ

- ・ IAM データベース認証：パスワードレスで接続（短期トークン）
- ・ KMS 暗号化（保管時、転送時）
- ・ VPC エンドポイント（RDS API 用）
- ・ Database Activity Streams：すべての DB アクティビティを Kinesis にストリーミング、監査統合

33.12 コストの内訳

- ・ インスタンス時間（Provisioned）または ACU 時間（Serverless v2）
- ・ ストレージ（GB-月、I/O Optimized で挙動変化）
- ・ I/O リクエスト（標準のみ）
- ・ バックアップストレージ（クラスタストレージサイズ超過分）
- ・ Global Database（追加リージョンの読み取り、レプリケート I/O）
- ・ Performance Insights（標準 7 日無料、長期は別料金）
- ・ RDS Proxy（vCPU 時間）

33.12.1 コスト最適化のコツ

- ・ Serverless v2 で 0 ACU まで落とす（対応バージョン）
- ・ Reserved Instance（Provisioned 1～3 年）
- ・ Aurora I/O Optimized を試算
- ・ リーダー数を最小化、必要時に増やす
- ・ Performance Insights 7 日で十分な場合は長期保存無効
- ・ Backtrack は MySQL のみで料金あり、不要なら無効

33.13 移行

7 章・21 章でも触れたが、Aurora への移行ハイライト：

33.13.1 Oracle / SQL Server → Aurora PostgreSQL

- AWS SCT でスキーマ・PL/SQL 変換
- DMS でデータ移行（Full Load + CDC）
- Babelfish で SQL Server からはアプリ無修正の場合も
- アプリ側 SQL の差異対応（ROWNUM → LIMIT、NVL → COALESCE 等）

33.13.2 MySQL → Aurora MySQL

互換性高く、移行は比較的容易。mysqldump + 復元 or DMS。

33.13.3 PostgreSQL → Aurora PostgreSQL

ほぼ無修正で移行可。pg_dump + 復元 or DMS。

33.14 モニタリング

- CloudWatch メトリクス：CPU、メモリ、接続数、レプリケーションラグ、ディスク I/O
- Performance Insights：SQL レベル
- Database Activity Streams：監査ログ
- Enhanced Monitoring：OS レベルメトリクス（CPU、ディスク、ネットワーク）
- スロークエリログ

CloudWatch アラームの典型しきい値：

- CPU > 80% で 5 分継続
- レプリケーションラグ > 30 秒
- ディスクキューデプス > 1
- 接続数 > インスタンス上限の 80%

33.15 Aurora vs RDS の判断

要件	推奨
新規・OSS 互換 (MySQL/PG)	Aurora
既存 RDS の延長	そのまま RDS、要件あれば Aurora 移行
SQL Server / Oracle	RDS（Aurora は MySQL/PG のみ）
極端な高可用性・グローバル	Aurora（Global Database）
単純・低コスト・最低限	RDS
Serverless 必須	Aurora Serverless v2 が現実的

33.16 よくあるトラブル

症状	対処
接続数枯渇	RDS Proxy、アプリ側プール、インスタンスサイズアップ
Failover で接続切れ	リトライロジック、クラスタエンドポイント使用、RDS Proxy
リーダーが追従遅延	書き込み負荷、リーダーサイズ、ネットワーク
Storage Full	128TiB 上限近い、データ整理、Aurora Limitless
I/O コスト高い	I/O Optimized 検討、クエリ最適化、キャッシュ
スロークエリ	Performance Insights、EXPLAIN、インデックス見直し
Babelfish で機能不足	互換マトリクス、PostgreSQL ネイティブで書き直し
Serverless v2 がスケールしない	Min/Max ACU、CPU/Mem の余地確認

34 コスト最適化と FinOps

10 章でコストの基本、28 章で Well-Architected の柱として軽く触れたコスト最適化を、本章では FinOps の方法論 と AWS の具体ツール を組み合わせて深掘りする。

34.1 FinOps とは

FinOps は **Finance + DevOps** の合成語。クラウド利用の財務管理を **エンジニアリング・財務・経営** の協業で回す方法論。FinOps Foundation が標準化を進めている。

34.1.1 FinOps の 3 原則

1. **Inform (可視化)** : 何にいくら使っているかを把握する
2. **Optimize (最適化)** : 無駄を削り、最適なリソース構成にする
3. **Operate (運用)** : 継続的なプロセス・文化として定着させる

34.1.2 FinOps のフェーズ

[Crawl 段階] 月次レポート、コスト按分タグ、無駄リソース掃除

↓

[Walk 段階] Savings Plans、Reserved Instance、予算アラート

↓

[Run 段階] 自動化、リアルタイム可視化、アーキテクチャ単位の最適化

34.2 AWS のコスト関連サービス

サービス	用途
Cost Explorer	可視化・分析 (13 か月、API 経由で 37 か月)
Budgets	予算設定とアラート (コスト・使用量・RI/SP)
Cost and Usage Report (CUR 2.0)	詳細データを S3 へ。Athena で深掘り
Cost Optimization Hub	推奨を一元集約・優先順位付け
Compute Optimizer	EC2 / EBS / Lambda / ASG / RDS / Fargate の右サイジング推奨
Trusted Advisor	コスト・パフォーマンス・セキュリティ・耐障害性チェック
Savings Plans	計算リソースの予約購入
Reserved Instances	従来型予約購入 (多くは Savings Plans 推奨)
Customer Carbon Footprint Tool	CO2 排出量可視化
Pricing Calculator	構成見積

34.3 コストの可視化

34.3.1 コスト配分タグ (Cost Allocation Tags)

リソースに付けたタグでコストを按分。**アカウント設定**で「Active」化しないとレポートに反映されない（アクティブ化日からのデータのみ）。

推奨タグ：

- Project / Application
- Environment (prod / staging / dev / sandbox)
- Owner / Team
- CostCenter
- ManagedBy (terraform / cdk / manual)

34.3.2 Cost Categories

複数のディメンション（アカウント・タグ・サービス）を **論理的にグループ** 化。「Marketing 部門」「データ基盤」「セキュリティ基盤」のような **ビジネス意味** で集計できる。

34.3.3 AWS Cost Explorer

13 か月分（API 経由で最大 37 か月）のコスト・使用量を分析。

- グループ化：サービス、リージョン、AZ、API オペレーション、タグ、Cost Categories
- フィルタ：複数条件
- 予測（Forecast）：ML ベースで今月末の予測
- 保存済みレポート、お気に入り
- 異常検知（Cost Anomaly Detection）

34.3.4 Cost Anomaly Detection

ML ベースで **予期しないコスト急増** を検知し、メール / Slack で通知。Monitor を「サービス」「アカウント」「タグ」「Cost Categories」で設定。

34.3.5 CUR (Cost and Usage Report 2.0)

最も詳細な利用データを **S3 にデータレイク形式で配信**。Glue カタログ自動登録、Athena / QuickSight でクエリ。

34.3.6 AWS Cost Optimization Hub

各種推奨（Savings Plans、Compute Optimizer、Idle Resources、Right-sizing）を **横断的に集約・優先順位付け**。組織レベルでも有効化可。

34.4 Right-Sizing（適正サイジング）

34.4.1 Compute Optimizer

EC2、Auto Scaling Group、EBS、Lambda、RDS、ECS Fargate に対し、過去 14 日のメトリクスから推奨サイズを算出。

```
aws compute-optimizer get-ec2-instance-recommendations
```

レコメンデーションは「現状のリスク」と「最適化後の節約見込み」がセット。GUI で一覧確認可。

34.4.2 手動チェックの観点

- CPU 使用率 < 40%（平均）→ ダウンサイズ候補
- メモリ使用率 < 40% → ダウンサイズ候補
- ネットワーク I/O < 50% → ダウンサイズ候補
- EBS Provisioned IOPS / Throughput が常時余裕 → gp3 に切替

34.5 購入オプション最適化

34.5.1 Compute Savings Plans

EC2、Fargate、Lambda 横断で 1～3 年予約。

- 最大 66% 割引
- インスタンスファミリー・サイズ・リージョン・OS を変更可
- 全 EC2 / Fargate / Lambda に自動適用

34.5.2 EC2 Instance Savings Plans

特定インスタンスファミリー・リージョンに縛り、最大 72% 割引。

34.5.3 Reserved Instances

従来型。RDS / ElastiCache / OpenSearch / Redshift で利用。EC2 では Savings Plans 推奨。

34.5.4 コミット戦略

- ベースライン消費の 60～70% を Savings Plans でカバー
- 残り 30% はオンデマンド + Spot
- ベースラインを急に下げない計画
- 3 年 No Upfront > 1 年 All Upfront > 1 年 No Upfront の順でお得（一般論）

34.5.5 Spot Instance

最大 90% 割引。中断耐性のあるワークロードに。

- EC2 Auto Scaling グループで Spot 混合
- Fargate Spot でコンテナタスク

- AWS Batch でバッチ処理
- EC2 Spot Fleet で複数インスタンスタイプ+複数 AZ
- EMR / SageMaker で計算ワークロード

34.6 不要リソースの掃除

定期的に確認するチェックリスト：

対象	探し方
停止 EC2 に紐づく EBS	Trusted Advisor、Cost Explorer の EBS
未アタッチ EBS	<code>aws ec2 describe-volumes --filters Name=status,Values=available</code>
未関連付け Elastic IP	<code>aws ec2 describe-addresses --query "Addresses[?AssociationId==null]"</code>
古い EBS スナップショット	Lifecycle Manager で自動削除
古い AMI	カスタム AMI の世代管理
使われていない NAT Gateway	VPC Flow Logs で通信実績確認
アイドル ELB	Cost Explorer + CloudWatch (Active Connections=0)
使われていない Lambda	Invocations =0 の関数
古い CloudWatch Logs グループ	保持期間設定、不要グループ削除
大量のマルチパートアップロードの破片	S3 ライフサイクル
非現行 S3 バージョン	S3 ライフサイクル

34.6.1 自動化

EventBridge Scheduler + Lambda で 夜間に検出 → タグ付け → 通知 → 一定期間後削除 のフローを組む。

34.7 サービス別の最適化ポイント

34.7.1 EC2

- 最新世代・Graviton 検討
- Auto Scaling で需要追従
- Spot 混合
- 停止可能なら夜間停止
- Compute Optimizer 反映
- EBS gp2 → gp3 移行（性能同等で安い）

34.7.2 RDS / Aurora

- Multi-AZ は本番のみ、検証は Single AZ

- Aurora Serverless v2（特に 0 ACU 自動ポーズ）
- リードレプリカ数の見直し
- **I/O Optimized**：I/O が多いなら検討
- **Reserved Instance**：1 年予約で 30～40%

34.7.3 Lambda

- **Graviton (ARM)**：x86 比 20% 安い
- **メモリ最適化**：Lambda Power Tuning
- **Provisioned Concurrency** は本当に必要な時間帯のみ
- **コールドスタート対策**でリクエストあたりコスト最適化

34.7.4 コンテナ (ECS / EKS / Fargate)

- **Fargate Spot / EC2 Spot**
- **Karpenter consolidation**
- **Cluster Autoscaler** 最小構成
- 不要な NAT Gateway 共有化

34.7.5 S3

- ライフサイクルで Standard → IA → Glacier → Deep Archive
- **Intelligent-Tiering**：アクセスパターン不明なら一律
- **S3 Storage Lens** で全社可視化
- 未完了マルチパート削除（ライフサイクル）
- 古いバージョン削除
- **Requester Pays**：パブリックデータの読み出しを利用者負担に

34.7.6 データ転送

- **VPC エンドポイント**（S3/DynamoDB は無料）
- **PrivateLink** で内部通信プライベート化
- **CloudFront** 経由で S3 オリジナル無料
- **リージョン間転送を避ける** 設計

34.7.7 CloudWatch

- **Logs 保持期間** 必須設定
- **Metrics の高頻度カスタムメトリクス** を絞る
- **Logs Insights クエリ** のスキャン量に注意
- **Container Insights / Application Insights** のサンプリング

34.7.8 Bedrock / SageMaker

- **Bedrock**：On-Demand → Provisioned Throughput の損益分岐
- **SageMaker Notebook / Endpoint** の停止忘れ防止
- **Spot Training**

- ・ Inferentia / Trainium 検討

34.8 予算管理

34.8.1 AWS Budgets

- ・ コスト予算：金額上限
- ・ 使用量予算：EC2 時間、データ転送量
- ・ Reservation 予算：RI/SP 利用率
- ・ Savings Plans 予算：SP 利用率と Coverage

通知：メール、SNS、Chatbot (Slack/Teams)。1 アカウントで **最大 100 個** の予算。

34.8.2 階層的予算設計

- ・ 組織全体予算：管理アカウントで月次総額
- ・ 環境別予算：prod / dev / sandbox
- ・ 部門別予算：CostCenter タグベース
- ・ プロジェクト別予算：Project タグベース

34.9 タグ戦略の運用

タグは **最初に決めて全リソースに強制** するのが鉄則。

- ・ Tag Policies (Organizations)：組織レベルで必須タグ・タグ値を強制
- ・ Service Control Policies：タグなしリソース作成を Deny
- ・ AWS Resource Groups Tag Editor：一括タグ付け・修正
- ・ AWS Config Rule：必須タグ違反を検知

34.10 組織的な FinOps

34.10.1 Cost Center モデル

- ・ Showback：使用部署にコストを **見える化** する（請求はしない）
- ・ Chargeback：使用部署に **請求書を回す**

最初は Showback で意識を高め、定着したら Chargeback に進む。

34.10.2 コストレビュー会議

- ・ 月次：前月実績、予算対比、異常検知、トレンド
- ・ 四半期：購入オプション最適化 (SP 更新)
- ・ 年次：全体戦略、契約見直し

参加者：FinOps チーム、エンジニア、財務、経営層。

34.11 持続可能性とコスト

10 章の **Customer Carbon Footprint Tool** と組み合わせ、CO2 削減 = 多くの場合 = コスト削減。

- Graviton 採用：CO2 削減 + コスト削減
- Spot 利用：遊休 HW 活用、CO2 削減
- データライフサイクル：低炭素な層へ

34.12 典型的な節約事例

施策	典型的な節約率
Compute Savings Plans 1 年 No Upfront	～30%
EC2 Spot（中断耐性ワークロード）	～70%
gp2 → gp3 EBS	～20%
Aurora Serverless v2 0 ACU	低頻度環境で 80% 超
S3 Intelligent-Tiering	10～30%
Right-sizing（Compute Optimizer）	10～25%
Graviton 採用（EC2 / Lambda）	10～20%
古い RI / SP の刷新	5～15%
NAT Gateway 削減（S3 GW Endpoint）	トラフィック次第で大幅
CloudWatch Logs 保持期間設定	ログ多めなら大

34.13 コスト最適化の優先順位

1. 見えない無駄を消す：未使用 EBS / EIP / NAT、保持期間なし Logs
2. **Right-sizing**：Compute Optimizer 推奨を反映
3. 購入オプション：Savings Plans でベースラインを覆う
4. **アーキテクチャ最適化**：Serverless 化、Graviton、S3 階層化
5. **組織的最適化**：FinOps 文化、定期レビュー、自動化

34.14 よくある罠

罠	対処
タグ付けを後からやる	最初から Tag Policy で強制
予算アラートだけで安心	予測ベースアラート、定期レビュー

罣	対処
Savings Plans を過剰に買う	ベースラインの 60～70% に留める
リザーブ買って即アーキ変更	3 年 RI は慎重に、1 年でリスクヘッジ
Cost Explorer を月末だけ見る	週次・日次の異常検知
削除を恐れて溜め込む	タグ + 自動化で一定期間後削除
Spot で本番停止	中断耐性設計、混合戦略
Bedrock / SageMaker で青天井	Budgets、利用上限、レート制限

35 可観測性と SRE

9 章で運用と監視の基本を扱った。本章では **可観測性 (Observability)** の概念と、AWS での **分散トレーシング**、**Application Signals**、**Container Insights**、**OpenTelemetry**、**SRE 的な SLI/SLO 運用** を深掘りする。

35.1 監視と可観測性の違い

- ・ **監視 (Monitoring)** : 事前に決めた指標が **予期した範囲** にあるかを継続チェック
- ・ **可観測性 (Observability)** : **予期しない問題** に対して、システムの内部状態を **外部から推測** できる性質

「予期しない問題が起こる」を前提に設計するのが現代システム。可観測性の 3 本柱：

柱	説明
Metrics (メトリクス)	数値時系列。集計済み、低コスト、可視化容易
Logs (ログ)	テキストイベント。詳細だがクエリ・コスト負荷高
Traces (トレース)	リクエストの分散経路。ボトルネック特定に強い

これらに加え、**Profiling** (CPU/メモリの詳細) と **Events** (重要事象) を含めて **5 本柱** とすることもある。

35.2 AWS の可観測性スタック

目的	主なサービス
メトリクス	CloudWatch Metrics, Managed Prometheus
ログ	CloudWatch Logs, OpenSearch Service
トレース	X-Ray, Managed Grafana, OpenTelemetry
APM・統合	CloudWatch Application Signals
外形監視	CloudWatch Synthetics
リアルユーザー監視	CloudWatch RUM
プロファイリング	CodeGuru Profiler
可視化	CloudWatch Dashboard, Managed Grafana, QuickSight
アラート・インシデント	SNS, EventBridge, Incident Manager, Chatbot

35.3 CloudWatch Metrics の使いこなし

35.3.1 Embedded Metric Format (EMF)

ログ出力と同時にメトリクスも出す JSON フォーマット。Lambda Powertools の Metrics モジュールが内部で使っている。


```
{
  "_aws": {
    "Timestamp": 1745231400000,
    "CloudWatchMetrics": [{
      "Namespace": "MyApp",
      "Dimensions": [["Service"]],
      "Metrics": [
        {"Name": "OrdersProcessed", "Unit": "Count"},
        {"Name": "OrderValue", "Unit": "None"}
      ]
    }]
  },
  "Service": "orders",
  "OrdersProcessed": 1,
  "OrderValue": 1234.56,
  "OrderId": "abc-123"
}
```

`PutMetricData` API を呼ばずに、ログ書き込みだけでメトリクスが生成される。コスト効率良。

35.3.2 Anomaly Detection

機械学習による **動的なしきい値**。曜日・時間帯のパターンを学習。

```
aws cloudwatch put-anomaly-detector \
  --namespace AWS/EC2 \
  --metric-name CPUUtilization \
  --dimensions Name=InstanceId,Value=i-0abc... \
  --stat Average
```

しきい値運用が難しい指標（リクエスト数、レイテンシ）で有効。

35.3.3 Composite Alarm

複数アラームを論理演算で合成。

```
aws cloudwatch put-composite-alarm \
  --alarm-name "ProdServiceUnhealthy" \
  --alarm-rule "ALARM(HighErrorRate) AND ALARM(HighLatency)"
```

アラート疲れ防止。

35.3.4 Metric Math

複数メトリクスを式で組み合わせて **派生メトリクス**。エラー率（`Errors / Invocations * 100`）など。

35.4 CloudWatch Logs

35.4.1 Logs Insights

```
fields @timestamp, @message, @logStream
| filter @message like /ERROR/ and userId = "user-123"
| stats count(*) by bin(5m)
| sort @timestamp desc
| limit 100
```

正規表現、JSON フィールド抽出 (`fields @message.userId as user`)、集計、ソート、グラフ化。

35.4.2 Live Tail

リアルタイムでログを流れるように見る。 `docker logs -f` の CloudWatch 版。

35.4.3 Subscription Filter

ログをリアルタイムで Kinesis / Firehose / Lambda に転送。OpenSearch / S3 への流し込みパイプラインの起点。

35.4.4 Vended Logs

VPC Flow Logs、Route 53 Resolver Logs、CloudFront Logs などを CloudWatch Logs / S3 / Firehose に出力。

35.4.5 コスト

- ・ 取り込み：\$0.50/GB（東京、Standard）
- ・ 保管：\$0.033/GB-月
- ・ Logs Insights：\$0.0050/GB スキャン

保持期間設定が必須、ログレベルの調整 と サンプリング で大幅削減できる。Standard と Infrequent Access (IA) クラスの選択も 2024～可能。

35.5 AWS X-Ray

分散トレース。

35.5.1 セグメントとサブセグメント

- ・ セグメント：1 サービス内のトレース単位
- ・ サブセグメント：セグメント内の細かい操作（DynamoDB 呼び出し、外部 HTTP 等）

X-Ray SDK / Lambda Powertools Tracer / OpenTelemetry いずれでも生成可。

35.5.2 サービスマップ

トレースから自動生成される **依存関係グラフ**。各ノードの平均レイテンシ、エラー率、スループットが見える。問題発生時の影響範囲特定が早い。

35.5.3 サンプリング

全トレースを取ると高コスト。X-Ray の **デフォルトサンプリングルール** は「最初の1リクエスト/秒 + 残り 5%」。アプリで上書き可。

35.6 Application Signals

CloudWatch の APM 機能 (2024 年 GA)。

35.6.1 主要機能

- **自動 SLI/SLO 設定**：レイテンシ、可用性、エラー率を自動収集
- **サービスカタログ**：マイクロサービス一覧
- **依存関係マップ**：X-Ray 統合のサービス相互参照
- **Burn Rate Alarm**：SLO の予算消費速度に基づくアラート
- **対応**：EC2 / ECS / EKS / Lambda (Lambda Powertools 経由)

35.6.2 SLO 設定の例

```
Service: orders-api
SLI: latency < 500ms (P99)
SLO: 99.5% over 7 days
Error budget: 0.5% × 10,080分 = 50.4分
Burn rate alarm: 1時間で全budget の 14.4倍消費
```

エラーバジレットの考え方を SRE 的に組み込める。

35.7 Container Insights

ECS / EKS / Kubernetes 向け。Pod / コンテナ / ノード単位の CPU / メモリ / ネットワーク / ディスクメトリクス。

35.7.1 Enhanced observability (2024～)

新世代。Prometheus メトリクス自動収集、より深い OS レベルメトリクス、コンテナ単位のドリルダウン。

35.7.2 料金注意

メトリクス数とログ量で課金。大規模クラスタでは **特定 namespace のみ有効**、サンプリングで節約。

35.8 AWS Distro for OpenTelemetry (ADOT)

OpenTelemetry (OTel) の AWS ディストリビューション。ベンダーロックイン回避、業界標準。

35.8.1 構成

- **ADOT Collector**：エージェント。Trace / Metric / Log を受信し変換・送信
- **ADOT SDK**：アプリに組み込んで自動計装
- 送信先：X-Ray、CloudWatch、Managed Prometheus、Managed Grafana、サードパーティ（Datadog、New Relic 等）

```
# ADOT Collector の設定例
receivers:
  otlp:
    protocols:
      grpc: { endpoint: 0.0.0.0:4317 }
      http: { endpoint: 0.0.0.0:4318 }
processors:
  batch:
exporters:
  awsxray:
  awsemf:
    namespace: MyApp
service:
  pipelines:
    traces: { receivers: [otlp], processors: [batch], exporters: [awsxray] }
    metrics: { receivers: [otlp], processors: [batch], exporters: [awsemf] }
```

OTel + ADOT は **将来の柔軟性** を確保する選択。

35.9 Managed Prometheus / Managed Grafana

OSS の Prometheus / Grafana をマネージドで。

35.9.1 Managed Prometheus

- スケーラブル、リテンション 150 日
- リモートライト経由でメトリクス収集
- PromQL 完全互換
- **Workspace** 単位

35.9.2 Managed Grafana

- ダッシュボード / アラート
- 多数のデータソース統合（CloudWatch、Prometheus、X-Ray、OpenSearch、Athena 等）
- IAM / SAML / Identity Center 認証

EKS と Prometheus エコシステムを使うなら **Managed Prometheus + Managed Grafana** が定番。

35.10 CloudWatch Synthetics

外形監視。Canary（Node.js / Python スクリプト）を定期実行して URL や API のヘルスを確認。

```
const { Synthetics } = require('Synthetics');

const apiCanary = async function () {
  const url = 'https://api.example.com/health';
  const response = await Synthetics.executeHttpStep('healthcheck', { url });
  if (response.statusCode !== 200) {
    throw new Error(`Status: ${response.statusCode}`);
  }
};

exports.handler = async () => {
  return await apiCanary();
};
```

CloudWatch アラームで失敗時に通知。

35.11 CloudWatch RUM (Real-User Monitoring)

実際のブラウザクライアントからパフォーマンス・エラーをメトリクス化。Core Web Vitals (LCP、FID、CLS)、JS エラー、ページロード時間。

35.12 Incident Manager

インシデント発生時の対応プロセス自動化。

- Response Plan (連絡経路、対応手順)
- Engagement (誰を呼ぶか、PagerDuty / Slack 統合)
- Runbook (Systems Manager Automation で対応操作)
- Post-incident analysis (テンプレートで振り返り)

CloudWatch アラーム → Incident Manager → 対応者呼び出し → Runbook 実行、というフロー。

35.13 SRE 的な運用

35.13.1 SLI / SLO / SLA

- SLI (Indicator) : 実測の指標 (例: 可用性 99.95%)
- SLO (Objective) : 目標 (例: 99.9% 保証する内部目標)
- SLA (Agreement) : 顧客との契約 (例: 99.5% 以下なら返金)

SLO は SLA より厳しく設定。バッファを内部で持つ。

35.13.2 Error Budget

SLO の余裕分を エンジニアリングチームが「使える」概念。

- 99.9% SLO → 30 日で 43.2 分のダウンタイム許容
- これを超えるとリリースを止め、信頼性向上に集中

- ・ 余裕があれば新機能リリースに使える

35.13.3 Burn Rate Alarm

Error Budget の消費速度でアラート。

1時間で 1日分 (30日 budget の $1/30 = 3.33\%$) 以上消費 → 重大アラート
 1時間で 6時間分以上消費 → 中程度アラート

短期と長期を組み合わせて 誤検知を減らす。Application Signals に組み込み済み。

35.13.4 Postmortem 文化

インシデント後に 非難なしの振り返り。

- ・ 何が起きたか
- ・ なぜ起きたか (5 Whys)
- ・ どう検知したか、どう対応したか
- ・ 再発防止策

Incident Manager のテンプレート活用。

35.14 オブザーバビリティの設計原則

- ・ 最初の 1 日からダッシュボード 1 枚作る
- ・ 構造化ログ (JSON) を最初から
- ・ 関連 ID を全ログに伝播
- ・ ヘルスチェックエンドポイント を必ず実装
- ・ ビジネスメトリクス を技術メトリクスと並べて見る
- ・ ログ・メトリクス・トレースの相互リンク を確立

35.15 コスト最適化

可観測性は 気を抜くと月数百万円 レベルになる。

- ・ ログ取り込み量を意識 (DEBUG ログを本番で出さない)
- ・ 保持期間 を必ず設定
- ・ Log Insights のクエリ範囲 を絞る
- ・ メトリクスのカーディナリティ に注意 (タグ次元が多すぎると爆発)
- ・ X-Ray サンプリング を用途別に
- ・ Container Insights は必要 namespace のみ

35.16 サードパーティ統合

- ・ Datadog : APM・ログ・メトリクスの統合プラットフォーム。AWS 連携豊富
- ・ New Relic : APM 強い、SaaS
- ・ Splunk : エンタープライズログ・SIEM

- **Honeycomb**：イベント指向、高カーディナリティに強い
- **Sumo Logic**：クラウドネイティブ SIEM

OTel + ADOT で **ベンダー乗り換え可能性** を確保しておくのが現代的。

35.17 よくある落とし穴

罠	対処
ログ垂れ流し	構造化、レベル別、サンプリング、保持期間
ダッシュボードがない	新規サービスはダッシュボード必須化
アラート疲れ	Composite Alarm、Severity 分け、Runbook 化
監視のための監視	SLO ベースに切替、ビジネス KPI と紐付け
トレースのサンプリング不足	99.9% は問題ない、エラー時は 100% サンプル
可観測性コスト爆発	月次レビュー、必要 namespace のみ、サンプリング
独自実装で属人化	Powertools / OTel 等の標準採用

36 AWS の最新動向と次の一步

本書の最終章として、2024～2026 年時点の AWS の主要トレンド、注目すべき新サービス、そして読者が 次に進むべき方向性 をまとめる。

36.1 2024～2026 年の主要トレンド

36.1.1 生成 AI の AWS 統合の加速

- Amazon Bedrock の急拡大：Claude、Nova、Llama、Mistral、Stability AI などのマルチモデル戦略
- Amazon Q シリーズの全面展開：Developer / Business / QuickSight / Connect
- AWS App Studio：自然言語からアプリ生成
- Bedrock Agents と Knowledge Bases で RAG の標準化
- MCP（Model Context Protocol）対応の進行：Bedrock や Claude API との連携が透過的に
- Bedrock Guardrails の高度化：プロンプトインジェクション対策、PII 自動マスキング

36.1.2 サーバーレスのさらなる成熟

- Lambda SnapStart の Java → Python / .NET 拡大
- Aurora Serverless v2 の 0 ACU 自動ポーズ
- EKS Auto Mode：K8s の運用負荷を Fargate 並みに
- DynamoDB の zero-ETL、Storage Class 自動化
- S3 Tables（Iceberg ネイティブ）

36.1.3 マルチリージョン・マルチアカウントの標準化

- Control Tower でランディングゾーンが一般化
- Cloud WAN で広域ネットワーク管理
- Multi-Region Keys（KMS）、Aurora Global Database、DynamoDB Global Tables の組み合わせ
- Resource Control Policies（RCP）：SCP のリソース版（2024～）

36.1.4 セキュリティの自動化

- GuardDuty Runtime Monitoring：EC2 / ECS / EKS のホスト内挙動
- GuardDuty Malware Protection：S3、EBS のマルウェアスキャン
- Security Hub の自動修復強化
- IAM Access Analyzer の未使用アクセス分析・カスタムポリシーチェック
- Verified Permissions（Cedar）のアプリ内認可
- AWS Verified Access でゼロトラスト

36.1.5 コスト・FinOps の本格化

- Cost Optimization Hub で組織横断

- **Compute Optimizer** の対象拡大 (RDS、ECS Fargate)
- **Cost Anomaly Detection** の精度向上
- **S3 Tables / Athena / Redshift Serverless** の zero-ETL でデータ基盤コスト見直し

36.1.6 持続可能性

- **Customer Carbon Footprint Tool** の利用拡大
- **Graviton (ARM)** の採用が標準に
- **Trainium / Inferentia** で ML の電力効率改善
- **AWS の再生可能エネルギー 100% 化目標 (2025～)**

36.1.7 エッジ・ハイブリッド

- **Outposts** の小型モデル
- **Local Zones** の都市拡大
- **Wavelength** で 5G エッジ

36.1.8 Quantum / Robotics / Industrial

- **Amazon Braket** (量子コンピューティング)
- **AWS RoboMaker** (ロボット開発)
- **AWS for Industries** (製造、ヘルスケア、金融、自動車)

36.2 注目の新サービス・機能 (2024～2026)

サービス / 機能	要点
Bedrock Knowledge Bases / Agents / Guardrails	生成 AI の本番活用基盤
Amazon Q Developer	IDE 内 AI コーディング支援
EKS Auto Mode	K8s の運用負荷を大幅低減
S3 Tables	Iceberg ネイティブテーブル
Aurora Serverless v2 0 ACU	低頻度 DB のコスト劇減
Lambda SnapStart for Python / .NET	Cold Start 改善
Aurora DSQL	PostgreSQL 互換、分散・サーバーレス・マルチリージョン Active-Active
DynamoDB Multi-Region Strong Consistency	Global Tables の強整合化
Resource Control Policies (RCP)	リソース側のガードレール (SCP の対)
Verified Access の機能拡張	ゼロトラスト VPN 代替
Security Hub Advanced	統合セキュリティビュー強化
Cost Optimization Hub	推奨集約・優先順位付け

サービス / 機能	要点
SageMaker Unified Studio	ML / 生成 AI / Glue / EMR / Redshift 統合 IDE
VPC IPAM	IP アドレス計画の自動化
Connectivity for VPCs (VPC Lattice)	サービス層のネットワーキング
Bedrock Custom Model Import	独自モデルを Bedrock API で

36.3 AWS リリースサイクルへの追従

AWS の新機能発表は **毎日** と言って過言ではない。情報を追う方法：

36.3.1 公式ソース

- AWS What's New：日次の新機能発表（RSS）
- AWS Blog：技術解説、深掘り
- AWS re:Invent（11 月末～12 月）：年次最大イベント、数百の新発表
- AWS Summit：地域・テーマ別カンファレンス
- AWS Heroes / Community Builders Blog：個人視点の解説

36.3.2 二次情報

- Last Week in AWS（Corey Quinn）：辛口だが洞察あり
- DevelopersIO（クラスメソッド）：日本語で圧倒的情報量
- Reddit r/aws、Hacker News
- re:Invent セッション動画（YouTube）

36.3.3 学習リソース（再掲）

29 章で詳述。Skill Builder、Adrian Cantrill、Stephane Maarek、JAWS-UG。

36.4 個人プロジェクトでのおすすめ構成

「AWS を学びたい」「ポートフォリオを作りたい」読者向けの、**Free Tier 内で完結** する典型構成：

36.4.1 1. サーバーレス Web アプリ（24 章）

S3 + CloudFront + ACM + API Gateway + Lambda + DynamoDB + Cognito

月コスト：数十円～数ドル。ほぼ常時 Free Tier 内。

36.4.2 2. 個人ブログ（Static Site Generator）

Hugo / Astro → S3 + CloudFront + Route 53 + ACM
 GitHub Actions (OIDC) → S3 sync + CF Invalidation

月コスト：100 円程度。

36.4.3 3. ML / 生成 AI 体験

Bedrock (On-Demand) + Knowledge Base (OpenSearch Serverless)
Lambda + API Gateway で簡易 Chatbot

月コスト：トークン量次第。週末プロジェクトなら数百円。

36.4.4 4. データ収集・分析

EventBridge Scheduler → Lambda → S3 (Bronze)
Glue Crawler + Athena で集計
QuickSight トライアル

月コスト：データ量次第、数百円～。

36.4.5 5. IoT / エッジ

ESP32 + AWS IoT Core MQTT → Lambda → DynamoDB / Timestream
QuickSight でダッシュボード

月コスト：数百円。

36.5 ケースに応じた参考アーキテクチャ集

業界・規模ごとの典型構成は、AWS Architecture Center と AWS Solutions Library で参照可能。

主なカテゴリ：

- E-Commerce
- Media / Video Streaming
- Healthcare
- Financial Services (PCI / 規制対応)
- IoT
- Gaming
- Mobile
- Web Application
- Data Lake
- ML / Generative AI

36.6 AWS と他クラウドの併用

実務では マルチクラウド / ハイブリッドクラウド も増える。

要件	組み合わせ
Google 系 SaaS と統合	GCP (BigQuery、Workspace) + AWS
Microsoft 系資産	Azure (Entra ID、AD) + AWS
Snowflake / Databricks	マルチクラウド SaaS + AWS
自社データセンターと統合	Direct Connect + Outposts + Storage Gateway
リージョン要件で他クラウド	DataSync、DMS で連携

ツール側は Terraform / Pulumi / Crossplane がマルチクラウド IaC の選択肢。

36.7 AWS とオープンソース

AWS は OSS の企業内利用基盤 として強い。代表例：

- EKS / Karpenter : Kubernetes
- MSK : Kafka
- OpenSearch : Elasticsearch フォーク
- Apache Iceberg : S3 Tables の基盤
- PostgreSQL / MySQL : Aurora 互換
- OpenTelemetry : ADOT
- Prometheus / Grafana : Managed
- Apache Spark : EMR、Glue、Athena Spark

「マネージドの OSS」という選択が、ベンダロックイン回避と運用負荷削減を両立する。

36.8 AWS パートナiershipとキャリア

36.8.1 AWS Partner Network (APN)

企業や個人が AWS のパートナーとして認定される制度。

- Consulting Partner : SIer、コンサル
- Technology Partner : ISV
- Training Partner : 教育
- MSP : マネージドサービス
- 個人は AWS Community Builder、AWS Hero、Heroes Distinguished でコミュニティ的認定

36.8.2 キャリアパス

- Cloud Engineer / DevOps
- Solutions Architect
- Data Engineer / ML Engineer
- Security Engineer
- FinOps Practitioner
- AWS Specialist Consultant

認定資格 + 実務経験 + ポートフォリオで市場価値が決まる。

36.9 学び続けるための 1 日 / 1 週間のリズム

36.9.1 毎日（15 分）

- AWS What's New を流し見
- 関心領域のブログを 1 記事

36.9.2 毎週（1～2 時間）

- 実機で 1 つ新機能を触る
- DevelopersIO や Last Week in AWS を読む
- コミュニティ Slack / Discord に顔を出す

36.9.3 毎月（半日）

- 自分の構成を Well-Architected Review
- Cost Explorer で前月コスト分析
- 1 つ深掘りトピックを選んで論文・公式 Whitepaper を読む

36.9.4 毎年（数日）

- re:Invent セッション 10～20 本視聴
- 1 つ新しい資格に挑戦
- ポートフォリオ更新

36.10 本書の総まとめ

本書は 36 章 / 約 500 ページにわたって、AWS の全体像を「広く、ある程度深く」扱った。各章で扱った内容を再構成すると以下のような学習軌跡になる。

ステージ	章
基礎	1～6 章（はじめに、アカウント、IAM、VPC、EC2、S3）
サービス横断	7～11 章（DB、コンテナ・サーバーレス、運用、セキュリティ、IaC）
専門サービス	12～22 章（DNS、CDN、ストレージ、メッセージ、データ、ML、認証、ネットワーク、セキュリティ、マルチアカウント、移行、IoT/メディア）
実践	23～26 章（パターン集、3 ハンズオン）
運用品質	27～29 章（DR、Well-Architected、認定資格）
深掘り	30～35 章（Lambda、DynamoDB、ECS/EKS、Aurora、コスト、可観測性）
総括	36 章（本章）

36.11 読者へのメッセージ

AWS は 巨大で常に動く システム。本書を読み終えた時点で、すべての細部を覚えている必要はない。重要なのは：

1. 全体像を把握していること（このサービスはどこに位置するか分かる）
2. 正しいドキュメントに辿り着ける こと（公式 docs を読みこなせる）
3. 実機で試せる こと（コンソールと CLI を動かせる）
4. 問題を切り分けられる こと（CloudWatch、CloudTrail、X-Ray を使える）
5. コストとセキュリティを意識 できること

本書はゴールではなく出発点である。手を動かし、コミュニティに参加し、実プロジェクトで試行錯誤することで、AWS の本当の理解が始まる。

最後に、AWS は 道具 に過ぎない。技術的優劣ではなく、**解きたい問題、届けたい価値** に照らして選び、使いこなす姿勢が一番大事である。本書がその一助となることを願って締めくくる。

37 ハンズオン 4：生成 AI チャットアプリ

24～26 章のハンズオンに続き、本章は **Bedrock + Knowledge Bases + API Gateway + Lambda + CloudFront** で社内ドキュメント RAG チャットボットを作る。生成 AI を本番運用品質で組み込む典型構成。

37.1 ゴール

- S3 に置いたマークダウン / PDF / DOCX を Knowledge Base が取り込み
- **Bedrock Claude** で質問に対して回答生成（引用付き）
- **Cognito** で社内ユーザー認証
- **CloudFront** + **S3** で React フロントエンド
- 全構成を **CDK**（TypeScript）で IaC 化
- **Guardrails** で安全性フィルタ

37.2 構成図



37.3 前提

- AWS アカウント、CDK 環境（24 章参照）
- Bedrock の **モデルアクセス申請** 完了（コンソール → Bedrock → Model access）
- 想定リージョン：**us-east-1** または **ap-northeast-1**（モデル提供状況を確認）
- 想定所要時間：3～5 時間

37.4 ステップ 1：プロジェクト初期化

```

mkdir -p ~/aws-handson/genai-chat && cd $_
npm init -y
npm install aws-cdk-lib constructs
  
```

37.5 ステップ 2 : Knowledge Base 用 S3 と OpenSearch Serverless

```
import { Stack, StackProps, RemovalPolicy } from 'aws-cdk-lib';
import { Construct } from 'constructs';
import * as s3 from 'aws-cdk-lib/aws-s3';
import * as iam from 'aws-cdk-lib/aws-iam';
import * as opensearchserverless from 'aws-cdk-lib/aws-opensearchserverless';
import * as bedrock from 'aws-cdk-lib/aws-bedrock';

export class KbStack extends Stack {
  constructor(scope: Construct, id: string, props?: StackProps) {
    super(scope, id, props);

    const docsBucket = new s3.Bucket(this, 'DocsBucket', {
      blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL,
      removalPolicy: RemovalPolicy.DESTROY,
      autoDeleteObjects: true,
      enforceSSL: true,
    });

    const collection = new opensearchserverless.CfnCollection(this, 'KbCollection', {
      name: 'kb-collection',
      type: 'VECTORSEARCH',
    });

    new opensearchserverless.CfnSecurityPolicy(this, 'EncPolicy', {
      name: 'kb-enc',
      type: 'encryption',
      policy: JSON.stringify({
        Rules: [{ ResourceType: 'collection', Resource: ['collection/kb-collection'] }],
        AWSOwnedKey: true,
      }),
    });

    const kbRole = new iam.Role(this, 'KbRole', {
      assumedBy: new iam.ServicePrincipal('bedrock.amazonaws.com'),
    });
    docsBucket.grantRead(kbRole);
    kbRole.addToPolicy(new iam.PolicyStatement({
      actions: ['aoss:APIAccessAll'],
      resources: ['*'],
    }));
    kbRole.addToPolicy(new iam.PolicyStatement({
      actions: ['bedrock:InvokeModel'],
      resources: [
        `arn:aws:bedrock:${this.region}::foundation-model/amazon.titan-embed-text-v2:0`,
      ],
    }));
  }
}
```



```

const kb = new bedrock.CfnKnowledgeBase(this, 'Kb', {
  name: 'company-kb',
  roleArn: kbRole.roleArn,
  knowledgeBaseConfiguration: {
    type: 'VECTOR',
    vectorKnowledgeBaseConfiguration: {
      embeddingModelArn: `arn:aws:bedrock:${this.region}::foundation-model/
amazon.titan-embed-text-v2:0`,
    },
  },
  storageConfiguration: {
    type: 'OPENSEARCH_SERVERLESS',
    opensearchServerlessConfiguration: {
      collectionArn: collection.attrArn,
      vectorIndexName: 'kb-index',
      fieldMapping: { vectorField: 'vector', textField: 'text', metadataField:
'metadata' },
    },
  },
});

new bedrock.CfnDataSource(this, 'DataSource', {
  knowledgeBaseId: kb.attrKnowledgeBaseId,
  name: 'docs',
  dataSourceConfiguration: {
    type: 'S3',
    s3Configuration: { bucketArn: docsBucket.bucketArn },
  },
  vectorIngestionConfiguration: {
    chunkingConfiguration: {
      chunkingStrategy: 'FIXED_SIZE',
      fixedSizeChunkingConfiguration: { maxTokens: 500, overlapPercentage: 20 },
    },
  },
});
}
}

```

実際は OpenSearch Serverless の Index 作成が手動 / カスタムリソース実装が必要だが、本章ではコンセプトに集中。

37.6 ステップ 3 : API Lambda

```

import { BedrockAgentRuntimeClient, RetrieveAndGenerateCommand } from '@aws-sdk/client-bedrock-agent-runtime';

const client = new BedrockAgentRuntimeClient({});

```

```

const KB_ID = process.env.KB_ID!;
const MODEL_ARN = process.env.MODEL_ARN!;

export const handler = async (event: any) => {
  const body = JSON.parse(event.body ?? '{}');
  const question = body.question;
  if (!question) return { statusCode: 400, body: JSON.stringify({ error: 'question
required' }) };

  const cmd = new RetrieveAndGenerateCommand({
    input: { text: question },
    retrieveAndGenerateConfiguration: {
      type: 'KNOWLEDGE_BASE',
      knowledgeBaseConfiguration: { knowledgeBaseId: KB_ID, modelArn: MODEL_ARN },
    },
  });

  try {
    const resp = await client.send(cmd);
    return {
      statusCode: 200,
      headers: { 'content-type': 'application/json' },
      body: JSON.stringify({
        answer: resp.output?.text,
        citations: resp.citations?.map(c => ({
          retrieved: c.retrievedReferences?.map(r => ({
            content: r.content?.text?.substring(0, 200),
            source: r.location?.s3Location?.uri,
          })),
        })),
      })),
    };
  } catch (e: any) {
    return { statusCode: 500, body: JSON.stringify({ error: e.message }) };
  }
};

```

37.7 ステップ 4 : API Gateway + Cognito

```

import { HttpApi, HttpMethod } from 'aws-cdk-lib/aws-apigatewayv2';
import { HttpUserPoolAuthorizer } from 'aws-cdk-lib/aws-apigatewayv2-authorizers';
import { HttpLambdaIntegration } from 'aws-cdk-lib/aws-apigatewayv2-integrations';
import { UserPool, UserPoolClient } from 'aws-cdk-lib/aws-cognito';

const userPool = new UserPool(this, 'Users', {
  selfSignUpEnabled: true,
  signInAliases: { email: true },
});

```

```
const upClient = new UserPoolClient(this, 'WebClient', { userPool });
const authorizer = new HttpUserPoolAuthorizer('Authz', userPool, { userPoolClients:
[upClient] });

const httpApi = new HttpApi(this, 'Api', { defaultAuthorizer: authorizer });
httpApi.addRoutes({
  path: '/api/chat',
  methods: [HttpMethod.POST],
  integration: new HttpLambdaIntegration('ChatInt', chatFn),
});
```

37.8 ステップ 5：フロントエンド（簡易版）

```
<!doctype html>
<html lang="ja">
<head><meta charset="utf-8"><title>社内ナレッジ Bot</title>
<style>
body { font-family: sans-serif; max-width: 800px; margin: 1em auto; padding: 0 1em; }
.chat { height: 60vh; overflow-y: scroll; border: 1px solid #ccc; padding: 1em; }
.msg { padding: 0.5em; border-radius: 0.5em; margin: 0.5em 0; }
.user { background: #cef; }
.bot { background: #efe; }
.cite { font-size: 0.9em; color: #666; }
input { width: 80%; padding: 0.5em; }
button { padding: 0.5em; }
</style></head>
<body>
<h1>社内ナレッジ Bot</h1>
<div id="chat" class="chat"></div>
<form id="form">
  <input id="q" placeholder="質問を入力..."><button>送信</button>
</form>
<script>
const TOKEN = localStorage.getItem('id_token');
const API = '/api/chat';
const chat = document.getElementById('chat');
document.getElementById('form').onsubmit = async (e) => {
  e.preventDefault();
  const q = document.getElementById('q').value;
  if (!q) return;
  appendMsg('user', q);
  document.getElementById('q').value = '';
  const r = await fetch(API, {
    method: 'POST',
    headers: { 'content-type': 'application/json', 'Authorization': `Bearer ${TOKEN}`
  },
  body: JSON.stringify({ question: q }),
});
```

```

    const data = await r.json();
    appendMsg('bot', data.answer, data.citations);
  };
  function appendMsg(role, text, citations) {
    const el = document.createElement('div');
    el.className = `msg ${role}`;
    el.innerHTML = `<div>${text}</div>`;
    if (citations) el.innerHTML += '<div class="cite">引用: ' + citations.flatMap(c =>
c.retrieved).map(r => r.source).join(', ') + '</div>';
    chat.appendChild(el);
    chat.scrollTop = chat.scrollHeight;
  }
</script>
</body>
</html>

```

37.9 ステップ 6 : Guardrails 適用

```

import { CfnGuardrail } from 'aws-cdk-lib/aws-bedrock';

const guardrail = new CfnGuardrail(this, 'Guardrail', {
  name: 'company-guardrail',
  blockedInputMessaging: 'この質問にはお答えできません。',
  blockedOutputsMessaging: 'この回答は提供できません。',
  contentPolicyConfig: {
    filtersConfig: [
      { type: 'SEXUAL', inputStrength: 'HIGH', outputStrength: 'HIGH' },
      { type: 'HATE', inputStrength: 'HIGH', outputStrength: 'HIGH' },
      { type: 'VIOLENCE', inputStrength: 'HIGH', outputStrength: 'HIGH' },
      { type: 'INSULTS', inputStrength: 'HIGH', outputStrength: 'HIGH' },
      { type: 'MISCONDUCT', inputStrength: 'HIGH', outputStrength: 'HIGH' },
      { type: 'PROMPT_ATTACK', inputStrength: 'HIGH', outputStrength: 'NONE' },
    ],
  },
  sensitiveInformationPolicyConfig: {
    piiEntitiesConfig: [
      { type: 'EMAIL', action: 'ANONYMIZE' },
      { type: 'PHONE', action: 'ANONYMIZE' },
      { type: 'CREDIT_DEBIT_CARD_NUMBER', action: 'BLOCK' },
    ],
  },
});

```

Lambda 側で `guardrailIdentifier` と `guardrailVersion` を設定して invoke する。

37.10 ステップ 7：取り込みジョブの起動

S3 にドキュメントをアップロードした後、Knowledge Base に取り込みを指示する必要がある。
EventBridge → Lambda で自動化：

```
import boto3, os
br = boto3.client('bedrock-agent')

def handler(event, context):
    br.start_ingestion_job(
        knowledgeBaseId=os.environ['KB_ID'],
        dataSourceId=os.environ['DS_ID'],
    )
```

37.11 ステップ 8：監視と Guardrails の効果検証

CloudWatch Metrics で：

- `Invocations` (API GW)
- `BedrockTokens` (カスタム EMF メトリクス)
- `GuardrailIntervened` (Guardrails が止めた回数)

X-Ray で：

- Lambda → Bedrock → KB → OpenSearch のレイテンシ内訳

37.12 ステップ 9：コスト確認

Cost Explorer でサービス別：

- **Bedrock**：Claude Sonnet で例えば 1 質問あたり \$0.003～\$0.01
- **OpenSearch Serverless**：時間課金、最低でも月 \$300 程度（OCU 課金）
- **Lambda / API Gateway**：数百円程度
- **S3 / CloudFront**：Free Tier 内で収まることが多い

OpenSearch Serverless が **最大コスト要因**。学習目的なら **Aurora pgvector** で代替する手もある（24 時間運用で月 \$60～70 程度）。

37.13 ステップ 10：後片付け

```
npx cdk destroy
```

OpenSearch Serverless の削除は数分かかる。Bedrock Knowledge Base は依存削除順に注意。

37.14 学んだこと

- `Bedrock RetrieveAndGenerate` で 1 コール RAG

- Knowledge Base + OpenSearch Serverless のセットアップ
- Cognito + JWT Authorizer で API 認証
- Guardrails で安全性フィルタ + PII マスキング
- EventBridge で取り込み自動化

37.15 発展課題

- Agents を使ってツール呼び出し（社内 API、計算）を統合
- Streaming Response：Lambda Function URL で SSE
- マルチターン会話：Conversation 履歴を DynamoDB で保持
- Hybrid Search：ベクトル + キーワード
- Citation UI：引用ソースのプレビュー、リンク
- A/B テスト：複数モデルで同質問を投げ評価
- Bedrock Evaluations で品質計測

37.16 トラブルシューティング

症状	対処
Model access denied	コンソールで Model access 申請、リージョン確認
Knowledge Base 取り込み失敗	S3 権限、ドキュメント形式（PDF/DOCX/MD/TXT/HTML）
OpenSearch Index 未作成	カスタムリソースで作成、または手動
Lambda Timeout	Bedrock 応答に最大数十秒。Lambda Timeout 60 秒以上
Guardrails で blocked	CloudWatch でブロック理由、しきい値調整
回答が引用元と矛盾	プロンプト調整、温度低下、別モデル試行
コスト想定外	OpenSearch Serverless OCU、Bedrock Provisioned 確認
トークン数オーバー	チャンクサイズ縮小、コンテキスト削減

38 CloudFormation と CDK 詳細

11 章で IaC ツールの俯瞰を扱った。本章では AWS 純正の CloudFormation と CDK を本格運用視点で深掘りする。スタック構造、Cross-stack 参照、Aspect、Custom Resource、StackSets、テスト、Drift 検出など。

38.1 CloudFormation の基本構造

```
AWSTemplateFormatVersion: '2010-09-09'
Description: My stack
Transform: AWS::Serverless-2016-10-31    # SAM の場合のみ

Parameters:
  Env:
    Type: String
    AllowedValues: [dev, staging, prod]
    Default: dev

Mappings:
  RegionMap:
    ap-northeast-1: { AmazonLinux: ami-0abc... }
    us-east-1:      { AmazonLinux: ami-0def... }

Conditions:
  IsProd: !Equals [!Ref Env, prod]

Resources:
  MyBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: !Sub "my-bucket-${AWS::AccountId}-${Env}"

Outputs:
  BucketArn:
    Value: !GetAtt MyBucket.Arn
    Export: { Name: !Sub "${AWS::StackName}-BucketArn" }
```

38.1.1 セクションの役割

- **Parameters**：実行時に与える変数
- **Mappings**：環境別の固定値テーブル
- **Conditions**：リソース作成の条件
- **Resources**：作成するリソース（必須）
- **Outputs**：他スタックや CLI から参照する値

38.2 Intrinsic Functions（組み込み関数）

関数	用途
<code>!Ref</code>	パラメータ・リソースの主要属性参照
<code>!GetAtt</code>	リソースの個別属性取得
<code>!Sub</code>	文字列補間
<code>!Join</code>	文字列結合
<code>!Split</code>	文字列分割
<code>!Select</code>	リストから要素取得
<code>!FindInMap</code>	Mappings から取得
<code>!If</code> / <code>!Equals</code> / <code>!And</code> / <code>!Or</code> / <code>!Not</code>	Conditions ロジック
<code>!ImportValue</code>	他スタックの Output を参照
<code>!Cidr</code>	サブネット CIDR 計算
<code>!Base64</code>	Base64 エンコード（UserData 等）

38.3 スタック設計戦略

38.3.1 モノリス vs ネスト vs Cross-stack

戦略	特徴
モノリス	1 スタックに全部。シンプルだが大きくなると重い・リソース上限
ネストスタック	親 → 子の階層。子は再利用可能なテンプレート
Cross-stack	<code>Export</code> / <code>ImportValue</code> で疎結合に
Stack Set	複数アカウント・複数リージョンへ展開

38.3.2 レイヤ分割の典型

[1. Network スタック]	VPC、Subnet、TGW、Direct Connect
[2. Security スタック]	IAM、KMS、SecurityHub、GuardDuty
[3. Storage スタック]	S3、RDS、DynamoDB
[4. Compute スタック]	ECS、Lambda、EC2
[5. App スタック]	各アプリケーション固有リソース

下から上に依存。下層変更時の影響範囲を絞れる。

38.4 Cross-stack 参照

```
# Network stack の Output
Outputs:
```



```
VpcId:
  Value: !Ref Vpc
  Export: { Name: !Sub "${AWS::StackName}-VpcId" }
```

```
# Compute stack でインポート
SubnetIds: !Split
  - ","
  - !ImportValue NetworkStack-PrivateSubnetIds
```

ただし Cross-stack 参照は **硬い結合**。Export を消そうとすると Import 側スタックでエラー。

38.4.1 SSM Parameter Store 経由

より柔軟な代替手段：

```
# Network stack で Parameter Store に書き出し
VpcIdParam:
  Type: AWS::SSM::Parameter
  Properties:
    Name: /network/vpc-id
    Type: String
    Value: !Ref Vpc

# 他スタックで動的参照
VpcId: '{{resolve:ssm:/network/vpc-id:1}}'
```

38.5 Custom Resource

CloudFormation でサポートされていないリソースや、外部 API を呼ぶ必要があるときに **Lambda を呼んでカスタム処理** する仕組み。

```
MyCustomResource:
  Type: Custom::MyLogic
  Properties:
    ServiceToken: !GetAtt MyCustomResourceFunction.Arn
    SomeParam: value
```

Lambda が Create / Update / Delete イベントを受け取り、CloudFormation に Success / Failed を返す。

CDK では `aws-cdk-lib/custom-resources` の `AwsCustomResource` で **AWS SDK 呼び出し** を直接書ける。

38.6 Drift 検出

実リソースとテンプレートの差分を検出。コンソールから手動実行 or API。

```
aws cloudformation detect-stack-drift --stack-name my-stack
aws cloudformation describe-stack-resource-drifts --stack-name my-stack
```

手動変更（コンソール操作）を検出して IaC に戻す運用。

38.7 StackSets

複数アカウント・複数リージョンに 1 テンプレートを展開。

38.7.1 2 つの権限モデル

- **Self-Managed**：管理者が IAM を各アカウントに準備
- **Service-Managed**：Organizations 連携で自動

```
aws cloudformation create-stack-set \
  --stack-set-name baseline \
  --template-body file://baseline.yaml \
  --permission-model SERVICE_MANAGED \
  --auto-deployment Enabled=true,RetainStacksOnAccountRemoval=false

aws cloudformation create-stack-instances \
  --stack-set-name baseline \
  --deployment-targets OrganizationalUnitIds=ou-abc-1234 \
  --regions ap-northeast-1 us-east-1
```

新規アカウント作成時に **自動デプロイ** される設定が便利。

38.8 AWS CDK 詳細

CDK は CloudFormation のラッパーだが、**プログラミング言語** で書ける利点大きい。

38.8.1 Construct の 3 層

層	説明
L1 (Cfn*)	CloudFormation 直マッピング (CfnBucket)
L2	パターン化された API (Bucket)
L3 (Pattern)	複数リソースを束ねた高水準 (ApplicationLoadBalancedFargateService)

新規開発は L2 中心、新サービスや細かい設定は L1 で補う。

38.8.2 App / Stack / Construct の階層

```
App
└─ Stack
```

```

└ Construct
  └ Resource (L1)

```

`Stack` 単位で CloudFormation スタックが生成される。`App` 全体で `cdk synth` → `cdk deploy`。

38.8.3 Cross-stack 参照 (CDK)

```

// 同 App 内なら自動的に Export/ImportValue が生成される
const networkStack = new NetworkStack(app, 'Network');
const computeStack = new ComputeStack(app, 'Compute', { vpc: networkStack.vpc });

```

CDK 側で依存関係を解決し、CloudFormation 側で Export/ImportValue として書き出す。

38.8.4 Tag の一括付与

```

import { Tags } from 'aws-cdk-lib';
Tags.of(app).add('Project', 'myapp');
Tags.of(app).add('Environment', env);

```

App ツリー全体に伝播。

38.8.5 Aspects

CDK ツリーを traverse して **横断的なルール** を適用する仕組み。

```

import { IAspect, Aspects, IConstruct } from 'aws-cdk-lib';
import { CfnBucket } from 'aws-cdk-lib/aws-s3';

class EnforceEncryption implements IAspect {
  visit(node: IConstruct) {
    if (node instanceof CfnBucket) {
      node.bucketEncryption = {
        serverSideEncryptionConfiguration: [{
          serverSideEncryptionByDefault: { sseAlgorithm: 'AES256' },
        }],
      };
    }
  }
}

Aspects.of(app).add(new EnforceEncryption());

```

組織共通のセキュリティルール強制、タグ強制、リソース命名規則チェックなどに。

38.8.6 cdk-nag

CDK ベストプラクティスを **自動チェック**。AWS Solutions / NIST 800-53 / HIPAA / PCI DSS 等のルールセット。

```
import { AwsSolutionsChecks } from 'cdk-nag';
Aspects.of(app).add(new AwsSolutionsChecks({ verbose: true }));
```

`cdk synth` 時に違反を出力。

38.8.7 CDK Pipelines

CDK 自身を **self-mutating** な CI/CD パイプライン で運用。

```
import * as pipelines from 'aws-cdk-lib/pipelines';

const pipeline = new pipelines.CodePipeline(this, 'Pipeline', {
  synth: new pipelines.ShellStep('Synth', {
    input: pipelines.CodePipelineSource.gitHub('owner/repo', 'main'),
    commands: ['npm ci', 'npm run build', 'npx cdk synth'],
  }),
});

pipeline.addStage(new MyAppStage(this, 'Dev', { env: { account: '111', region: 'ap-northeast-1' } }));
pipeline.addStage(new MyAppStage(this, 'Prod', { env: { account: '222', region: 'ap-northeast-1' } }), {
  pre: [new pipelines.ManualApprovalStep('PromoteToProd')],
});
```

複数アカウント・複数リージョン・段階デプロイを宣言的に。

38.9 テスト

38.9.1 1. snapshot テスト

```
import { Template } from 'aws-cdk-lib/assertions';

test('matches snapshot', () => {
  const stack = new MyStack(new App(), 'Test');
  const template = Template.fromStack(stack);
  expect(template.toJSON()).toMatchSnapshot();
});
```

予期せぬ差分を検知。

38.9.2 2. ファインマッチング

```
import { Match } from 'aws-cdk-lib/assertions';

test('S3 bucket has versioning', () => {
  const stack = new MyStack(new App(), 'Test');
```

```
const template = Template.fromStack(stack);
template.hasResourceProperties('AWS::S3::Bucket', {
  VersioningConfiguration: Match.objectLike({ Status: 'Enabled' }),
});
});
```

38.9.3 3. 統合テスト

`@aws-cdk/integ-tests-alpha` で実 AWS にデプロイしてアサート。`integ-runner` で実行。

38.10 デプロイ戦略

38.10.1 Hotswap

```
cdk deploy --hotswap
```

CloudFormation を経由せずに Lambda コード・Step Functions 定義などを直接更新。開発ループを高速化（本番では使わない）。

38.10.2 Watch

```
cdk watch
```

ファイル変更を検知して自動 deploy（Hotswap 込み）。

38.10.3 Bootstrap モダン化

```
cdk bootstrap aws://<account>/<region>
```

CDK 用の S3 / IAM Role / KMS Key が作られる。組織レベルでカスタマイズ（CdkBootstrap-Modern）。

38.11 CDK と Terraform の選び方

要件	推奨
AWS だけ・型安全・プログラミング	CDK
マルチクラウド・既存 Terraform 資産	Terraform
サーバーレス特化	SAM or CDK
チームが Java/.NET/Python に習熟	CDK

要件	推奨
HashiCorp エコシステム (Vault、Vagrant 等)	Terraform
大量の同様リソース展開	CDK の loop / Terraform の count 両方可
GitOps、宣言重視	Terraform / OpenTofu

両方を併用する組織も多い：CDK で AWS 基盤、Terraform で SaaS / マルチクラウド連携。

38.12 ベストプラクティス

- Stack を環境変数で分離：dev/staging/prod を別 Stack
- Construct を再利用ライブラリ化：社内共通 Construct を npm package で配布
- cdk-nag / cfn-guard / Checkov で静的検査
- PR ごとに `cdk diff` を CI で表示してレビュー
- デプロイは CI/CD 経由、ローカルからの直接 deploy を本番禁止
- タグを App 単位で必ず付与
- Outputs は最小限、SSM Parameter Store の方が柔軟
- Stack Set / CDK Pipelines でマルチアカウント
- State / Drift の定期チェック

38.13 制限値の頭出し

項目	上限
スタックあたりリソース数	500（緩和申請可）
スタックあたり Outputs	200
スタックあたり Parameters	200
スタックあたり Mappings	200
テンプレートサイズ（直接アップロード）	51,200 バイト
テンプレートサイズ（S3 経由）	1MB

38.14 よくある罠

罠	対処
Cross-stack Export を消せない	Import 側を先に変更してから Export 削除
スタック削除で IAM Role 消えない	<code>AssumeRolePolicy</code> 競合、依存リソース残存
Drift が大量	手動変更を IaC に取り込み、SCP で手動禁止
<code>cdk destroy</code> で残るリソース	<code>removalPolicy: DESTROY</code> 、 <code>autoDeleteObjects</code>

罣	対処
ROLLBACK_FAILED ステート	CloudFormation ヘルプ参照、必要なら手動削除
カスタムリソースの Stuck	Lambda timeout、SUCCESS / FAILED 必ず返す
Replacement で本番停止	UpdateReplacePolicy: Retain、Replacement プロパティ把握
Stack Set デプロイ失敗	対象アカウントの IAM Role、リージョンの opt-in

39 Terraform 詳細

Terraform / OpenTofu は AWS だけでなく、マルチクラウド・SaaS・自社プロビジョニング全般に使える IaC の事実上標準。本章では state 管理、モジュール、ワークフロー、テスト、組織運用までを深掘りする。

39.1 Terraform vs OpenTofu

2023 年 8 月、HashiCorp が Terraform を BSL (Business Source License) に変更したのを契機に、コミュニティが OpenTofu としてフォーク。

項目	Terraform	OpenTofu
ライセンス	BSL (商用制約)	MPL 2.0 (OSS)
機能	HashiCorp 拡張あり	OSS のみ + 独自拡張
互換	v1.5 まで OpenTofu と互換	v1.6 以降は分岐
HCP / Cloud	HashiCorp Cloud Platform	— (独自にどうぞ)
State 暗号化	—	v1.7+ で State 暗号化機能
Provider	HashiCorp 公式 + コミュニティ	Terraform Registry も利用可

新規プロジェクトは要件次第。OpenTofu は OSS 純度・将来性、Terraform は HCP 統合・成熟度。

39.2 基本構造

```
my-project/
├ main.tf
├ variables.tf
├ outputs.tf
├ versions.tf
├ terraform.tfvars
├ modules/
│   ├── vpc/
│   └── alb/
└ environments/
    ├── dev/
    └── prod/
```

```
# versions.tf
terraform {
  required_version = ">= 1.10"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.70" }
  }
  backend "s3" {
```



```

    bucket      = "my-tfstate"
    key         = "myapp/terraform.tfstate"
    region      = "ap-northeast-1"
    use_lockfile = true      # Terraform 1.10+ S3 ネイティブロック
    encrypt     = true
  }
}

provider "aws" {
  region = "ap-northeast-1"
  default_tags {
    tags = {
      Project      = "myapp"
      Environment = var.env
      ManagedBy    = "terraform"
    }
  }
}

```

39.3 State 管理

39.3.1 Local backend vs Remote backend

- **Local** : terraform.tfstate をローカルに置く。個人検証のみ
- **Remote** : S3 / HCP / Consul / Postgres 等。複数人 / CI 必須

39.3.2 S3 backend + ロック

Terraform 1.10 以降、S3 ネイティブロック (`use_lockfile = true`) が GA。それ以前は DynamoDB テーブルを併用していた。

```

backend "s3" {
  bucket      = "tfstate-prod"
  key         = "network/terraform.tfstate"
  region      = "ap-northeast-1"
  use_lockfile = true      # 1.10+ 推奨
  # 旧形式 (1.9以前):
  # dynamodb_table = "tfstate-lock"
  encrypt     = true
  kms_key_id   = "arn:aws:kms:..."
}

```

39.3.3 State の論理分割

サービス・チーム・環境ごとに State を分けるのが鉄則。

- **環境分離** : dev / staging / prod で別 State
- **レイヤ分離** : network / data / compute / app

- ・ **ライフサイクル分離**：頻繁に変わるもの vs 静的なもの

S3 backend keys:

```
network/dev/terraform.tfstate
network/prod/terraform.tfstate
app/api/dev/terraform.tfstate
app/api/prod/terraform.tfstate
```

39.3.4 State 操作（注意して使う）

```
terraform state list
terraform state show aws_s3_bucket.app
terraform state mv aws_s3_bucket.app aws_s3_bucket.legacy
terraform state rm aws_s3_bucket.legacy      # state からのみ削除（実体は残る）
terraform state pull > backup.tfstate
terraform import aws_s3_bucket.app my-bucket  # 既存リソースを state に取り込み
```

39.3.5 Workspace

```
terraform workspace new dev
terraform workspace select prod
```

State を `terraform.tfstate.d/<workspace>/` に分離。シンプルな環境分けに使えるが、複雑な環境差異には不向き。ディレクトリ分離 + 別 State の方が保守しやすい。

39.4 モジュール

`modules/<name>/{main.tf,variables.tf,outputs.tf}` の構造。

39.4.1 呼び出し

```
module "vpc" {
  source = "./modules/vpc"
  cidr   = "10.0.0.0/16"
  azs    = ["ap-northeast-1a", "ap-northeast-1c"]
}
```

39.4.2 Source の種類

- ・ ローカル： `./modules/vpc`
- ・ Git： `git::https://example.com/modules.git//vpc?ref=v1.2.3`
- ・ Terraform Registry： `terraform-aws-modules/vpc/aws`
- ・ S3 / GCS： `s3::https://s3-...`

39.4.3 コミュニティモジュール

terraform-aws-modules/* が定番。

```
module "vpc" {
  source      = "terraform-aws-modules/vpc/aws"
  version    = "~> 5.0"
  name       = "my-vpc"
  cidr       = "10.0.0.0/16"
  azs        = ["ap-northeast-1a", "ap-northeast-1c"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24"]
  enable_nat_gateway = true
}
```

実装の手間を大幅に削減。バージョン pin が重要。

39.4.4 自社モジュールの設計指針

- **Single Responsibility** : 1 モジュール = 1 責務
- **最小限の Variable** : 不要なオプション乱発しない
- **Default 値** で使いやすく
- **Output** で必要な属性を公開
- **バージョンニング** : Git tag で管理
- **README と examples ディレクトリ** : 使い方を必ず添える

39.5 Provider

```
provider "aws" {
  region = "ap-northeast-1"
  alias  = "tokyo"
}

provider "aws" {
  region = "us-east-1"
  alias  = "us-east"    # CloudFront / ACM 用
}

resource "aws_acm_certificate" "cf" {
  provider = aws.us-east
  ...
}
```

複数リージョン・複数アカウントを 1 State で扱う場合に。

39.5.1 Cross-account (Assume Role)

```

provider "aws" {
  region = "ap-northeast-1"
  assume_role {
    role_arn      = "arn:aws:iam::123456789012:role/TerraformRole"
    session_name = "terraform"
  }
}

```

39.6 ワークフロー

39.6.1 コマンド一覧

```

terraform init      # provider / module / backend 初期化
terraform fmt       # コード整形
terraform validate  # 構文チェック
terraform plan      # 差分プレビュー
terraform apply     # 適用
terraform destroy   # 全削除
terraform output    # Outputs 表示
terraform refresh   # 実リソースから state 更新 (plan に統合済み)
terraform taint     # リソースを再作成対象にマーク (非推奨、state replace に置換)

```

39.6.2 Plan / Apply の本格運用

```

terraform plan -out=plan.tfplan
terraform apply plan.tfplan

```

`-out` で plan 結果を保存し、Apply 時にそれを再利用。CI/CD で **レビュー後の plan** を厳密に適用するパターン。

39.7 CI/CD 統合

39.7.1 GitHub Actions (OIDC + Plan/Apply)

```

name: terraform
on:
  pull_request:
  push: { branches: [main] }
permissions:
  id-token: write
  contents: read
  pull-requests: write
jobs:
  plan:

```

```

runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: hashicorp/setup-terraform@v3
    with: { terraform_version: 1.10.0 }
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::...:role/github-tf
      aws-region: ap-northeast-1
  - run: terraform init
  - run: terraform plan -out=plan.tfplan
  - uses: actions/upload-artifact@v4
    with: { name: tfplan, path: plan.tfplan }

apply:
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  needs: plan
  runs-on: ubuntu-latest
  environment: prod
  steps:
    - uses: actions/checkout@v4
    - uses: actions/download-artifact@v4
      with: { name: tfplan }
    - uses: hashicorp/setup-terraform@v3
    - uses: aws-actions/configure-aws-credentials@v4
      with: { role-to-assume: arn:aws:iam::...:role/github-tf, aws-region: ap-
northeast-1 }
    - run: terraform init
    - run: terraform apply plan.tfplan

```

`environment: prod` で GitHub の承認待ちを挟める。

39.7.2 HCP Terraform / Terraform Enterprise

HashiCorp 公式の SaaS / オンプレ管理。State 管理、Plan/Apply 自動化、コスト見積、Sentinel ポリシー、Variables / Workspaces 管理を統合提供。

39.7.3 Atlantis / OpenTofu のコミュニティツール

PR ごとに `atlantis plan` でコメント、承認後に `atlantis apply`。OSS で自前運用可。

39.8 テスト

39.8.1 terraform test (v1.6+)

```

# tests/vpc.tfctest.hcl
run "create_vpc" {
  command = plan

```

```
variables {  
  cidr = "10.0.0.0/16"  
}  
  
assert {  
  condition      = aws_vpc.this.cidr_block == "10.0.0.0/16"  
  error_message = "CIDR mismatch"  
}  
}
```

40 Terratest (Go ベース)

実 AWS にデプロイして E2E テスト。本格的な検証に。

40.1 Drift と State の整合

40.1.1 Drift Detection

```
terraform plan -refresh-only
```

実環境と state の差分を確認（リソース更新は加えない）。手動変更を検知。

40.1.2 State Refresh と Lock

`terraform refresh` は内部的に呼ばれる。State Lock がかかる（DynamoDB / S3 lockfile）ので、複数同時実行は安全。

40.2 セキュリティ

- State に機密情報が含まれる：S3 + KMS 暗号化、IAM で読み取り権限を絞る
- Provider credentials：環境変数 / IAM Role / OIDC、ハードコード禁止
- 秘密の入力：Variables の `sensitive = true`、HCP Variables、Vault との統合
- tfsec / checkov / trivy / OPA / Sentinel：静的解析でセキュリティ違反検知

```
tfsec .  
checkov -d .
```

40.3 コスト

- Infracost：Plan からコスト見積
- HCP Terraform にも統合あり

```
infracost breakdown --path .
```

PR ごとに「この変更で月 +\$120」のような表示が可能。

40.4 高度な機能

40.4.1 Dynamic Block

```
resource "aws_security_group" "app" {  
  name = "app"
```

```

vpc_id = var.vpc_id

dynamic "ingress" {
  for_each = var.allowed_ports
  content {
    from_port = ingress.value.port
    to_port   = ingress.value.port
    protocol  = "tcp"
    cidr_blocks = ingress.value.cidrs
  }
}
}

```

40.4.2 for_each / count

```

resource "aws_subnet" "public" {
  for_each = toset(var.azs)
  vpc_id   = aws_vpc.this.id
  availability_zone = each.value
  cidr_block = cidrsubnet(var.cidr, 8, index(var.azs, each.value))
}

```

`for_each` は `map` / `set`、`count` は `number`。`for_each` の方が安全（リソースが Move しても影響少）。

40.4.3 Locals / Functions

```

locals {
  common_tags = {
    Environment = var.env
    Project     = "myapp"
  }
  azs = data.aws_availability_zones.available.names
}

```

40.4.4 Data Source

```

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["al2023-ami-*-x86_64"]
  }
}

```


実環境を **読み取って** 使う。`aws_caller_identity`、`aws_region`、`aws_iam_policy_document` などが頻出。

40.4.5 Lifecycle

```
resource "aws_db_instance" "this" {
  # ...
  lifecycle {
    prevent_destroy      = true           # 削除を防ぐ
    create_before_destroy = true         # 置き換え時、新を先に作る
    ignore_changes       = [password]    # 指定属性の変更を無視
  }
}
```

40.5 組織運用パターン

40.5.1 Mono-repo vs Multi-repo

- **Mono-repo**：すべての IaC を 1 リポジトリ。検索性高、変更影響を一元把握
- **Multi-repo**：チーム / サービス単位で分離。権限・CI/CD を独立化

40.5.2 ディレクトリ構造の典型

```
infra/
├ modules/           共通モジュール
├ live/
│   ├── network/
│   │   ├── dev/
│   │   └── prod/
│   ├── data/
│   └── apps/
│       ├── web/
│       │   ├── dev/
│       │   └── prod/
│       └── batch/
└ tools/             スクリプト・helper
```

`live/<layer>/<env>/` の各ディレクトリが **独立した State**。

40.5.3 Terragrunt

Terraform のラッパー。DRY 化（重複削減）と環境別設定の管理を強化。

```
# terragrunt.hcl
remote_state {
  backend = "s3"
  config = {
```

```

    bucket = "tfstate-${get_aws_account_id()}"
    key     = "${path_relative_to_include()}/terraform.tfstate"
    region = "ap-northeast-1"
  }
}

```

`run-all` で全環境一括実行も可。組織が大きくなるなら検討。

40.6 AWS Provider のヒント

- ・リソース名は API 名を意識（`aws_instance` → `RunInstances`）
- ・バージョン pin は `~> 5.70` のように厳密に
- ・Default Tags（provider レベル）でタグ漏れ防止
- ・Aliases でマルチリージョン
- ・新サービスはリリース直後 Provider が遅れることがある

40.7 よくあるトラブル

症状	対処
State Lock 失敗	DynamoDB / S3 lockfile 確認、 <code>terraform force-unlock</code> （慎重に）
Plan が大量変更を出す	Provider バージョン違い、refresh 結果、手動変更
Apply が AccessDenied	IAM ロール、Assume Role、KMS 鍵権限
Cycle error	リソース間の循環依存、間に <code>depends_on</code> を整理
既存リソースを取り込みたい	<code>terraform import</code> 、新形式の <code>import {}</code> ブロック (v1.5+)
State 破損	S3 versioning でロールバック、定期バックアップ習慣
for_each で order 変わる	集合型なので順序保証なし、index/key を安定化
モジュール更新で破壊的変更	バージョン pin、changelog 確認、staging で検証

41 CI/CD と DevOps

11 章で IaC、38～39 章で CloudFormation/CDK/Terraform を扱った。本章では **継続的インテグレーション・継続的デリバリ** の AWS 上での実装を、純正 CodeCatalyst / CodePipeline と GitHub Actions の比較を含めて深掘りする。

41.1 CI/CD の全体像

```
[Git push] → CI (ビルド・テスト・スキャン) → Artifact
      ↓
      CD (dev → staging → prod デプロイ)
      ↓
      Observability (メトリクス・トレース)
      ↓
      Feedback Loop (issue / metric / SLO)
```

41.2 AWS の CI/CD サービス群

サービス	用途
CodeCommit	Git ホスティング (新規受付終了の方向)
CodeBuild	マネージドビルド
CodeDeploy	EC2 / Lambda / ECS のデプロイメント
CodePipeline	パイプラインオーケストレーション
CodeArtifact	npm / pip / maven などのアーティファクトリ
CodeGuru	コードレビュー・プロファイラ (Q Developer に統合進行)
CodeCatalyst	統合 DevOps プラットフォーム (CodeCommit 後継的位置)
Amazon Q Developer	生成 AI コーディング支援

実務では **GitHub + GitHub Actions** が圧倒的シェア。AWS 純正は **Code** シリーズと CodeCatalyst。

41.3 GitHub Actions + AWS (推奨)

41.3.1 OIDC でアクセスキー不要

3 章で触れた通り、長期キーを置かずに **Assume Role** で AWS 操作。

```
permissions:
  id-token: write
  contents: read

jobs:
```

```

deploy:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: arn:aws:iam::123456789012:role/gha-deploy
        aws-region: ap-northeast-1
    - run: aws sts get-caller-identity

```

IAM ロールの信頼ポリシー：

```

{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::123456789012:oidc-provider/
token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
        "token.actions.githubusercontent.com:sub": "repo:owner/repo:ref:refs/heads/
main"
      }
    }
  }]
}

```

`sub` は **StringEquals + フルパス指定** が安全。ワイルドカードは fork PR で乗っ取りリスク。

41.3.2 典型的な CI Workflow

```

name: ci
on:
  pull_request:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }

```

```

- run: npm ci
- run: npm run lint
- run: npm test -- --coverage
- uses: codecov/codecov-action@v4

build:
  needs: test
  runs-on: ubuntu-latest
  permissions: { id-token: write, contents: read }
  steps:
    - uses: actions/checkout@v4
    - uses: aws-actions/configure-aws-credentials@v4
      with: { role-to-assume: arn:aws:iam::...:role/gha-ci, aws-region: ap-
northeast-1 }
    - uses: aws-actions/amazon-ecr-login@v2
    - run: |
        docker build -t myapp:${{ github.sha }} .
        docker tag myapp:${{ github.sha }} 123456789012.dkr.ecr.ap-
northeast-1.amazonaws.com/myapp:${{ github.sha }}
        docker push 123456789012.dkr.ecr.ap-northeast-1.amazonaws.com/myapp:
${{ github.sha }}

```

41.3.3 段階的 CD (Environment 機能)

```

deploy-staging:
  needs: build
  environment: staging          # GitHub Environment
  runs-on: ubuntu-latest
  steps:
    - run: aws ecs update-service --cluster c --service s --force-new-deployment

deploy-prod:
  needs: deploy-staging
  environment: prod            # 承認待ち (Environment Protection Rules)
  runs-on: ubuntu-latest
  steps:
    - run: aws ecs update-service --cluster c --service s --force-new-deployment

```

`environment: prod` で **承認者リスト** を設定すれば、手動承認後にだけ実行される。

41.4 AWS CodePipeline

純正のパイプラインオーケストレーション。Source → Build → Deploy のステージ構成。

```

# CDK での定義例
import * as codepipeline from 'aws-cdk-lib/aws-codepipeline';
import * as actions from 'aws-cdk-lib/aws-codepipeline-actions';

```

```
import * as codebuild from 'aws-cdk-lib/aws-codebuild';

const pipeline = new codepipeline.Pipeline(this, 'Pipeline', {
  pipelineName: 'myapp',
  pipelineType: codepipeline.PipelineType.V2,    // V2 推奨
});

pipeline.addStage({
  stageName: 'Source',
  actions: [new actions.GitHubSourceAction({
    actionName: 'GitHub',
    owner: 'owner', repo: 'repo', branch: 'main',
    oauthToken: SecretValue.secretsManager('github-token'),
    output: new codepipeline.Artifact('SrcArtifact'),
  })],
});

pipeline.addStage({
  stageName: 'Build',
  actions: [new actions.CodeBuildAction({
    actionName: 'Build',
    project: new codebuild.PipelineProject(this, 'Build', {
      buildSpec: codebuild.BuildSpec.fromSourceFilename('buildspec.yml'),
    }),
    input: srcArtifact,
    outputs: [buildArtifact],
  })],
});

pipeline.addStage({
  stageName: 'DeployProd',
  actions: [new actions.ManualApprovalAction({ actionName: 'Approve' })],
});
```

41.4.1 CodePipeline V2 の利点

- **Trigger Filter** : PR イベント、ファイルパスフィルタ
- **Variables** : パイプライン全体で変数共有
- **Stage Conditions** : 失敗・成功時の条件アクション

41.5 AWS CodeBuild

マネージドビルド環境。 `buildspec.yml` で定義。

```
version: 0.2
phases:
  install:
    runtime-versions:
```

```

    nodejs: 20
  pre_build:
    commands:
      - npm ci
  build:
    commands:
      - npm test
      - npm run build
  artifacts:
    files: ['dist/**/*', 'package.json']
  cache:
    paths: ['node_modules/**/*']

```

41.5.1 ビルド環境

- AWS マネージドイメージ (Amazon Linux、Ubuntu、Windows)
- カスタムイメージ (ECR)
- Compute タイプ: BUILD_GENERAL1_SMALL〜LARGE、ARM、GPU、Lambda
- VPC 内ビルド (プライベートリソースアクセス)

41.5.2 バッチビルド

- **Build matrix**: 複数バージョン並列
- **Build graph**: 依存関係付き並列

41.6 AWS CodeDeploy

EC2 / Lambda / ECS のデプロイメント。

41.6.1 EC2 デプロイ

- **In-place**: 既存インスタンスを順次更新
- **Blue/Green**: 新環境を作って切替 (ELB 連携)

appspec.yml :

```

version: 0.0
os: linux
files:
  - source: /
    destination: /opt/myapp
hooks:
  BeforeInstall:
    - location: scripts/before_install.sh
      timeout: 300
  AfterInstall:
    - location: scripts/after_install.sh
  ApplicationStart:

```

```
- location: scripts/start.sh
ValidateService:
- location: scripts/health_check.sh
  timeout: 60
```

41.6.2 ECS デプロイ

- **Rolling** (標準)
- **Blue/Green** (CodeDeploy 連携) : Listener 切替で即時切替、ロールバック容易

```
# appspec.yaml (ECS Blue/Green)
version: 0.0
Resources:
- TargetService:
  Type: AWS::ECS::Service
  Properties:
    TaskDefinition: <TASK_DEFINITION>
    LoadBalancerInfo:
      ContainerName: app
      ContainerPort: 8080
```

41.6.3 Lambda デプロイ

- **Canary** : 10% を 5 分、その後 100%
- **Linear** : 10% ずつ 10 分で
- **AllAtOnce** : 即時

CloudWatch アラームと連動して、エラー率が高ければ自動ロールバック。

41.7 Amazon CodeCatalyst

統合 DevOps プラットフォーム (GitHub Codespaces / GitLab 的)。

- **Project** : Source repo + Build / CD + Issue 管理
- **Workflows** : ビルド・デプロイ定義 (YAML)
- **Dev Environment** : クラウド開発環境 (Cloud9 後継)
- **Blueprint** : プロジェクト雛形
- **Identity** : AWS Builder ID で個人アカウント

CodeCommit が新規受付停止の方向で、CodeCatalyst が後継的位置。ただし GitHub の地位は変わらず、補完的な選択肢。

41.8 Amazon Q Developer

生成 AI 開発支援。

- **IDE プラグイン** (VSCode、JetBrains、Visual Studio)
- **コード生成・補完** : 自然言語コメントからコード提案

- ・コード説明・リファクタリング
- ・単体テスト生成
- ・セキュリティスキャン
- ・Q Code Transformation : Java バージョンアップ、.NET 移行などの自動コード変換
- ・AWS CLI / Console での Q Chat : 「これはなぜ動かない？」を AWS リソース文脈で回答

GitHub Copilot 競合。AWS 統合が強み。

41.9 セキュリティスキャン

CI/CD パイプラインに組み込む静的・動的スキャン。

ツール	対象
Amazon Inspector	ECR / Lambda / EC2 の脆弱性スキャン
Snyk / Dependabot / Renovate	依存パッケージの脆弱性
Trivy / Grype	コンテナイメージスキャン (GHA で)
tfsec / checkov / Trivy IaC	IaC の設定ミス検知
Semgrep / Bandit / ESLint security rules	SAST
GitHub Advanced Security	シークレットスキャン、CodeQL
AWS Secrets Manager + git-secrets	シークレット流出防止
cdk-nag	CDK ベストプラクティス

PR ブロッキングと「警告のみ」を切り分けて運用。

41.10 Artifact 管理

41.10.1 ECR

コンテナイメージ。組織内共有、レプリケーション、脆弱性スキャン統合。

41.10.2 CodeArtifact

npm / pip / maven / NuGet / Cargo パッケージのプライベートリポジトリ。ECR の言語パッケージ版。

41.10.3 S3

汎用アーティファクト (zip、tar、ML モデル、データセット)。バージョニング前提。

41.11 デプロイ戦略

戦略	動作
Rolling	古い→新しいを段階入れ替え
Blue/Green	新環境を立てて切替、即ロールバック可
Canary	一部ユーザーに新版、徐々に拡大
Feature Flag	コードはデプロイ済み、フラグで有効化
A/B Testing	ユーザーセグメントで使い分け、計測
Shadow	本番トラフィックを コピー して新版で実行（破壊しない）

41.11.1 AppConfig (Feature Flag)

AWS のフィーチャーフラグサービス。

```
import boto3
ac = boto3.client('appconfigdata')
session = ac.start_configuration_session(
    ApplicationIdentifier='myapp',
    EnvironmentIdentifier='prod',
    ConfigurationProfileIdentifier='features'
)
config =
ac.get_latest_configuration(ConfigurationToken=session['InitialConfigurationToken'])
features = json.loads(config['Configuration'].read())
if features.get('new_checkout'):
    return new_checkout()
else:
    return old_checkout()
```

デプロイ戦略：Linear / Canary / All at once。CloudWatch アラーム連動で **自動ロールバック**。

41.12 マルチアカウントデプロイ

```
[CI Account] → CodePipeline / GHA
    ↓ Assume Role
[Dev Account] → デプロイ
    ↓
[Staging Account] → デプロイ
    ↓ 承認
[Prod Account] → デプロイ
```

各アカウントに **デプロイ用 IAM ロール**（Permissions Boundary 付き）を用意し、CI から Assume。CDK Pipelines はこの構造をネイティブサポート。

41.13 モニタリング・通知

- CloudWatch Logs / Metrics でビルド・デプロイ履歴
- SNS → Chatbot で Slack / Teams 通知
- デプロイ失敗時の自動ロールバック（CodeDeploy のアラーム連動）
- DORA Metrics : Deployment Frequency / Lead Time / MTTR / Change Failure Rate を計測

41.14 ベストプラクティス

- PR ごとに CI 必須、main 直 push 禁止
- 自動テスト・静的解析・セキュリティスキャンを CI で
- 本番デプロイは承認 + Canary
- CloudWatch アラームと連動した自動ロールバック
- シークレットは Secrets Manager / Parameter Store、リポジトリに置かない
- OIDC でキーレス
- マルチアカウント分離（Dev / Staging / Prod）
- Feature Flag で デプロイとリリースを分離
- DORA Metrics で改善サイクルを回す

41.15 よくある罠

罠	対処
長期 IAM キーを GHA に置く	OIDC で AssumeRole に切替
GHA の sub にワイルドカード	StringEquals でフルパス固定
Production を main 直 push	Environment Protection Rules + 承認者
テストなしのデプロイ	ユニット + 統合 + smoke を最低限
Feature Flag のフラグ放置	定期棚卸し、TTL 付き
モノリスの Big Bang デプロイ	Canary、Blue/Green、Feature Flag 分割
Rollback できない DB マイグレーション	Backward-compatible スキーマ変更を徹底
Pipeline の手動承認だけ	自動 Smoke テスト + アラーム自動ロールバック

41.16 DevOps 文化

CI/CD ツールは 目的ではなく手段。文化面では：

- 小さく頻繁にリリース：1 日複数回デプロイ
- Trunk-based Development：長期分岐を避ける
- Feature Flag で master を常にリリース可能に
- Postmortem 文化：Blameless、共有
- SRE / DevOps エンジニアと開発者の協業
- 自動化を後回しにしない（手作業はバグの温床）

技術導入だけでなく、**組織の意思決定スピード** も同時に変わる必要がある。

42 マルチテナント SaaS 設計

複数の顧客（テナント）に同じシステムを提供する SaaS の設計パターンを、AWS 上での実装と合わせて扱う。テナント分離戦略、課金、オンボーディング、運用、セキュリティを横断する。

42.1 マルチテナンシーの 3 モデル

モデル	概要
Silo	テナントごとに独立リソース（VPC・DB・アカウント単位）
Pool	全テナントで共通リソース、論理分離
Bridge	一部 Pool、一部 Silo（ハイブリッド）

42.1.1 Silo モデル

Tenant A → 独立 AWS アカウント / VPC / DB
 Tenant B → 独立 AWS アカウント / VPC / DB

メリット：

- ・強い分離性、相互影響なし
- ・リソース上限・課金がテナント単位で明確
- ・規制要件への対応が楽

デメリット：

- ・運用コスト・人件費が高い
- ・ベースインフラが各テナントに発生
- ・アップデート展開に時間がかかる

主に **エンタープライズ顧客、高規制業界**（金融・医療）向け。

42.1.2 Pool モデル

1つの ECS / EKS / Lambda + 1 DynamoDB / Aurora
 全テナントが同じインフラ上で動作
 論理的にテナント ID で分離

メリット：

- ・規模の経済で低コスト
- ・アップデート反映が早い
- ・リソース効率が良い

デメリット：

- ・**Noisy Neighbor**（特定テナントが大量消費すると他に影響）
- ・分離強度が低い
- ・「うちは何リソース使っている」が見えにくい

主に 中小企業向け SaaS、FreeTier がある B2C 向け。

42.1.3 Bridge モデル

Compute は Pool (共通 ECS)
DB は Silo (テナントごとに別 RDS)
ストレージは Pool (共通 S3、プレフィクス分離)

実用上ほとんどの SaaS は Bridge。「分離強度が必要な層は Silo、効率重視は Pool」。

42.2 AWS SaaS Builder Toolkit / SaaS Boost

AWS 公式の SaaS リファレンス。CDK ベースで Silo / Pool / Bridge 各モデルの構成例を提供。学習・PoC のスタート地点として有用。

42.3 テナント識別と分離

42.3.1 テナント ID の埋め込み

JWT クレーム → 全リクエストに `tenant_id` を伝播。

```
# Lambda での例
def handler(event, context):
    claims = event['requestContext']['authorizer']['jwt']['claims']
    tenant_id = claims['custom:tenant_id']
    # 以降、すべてのクエリに tenant_id を含める
    item = ddb.get_item(Key={'PK': f'TENANT#{tenant_id}#USER#{user_id}'})
```

42.3.2 ABAC でデータ分離

IAM の `Condition` で `aws:PrincipalTag/TenantId` を使い、テナントの DynamoDB 行・S3 プレフィクスへのアクセスを限定。

```
{
  "Effect": "Allow",
  "Action": ["dynamodb:GetItem", "dynamodb:Query"],
  "Resource": "arn:aws:dynamodb:*:*:table/Tenants",
  "Condition": {
    "ForAllValues:StringEquals": {
      "dynamodb:LeadingKeys": ["${aws:PrincipalTag/TenantId}"]
    }
  }
}
```

```
{
  "Effect": "Allow",
```

```

    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::saas-data/*",
    "Condition": {
        "StringLike": {
            "s3:prefix": ["${aws:PrincipalTag/TenantId}/*"]
        }
    }
}

```

Cognito Pre Token Generation Lambda で `tenant_id` を **セッションタグ** に埋める。

42.3.3 Lambda での Assume Role-by-tenant

リクエストごとに **テナント専用の限定ロール** を Assume :

```

sts = boto3.client('sts')
resp = sts.assume_role(
    RoleArn='arn:aws:iam::...:role/TenantBaseRole',
    RoleSessionName=f'tenant-{tenant_id}',
    Tags=[{'Key': 'TenantId', 'Value': tenant_id}],
    PolicyArns=[],
)
tenant_creds = resp['Credentials']
# 以降のリソース操作はこの一時認証で

```

これで **コードバグでもクロステナント漏洩を防止** できる。

42.4 データストアの分離パターン

42.4.1 DynamoDB

パターン	説明
テナントごとに別テーブル	Silo。テナント数大なら管理大変
PK プレフィクスにテナント ID	Pool。 <code>TENANT#X#USER#Y</code>
テーブル + GSI で混合	Pool かつ多角的アクセスパターン

Pool パターンが圧倒的に運用コスト低い。Single Table Design (31 章) と相性◎。

42.4.2 Aurora / RDS

パターン	説明
Cluster ごと別	完全分離。料金高
Database ごと別 (同一 Cluster)	中庸。 <code>CREATE DATABASE</code> で論理分離
Schema ごと別	PostgreSQL の <code>SET search_path</code>

パターン	説明
Row Level Security	すべて 1 テーブルに、 <code>tenant_id</code> 列で分離

PostgreSQL の Row Level Security (RLS) が最も柔軟：

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON orders
  USING (tenant_id = current_setting('app.current_tenant')::uuid);

-- アプリ接続時
SET app.current_tenant = 'tenant-uuid';
```

42.4.3 S3

```
s3://my-saas-data/
  tenants/
    tenant-A/
      orders/
      uploads/
    tenant-B/
      orders/
      uploads/
```

プレフィクスでの分離 + IAM Condition。バケットポリシーは 1 つで済む。

42.5 認証・ID 管理

42.5.1 Cognito User Pool 戦略

パターン	説明
テナントごと別 User Pool	完全分離。テナント追加でプール作成オーバーヘッド
1つの User Pool でカスタム属性	<code>custom:tenant_id</code> でラベル付け、Pre Token で JWT に
Identity Federation	テナントの IdP (SAML/OIDC) 連携、企業向け

Cognito User Pool は 1 リージョンで最大 1000 プール なので Silo はテナント数次第で破綻。

42.5.2 マルチテナント JWT

```
{
  "sub": "user-uuid",
  "email": "user@example.com",
  "custom:tenant_id": "tenant-uuid",
  "custom:roles": ["admin"],
  "iss": "https://cognito-idp...",
```



```
"aud": "client-id"
}
```

API Gateway JWT Authorizer で検証 → Lambda で `tenant_id` を取り出し、すべての処理に伝播。

42.6 オンボーディング

新規テナント登録時の自動化フロー：

```
[新規申し込み]
→ Lambda (onboarding)
  ↳ Cognito にテナント管理者ユーザー作成
  ↳ DynamoDB に Tenant メタデータ
  ↳ S3 プレフィクス作成
  ↳ Silo の場合: CloudFormation StackSet で専用リソース作成
  ↳ サブドメイン割当 (Route 53)
  ↳ ウェルカムメール送信
```

Step Functions で長時間ワークフローとして組むのが定石。

42.6.1 Tier 別オンボーディング

- **Free / Starter** : Pool に追加するだけ (数秒)
- **Standard** : Pool + 専用 DB schema
- **Enterprise / Premium** : Silo (専用 VPC / DB / 場合によってアカウント)

42.7 課金とメータリング

42.7.1 利用量計測

各テナントの使用量を継続収集：

```
# CloudWatch EMF メトリクスでテナント別の使用量を出力
metrics.add_dimension(name='TenantId', value=tenant_id)
metrics.add_metric(name='APICall', unit='Count', value=1)
metrics.add_metric(name='StorageBytes', unit='Bytes', value=size)
```

CloudWatch → Athena / QuickSight で月次集計。

42.7.2 AWS Marketplace 統合

SaaS を AWS Marketplace で販売する場合：

- **Subscription** : 固定月額
- **Contract Pricing** : 1～3 年契約
- **Pay-as-you-go (Metered)** : 使用量に応じて
- **Free Trial**

利用量は AWS Marketplace Metering Service API で送信。請求は AWS が代行。

42.7.3 Stripe / Recurly 等の外部請求基盤

AWS Marketplace 以外の場合、Stripe / Recurly / Chargebee と連携。

42.8 Tier 別リソース割当

```
Free Tier:    Pool, レート 100 req/min, ストレージ 1 GB
Standard:    Pool, レート 1,000 req/min, ストレージ 100 GB
Enterprise:  Bridge / Silo, 専用 DB, SLA 99.95%
```

42.8.1 レート制限

API Gateway **Usage Plan** + API Key (テナントごと)。または ALB / WAF レート制限。Lambda 同時実行数 **Reserved Concurrency** で Tier 分離。

42.8.2 物理分離 (Tier 別)

- Free / Standard：共通 Lambda + DynamoDB (On-Demand)
- Enterprise：専用 Lambda (Provisioned Concurrency) + 専用 Aurora

42.9 Noisy Neighbor 対策

Pool モデルの最大の課題。

- レート制限：API Gateway / WAF / Lambda Reserved Concurrency
- **DynamoDB 書き込みシャーディング**：高負荷テナントのキー分散
- **Cell-based Architecture**：複数の小規模 Pool に分け、テナントを Cell に割当 (特定 Cell 障害が他 Cell に波及しない)
- **モニタリング**：テナント別の SLA 違反検知 → 別 Cell / Tier 移動

42.9.1 Cell-based Architecture

```
Region: ap-northeast-1
├ Cell-1 (Pool: 1,000 テナント収容)
│   ├── ECS, DynamoDB, ElastiCache
│   └── Tenants: A, B, C, ...
├ Cell-2
│   └── Tenants: D, E, F, ...
└ Cell-N
    └── Tenants: ...
```

ルーティング層 (CloudFront → Lambda@Edge) でテナント → Cell マッピング

Cell 単位で 分離・スケール・障害爆発半径制限。

42.10 観測性

指標	収集方法
テナント別 API リクエスト	EMF メトリクス (TenantId Dimension)
テナント別エラー率	CloudWatch Logs Insights で集計
テナント別レスポンス時間	X-Ray + Application Signals
テナント別ストレージ使用量	S3 Storage Lens、DynamoDB Metrics
テナント別コスト	Cost Allocation Tags + Cost Categories
SLA 違反検知	テナント別 Composite Alarm

42.10.1 カーディナリティに注意

CloudWatch Metrics の **Dimension** にテナント ID を入れると、テナント数 × メトリクス数の組み合わせ爆発で **コスト高騰**。

対策：

- 上位 N テナント（高トラフィック）だけ Dimension に
- Logs Insights / Athena 経由で集計
- Datadog 等の高カーディナリティ対応 SaaS を検討

42.11 セキュリティ

42.11.1 Tenant Isolation のテスト

以下を **自動テスト** に組み込む：

- Tenant A の認証で Tenant B のリソースにアクセス → 失敗を確認
- IAM Policy Simulator で疑似シナリオ
- **Continuous Penetration Testing**

42.11.2 暗号化

- **KMS Multi-tenant key vs Tenant-specific key**
- Tenant 別 key (CMK)：強い分離、コスト・運用増
- 共通 key + Encryption Context にテナント ID：実質テナント分離、運用低

```
kms.encrypt(
    KeyId='alias/saas',
    Plaintext=data,
    EncryptionContext={'TenantId': tenant_id}
)
```

復号時に Encryption Context が一致しないと失敗 → クロステナント漏洩防止。

42.11.3 コンプライアンス

- ・ **データ residency**：テナントのリージョン要件（GDPR、APPI、HIPAA）
- ・ **監査**：CloudTrail にテナント操作を記録、Audit Manager で証跡化
- ・ **PII / PCI**：Macie で機密データ検出

42.11.4 Per-Tenant の権限境界

各テナントの管理者ロールに **Permissions Boundary** を設定し、テナント外への権限拡張を物理的に防止。

42.12 運用

42.12.1 マイグレーション戦略

- ・ **Dual Write**：旧スキーマと新スキーマの両方に書き、徐々に切替
- ・ **Feature Flag** でテナント単位ロールアウト
- ・ **Canary Tenant**：少数の信頼できるテナントで先行検証

42.12.2 バックアップとリストア

Pool モデルでは「特定テナントだけリストア」が難しい。対策：

- ・ **DynamoDB**：テナント別 PITR + ETL で別テーブル復元
- ・ **Aurora**：論理バックアップ（`pg_dump` の WHERE 句）
- ・ **イベントソーシング**：イベントログから再構築

42.12.3 テナント解約・退会

- ・ データを **エクスポート** して提供（GDPR の **Right to Data Portability**）
- ・ 一定期間後に削除（**Right to be Forgotten**）
- ・ 削除実行を CloudTrail / Logs に記録

42.13 参考実装

AWS SaaS Factory Reference Architecture：CDK ベースで Pool / Silo / Bridge の各モデル実装が公開。

ECS Pool モデル / EKS マルチテナント / DynamoDB Single Table の SaaS 例が GitHub で参照可能。

42.14 設計判断のチェックリスト

- ・ [] テナント分離レベル（Silo / Pool / Bridge）を文書化
- ・ [] テナント ID の伝播経路（JWT → アプリ → DB → ログ）を設計
- ・ [] ABAC / Permissions Boundary でクロステナント防止
- ・ [] Tier 別の SLA / リソース割当・課金モデル

- [] Noisy Neighbor 対策（レート制限、Cell）
- [] テナント別観測性（カーディナリティ管理）
- [] オンボーディング・解約の自動化
- [] バックアップ・リストアのテナント単位対応
- [] コンプライアンス（リージョン、データ residency）
- [] テナント分離の自動テスト

42.15 よくある罠

罠	対策
初期から Silo で破綻	Pool 始まり、必要時に Tier 別に Silo に
テナント ID 伝播の漏れ	型レベルで強制（TypeScript の Branded Type 等）
Noisy Neighbor で SLA 違反	Cell、レート制限、Tier 移動
Cognito Pool 上限	カスタム属性で Pool 共用
メトリクスのカーディナリティ爆発	Top-N、Logs Insights、Datadog
テナントデータの誤露出	自動分離テスト、ABAC
リストアが個別不可	イベントソーシング、PITR + ETL
課金モデルの後付け変更	最初から Cost Allocation Tags、メータリング

43 AWS チートシート

本書の最終資料章として、実務で頻出する数字・ID・コマンド・コンソールの場所を **早見表** としてまとめる。本書を辞書的に使うときの索引としても活用できる。

43.1 リージョンコード

リージョン	コード	主な用途
東京	ap-northeast-1	国内サービス標準
大阪	ap-northeast-3	国内 DR
ソウル	ap-northeast-2	日韓ビジネス
シンガポール	ap-southeast-1	東南アジア
シドニー	ap-southeast-2	豪州
ジャカルタ	ap-southeast-3	opt-in 必要
ムンバイ	ap-south-1	インド
バージニア北部	us-east-1	新サービス先行・グローバル
オハイオ	us-east-2	
オレゴン	us-west-2	
N. カリフォルニア	us-west-1	
アイルランド	eu-west-1	欧州標準
フランクフルト	eu-central-1	GDPR 準拠
ロンドン	eu-west-2	
パリ	eu-west-3	
ストックホルム	eu-north-1	
サンパウロ	sa-east-1	南米
カナダ中部	ca-central-1	
中国（北京）	cn-north-1	Sinnet 運営、別アカウント
中国（寧夏）	cn-northwest-1	NWCD 運営、別アカウント
GovCloud (US-East/West)	us-gov-east-1 / us-gov-west-1	米国政府向け、別アカウント

43.2 AWS 主要サービス価格感（東京、目安）

価格は変動するため 2026 年 4 月時点の目安。最新は公式 Pricing で要確認。

43.2.1 EC2

インスタンス	オンデマンド/時	月額（24/7）
t3.nano	\$0.0068	\$5
t3.micro	\$0.0136	\$10
t3.small	\$0.0272	\$20

インスタンス	オンデマンド/時	月額 (24/7)
t3.medium	\$0.0544	\$40
t3.large	\$0.1088	\$80
m7i.large	\$0.124	\$90
m7g.large (Graviton)	\$0.1088	\$80
c7i.xlarge	\$0.245	\$180
r7i.large	\$0.182	\$133

Savings Plans 1 年で約 30%、3 年で約 50% 引き。

43.2.2 ストレージ・ネットワーク

項目	料金 (月、東京)
EBS gp3	\$0.096/GB
EBS gp2	\$0.12/GB
EFS Standard	\$0.36/GB
S3 Standard	\$0.025/GB
S3 Standard-IA	\$0.0138/GB + アクセス料
S3 Glacier Deep Archive	\$0.002/GB
パブリック IPv4	\$0.005/時 (約 \$3.6/月)
Elastic IP (未使用)	\$0.005/時
NAT Gateway	\$0.062/時 + \$0.062/GB
ALB	\$0.0243/時 + LCU
NLB	\$0.0243/時 + NLCU
CloudFront データ転送	\$0.114/GB (最初の 10TB)
インターネットへの送信	\$0.114/GB (最初の 10TB)
Route 53 ホストゾーン	\$0.50/月
Route 53 クエリ	\$0.40/100 万

43.2.3 サーバーレス

項目	料金
Lambda リクエスト	\$0.20/100 万
Lambda 実行 (x86)	\$0.0000166667/GB 秒
Lambda 実行 (ARM/Graviton)	\$0.0000133334/GB 秒
Lambda Free Tier	月 100 万リクエスト・40 万 GB 秒 (永続)
API Gateway HTTP API	\$1.00/100 万リクエスト
API Gateway REST API	\$3.50/100 万リクエスト
DynamoDB On-Demand 書込	\$1.4221/100 万 WRU
DynamoDB On-Demand 読込	\$0.2845/100 万 RRU

項目	料金
DynamoDB ストレージ	\$0.285/GB-月
Step Functions Standard	\$0.025/1,000 ステート遷移
Step Functions Express	\$1/100 万リクエスト + 実行時間

43.2.4 AI / ML

項目	料金 (目安)
Bedrock Claude Sonnet (input)	\$3/100 万トークン
Bedrock Claude Sonnet (output)	\$15/100 万トークン
Bedrock Claude Haiku (input)	\$0.25/100 万トークン
Bedrock Nova Pro (input)	\$0.80/100 万トークン
Bedrock Titan Embeddings v2	\$0.02/100 万トークン
SageMaker ml.m5.large	\$0.115/時
OpenSearch Serverless OCU	\$0.24/時/OCU (最低 4 OCU)
Rekognition 画像分析	\$0.001/画像 (最初の 100 万)
Translate	\$15/100 万文字
Transcribe	\$0.024/分

43.3 主要 ARN フォーマット

```

arn:aws:s3:::my-bucket
arn:aws:s3:::my-bucket/path/to/key
arn:aws:ec2:ap-northeast-1:123456789012:instance/i-0abcd1234
arn:aws:ec2:ap-northeast-1:123456789012:vpc/vpc-0abcd1234
arn:aws:iam::123456789012:user/alice
arn:aws:iam::123456789012:role/MyRole
arn:aws:iam::aws:policy/AdministratorAccess
arn:aws:lambda:ap-northeast-1:123456789012:function:my-fn
arn:aws:lambda:ap-northeast-1:123456789012:function:my-fn:1
arn:aws:dynamodb:ap-northeast-1:123456789012:table/my-table
arn:aws:rds:ap-northeast-1:123456789012:db:my-db
arn:aws:sns:ap-northeast-1:123456789012:my-topic
arn:aws:sqs:ap-northeast-1:123456789012:my-queue
arn:aws:secretsmanager:ap-northeast-1:123456789012:secret:my-secret-AbCdEf
arn:aws:kms:ap-northeast-1:123456789012:key/12345678-1234-1234-1234-123456789012
arn:aws:bedrock:ap-northeast-1::foundation-model/anthropic.claude-sonnet-4-6-20251201-v1:0

```


43.4 必須リソースの上限値（クォータ）

項目	デフォルト上限
VPC 数 / リージョン	5
サブネット数 / VPC	200
SG 数 / VPC	2,500
SG ルール / SG	60 / 60 (in/out)
ENI / SG	5 (per ENI)
Elastic IP / リージョン	5
EC2 vCPU / オンデマンド (Standard)	リージョンごとに異なる、申請必要
Lambda 同時実行	1,000
Lambda ENV 変数 合計	4 KB
Lambda 実行時間	15 分
DynamoDB テーブル / リージョン	2,500
API Gateway API / リージョン	600
S3 バケット / アカウント	100（緩和申請可、最大 1,000）
CloudFormation スタック / リージョン	2,000
スタック内リソース	500
ALB / リージョン	50
Cognito User Pool / リージョン	1,000
IAM ユーザー / アカウント	5,000
IAM ロール / アカウント	1,000
IAM 管理ポリシー / アカウント	1,500

緩和は Service Quotas コンソールから申請可。

43.5 AWS CLI チートシート

```
# 認証情報の確認
aws sts get-caller-identity
aws configure list

# プロファイル指定
aws s3 ls --profile dev
export AWS_PROFILE=dev

# リージョン指定
aws ec2 describe-instances --region ap-northeast-1
export AWS_REGION=ap-northeast-1

# 出力フォーマット
aws ec2 describe-instances --output json
```

```

aws ec2 describe-instances --output table
aws ec2 describe-instances --output text
aws ec2 describe-instances --query 'Reservations[].Instances[].InstanceId'

# ページング自動・無効
aws ec2 describe-instances --no-paginate
aws ec2 describe-instances --max-items 10

# Identity Center ログイン
aws configure sso
aws sso login --profile dev

# CloudShell
aws cloudshell # コンソールから

```

43.5.1 よく使うコマンド

```

# S3
aws s3 ls
aws s3 cp file.txt s3://bucket/
aws s3 sync ./dist s3://bucket/site/ --delete
aws s3 rm s3://bucket/key

# EC2
aws ec2 describe-instances
aws ec2 start-instances --instance-ids i-0abc
aws ec2 stop-instances --instance-ids i-0abc
aws ec2 terminate-instances --instance-ids i-0abc
aws ec2 modify-instance-metadata-options --instance-id i-0abc --http-tokens required

# IAM
aws iam list-users
aws iam list-roles
aws iam get-role --role-name MyRole
aws iam list-attached-role-policies --role-name MyRole

# Lambda
aws lambda list-functions
aws lambda invoke --function-name my-fn --payload '{}' /tmp/out.json
aws lambda update-function-code --function-name my-fn --zip-file fileb://func.zip

# DynamoDB
aws dynamodb scan --table-name my-table --max-items 10
aws dynamodb get-item --table-name my-table --key '{"id":{"S":"1"}}'
aws dynamodb put-item --table-name my-table --item '{"id":{"S":"1"},"name":{"S":"a"}}'

# CloudWatch Logs
aws logs tail /aws/lambda/my-fn --follow

```

```

aws logs filter-log-events --log-group-name /aws/lambda/my-fn --filter-pattern ERROR
aws logs start-query --log-group-name /aws/lambda/my-fn --start-time ... --end-time ...
--query-string "fields @timestamp, @message | filter @message like /ERROR/"

# SSM
aws ssm start-session --target i-0abc
aws ssm get-parameter --name /myapp/db_password --with-decryption
aws ssm put-parameter --name /myapp/x --type String --value "hello"

# CloudFormation / CDK
aws cloudformation describe-stacks --stack-name my-stack
aws cloudformation describe-stack-events --stack-name my-stack
aws cloudformation delete-stack --stack-name my-stack
npx cdk deploy
npx cdk diff
npx cdk destroy

# Cost
aws ce get-cost-and-usage --time-period Start=2026-04-01,End=2026-04-30 --granularity
MONTHLY --metrics UnblendedCost

```

43.6 コンソールの「どこにあるか」早見表

やりたいこと	場所
請求情報	右上アカウント名 → Billing and Cost Management
予算アラート設定	Billing → Budgets
Free Tier 確認	Billing → Free Tier
サポートケース	右上の? → AWS Support
リソース棚卸し	Resource Groups → Tag Editor
全リージョンのリソース	Tag Editor で「すべてのリージョン」
サービス上限緩和	Service Quotas
新規アカウント作成	Organizations → アカウント追加
Identity Center ログイン URL	IAM Identity Center → Settings → User portal URL
自分の ARN	IAM → Users / 右上ドロップダウン / aws sts get-caller-identity
GuardDuty 有効化	GuardDuty → 有効にする
CloudTrail 履歴	CloudTrail → イベント履歴

43.7 セキュリティ初期セットアップ チェックリスト

新規アカウント開設時に 最初の 30 分 で：

- [] ルートユーザーに passkey（または ハードウェアキー） MFA
- [] ルートユーザーのアクセスキーは作らない／削除

- [] AWS Budgets で月次予算アラート（実測 80%、予測 100%）
- [] Free Tier 使用量アラート有効化
- [] IAM Identity Center を有効化（または作業用 IAM ユーザー + MFA）
- [] CloudTrail 全リージョン有効、S3 に保管、ログファイル整合性検証
- [] GuardDuty 全リージョン有効
- [] IAM Access Analyzer 有効
- [] EBS デフォルト暗号化を全リージョンで ON
- [] S3 アカ운トレベル Block Public Access ON
- [] SCP で利用リージョン制限（東京 + 大阪のみ等）
- [] AWS CLI が動作（`aws sts get-caller-identity`）

43.8 トラブルシューティング 早見表

症状	まず見るべき場所
請求が多い	Cost Explorer → サービス別 → リージョン別、Cost Anomaly Detection
権限エラー	CloudTrail → eventName で検索、IAM Policy Simulator
API スロットル	CloudWatch メトリクス、Service Quotas
DNS が解決しない	dig、Route 53 hosted zone、レジストラ NS
SSL エラー	ACM 証明書、CloudFront は us-east-1 か
接続できない（EC2/RDS）	SG、NACL、ルートテーブル、VPC エンドポイント
Lambda がエラー	CloudWatch Logs <code>/aws/lambda/<name></code>
API Gateway 504	Lambda タイムアウト 29 秒、バックエンド遅延
S3 403	Block Public Access、バケットポリシー、IAM、署名付き URL
CloudFormation 失敗	Events タブ、ROLLBACK_FAILED ステート
request signature mismatch	時刻同期、AWS_ACCESS_KEY_ID 設定

43.9 Pricing Calculator URL

新サービスや構成変更前に **Pricing Calculator** で見積：

- 公式：`calculator.aws`
- アカウントなしでも使える
- 構成を保存・共有可能
- 月額・年額・3 年想定で表示

43.10 公式ドキュメントの場所

- **General Reference**：`docs.aws.amazon.com/general/latest/gr/`
- **Service Endpoints & Quotas**：上記内
- **Whitepapers**：`aws.amazon.com/whitepapers/`
- **Architecture Center**：`aws.amazon.com/architecture/`

- AWS What's New : aws.amazon.com/new/
- AWS Status : status.aws.amazon.com
- Blog : aws.amazon.com/blogs/

43.11 緊急時の連絡先

- AWS Support（コンソール内ケース作成）
- AWS Health Dashboard：障害情報
- Trusted Advisor：Business / Enterprise サポートで全推奨が見える
- Account Manager（Enterprise Support 以上）

本章は本書の **最終資料**。本文中で参照するときの索引として活用してほしい。